



Resumen bibliografía

🕒 Created	@September 7, 2021 8:53 PM
▼ Class	Introducción a los Sistemas Distribuidos
▼ Type	Resumen
☑ Completed	✓
▼ Status	Materia Aprobada

Capítulo 1: Computer Networks and the Internet

La internet pública es UN tipo de computer network (red de computadoras).

La internet es una red que conecta billones de computadoras (o dispositivos) alrededor del mundo. La internet es los equipos, enlaces y protocolos que dan soporte.

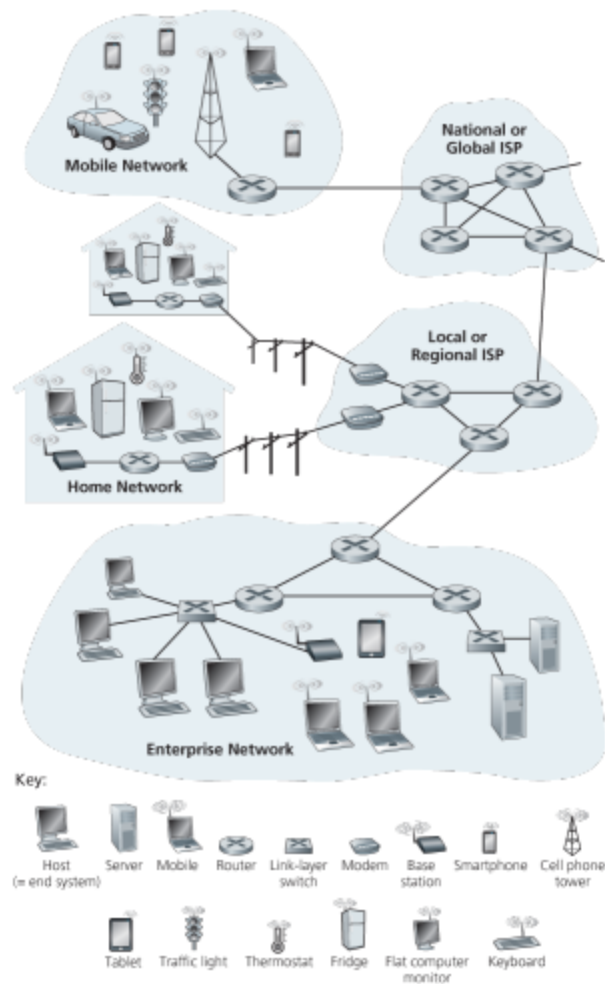


Figure 1.1 Some pieces of the Internet

Los end systems -o hosts- (computadoras y dispositivos) se conectan entre si por una red de enlaces (communication links) y packet switches.

Los enlaces transmiten datos en forma de paquetes desde un sistema a otro (los paquetes se arman-desarman a los datos originales).

Un switch recibe un paquete por una de sus entradas y lo destina a una de sus salidas. Hoy en día, los dos tipos más comunes son: routers y link-layer switches (conmutador de capa de enlace).

Al camino que hace un paquete desde que sale y llega a destino, se lo llama route o path.

Thus, in many ways, packets are analogous to trucks, communication links are analogous to highways and roads, packet switches are analogous to intersections, and end systems are analogous to buildings. Just as a truck takes a path through the transportation network, a packet takes a path through a computer network.

Los hosts acceden a Internet a través de ISPs (Internet Service Providers) [pueden ser ISPs residenciales como el cable local o compañías de teléfono, ISPs corporativas, universidades, etc]

Cada ISP es una red con sus enlaces y switches.

Las ISPs también proveen acceso a Internet a proveedores de contenido, conectando sitios web y servidores de video a Internet.

Como Internet se trata de conectar todos los hosts entre sí, las ISPs deben estar interconectadas. Se conectan mediante ISPs "superiores" (de tier superior) que consisten en routers interconectados por enlaces de fibra óptica de alta velocidad.

Cada ISP funciona independientemente.

Todo lo que conforma a la Internet se rige bajo **protocolos** que controlan el envío y recepción de información.

Internet es también una infraestructura que provee servicios a aplicaciones.

The applications are said to be distributed applications, since they involve multiple end systems that exchange data with each other. Importantly, Internet applications run on end systems—they do not run in the packet switches in the network core. Although packet switches facilitate the exchange of data among end systems, they are not concerned with the application that is the source or sink of data

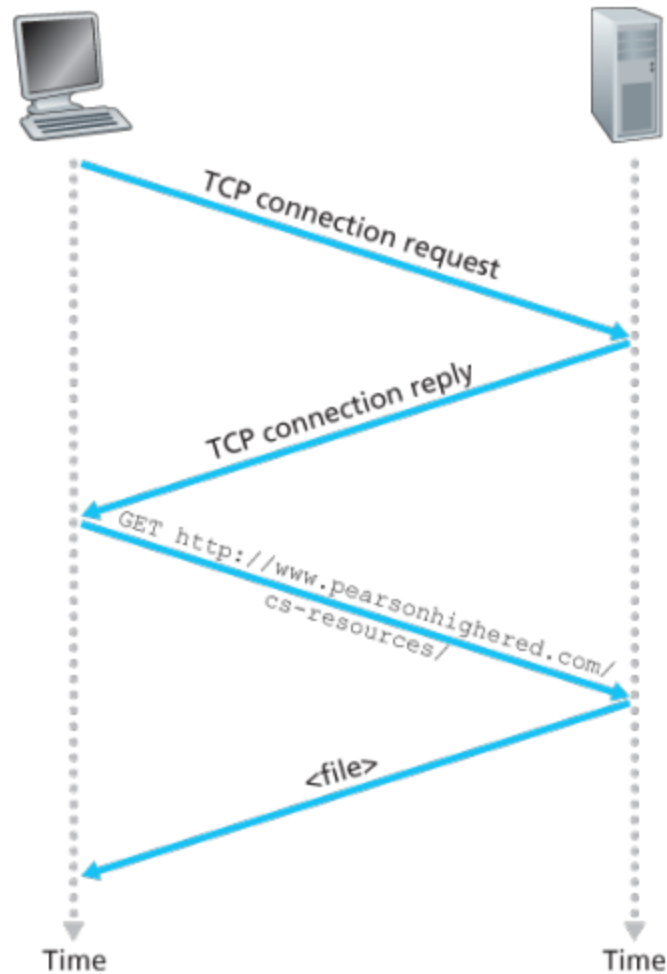
Cómo hace un programa que está corriendo en un determinado sistema para indicarle a Internet que tiene que enviar cierta información a otro programa que está corriendo

en otro sistema distinto? Para eso todos los hosts cuentan con un *socket interface* que lo especifica. Esta interfaz es una serie de reglas que el sistema que envía debe seguir para que internet pueda enviar la información a destino.

Suppose Alice wants to send a letter to Bob using the postal service.

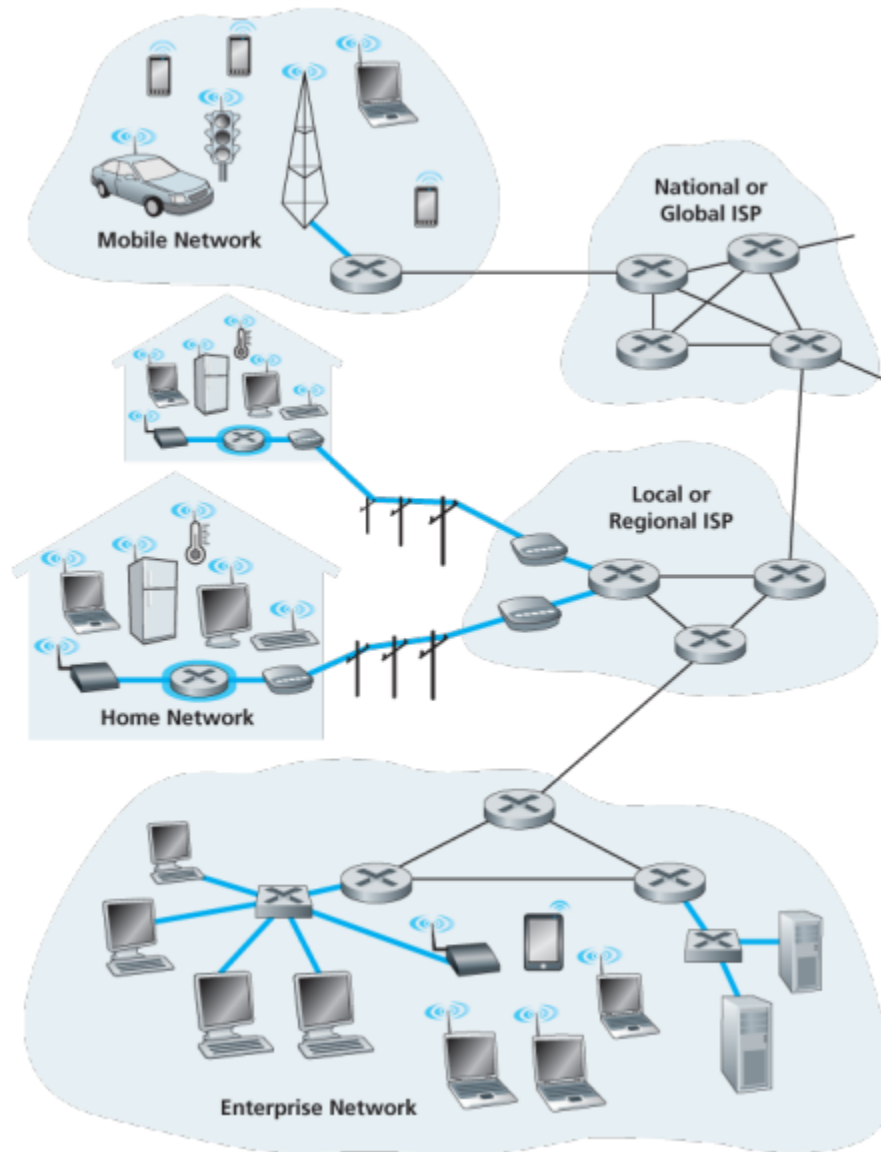
Alice, of course, can't just write the letter (the data) and drop the letter out her window. Instead, the postal service requires that Alice put the letter in an envelope; write Bob's full name, address, and zip code in the center of the envelope; seal the envelope; put a stamp in the upper-right-hand corner of the envelope; and finally, drop the envelope into an official postal service mailbox. Thus, the postal service has its own "postal service interface," or set of rules, that Alice must follow to have the postal service deliver her letter to Bob. In a similar manner, the Internet has a socket interface that the program sending data must follow to have the Internet deliver the data to the program that will receive the data.

Qué es y qué hace un protocolo?



A protocol defines the format and the order of messages exchanged between two or more communicating entities, as well as the actions taken on the transmission and/or receipt of a message or other event.

Las redes de acceso son las que conectan físicamente un host al primer router en un path que va desde el host original hasta cualquier otro host destino.



Ej. DSL (Digital Subscriber Line) → modem que usa la línea telefónica data tal que una llamada telefónica y una conexión a Internet pueden compartir el enlace al mismo tiempo.

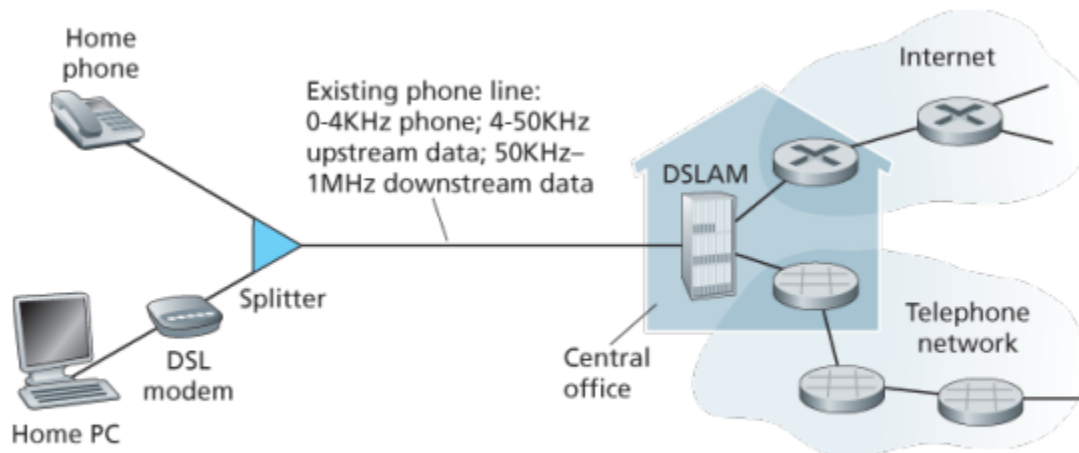


Figure 1.5 DSL Internet access

Otro ej. es el cable. Es la misma idea pero en vez de con el teléfono, se comparte con el cable de la señal de televisión. Una característica importante es que es un medio de transmisión compartido.

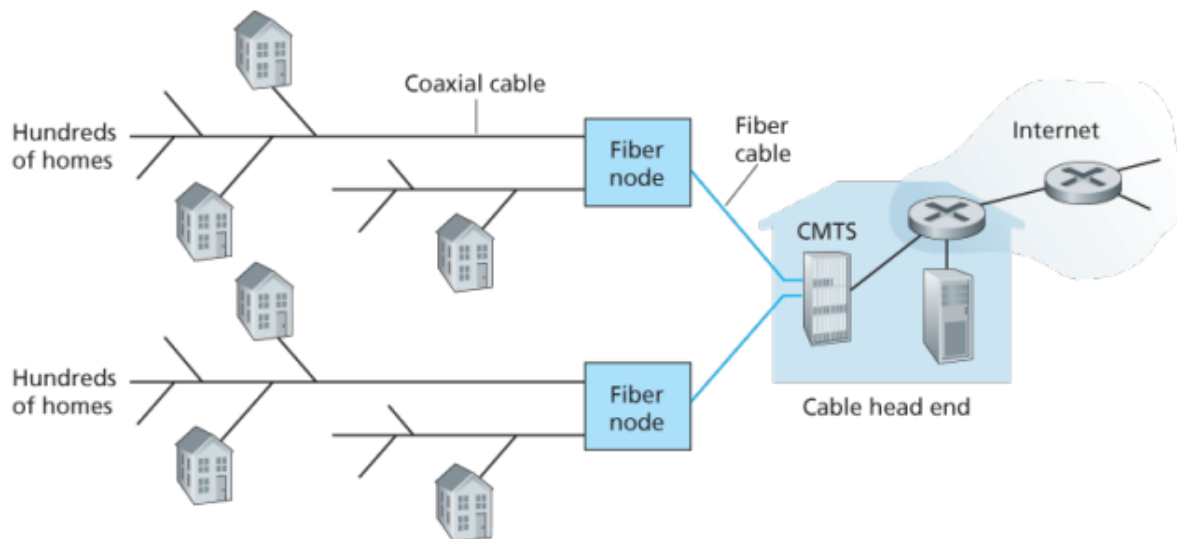


Figure 1.6 A hybrid fiber-coaxial access network

In particular, every packet sent by the head end travels downstream on every link to every home and every packet sent by a home travels on the upstream channel to the head end. For this reason, if several users are simultaneously downloading a video file on the

downstream channel, the actual rate at which each user receives its video file will be significantly lower than the aggregate cable downstream rate. On the other hand, if there are only a few active users and they are all Web surfing, then each of the users may actually receive Web pages at the full cable downstream rate, because the users will rarely request a Web page at exactly the same time.

Otras: Wi-Fi, Ethernet, 3G, 4G, FTTH

La información (bits) viaja a través de medios físicos que pueden ser cable trenzado de cobre, cable coaxial, fibra óptica, satélite, canal de radio. Los medios se pueden clasificar en guiados y no guiados. Los guiados son los que viajan a través de un medio sólido (como fibra óptica), los no guiados son ondas que se propagan (como satélite).

El núcleo (core) de la red es el conjunto de switches y enlaces que interconectan los hosts.

Para enviar un mensaje desde una fuente a un destino, la fuente divide los mensajes largos en pedazos de información más pequeños → paquetes. Los paquetes se transmiten mediante un enlace cuyo rate es la cantidad de bits del paquete ($L[\text{bits}]$) sobre el rate del enlace ($R [\text{bits/s}]$).

La mayoría de los switches usan transmisión "store-and-forward", esto es que el switch tiene que recibir **todo** el paquete antes de poder transmitir el primer bit a la salida. Por ejemplo, si se quiere enviar información de A a B: la fuente A empieza a transmitir en el tiempo 0, al tiempo L/R va a haber transmitido todos los paquetes al router (despreciando el tiempo de propagación), recién ahí que ya recibió todo el paquete, el router puede empezar a transferirlo al destino B, al tiempo $2L/R$ B va a haber recibido el paquete. Si, en cambio, el switch hubiese transmitido todo apenas le llegaba, el tiempo total de demora hubiese sido L/R .

Todos los switches tienen un buffer de salida donde van almacenando los paquetes que llegan de varios enlaces y que todavía no pudieron ser transmitidos porque el respectivo enlace está transportando otros paquetes. Esto genera otro tiempo de demora que depende del nivel de congestión de la red. Puede pasar que un paquete

nuevo se encuentre con el buffer lleno y en ese caso se pierden paquetes (el nuevo o alguno de los que estaba ahí esperando, depende).

Cómo sabe el router a qué enlace debe mandar un paquete? Por la IP, cada host tiene una IP, al enviar un paquete el host fuente indica la IP del host destino. Cada router tiene una tabla (forwarding table) que mapea las direcciones destino con sus enlaces salientes.

For example, suppose Joe is driving from Philadelphia to 156 Lakeside Drive in

Orlando, Florida. Joe first drives to his neighborhood gas station and asks how to get to 156 Lakeside Drive in Orlando, Florida. The gas station attendant extracts the Florida portion of the address and tells Joe that he needs to get onto the interstate highway I-95 South, which has an entrance just next to the gas station. He also tells Joe that once he enters Florida, he should ask someone else there. Joe then takes I-95 South until he gets to Jacksonville, Florida, at which point he asks another gas station attendant for directions. The attendant extracts the Orlando portion of the address and tells Joe that he should continue on I-95 to Daytona Beach and then ask someone else. In Daytona Beach, another gas station attendant also extracts the Orlando portion of the address and tells Joe that he should take I-4 directly to Orlando. Joe takes I-4 and gets off at the Orlando exit. Joe goes to another gas station attendant, and this time the attendant extracts the Lakeside Drive portion of the address and tells Joe the road he must follow to get to Lakeside Drive. Once Joe reaches Lakeside Drive, he asks a kid on a bicycle how to get to his destination. The kid extracts the 156 portion of the address and points to the house. Joe finally reaches his ultimate destination. In the above analogy, the gas station attendants and kids on bicycles are analogous to routers

Existen protocolos de "routing" para setear automáticamente las tablas de forwarding. Un protocolo de estos podría, por ejemplo, determinar el camino más corto desde cada router a cada destino para configurar las tablas.

Hasta ahora hablamos de packet switching, pero también existe otro approach que es circuit switching (ej. telefonos). En este tipo de redes se reservan los resources que se necesitan para transportar la información mientras dure la comunicación entre hosts (en el otro approach se usaba a demanda). Al reservarse los recursos, se garantiza la velocidad de transmisión. La desventaja es que en tiempos "muertos" hay muchos circuitos sin usarse, lo cual es como desperdicio. Por ejemplo, cuando en una llamada nadie habla, la linea esta siendo ocupada al pedo.

As a simple analogy, consider two restaurants, one that requires reservations and another that neither requires reservations nor accepts them. For the restaurant that requires reservations, we have to go through the hassle of calling before we leave home. But when we arrive at the restaurant we can, in principle, immediately be seated and order our meal. For the restaurant that does not require reservations, we don't need to bother to reserve a table. But when we arrive at the restaurant, we may have to wait for a table before we can be seated.

En terminos generales, packet switching es más eficiente. Pensar que si tienes un porcentaje alto de usuarios inactivos, con circuit switching se desperdicia mucho tiempo y recursos.

Internet es una red de redes, las ISPs que conectan a los hosts tambien tiene que ser conectadas entre ellas. Modelo de tiers. Arriba de todo estan los ISPs Tier 1, que deben ser una 12, son globales.

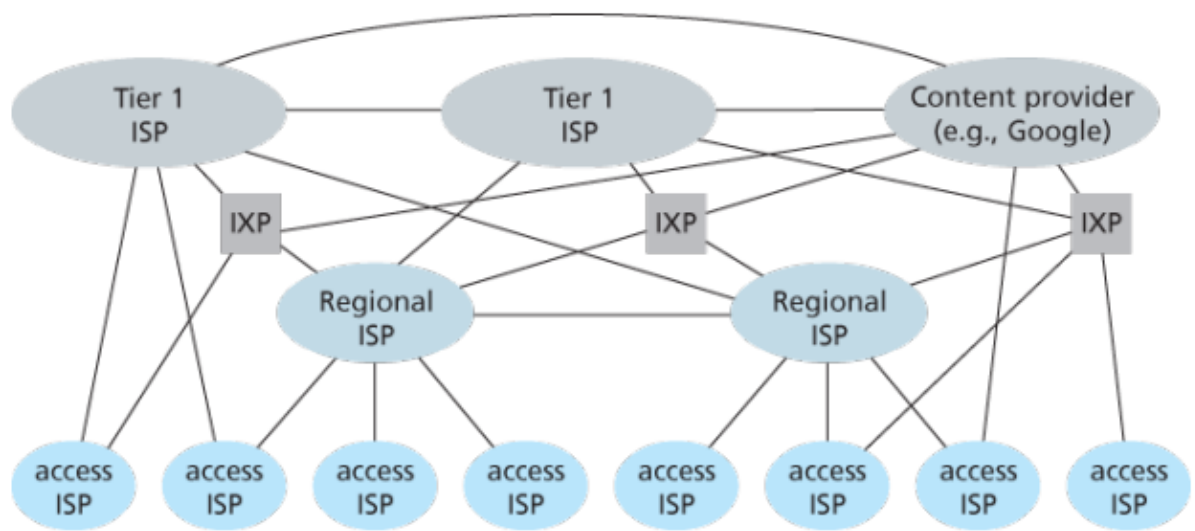


Figure 1.15 Interconnection of ISPs

Algunos problemas que surgen en Internet son: demoras, rendimiento limitado y pérdida de paquetes (información).

El tiempo total que lleva transportar un paquete de información de un sistema a otro es la suma de los siguientes tiempos: tiempo de procesamiento, tiempo esperando en el buffer, tiempo de inserción al medio y tiempo de transmisión (tiempo que voy a pasar en el medio).

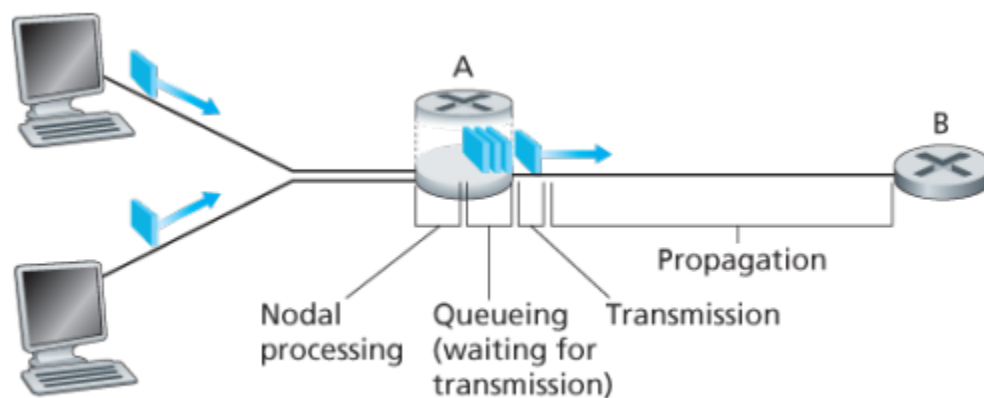


Figure 1.16 The nodal delay at router A

El tiempo de inserción al medio se calcula como d/c , donde d es la distancia que está en el medio y c la velocidad de transmisión del medio.

El tiempo de transmisión se calcula como L/R , donde L es la cantidad de bits que se transmiten y R el rate de transmisión.

Traffic intensity: $L \cdot a / R$, donde L es la cantidad de bits [bits], a es [packets/sec] y R es [bits/sec]. Si el resultado es mayor a 1, se dice que la tasa promedio a la que arriban los bits, excede la tasa promedio de inserción en el medio, en tal caso la cola del buffer va a crecer indefinidamente y el tiempo de demora en el buffer va a tender a infinito.

Golden rule: Design your system so that the traffic intensity is no greater than 1

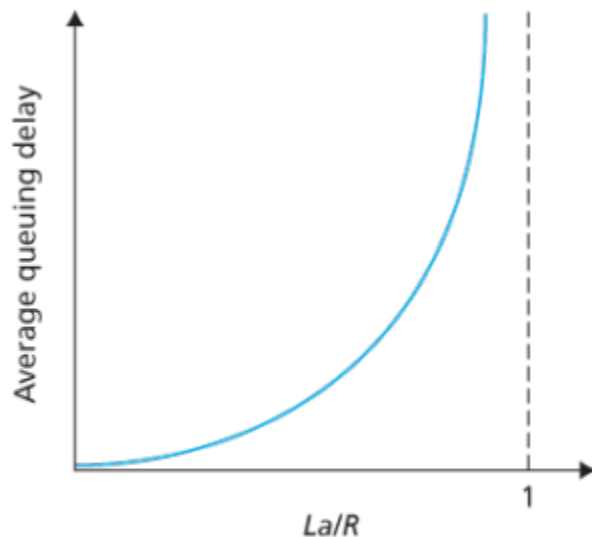


Figure 1.18 Dependence of average queuing delay on traffic intensity

El rendimiento instantáneo en cualquier instante de tiempo es la tasa (en bits/sec) a la que el host destino recibe los datos. Hoy en día el principal limitante del rendimiento de Internet son las redes de acceso.

Los protocolos de redes están organizados en capas. Cada protocolo pertenece a una capa. Cada capa provee de un servicio a la capa superior.

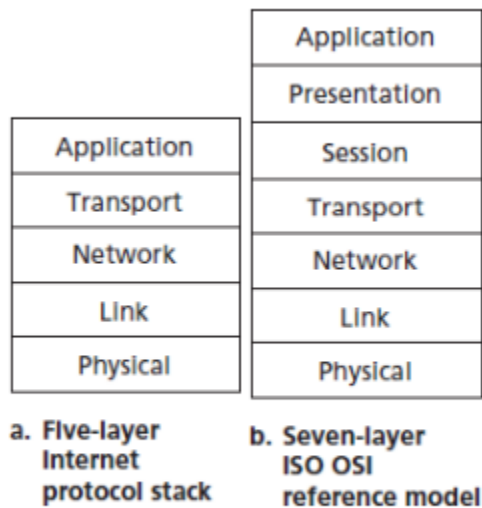


Figure 1.23 The Internet protocol stack (a) and OSI reference model (b)

La capa de aplicaciones es donde residen las redes de aplicaciones y sus protocolos. Por ejemplo, HTTP, SMTP, VPN, DNS, FTP, SSH, DHCP, LDAP. Se distribuye en múltiples hosts tal que intercambian mensajes entre ellos usando algún protocolo.

La capa de transporte dice es la que lleva esos mensajes de una aplicación a otra. Existen dos protocolos: TCP y UDP. Nos referimos a un paquete de capa de transporte como un **segmento**.

La capa de red provee el servicio que entrega un segmento a la capa de transporte destino. Existe un único protocolo: IP.

La capa de enlace mueve paquetes de un nodo (host o router) a otro. Ejemplos: ethernet, wi-fi, bluetooth, NFC, 4G, cable modem, ADSL.

La capa física es la que mueve cada bit a través del enlace. Ejemplos: fibra óptica, cable coaxil, UTP.

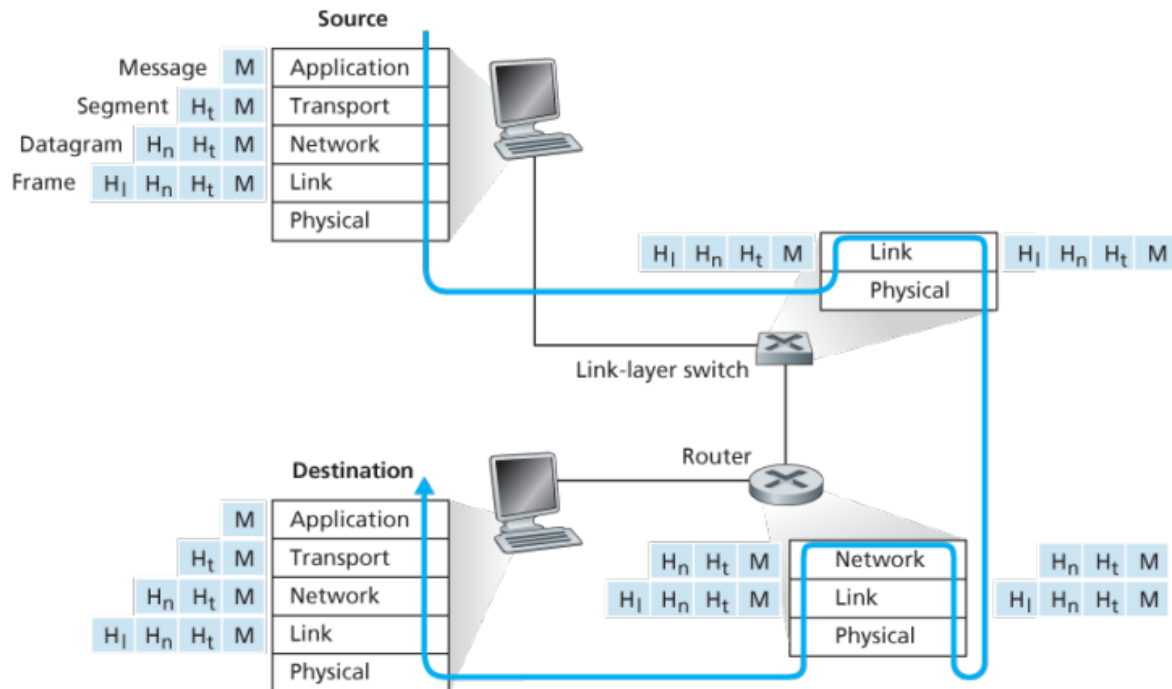


Figure 1.24 Hosts, routers, and link-layer switches; each contains a different set of layers, reflecting their differences in functionality

Tipos de ataques de seguridad:

- DoS (denial-of-service) → se divide en tres categorías: vulnerability attack (se aprovecha de vulnerabilidades), connection flooding (abre muchas conexiones, saturando la red), bandwidth flooding (envía muchos paquetes, saturando los enlaces)
- Packet sniffers
- Masquerade (hacerse pasar por otro usuario, cambiando la dirección)
- Man in the middle

Capítulo 2: Capa de aplicaciones

El primer paso en el desarrollo de una aplicación es su arquitectura. Los dos paradigmas predominantes son: **cliente-servidor** y **peer-to-peer** (P2P).

En una arquitectura cliente-servidor hay un host que sirve de servidor, que está siempre encendido, atendiendo solicitudes de otros hosts denominados clientes. Ejemplos:

Web, FTP, Telnet, email.

Características de la arquitectura cliente-servidor:

- Los clientes no interactúan entre sí, no pueden comunicarse directamente, sino que lo hacen a través del servidor.
- El servidor tiene una dirección IP fija y conocida tal que un cliente **siempre** puede contactarlo.

En general, en esta arquitectura no alcanza con un solo servidor para atender todas las solicitudes de clientes, por eso existen los data centers.

En una arquitectura P2P no se depende de servidores, sino que se basa en la comunicación directa entre pares de hosts (peers). Estos *peers* son los dispositivos controlados por los usuarios finales (smartphones, tablets, notebooks). Ejemplos: BitTorrent, Skype.

Existen aplicaciones que utilizan ambas arquitecturas, por ejemplo en servicios de mensajería donde las direcciones IP se trackean mediante servidores pero los mensajes son enviados directamente de usuario a usuario.

Una de las ventajas de P2P es su *self-scalability*, en el caso de una aplicación de file-sharing cada host suma capacidad de servicio a la aplicación al distribuir archivos para otros peers. También se dice que es eficiente económicamente ya que no requiere de una cantidad significativa de servidores y ancho de banda. Las desventajas son que tiene problemas de seguridad, *reliability* y performance.

Las aplicaciones en la red consisten de pares de **procesos** que se envían mensajes entre sí a través de la red. Por ejemplo, en la Web un cliente browser intercambia mensajes con el servidor web; en una aplicación de file-sharing (p2p), un archivo es transferido desde un proceso en un peer a otro proceso en otro peer. En este último caso, se dice que el cliente es el que descarga el archivo y el servidor el que lo sube.

En cualquier comunicación entre un par de procesos, se dice que el proceso que inicia la comunicación es el cliente y el que espera a ser contactado para empezar la comunicación es el servidor.

Un proceso envía y recibe mensajes de la red a través de una interfaz llamada **socket**.

A process is analogous to a house and its socket is analogous to its door. When a process wants to send a message to another process on another host, it shoves the message out its door (socket). This sending process assumes that there is a transportation infrastructure on the other side of its door that will transport the message to the door of the destination process. Once the message arrives at the destination host, the message passes through the receiving process's door (socket), and the receiving process then acts on the message.

El socket es la interfaz entre la capa de aplicaciones y la capa de transporte dentro de un host. También se puede pensar como la API (Application Programming Interface) entre la aplicación y la red.

El desarrollador de la aplicación solo tiene control sobre los siguientes aspectos del socket:

- La elección del protocolo de transporte
- La capacidad de ajustar algunos parámetros de la capa de transporte, como el tamaño del buffer y el tamaño de los segmentos.

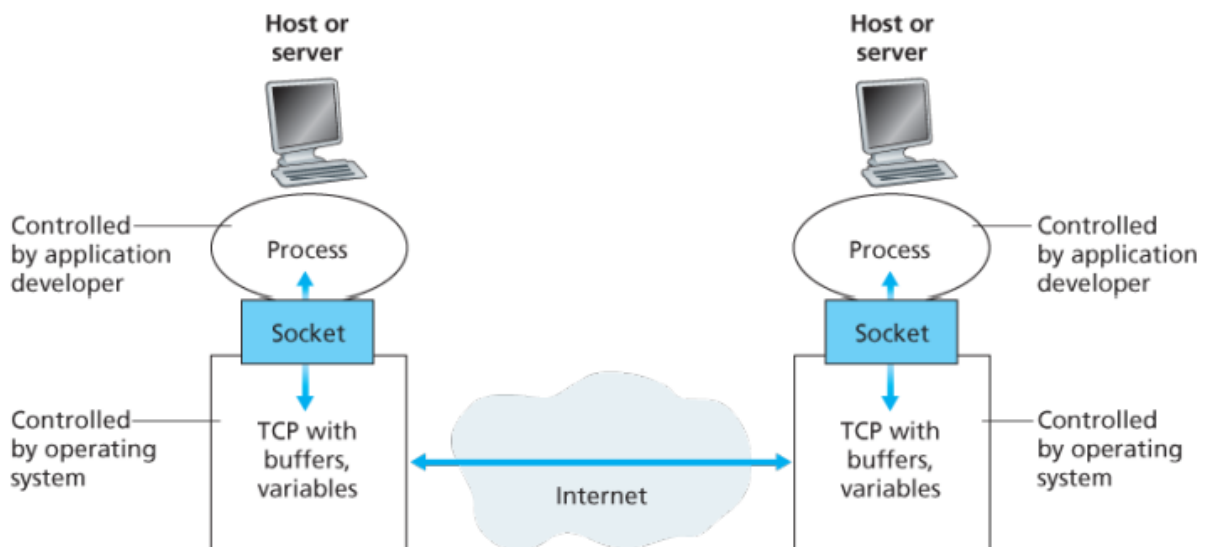


Figure 2.3 Application processes, sockets, and underlying transport protocol

Para que un proceso pueda enviarle algo a otro proceso, este último tiene que tener una dirección.

Para identificar al proceso recibido, se necesita la dirección del host (IP) y un identificador del proceso en el host destino (puerto).

La aplicación manda mensajes a través del socket. Del otro lado del socket está el protocolo de la capa de transporte que tiene que hacer llegar esos mensajes al socket del proceso receptor. El desarrollador de la aplicación debe elegir el protocolo que mejor se adapte a sus necesidades.

Qué servicios ofrece el protocolo de la capa de transporte a las aplicaciones que lo invocan?

- Transferencia de datos confiable (**reliable data transfer**): Se dice que es confiable cuando puede garantizar que no se pierdan o corrompan paquetes de datos.
- Caudal (**throughput**): Es la velocidad a la que el proceso origen puede entregarle bits al proceso receptor. Como el ancho de banda se comparte con otras sesiones, el rendimiento puede variar. Existen aplicaciones (bandwidth-sensitive applications) que piden garantía de un rendimiento mínimo (por ej. aplicaciones multimedia). Otras (elastic applications) usan lo que esté disponible (por ej. mail y transferencia de archivos).
- **Timing**: Por ejemplo, se podría garantizar que ningún bit tarde más de X ms en llegar a destino (interactive real-time applications, teleconferencias, juegos multijugador).
- **Seguridad**: Por ejemplo, se podría encriptar toda la data enviada.

Application	Data Loss	Throughput	Time-Sensitive
File transfer/download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic (few kbps)	No
Internet telephony/ Video conferencing	Loss-tolerant	Audio: few kbps–1Mbps Video: 10 kbps–5 Mbps	Yes: 100s of msec
Streaming stored audio/video	Loss-tolerant	Same as above	Yes: few seconds
Interactive games	Loss-tolerant	Few kbps–10 kbps	Yes: 100s of msec
Smartphone messaging	No loss	Elastic	Yes and no

Figure 2.4 Requirements of selected network applications

Internet provee dos protocolos de transporte: TCP y UDP.

El modelo de servicio **TCP** de entrada provee dos servicios: *connection-oriented* (establece una conexión entre las dos entidades antes de que empiece el flujo de datos - handshaking procedure) y *reliable data transfer*. Además, cuenta con un mecanismo para controlar la congestión. Para proveer seguridad, se lo puede mejorar con SSL en esta capa de aplicación.

El modelo UDP ofrece servicios mínimos. Es connectionless (no handshaking), unreliable data transfer (hasta podría pasar que los datos lleguen en distinto orden al que se enviaron), no tiene un mecanismo para controlar la congestión.

Observación: hoy en día Internet no da garantías de timing ni rendimiento; sin embargo, suele funcionar bien.

Un protocolo de la capa de aplicaciones define cómo intercambiar mensajes entre procesos. En particular:

- Los tipos de mensajes a intercambiar
- La sintaxis de los mensajes
- La información de los campos que se envían en el mensaje

- Reglas para determinar a un proceso cuándo y cómo enviar y recibir mensajes

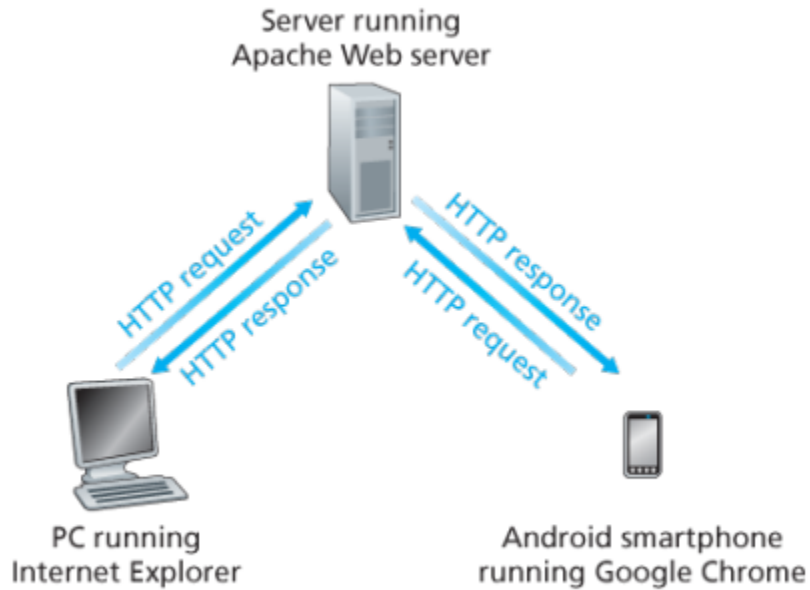
The Web is a client-server application that allows users to obtain documents from Web servers on demand. The Web application consists of many components, including a standard for document formats (that is, HTML), Web browsers (for example, Firefox and Microsoft Internet Explorer), Web servers (for example, Apache and Microsoft servers), and an application-layer protocol. The Web's application-layer protocol, HTTP, defines the format and sequence of messages exchanged between browser and Web server. Thus, HTTP is only one piece (albeit, an important piece) of the Web application

HTTP es el protocolo de la capa de aplicación de la Web. Se implementa en dos programas: en el cliente y en el servidor. El programa cliente y el programa servidor interactúan entre sí intercambiando mensajes HTTP. HTTP define la estructura de los mensajes y cómo deben enviarlos tanto cliente como servidor.

Una **página web** consiste de objetos. Un objeto en este contexto, es un archivo accesible por una URL (puede ser una imagen JPEG, un archivo HTML, un applet de Java, etc.). Cada URL se compone por dos elementos: el *hostname* del servidor que aloja el objeto y el *path* al objeto.

HTTP define cómo los clientes web deben solicitar páginas web al servidor web y cómo los servidores deben enviar dichas páginas al cliente.

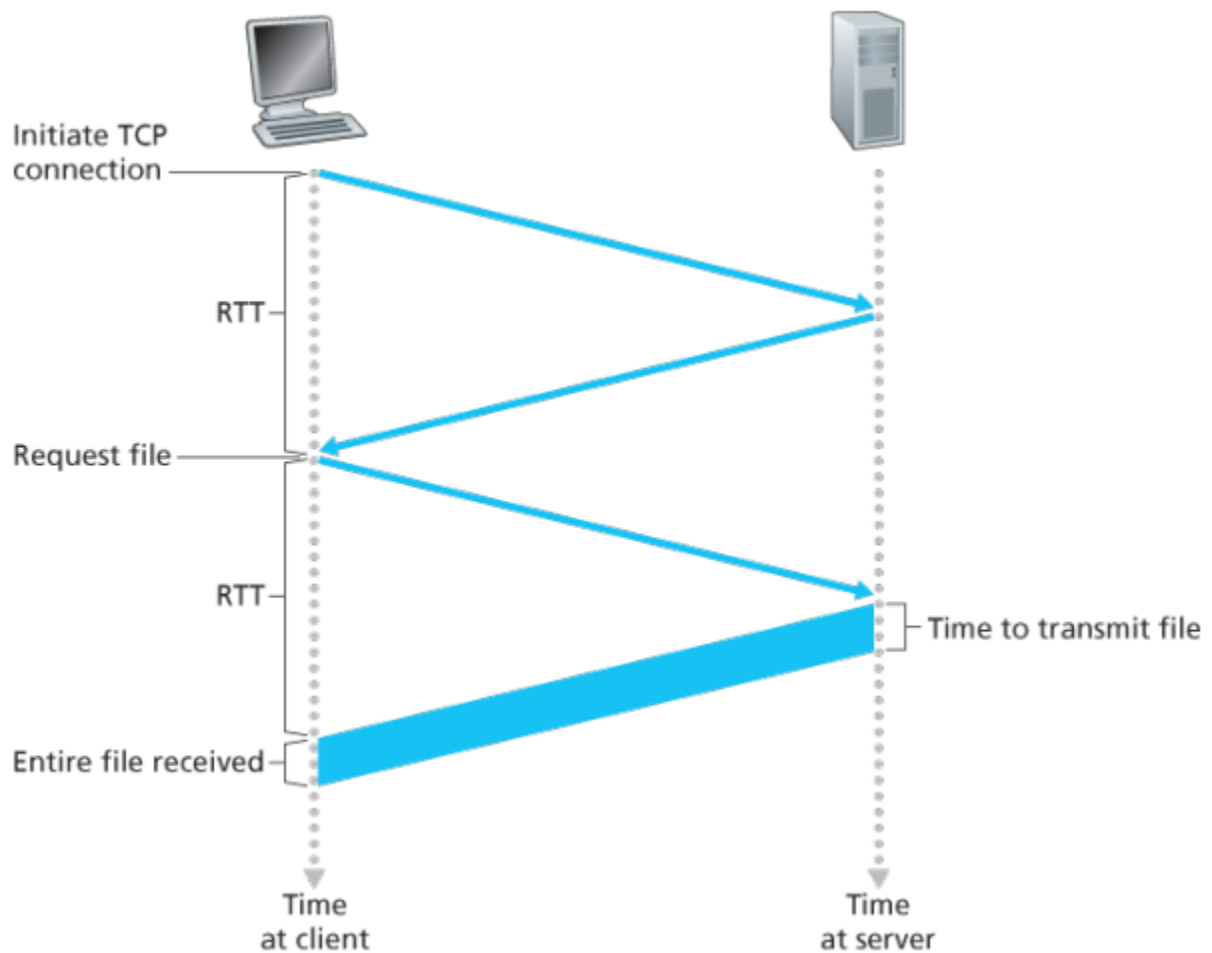
HTTP utiliza TCP como protocolo de transporte.



Observación: el servidor responde solicitudes sin almacenar ningún tipo de información sobre el cliente → **stateless protocol**. Responde a demanda.

Conexiones persistentes son aquellas donde todas las solicitudes se realizan bajo una misma conexión TCP, si se utilizan conexiones TCP separadas por cada par de request/response se dice que es una **conexión no persistente**. Esta es una decisión que debe tomar el desarrollador de la aplicación.

Haciendo una estimación rápida del tiempo total que lleva solicitar un archivo con una conexión, obtenemos 2 RTT (round-trip time - tiempo que tarde en ir y volver). 1 RTT de handshaking y 1 al solicitar y obtener la respuesta.



Una desventaja de las conexiones no persistentes es el hecho de que tenga que crearse y mantenerse una nueva conexión por cada objeto solicitado. Por cada conexión se aloca buffers y variables, entonces puede resultar caro. Además, cada objeto tiene una demora de 2 RTTs.

En conexiones persistentes, se deja abierta la conexión y normalmente, el servidor HTTP la cierra luego de cierta cantidad de tiempo sin haber recibido nada.

Un mensaje de solicitud típico:

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

La primera línea se denomina *request line*, las siguientes son los *headers*. La request line se compone por tres partes: el método, la URL y la versión HTTP. El método puede ser GET, PUT, POST, DELETE o HEAD.

Host indica dónde reside el objeto. *Connection: close* indica que debe cerrarse la conexión luego de responder. *User-agent* el browser que está haciendo la solicitud. *Accept-language* indica el idioma preferente del objeto.

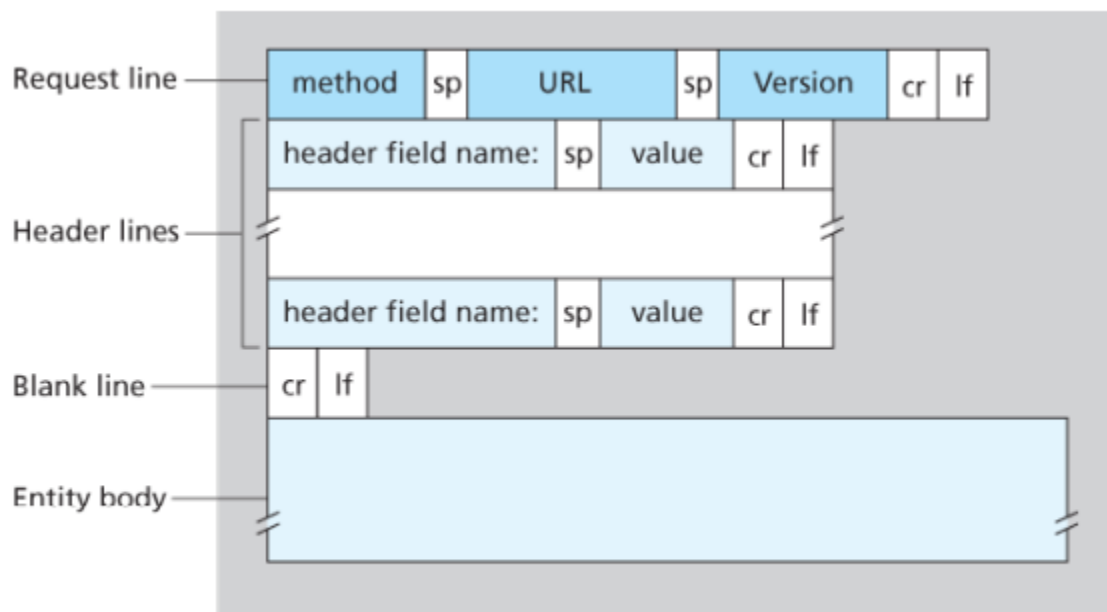


Figure 2.8 General format of an HTTP request message

Mensaje típico de respuesta:

```

HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)

```

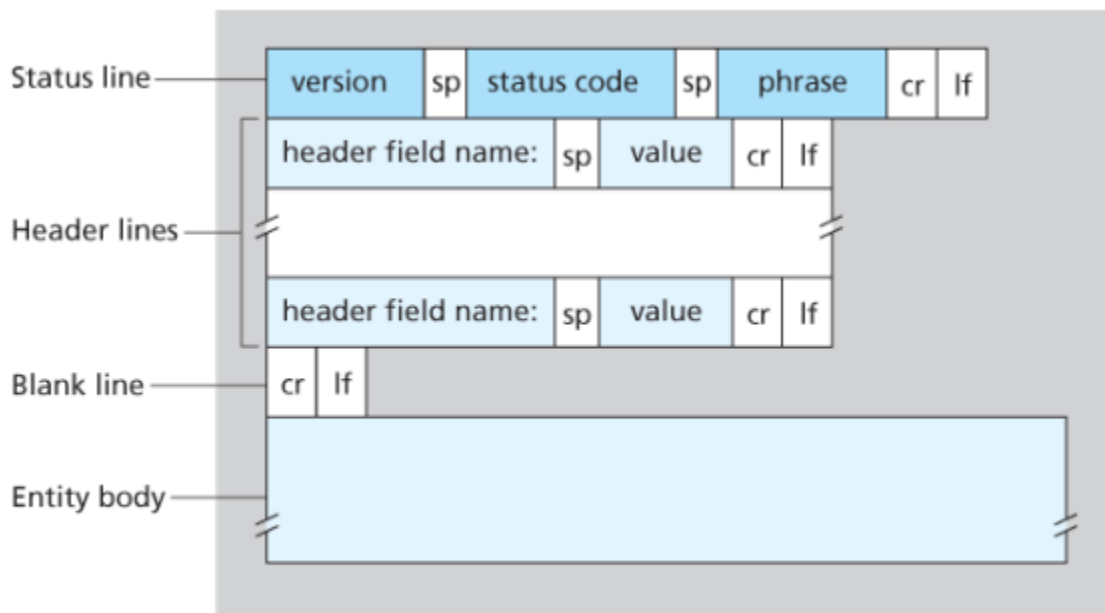


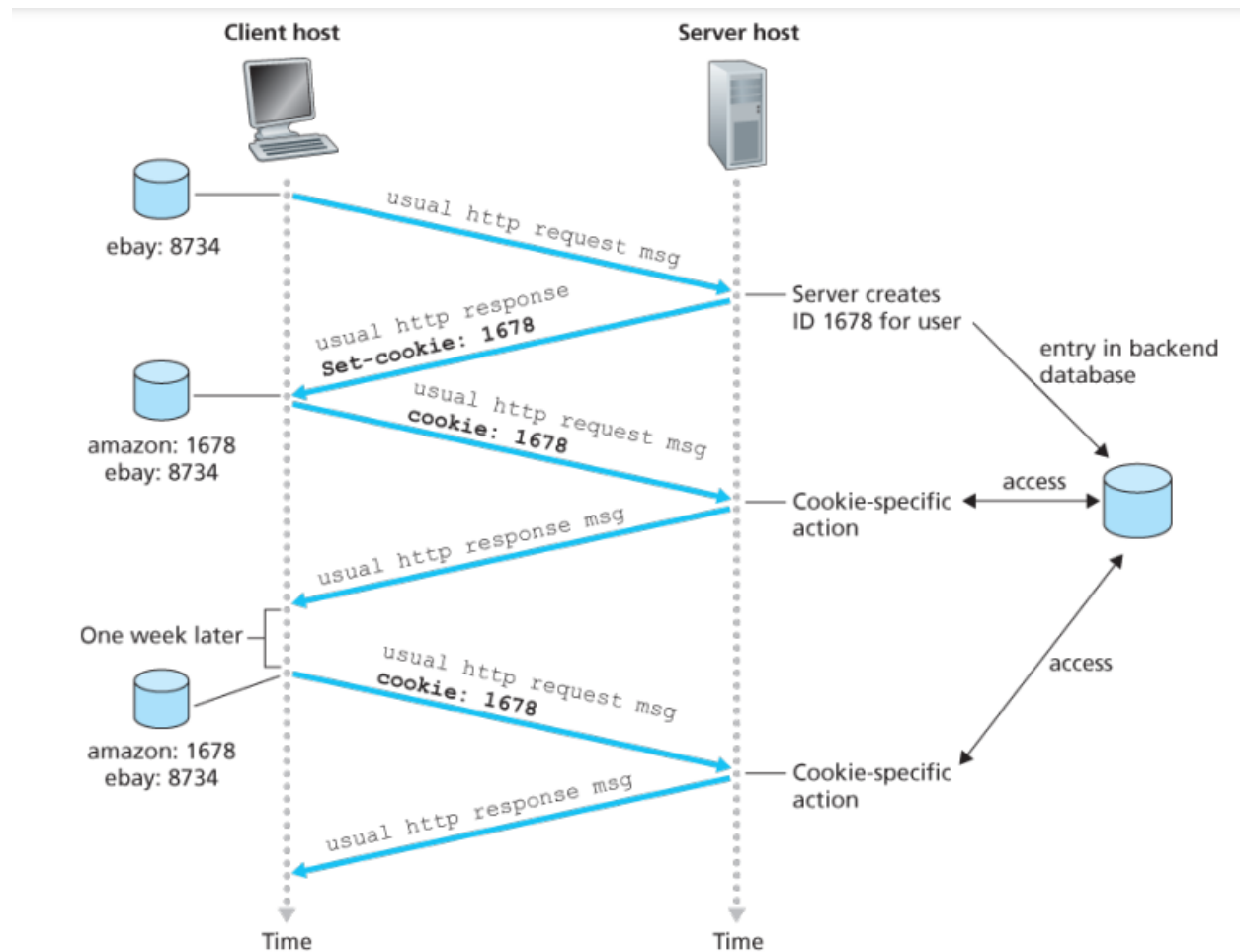
Figure 2.9 General format of an HTTP response message

Algunos códigos de error comunes:

- 200 OK
- 301 Moved Permanently: la nueva URL se especifica en el campo *Location*.
- 400 Bad Request: el servidor no pudo interpretar la solicitud. Suele ser un error del cliente.
- 404 Not Found
- 505 HTTP Version Not Supported

Si bien HTTP es stateless, hay casos donde se necesita identificar usuarios ya sea para restringirles acceso o para cambiar el funcionamiento según el tipo de usuario. Para ello, se utilizan las **Cookies**.

Una cookie está compuesta por: un header en el mensaje de respuesta HTTP, un header en el mensaje de solicitud HTTP, un archivo que se almacena en el host pero lo controla el browser del usuario y una base de datos backend en el sitio web.



Una caché web (o servidor proxy) es una entidad de red que satisface solicitudes HTTP en nombre de un servidor web de origen. La caché web tiene su propio disco de almacenamiento donde mantiene copias de los objetos más recientemente consultados. Es configurable desde el browser.

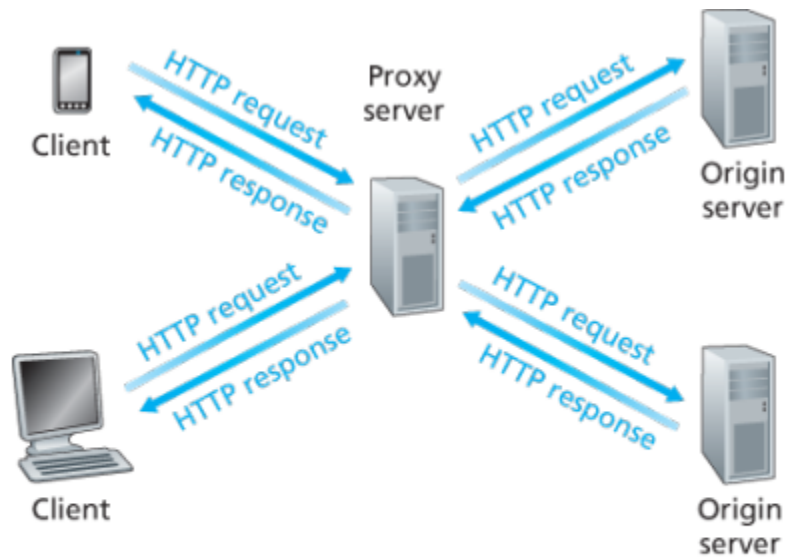


Figure 2.11 Clients requesting objects through a Web cache

1. El browser establece una conexión TCP con el Web cache y le solicita algún objeto
2. El Web caché verifica si tiene almacenada una copia del objeto solicitado. Si la tiene, la devuelve en una respuesta HTTP.
3. Si el web caché no tiene una copia almacenada, establece una conexión TCP con el servidor original y le envía la solicitud.
4. Cuando recibe la respuesta del servidor, guarda una copia del objeto y envía otra copia al cliente.

Dos ventajas importantes: reduce el tiempo de respuesta a solicitudes, pueden reducir significativamente el tráfico en los enlaces de acceso (esto reduce costos porque se ahorra tener que upgradear el ancho de banda).

Algo negativo que podría pasar es que el objeto cacheado no esté actualizado, para eso existe un mecanismo que permite a la caché verificar que sus objetos almacenados estén al día → **Conditional GET**. Se dice que una solicitud HTTP es un conditional get si: la solicitud usa el método GET y si la solicitud incluye el header *If-Modified-Since*. Lo que hace el GET condicional, es pedirle el objeto al servidor sólo si el objeto fue modificado desde la fecha indicada en *If-Modified-Since*.

El **sistema de mails** se compone por tres elementos: user agents, mail servers y el Simple Mail Transfer Protocol (SMTP).

User agents son los agentes que permiten leer, reenviar, responder, guardar y redactar mensajes; por ejemplo, Microsoft Outlook.

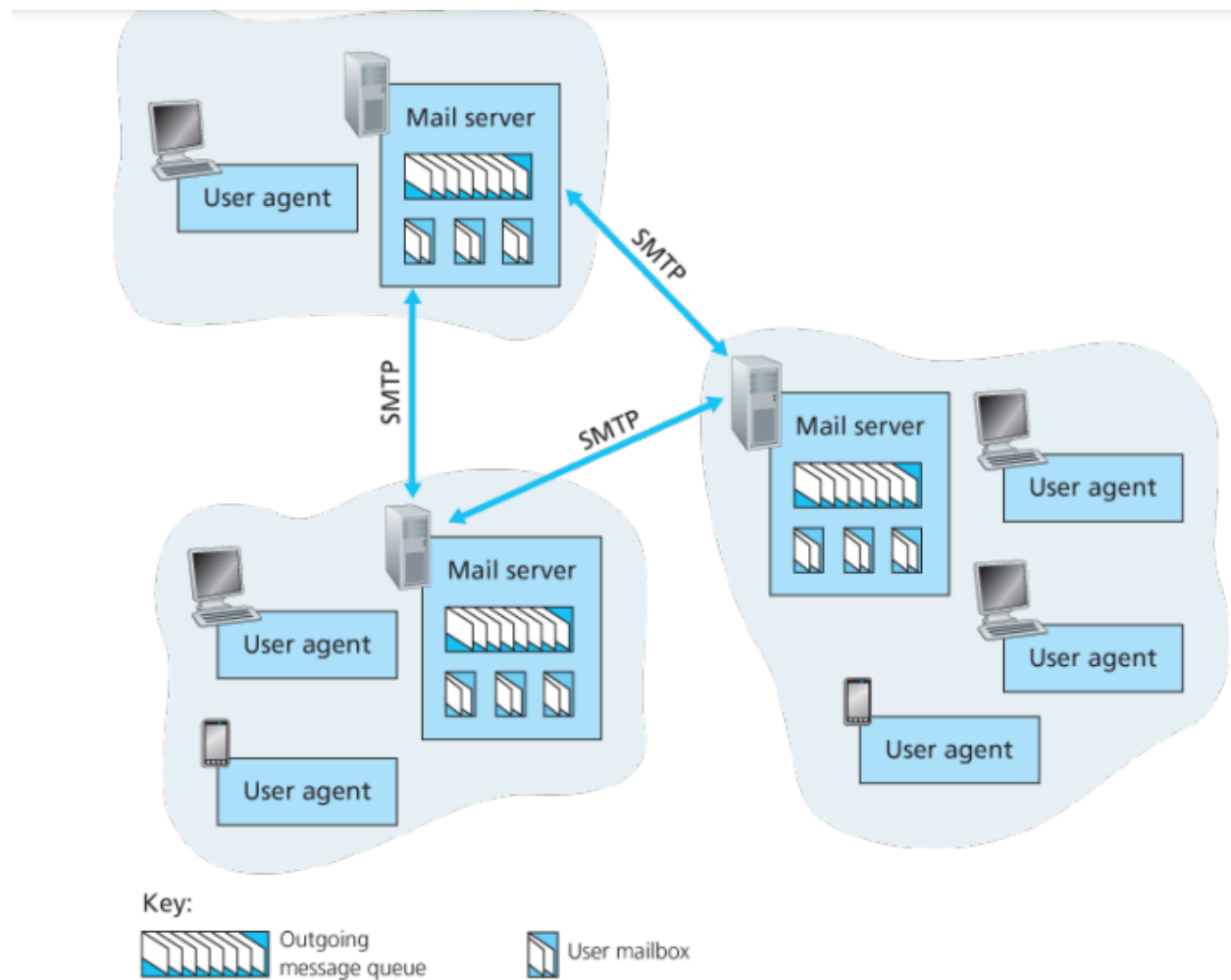


Figure 2.14 A high-level view of the Internet e-mail system

El protocolo SMTP usa el servicio de transferencia de datos confiable de TCP para transferir mails del servidor de mails del sender al servidor de mails del recipient.

Suppose Alice wants to send Bob a simple ASCII message.

1. Alice invokes her user agent for e-mail, provides Bob's e-mail address (for example, bob@some school.edu), composes a

message, and instructs the user agent to send the message.

2. Alice's user agent sends the message to her mail server, where it is placed in a message queue.

3. The client side of SMTP, running on Alice's mail server, sees the message in the message queue. It opens a TCP connection to an SMTP server, running on Bob's mail server.

4. After some initial SMTP handshaking, the SMTP client sends Alice's message into the TCP connection.

5. At Bob's mail server, the server side of SMTP receives the message. Bob's mail server then places the message in Bob's mailbox.

6. Bob invokes his user agent to read the message at his convenience

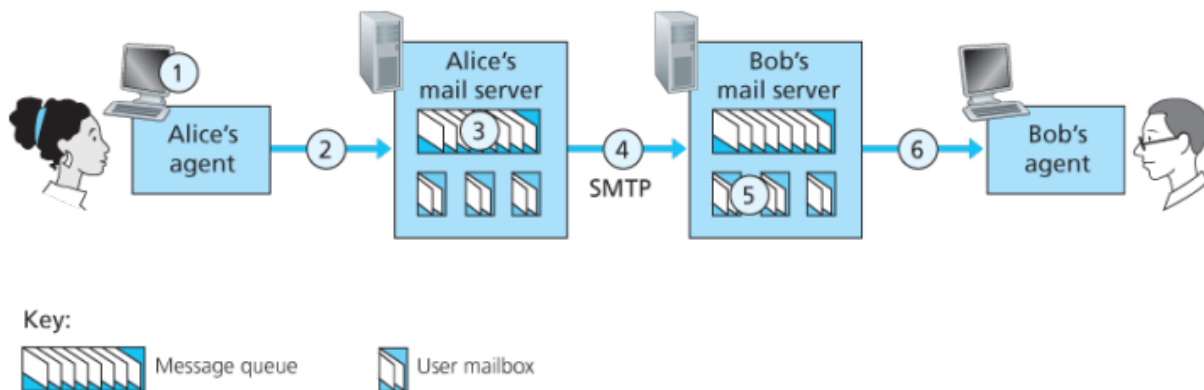


Figure 2.15 Alice sends a message to Bob

Primero, el cliente SMTP establece una conexión TCP al puerto 25 del servidor SMTP. Si el servidor está caído, vuelve a intentar más tarde. Una vez establecida la conexión, cliente y servidor hacen un hand-shaking donde el cliente indica el mail del sender y el del recipient. Luego, el cliente envía el mensaje. Se repite el proceso si el cliente necesita enviar más mensajes, sino se cierra la conexión.

Ejemplo:

```

// S: Servidor
// C: Cliente

S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection

```

Una diferencia importante entre HTTP y SMTP es que HTTP es un **pull protocol**, los usuarios obtienen información de un servidor web (la conexión TCP la inicia quien quiere recibir algo), SMTP es un **push protocol**, el mail server manda información a otro mail server (la conexión TCP la inicia quien quiere enviar algo). Una segunda diferencia es que SMTP requiere que el cuerpo del mensaje esté en formato 7-bit ASCII. Otra diferencia es que HTTP devuelve cada objeto encapsulado en una respuesta distinta, SMTP devuelve todos los objetos en un mismo mensaje.

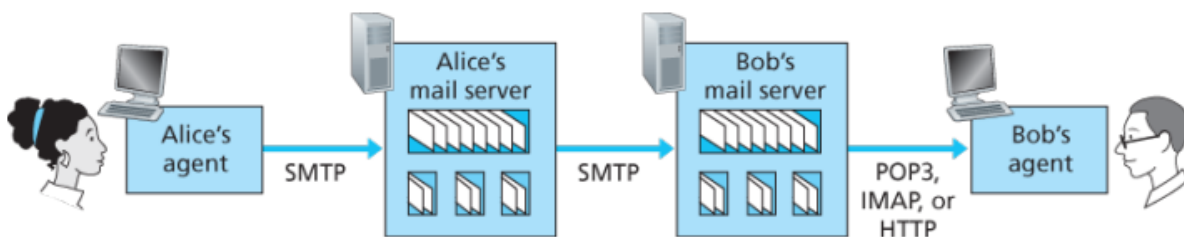


Figure 2.16 E-mail protocols and their communicating entities

Cuando se envía un email de un usuario a otro, lo que pasa es que desde el agente del emisor se empuja el mensaje al servidor del usuario via SMTP, esto tiene la ventaja de que si el servidor del destino está caído se puede reintentar cada X minutos, ya que el servidor origen siempre está encendido. Luego, de nuevo por SMTP se manda el mensaje al servidor del usuario destino. Finalmente, el email viaja del servidor destino

al agente destino mediante algún protocolo de mail access como puede ser POP3, IMAP o HTTP (Yahoo, Gmail, etc).

POP3 comienza cuando el agente abre una conexión TCP con el servidor en el puerto 110. Luego avanza en tres fases: autorización, transacción y actualización.

Ejemplo:

```
// Autorizacion
telnet mailServer 110
+OK POP3 server ready
user bob
+OK
pass hungry
+OK user successfully logged on

// Transacción
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: (blah blah ...
S: .....
S: .....blah)
S: .
C: dele 1
C: retr 2
S: (blah blah ...
S: .....
S: .....blah)
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

Este protocolo tiene problemas para acceder a los datos desde distintos equipos, por eso surge **IMAP**. IMAP asocia cada mensaje con una carpeta y ofrece comandos para poder manipular dichas carpetas. A diferencia de POP3, almacena información del usuario entre sesiones.

Hay dos maneras de identificar a un host: por su nombre (hostname) o por su IP. El **DNS** (Domain Name System) se encarga de traducir uno a otro. El DNS es una base de datos distribuida implementada en una jerarquía de servidores DNS y es un

protocolo de capa de aplicación que permite a los hosts consultar la base de datos distribuida.

Un ejemplo con HTTP:

1. El browser extrae el hostname de la URL y se lo pasa al cliente DNS
2. El cliente DNS envía una consulta con el hostname al servidor DNS
3. Eventualmente, el cliente DNS recibe una respuesta con la dirección IP del hostname
4. Una vez que el browser tiene la IP, puede abrir la conexión TCP con el servidor HTTP

Observación: esta consulta del DNS agregar un pequeño delay.

Otros servicios que ofrece DNS:

- **Host aliasing:** un host puede tener uno o más alias si su hostname es complicado. DNS puede usarse para obtener el hostname real a partir de un alias.
- **Mail server aliasing:** de forma similar, el hostname de un correo electrónico puede ser complicado (por ejemplo que yahoo.com sea un alias de relay1.west-coast.yahoo.com).
- **Load distribution:** sitios con mucho tránsito o datos en general se distribuyen en varios servidores; tal que cada servidor corre en un host distinto y con distinta IP, o sea que una serie de IPs resultan asociadas a un hostname. La base de datos de DNS almacena este set de IPs.

Desventajas de una base de datos DNS centralizada:

- A single point of failure. Si se cae, se cae todo internet.
- Traffic volume.
- Distant centralized database.
- Maintenance.

Por eso, la base de datos se distribuye entre varios servidores.

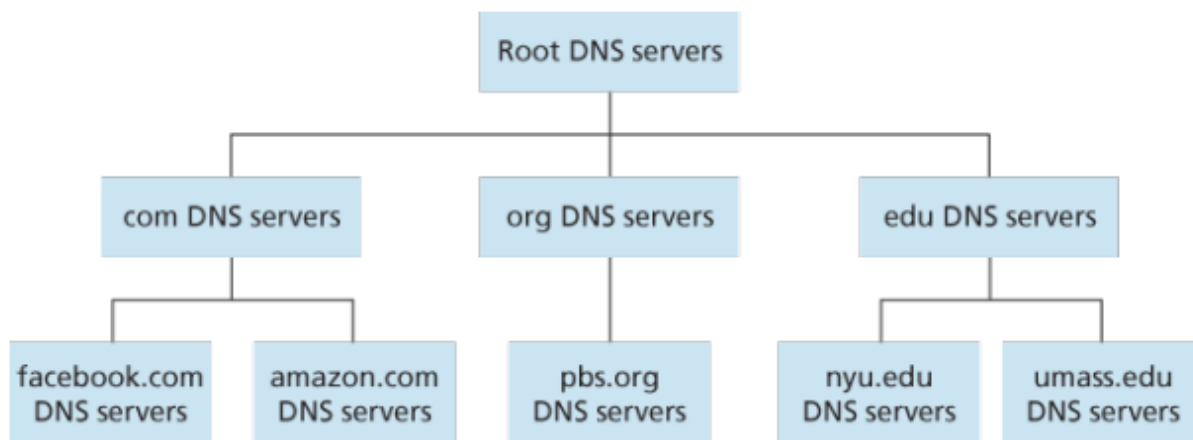


Figure 2.17 Portion of the hierarchy of DNS servers

Suppose the host `cse.nyu.edu` desires the IP address of `gaia.cs.umass.edu`. Also suppose that NYU's local DNS server for `cse.nyu.edu` is called `dns.nyu.edu` and that an authoritative DNS server for `gaia.cs.umass.edu` is called `dns.umass.edu`. As shown in Figure 2.19, the host `cse.nyu.edu` first sends a DNS query message to its local DNS server, `dns.nyu.edu`. The query message contains the hostname to be translated, namely, `gaia.cs.umass.edu`. The local DNS server forwards the query message to a root DNS server. The root DNS server takes note of the `edu` suffix and returns to the local DNS server a list of IP addresses for TLD servers responsible for `edu`. The local DNS server then resends the query message to one of these TLD servers. The TLD server takes note of the `umass.edu` suffix and responds with the IP address of the authoritative DNS server for the University of Massachusetts, namely, `dns.umass.edu`. Finally, the local DNS server resends the query message directly to `dns.umass.edu`, which responds with the IP address of `gaia.cs.umass.edu`.

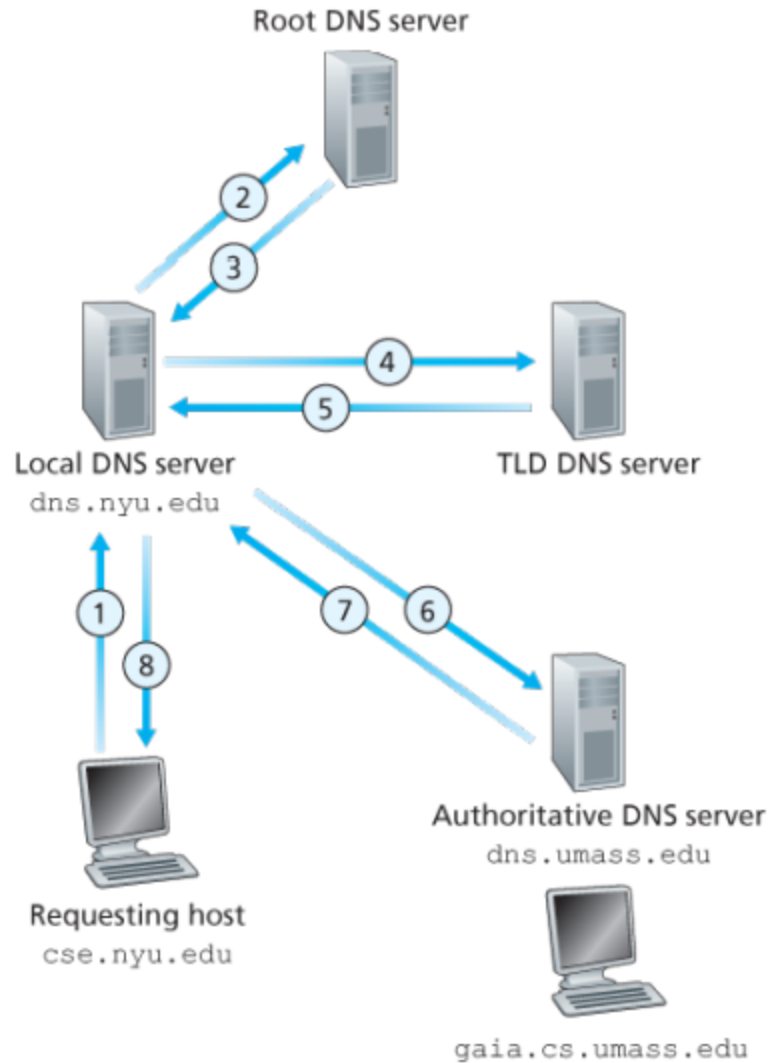


Figure 2.19 Interaction of the various DNS servers

DNS usa una caché para reducir el delay y la cantidad de mensajes DNS.

Los servidores DNS almacenan **resource records** (RRs). Un RR es una tupla de cuatro campos: (*Name*, *Value*, *Type*, *TTL*). TTL es el time to live, determina cuándo debería ser eliminado de la caché. El significado de *Name* y *Value* depende del valor de *Type*:

- Si **Type=A**, *Name* es un hostname y *Value* su IP. Ejemplo: (relay1.bar.foo.com, 145.37.93.126, A)
- Si **Type=NS**, *Name* es un dominio y *Value* el hostname de un servidor DNS de mayor jerarquía que sabe cómo obtener la IP. Ejemplo: (foo.com, dns.foo.com, NS)

- Si **Type=CNAME**, *Value* es el hostname canónico para el alias *Name*. Ejemplo: (foo.com, relay1.bar.foo.com, CNAME)
- Si **Type=MX**, *Value* es el nombre canónico de un mail server con el alias *Name*. Ejemplo: (foo.com, mail.bar.foo.com, MX)

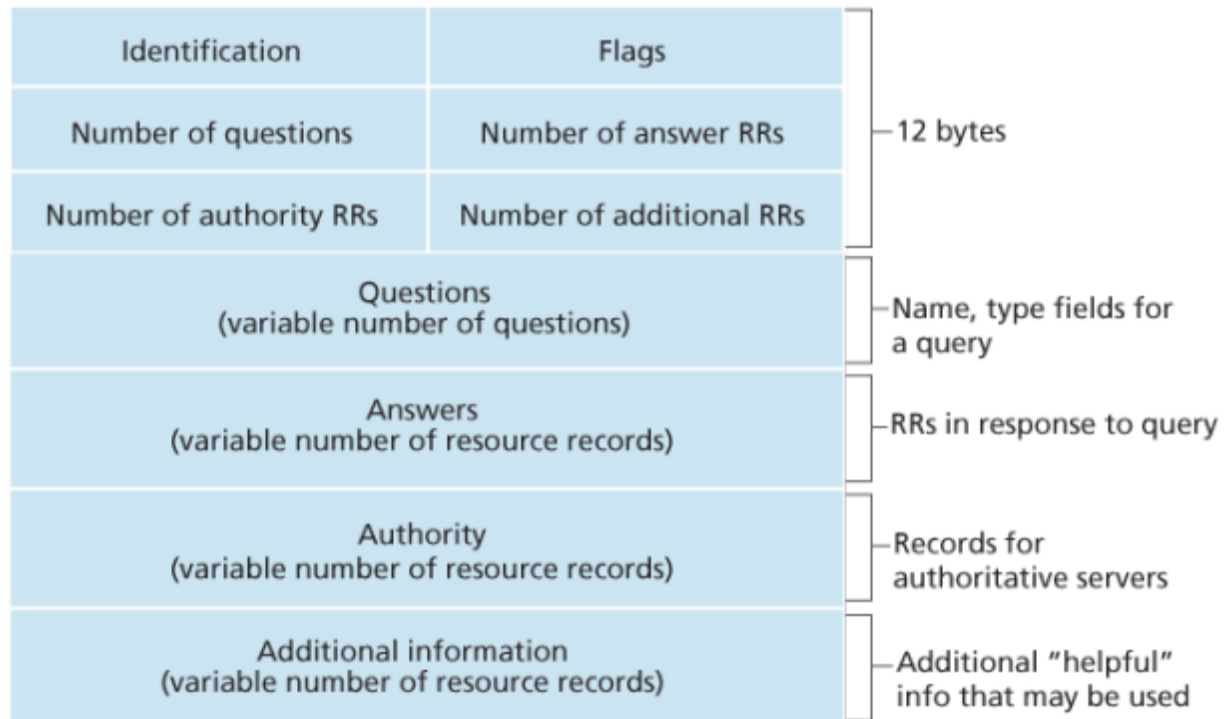
If a DNS server is authoritative for a particular hostname, then the DNS server will contain a Type A record for the hostname. (Even if the DNS server is not authoritative, it may contain a Type A record in its cache.) If a server is not authoritative for a hostname, then the server will contain a Type NS record for the domain that includes the hostname; it will also contain a Type A record that provides the IP address of the DNS server in the Value field of the NS record. As an example, suppose an edu TLD server is not authoritative for the host gaia.cs.umass.edu . Then this server will contain a record for a domain that includes the host gaia.cs.umass.edu , for example, (umass.edu, dns.umass.edu, NS) . The edu TLD server would also contain a Type A record, which maps the DNS server dns.umass.edu to an IP address, for example, (dns.umass.edu, 128.119.40.111, A) .

Formato de mensajes DNS:

- Los primeros 12 bytes componen el *header*. Los primeros 16 bits son el identificador de la query; este identificador se envía con la respuesta para poder matchear las respuestas recibidas con la query que se envió. Flags posibles: 1 bit para indicar si el mensaje es query(0) o reply(1), 1 bit para indicar en el reply si el DNS es authoritative para algun nombre consultado, 1 bit para solicitar al servidor que trabaje recursivamente cuando no encuentre el record, 1 bit para indicar que el servidor soporta recursión.
- La sección de preguntas contiene información sobre la consulta que se está haciendo. Incluye el nombre que se está consultando y el tipo de consulta (type a,

mx, etc.).

- La sección de respuestas contiene los RRs resultantes del nombre consultado.
- La sección de autoridades contiene información sobre otros servidores con autoridad.



Suponiendo que quisiéramos distribuir un archivo grande, como una nueva distribución de Linux por ejemplo, desde un servidor a varios hosts; en una arquitectura cliente-servidor, el servidor debería mandarle una copia a cada uno de los hosts, pero esto resultaría en una carga importante en el servidor y requeriría de mucho ancho de banda. En cambio, si se hace con la arquitectura **P2P**, cada *peer* puede redistribuir una porción del archivo a otros peers, distribuyendo la carga del servidor. Aplicaciones con P2P pueden ser self-scaling.

BitTorrent es un ejemplo de aplicación P2P. Se dice que la colección de peers que participan en la distribución de un archivo es un torrent. Mientras un peer descarga porciones de algún archivo, sube también porciones del archivo a otros peers. Una vez que un usuario descargó el archivo completo, puede retirarse del torrent (selfishly) o quedarse y seguir subiendo partes del archivo para otros peers.

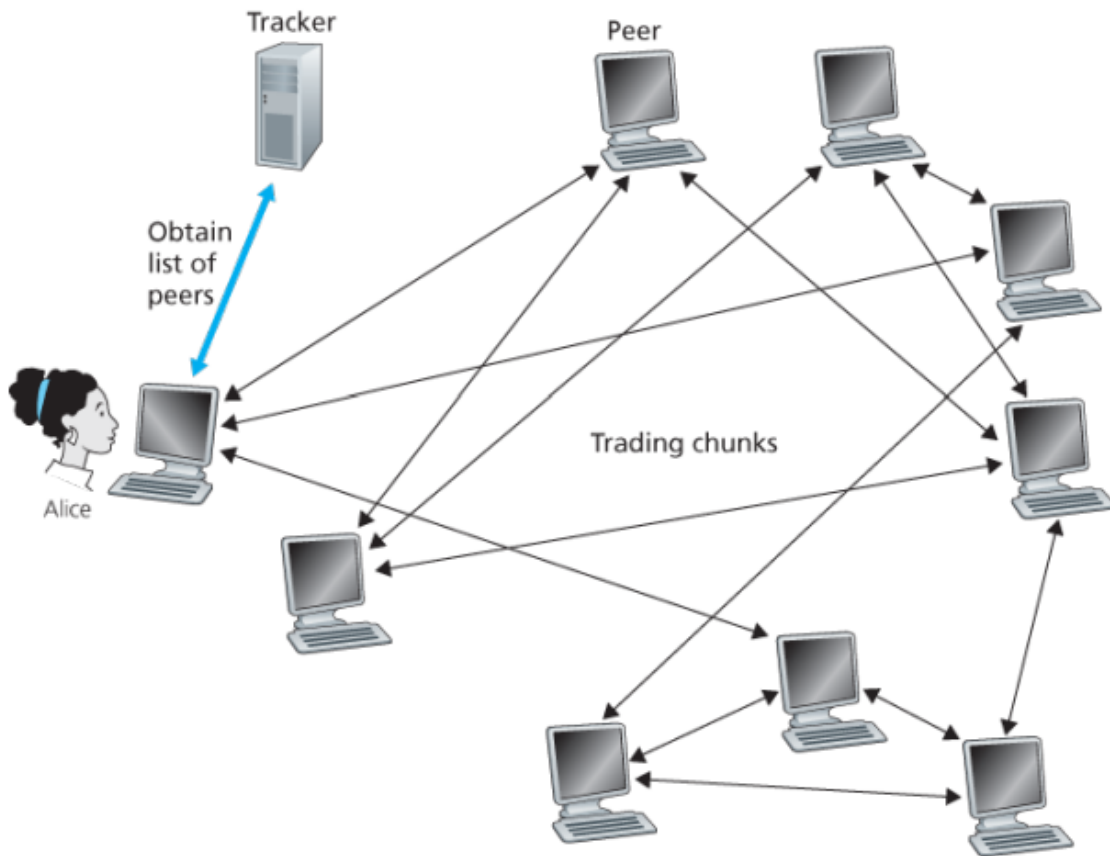


Figure 2.24 File distribution with BitTorrent

Cada torrent tiene un nodo llamado *tracker*. El tracker lleva registro de los peers que están participando del torrent. Cuando aparece un nuevo peer, el tracker selecciona un subconjunto de peers y le envía al nuevo las IPs de dicho subconjunto. Con estas IPs, el nuevo peer puede establecer una conexión TCP con cada uno. Con el tiempo habrá peers que se vayan y otros nuevos, las conexiones van fluctuando. Periódicamente, a partir de estas conexiones TCP se consulta a los otros peers vecinos qué porciones del archivo tienen; se comparan los propios con los ajenos y se piden los que no tenga. Para decidir a qué vecinos pedirles datos se utiliza el mecanismo **rarest first**, la idea es pedir los bits menos repetidos entre los vecinos así se distribuyen equitativamente las partes.

Para decidir a qué vecinos enviarles información solicitada, se le da prioridad a aquellos vecinos que están compartiéndole datos a más velocidad pero también incluye a uno seleccionado de manera random.

En servicios que proveen contenidos multimedia, un primer approach que se podría pensar es que tengan un data center con toda la información almacenada allí y que hagan el stream desde ahí a cualquier parte del mundo. El problema con eso es que: 1. si el cliente está muy lejos del data center, los paquetes van a tener que pasar por un montón de enlaces y routers, y eso puede terminar en delays muy feos; 2. un video popular va a estar enviándose muchas veces sobre los mismos enlaces, esto gasta mucho bandwidth y es costoso; 3. un solo data center es un single point of failure, si se cae, se cae todo.

Como solución, aparecen las **CDN** (content distribution networks). Una CDN tiene múltiples servidores distribuidos por todo el mundo, tal que intenta redirigir las solicitudes de cada cliente al CDN que le provea la mejor experiencia.

Como algunos videos no son muy populares, algo que hacen muchos CDNs es guardar los videos en algunos clusters, si un cluster necesita un video que no tiene, entonces se lo pide a otro y se guarda una copia local al mismo tiempo que hace streaming del video.

Muchas CDNs aprovechan DNS para redirigir solicitudes. Por ejemplo: un proveedor NetCinema distribuye sus videos mediante KingCDN, en el sitio de NetCinema las urls tienen la forma `http://video.netcinema.com/6Y7B23V`. Lo que sucede:

1. El usuario ingresa a la url y el host envia la solicitud DNS para `video.netcinema.com`
2. El DNS local del usuario envía la solicitud al servidor de mayor jerarquía de NetCinema. Este último, en vez de devolver la IP, devuelve otro hostname del dominio de KingCDN, podría ser por ej. `a1105.kingcdn.com`
3. Ahora el DNS local envía una segunda query pero esta vez al servidor DNS de KingCDN y este sí devuelve la IP.

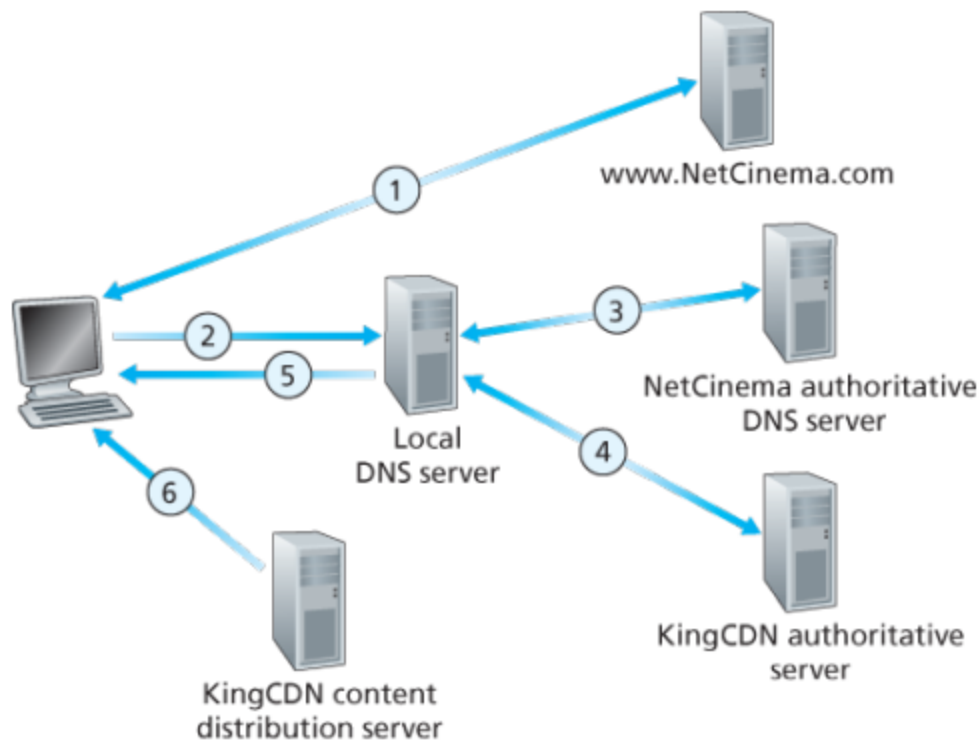


Figure 2.25 DNS redirects a user's request to a CDN server

Capítulo 3: Transport Layer

La capa de transporte provee *comunicación lógica* entre aplicaciones. Comunicación lógica se refiere a que desde la perspectiva de la aplicación, es como si los hosts estuvieran directamente conectados, pero físicamente no es así porque en el medio hay routers y enlaces.

Los protocolos de capa de transporte se implementan en los hosts, pero no en los routers. Convierten los mensajes a enviar en paquetes que se conocen como *segments*. Este segment se le pasa a la capa de red, que lo transforma en un *datagram* y luego se envía al destino.

Así como la capa de transporte provee comunicación lógica entre aplicaciones (procesos), la capa de red provee comunicación lógica entre hosts.

Consider two houses, one on the East Coast and the other on the West Coast, with each house being home to a dozen kids. The kids

in the East Coast household are cousins of the kids in the West Coast household. The kids in the two households love to write to each other—each kid writes each cousin every week, with each letter delivered by the traditional postal service in a separate envelope. Thus, each household sends 144 letters to the other household every week. (These kids would save a lot of money if they had e-mail!) In each of the households there is one kid—Ann in the West Coast house and Bill in the East Coast house—responsible for mail collection and mail distribution. Each week Ann visits all her brothers and sisters, collects the mail, and gives the mail to a postal-service mail carrier, who makes daily visits to the house. When letters arrive at the West Coast house, Ann also has the job of distributing the mail to her brothers and sisters. Bill has a similar job on the East Coast. In this example, the postal service provides logical communication between the two houses—the postal service moves mail from house to house, not from person to person. On the other hand, Ann and Bill provide logical communication among the cousins—Ann and Bill pick up mail from, and deliver mail to, their brothers and sisters. Note that from the cousins' perspective, Ann and Bill are the mail service, even though Ann and Bill are only a part (the end-system part) of the end-to-end delivery process. This household example serves as a nice analogy for explaining how the transport layer relates to the network layer:

- application messages: letters in envelopes
- processes: cousins
- hosts (also called end systems): houses
- transport-layer protocol: Ann and Bill
- network-layer protocol: postal service (including mail carriers)

El protocolo de transporte mueve la información hasta el borde de la red (o sea, la capa de red) y viceversa pero no tiene nada que ver en cómo esa información viaja dentro de la red.

Algunos de los servicios que ofrece un protocolo de transporte están limitados por los servicios de la capa de red. Si la capa de red no puede garantizar los tiempos de demora o el bandwidth, tampoco lo puede hacer la capa de transporte.

El protocolo de red de Internet es IP. El modelo de servicio de IP es *best-effort delivery service*. Esto quiere decir que hace todo lo posible para entregar los segments, pero no garantiza nada. Por eso se dice que IP es un servicio unreliable.

La principal responsabilidad de los protocolos UDP y TCP es la de extender el servicio de entrega entre dos hosts a un servicio de entrega entre dos procesos corriendo en dichos hosts. Esta extensión se llama **transport-layer multiplexing** and **demultiplexing**.

Además proveen de un chequeo de errores en los headers de los segmentos.

UDP solo garantiza esas dos cosas (por eso se dice que es unreliable). TCP provee más servicios, tal como la transferencia confiable de datos y el control de congestión.

La capa de transporte no le entrega los datos directamente al proceso, sino que al socket. Cada socket tiene un *id*. Para saber a qué socket debe entregarle, cada segmento tiene un set de campos donde se especifica el id del socket. Esta acción de entregar datos de la capa de transporte al socket es **demultiplexing**. La acción de juntar pedazos de datos de diferentes sockets, encapsularlos con un header para crear segmentos y pasarselos a la capa de red, es **multiplexing**.

En el header deben estar seteados los campos: **source port number field** y **destination port number field**. Así al hacer el demultiplexing, la capa de transporte se fija en el destination port a qué socket debe mandar el paquete. El source port sirve por si se quiere enviar algo de retorno.

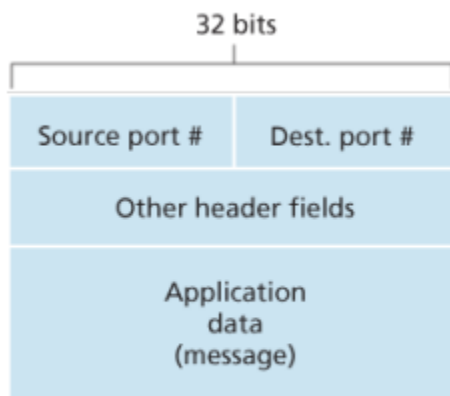
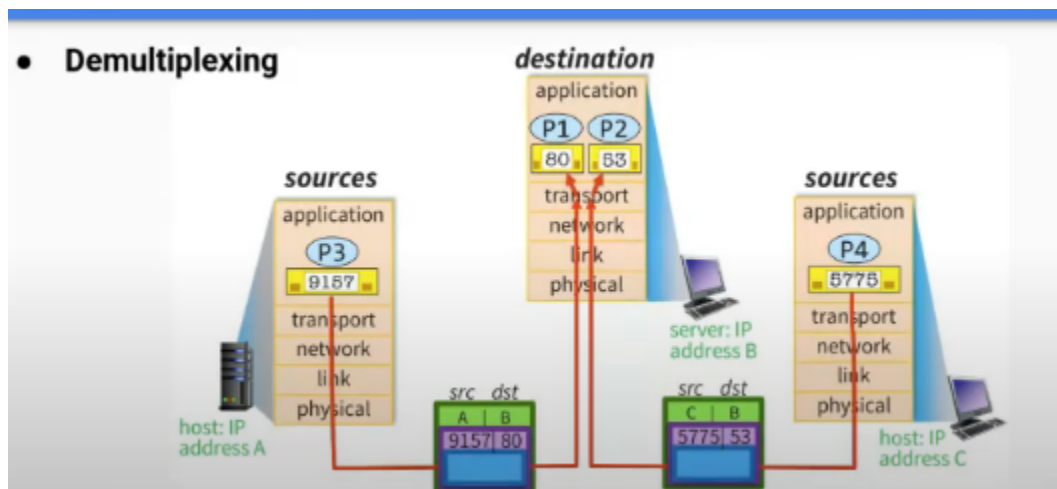


Figure 3.3 Source and destination port-number fields in a transport-layer segment

Un socket TCP se identifica por cuatro campos: **source IP address**, **source port number**, **destination IP address** y **destination port number**. A diferencia de UDP, si llegan dos segmentos con distinto source IP o source port, van a ser dirigidos a distintos sockets.



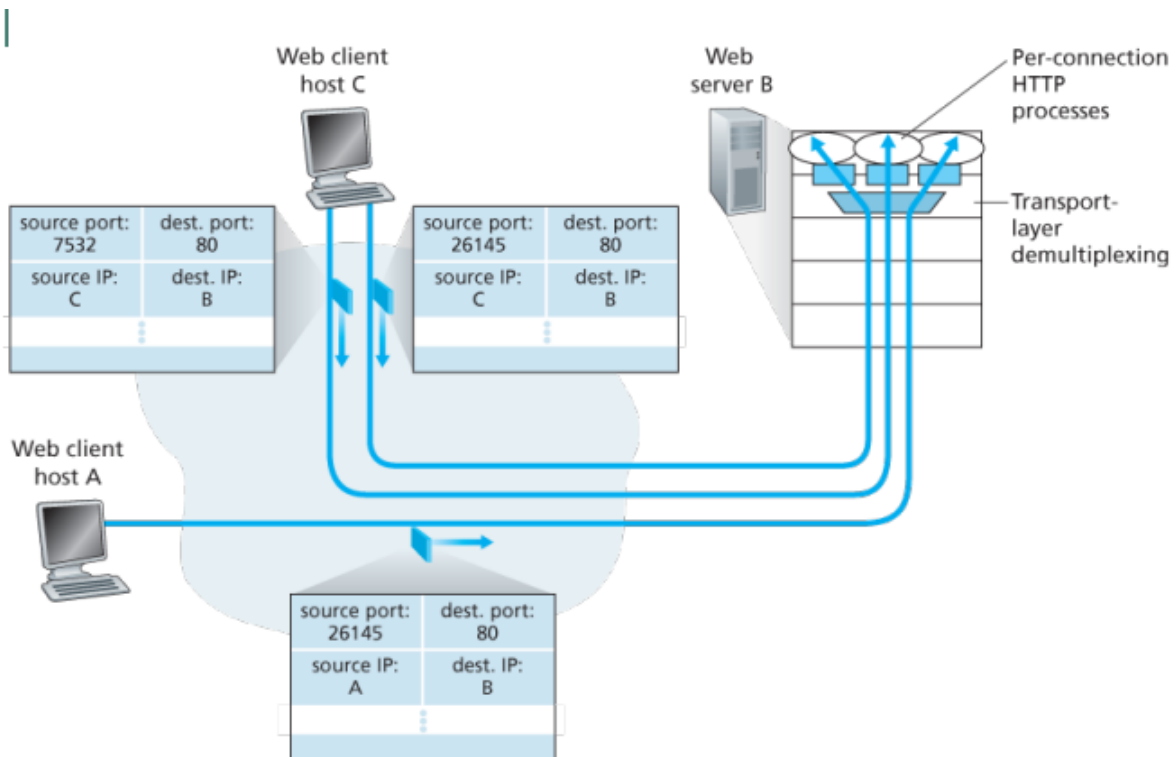
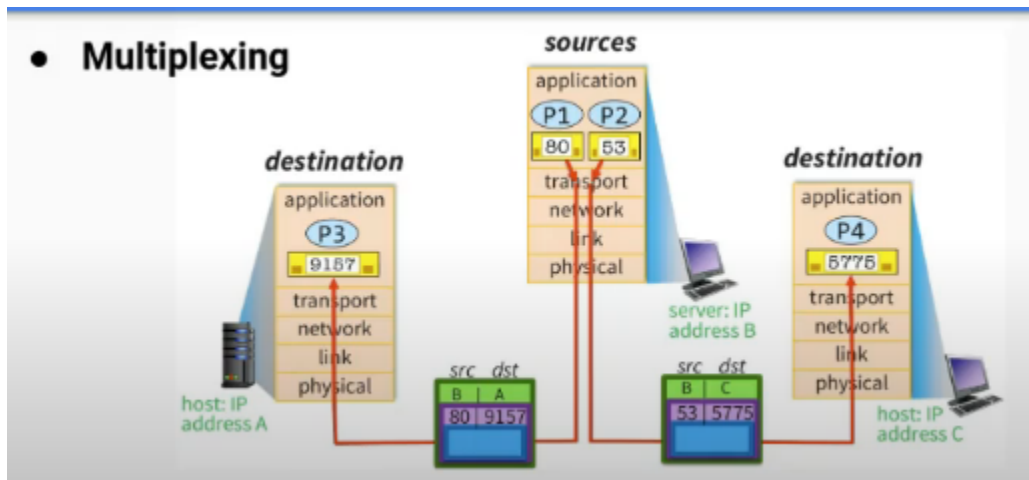


Figure 3.5 Two clients, using the same destination port number (80) to communicate with the same Web server application

Recordar que **UDP** es connectionless (no hace handshaking antes de enviar un segmento). El servicio que ofrece es mínimo y no le agrega nada al protocolo IP.

DNS es un ejemplo de una aplicación que suele usar UDP. DNS envía una query y si no obtiene respuesta puede intentar enviarla de nuevo, intentar enviarla a otro servidor o informar a la aplicación que no pudo obtener respuesta.

Por qué un developer elegiría UDP antes que TCP?

- **Más control sobre qué datos se envían y cuándo.** UDP apenas un proceso envía información, la empaqueta y la manda; TCP tiene el sistema de control de congestión que lo deja esperando si están muy cargados los enlaces. Además, como TCP garantiza que todos los paquetes lleguen y lleguen ordenados, intenta reenviar la información hasta confirmar que llegó bien, independientemente del tiempo que eso demore. Para aplicaciones de tiempo real, eso no está bueno.
- **No establece ninguna conexión.** Establecer una conexión introduce delays.
- **No mantiene estados de conexión.** Esto permite que se puedan mantener más clientes activos en una aplicación que corre con UDP.
- **Headers con bajo overhead.** TCP tiene 20 bytes header overhead, UDP tiene 8 bytes.

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Web	HTTP	TCP
File transfer	FTP	TCP
Remote file server	NFS	Typically UDP
Streaming multimedia	typically proprietary	UDP or TCP
Internet telephony	typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP
Name translation	DNS	Typically UDP

Figure 3.6 Popular Internet applications and their underlying transport protocols

Cabe mencionar que es posible hacer que una aplicación que se comunica con UDP transporte datos de manera confiable. Para eso debe construirse esa *reliability* desde la misma aplicación.

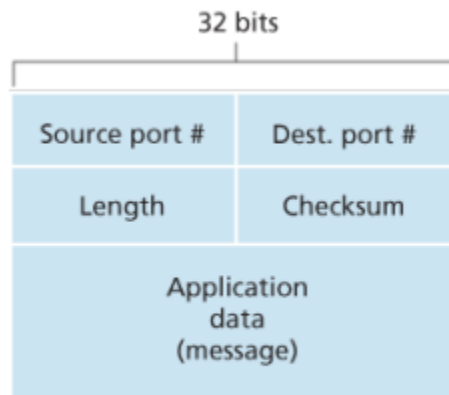
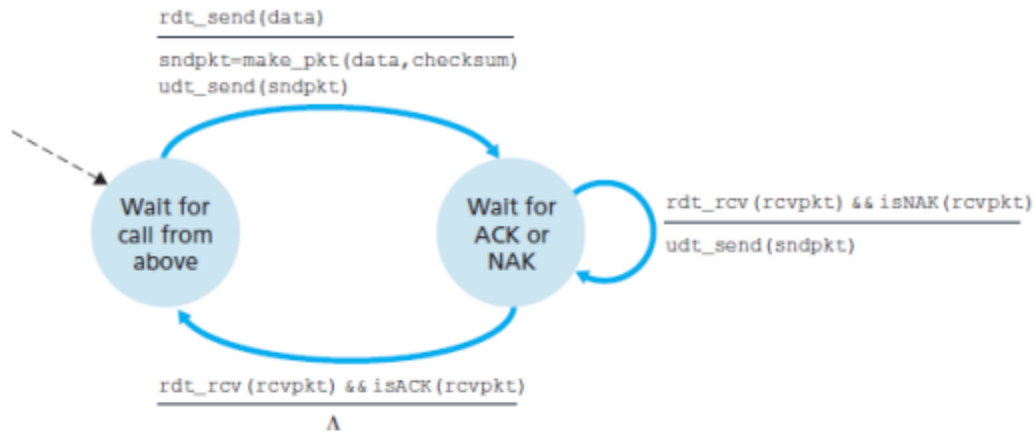


Figure 3.7 UDP segment structure

El *checksum* se usa para la detección de errores, para saber si bits dentro del segmento fueron alterados durante el transporte. Al hacer el envío, se hace el complemento a 1 de la suma de las 16-bit words del segmento y se guarda en el campo de checksum. Al recibir el paquete, se suman todos los campos de nuevo pero incluyendo el checksum. Si no hubo errores, el resultado va a ser todos 1s, si hay algún 0 es porque hubo un error en el transporte.

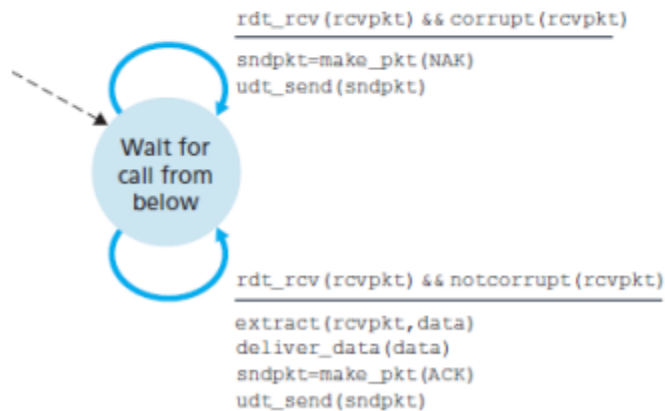
Para el análisis que sigue se asume que los paquetes de datos se entregan en el orden que fueron enviados, con la posibilidad de que alguno se pierda en el camino.

ARQ (Automatic Repeat reQuest) **protocols**: se basan en retransmitir mensajes cuando estos no fueron recibidos correctamente. Por cada mensaje el receptor da el OK o pide que se le repita. Requiere 3 capacidades: detección de errores, feedback del receptor y retransmisión.



a. rdt2.0: sending side

Figure 3.10 *rdt2.0* – A protocol for a channel with bit errors



b. rdt2.0: receiving side

Observación: no se le envía ningún otro mensaje hasta no recibir el OK del último (ACK=OK, NAK=NOT OK).

Se lo conoce como **stop-and-wait** protocol.

Una falla de este modelo es que no considera que el paquete que envía la señal de ACK o NAK podría estar corrompido. Una solución puede ser reenviar la información, esto lleva a paquetes duplicados ya que el receptor a priori no sabe si lo que está recibiendo es una retransmisión o es información nueva. Para solucionar los paquetes duplicados se agrega un nuevo campo con un número de secuencia (**sequence number**) a la data del paquete, así puede compararlo con el del paquete anterior para saber si es el mismo o es uno nuevo.

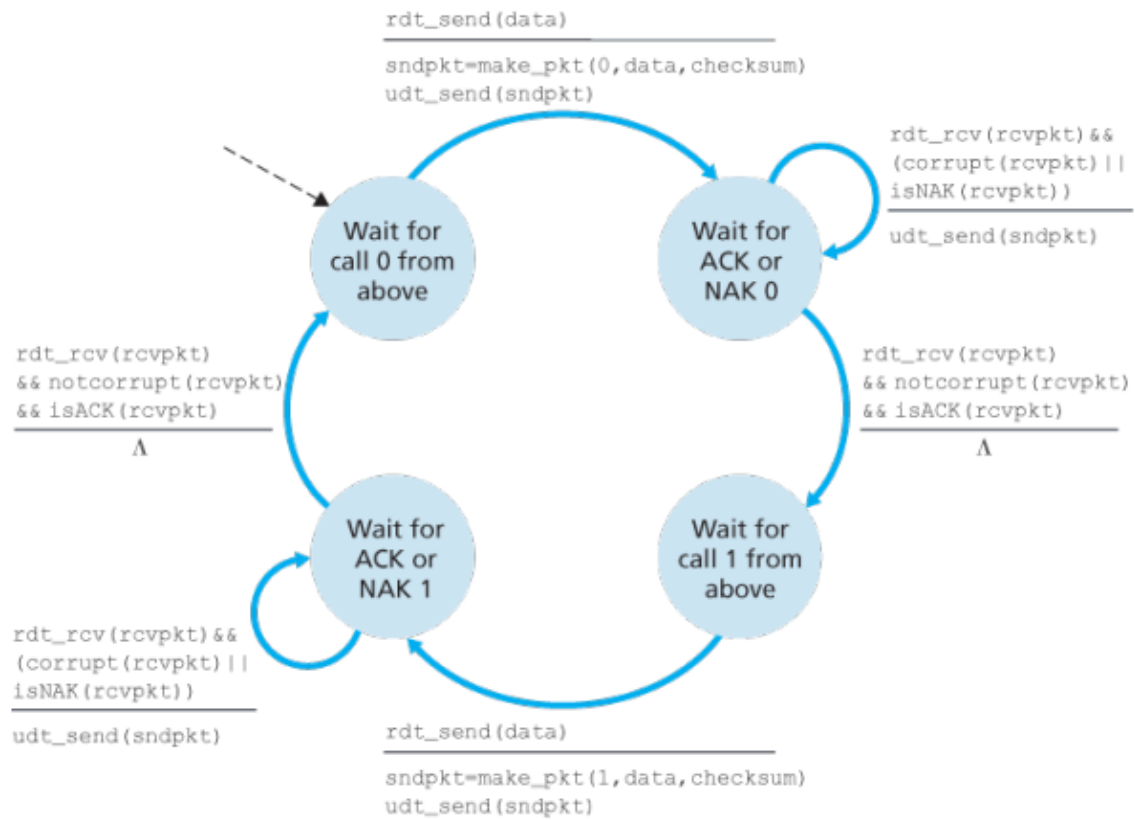


Figure 3.11 *rdt2.1* sender

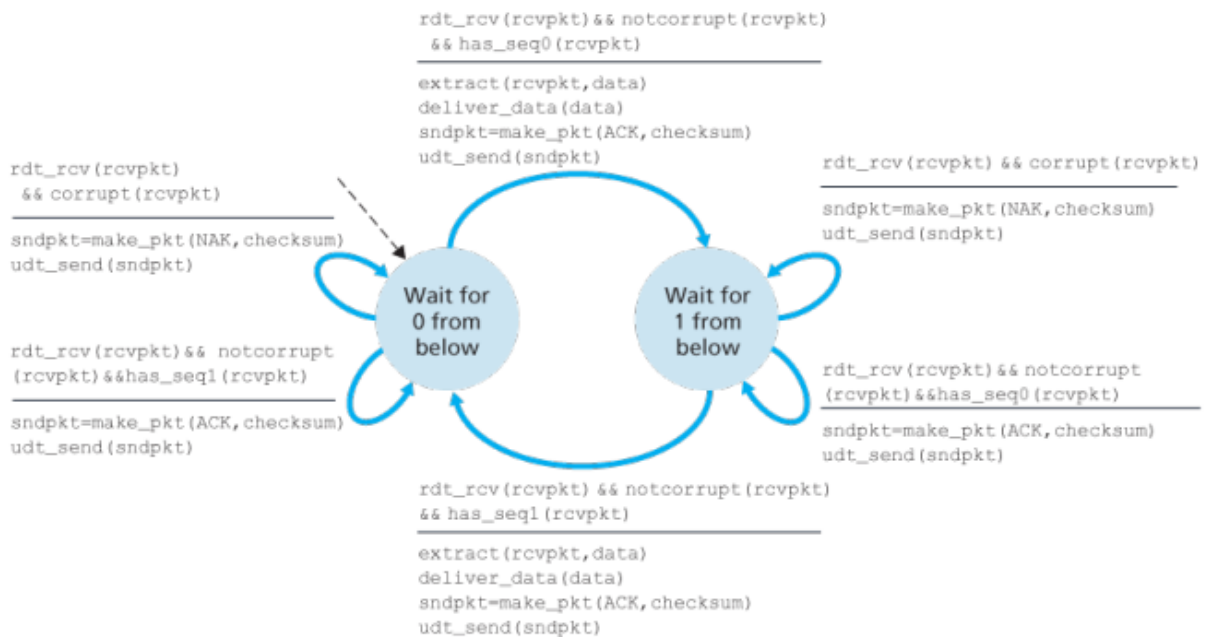


Figure 3.12 *rdt2.1* receiver

En el siguiente modelo se hace una pequeña modificación donde en vez de mandar señales NAK, se manda una señal ACK por el último paquete bien recibido. Un sender que recibe 2 ACKs por el mismo paquete va a saber que el receptor no recibió bien el paquete que le sigue al que fue recibido doble. Entonces, ahora el receptor tiene que incluir el número de secuencia del paquete reconocido por ACK y el enviador tiene que chequearlo.

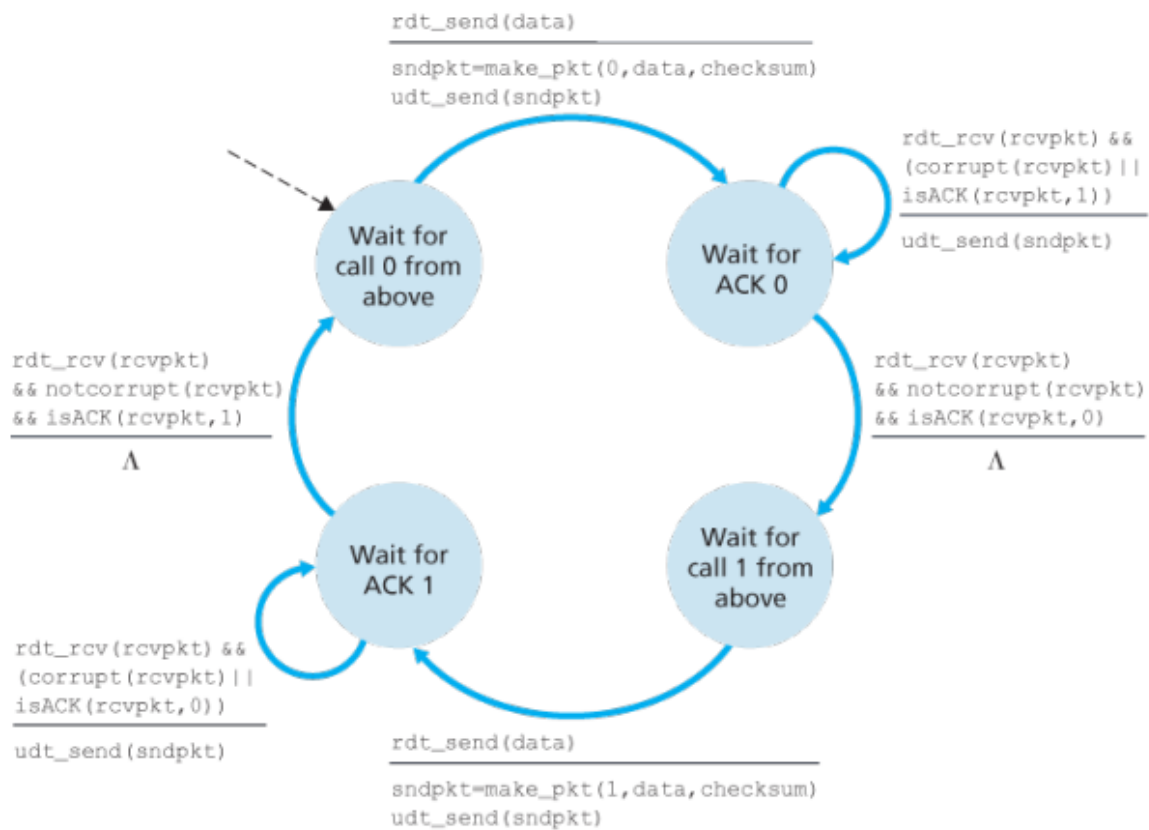


Figure 3.13 *rdt2.2* sender

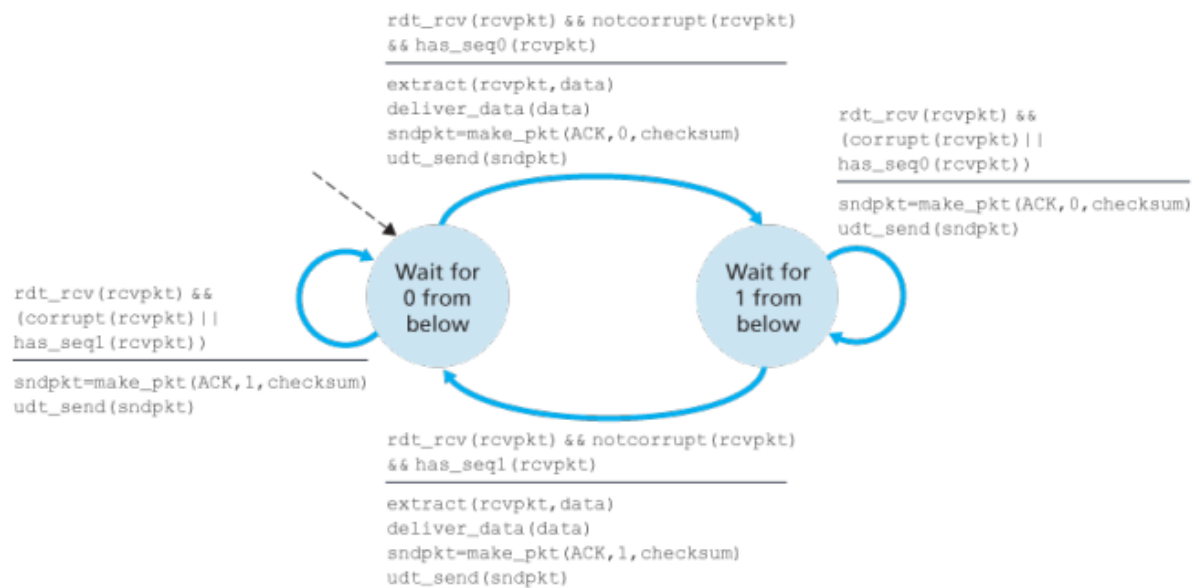


Figure 3.14 *rdt2.2* receiver

Los modelos ilustrados hasta ahora no contemplan que se pueda perder paquetes. En el caso que se pierda información, se puede agregar un mecanismo tal que después de cierta cantidad de tiempo sin recibir nada retransmita el mensaje. Es difícil estimar el tiempo de espera para considerar un paquete como perdido, es un tiempo que establece el enviador. Puede pasar que un paquete tome más tiempo de lo normal y en ese caso se van a tener paquetes duplicados, pero eso ya se soluciona con lo desarrollado anteriormente. La funcionalidad que se agrega es la posibilidad del enviador de: comenzar un timer luego de enviar un paquete, responder a una interrupción de ese timer y detener el timer.

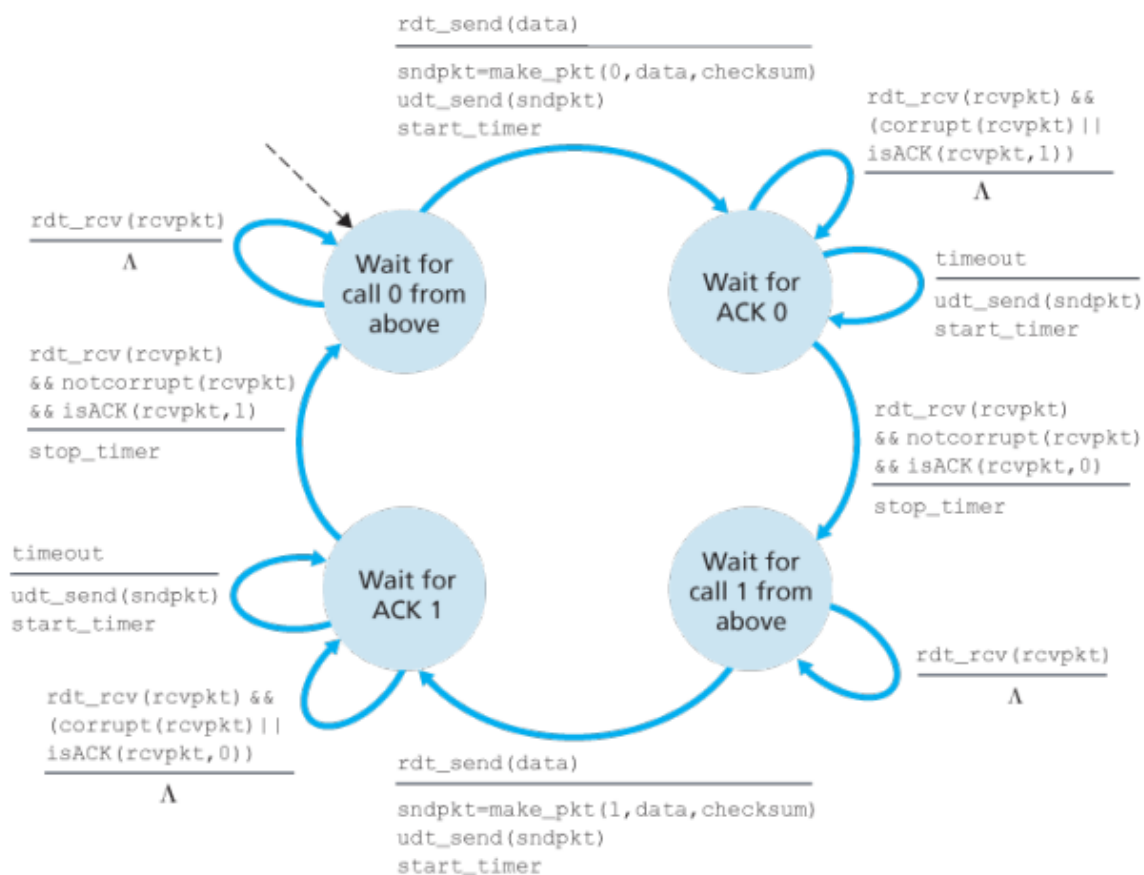
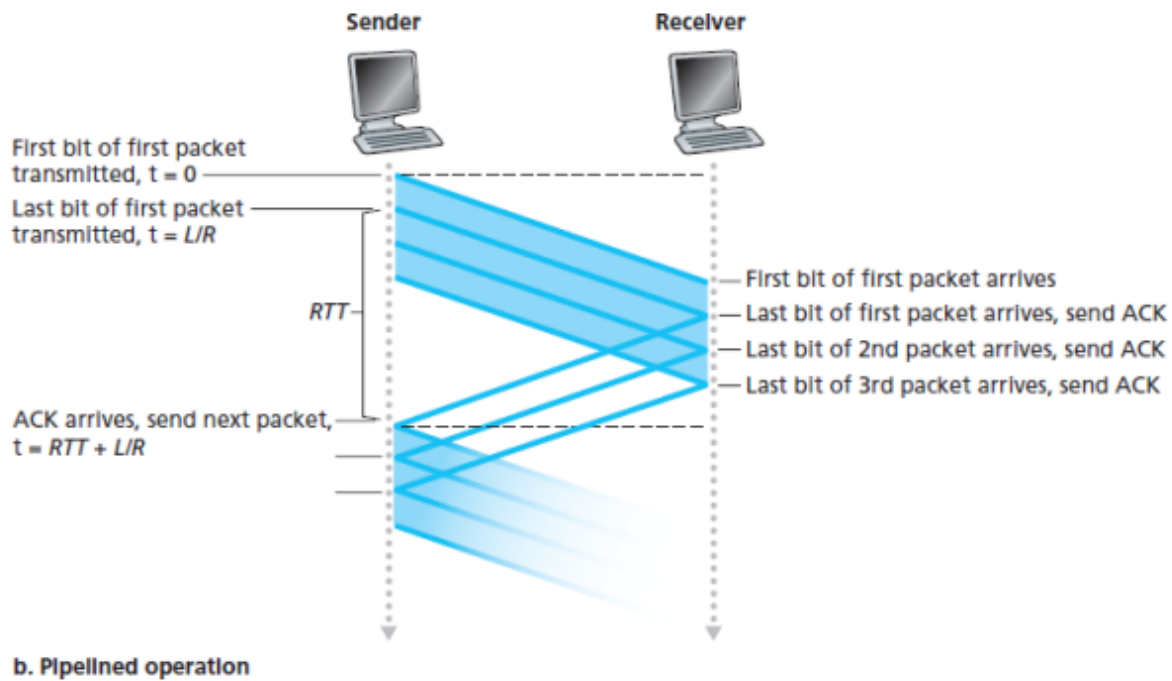


Figure 3.15 *rdt3.0* sender

El protocolo rdt3.0 cumple su función pero no tiene buena performance debido al protocolo stop-and-wait. La solución es que se permita enviar muchos paquetes sin tener que esperar que se reconozca cada uno → **pipelining**.



Las consecuencias son:

- Se debe incrementar el rango de números de secuencia.
- El enviador y el receptor probablemente tengan que almacenar en el buffer más de un paquete. Minimamente el buffer va a tener que guardar los paquetes que envió pero que todavía no sabe que fueron recibidos.
- El rango de números de secuencia va a depender de cómo responda el protocolo a la pérdida, corrupción o demora de paquetes. Existen dos approaches para el manejo de errores con pipelines:
 - **Go-Back-N (GBN):**
El sender tiene permitido enviar múltiples paquetes sin necesidad de esperar reconocimiento por ellos, pero tiene la limitación de que solo puede tener una cantidad máxima N de paquetes sin reconocer en el pipeline.

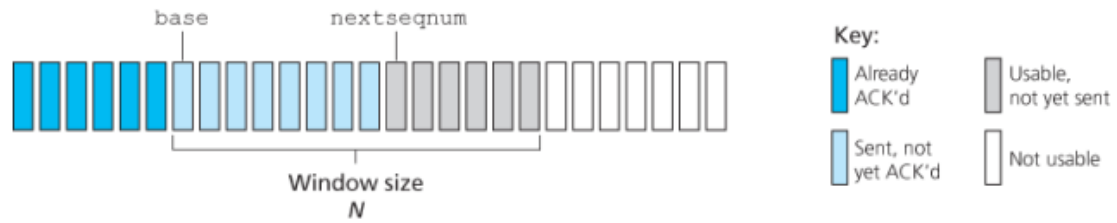


Figure 3.19 Sender's view of sequence numbers in Go-Back-N

Se suele referir a N como window size y a GBN como sliding-window protocol.

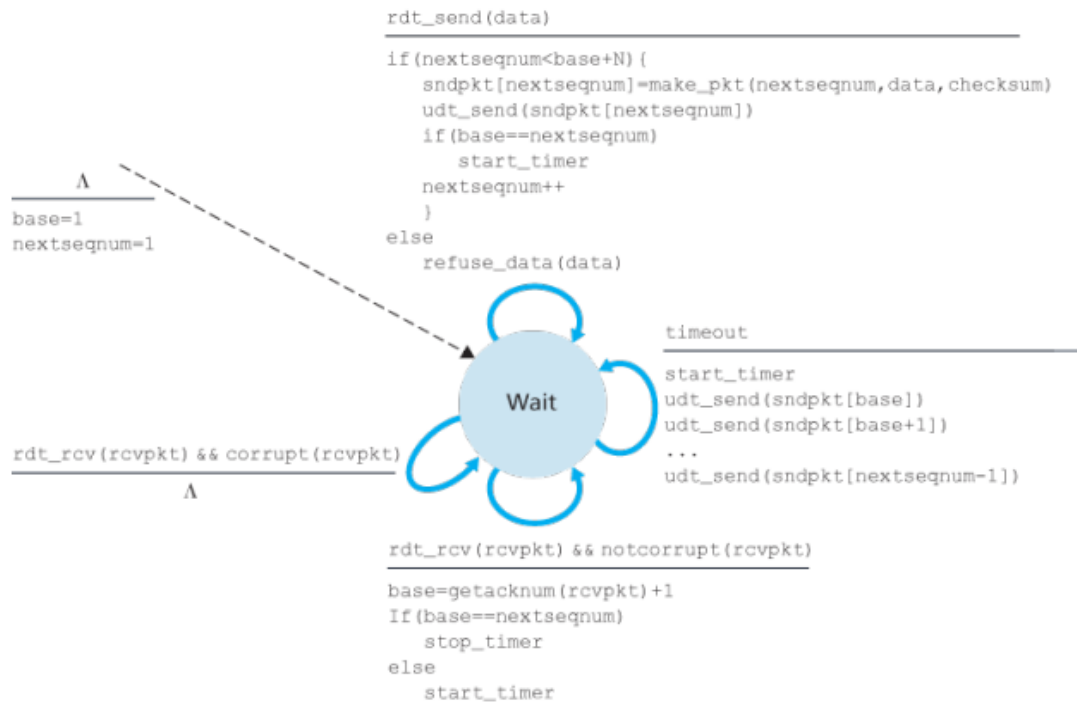


Figure 3.20 Extended FSM description of the GBN sender

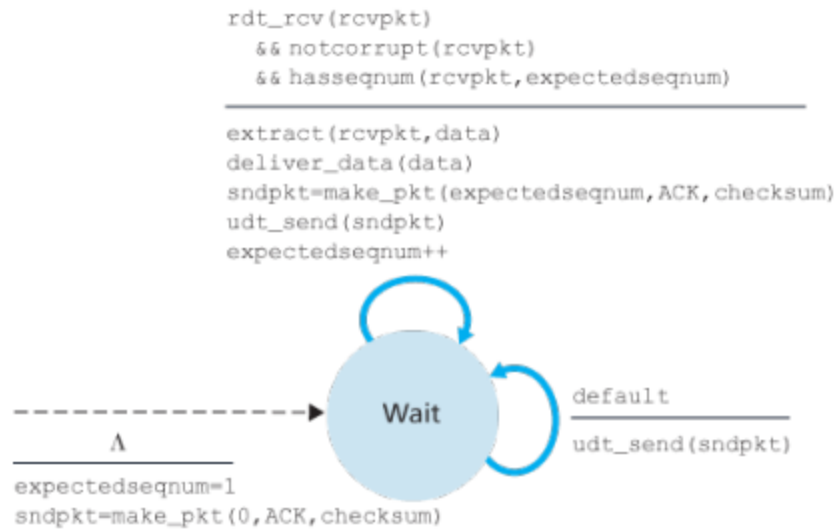


Figure 3.21 Extended FSM description of the GBN receiver

El sender tiene que responder a 3 tipos de eventos: **1.** La invocación "desde arriba". Cuando se llama `rdt_send()` se tiene que chequear que la "ventana" no esté completa (o sea que no haya N paquetes sin reconocer). En una implemetación real, habría un buffer o algún flag/semáforo para que sólo deje llamar a `rdt` cuando no esté lleno. **2.** Recepción de un ACK. El reconocimiento de la recepción de paquetes es acumulativo, o sea se reconoce a todos los paquetes con número de secuencia $\geq n$. **3.** Evento de timeout. Se tiene un solo timer para el ultimo paquete que fue transmitido pero no reconocido. Si se recibe una señal ACK pero todavía hay paquetes que no se reconocieron, se reinicia el timer. Si ya se reconocieron todos los paquetes, se detiene el timer.

Del lado del receptor. Si recibe bien y en orden un paquete con número de secuencia n , envía un ACK por los n paquetes y entrega el paquete a la capa superior. Si no recibe bien algún paquete o si llega desordenado, lo descarta y envía un ACK por el último paquete recibido. Hacer esto tiene la ventaja de la simplicidad en el buffer del receptor y la desventaja de que se podría estar descartando un paquete bueno sólo por estar desordenado y podría pasar que en la retransmisión se corrompa o lo que fuere y se tenga que volver a pedir.

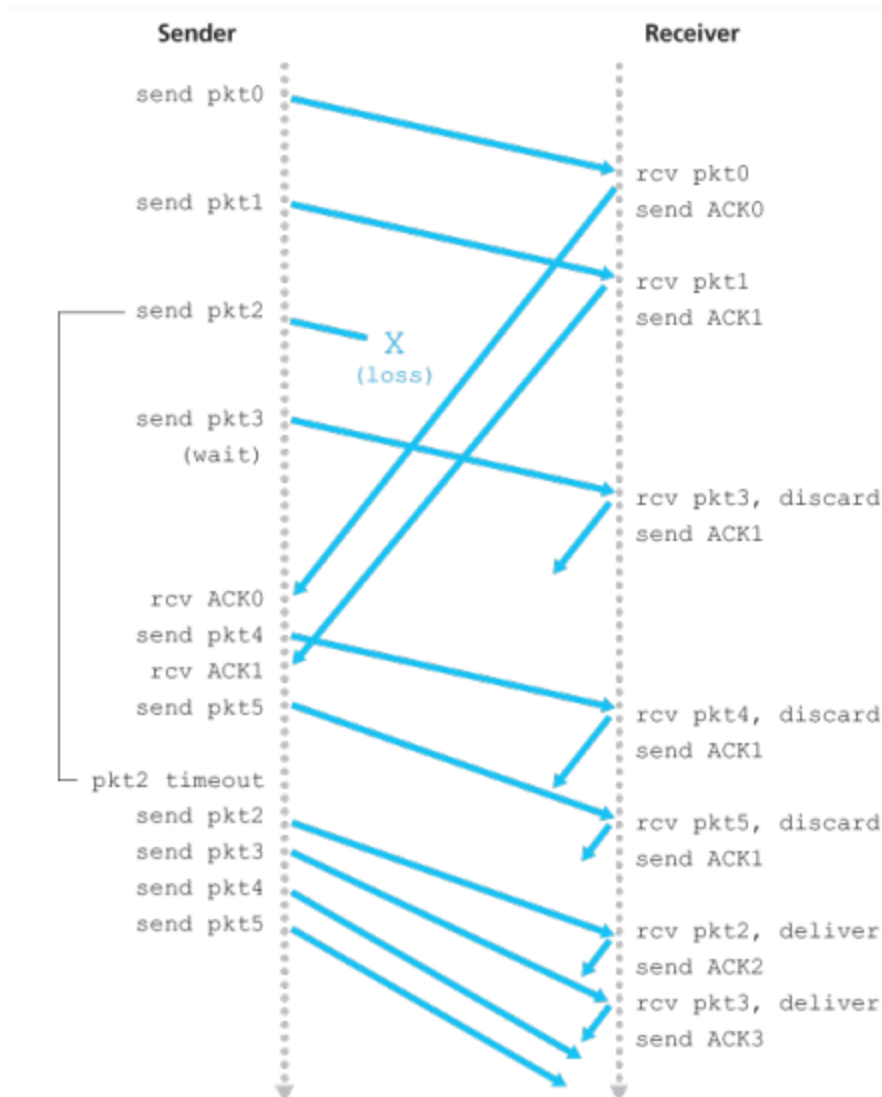


Figure 3.22 Go-Back-N in operation

- **Selective repeat:**

Hay casos donde GBN puede tener problemas de performance, por ejemplo cuando se tiene una ventana grande y hay mucho bandwidth-delay va a haber muchos paquetes en el pipeline y ya con que alguno tenga un error, se va a tener que retransmitir todo. El protocolo selective repeat evita tener que retransmitir paquetes innecesarios, retransmite solo los que sospecha que se hayan recibido con errores. Esto implica que se tengan que reconocer los paquetes individualmente.

El receptor reconoce un paquete esté o no en orden. Los que estén desordenados los guarda en un buffer hasta que llegue algún paquete con

número de secuencia menor y ahí los envía ordenados en un batch a la capa de arriba.

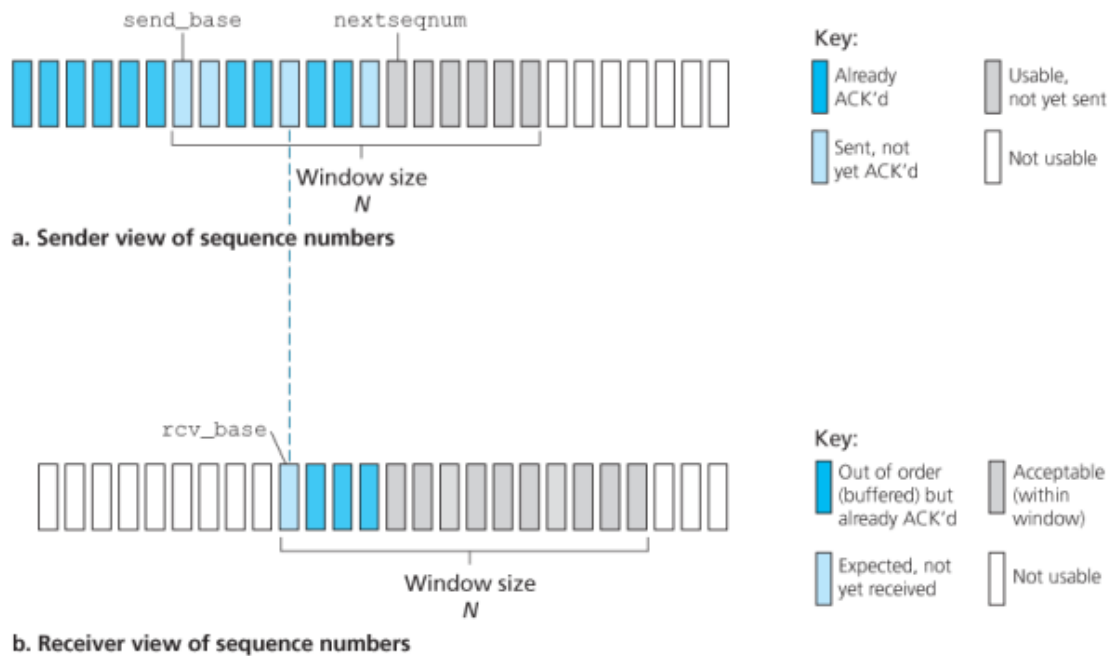


Figure 3.23 Selective-repeat (SR) sender and receiver views of sequence-number space

1. *Data received from above.* When data is received from above, the SR sender checks the next available sequence number for the packet. If the sequence number is within the sender's window, the data is packetized and sent; otherwise it is either buffered or returned to the upper layer for later transmission, as in GBN.
2. *Timeout.* Timers are again used to protect against lost packets. However, each packet must now have its own logical timer, since only a single packet will be transmitted on timeout. A single hardware timer can be used to mimic the operation of multiple logical timers [Varghese 1997].
3. *ACK received.* If an ACK is received, the SR sender marks that packet as having been received, provided it is in the window. If the packet's sequence number is equal to `send_base`, the window base is moved forward to the unacknowledged packet with the smallest sequence number. If the window moves and there are untransmitted packets with sequence numbers that now fall within the window, these packets are transmitted.

Figure 3.24 SR sender events and actions

1. *Packet with sequence number in $[rcv_base, rcv_base+N-1]$ is correctly received.* In this case, the received packet falls within the receiver's window and a selective ACK packet is returned to the sender. If the packet was not previously received, it is buffered. If this packet has a sequence number equal to the base of the receive window (rcv_base in Figure 3.22), then this packet, and any previously buffered and consecutively numbered (beginning with rcv_base) packets are delivered to the upper layer. The receive window is then moved forward by the number of packets delivered to the upper layer. As an example, consider Figure 3.26. When a packet with a sequence number of $rcv_base=2$ is received, it and packets 3, 4, and 5 can be delivered to the upper layer.
2. *Packet with sequence number in $[rcv_base-N, rcv_base-1]$ is correctly received.* In this case, an ACK must be generated, even though this is a packet that the receiver has previously acknowledged.
3. *Otherwise.* Ignore the packet.

Figure 3.25 SR receiver events and actions

Observación importante: no siempre el sender y el receiver van a tener la misma vista de qué paquetes se recibieron bien, o sea, no siempre van a coincidir las ventanas.

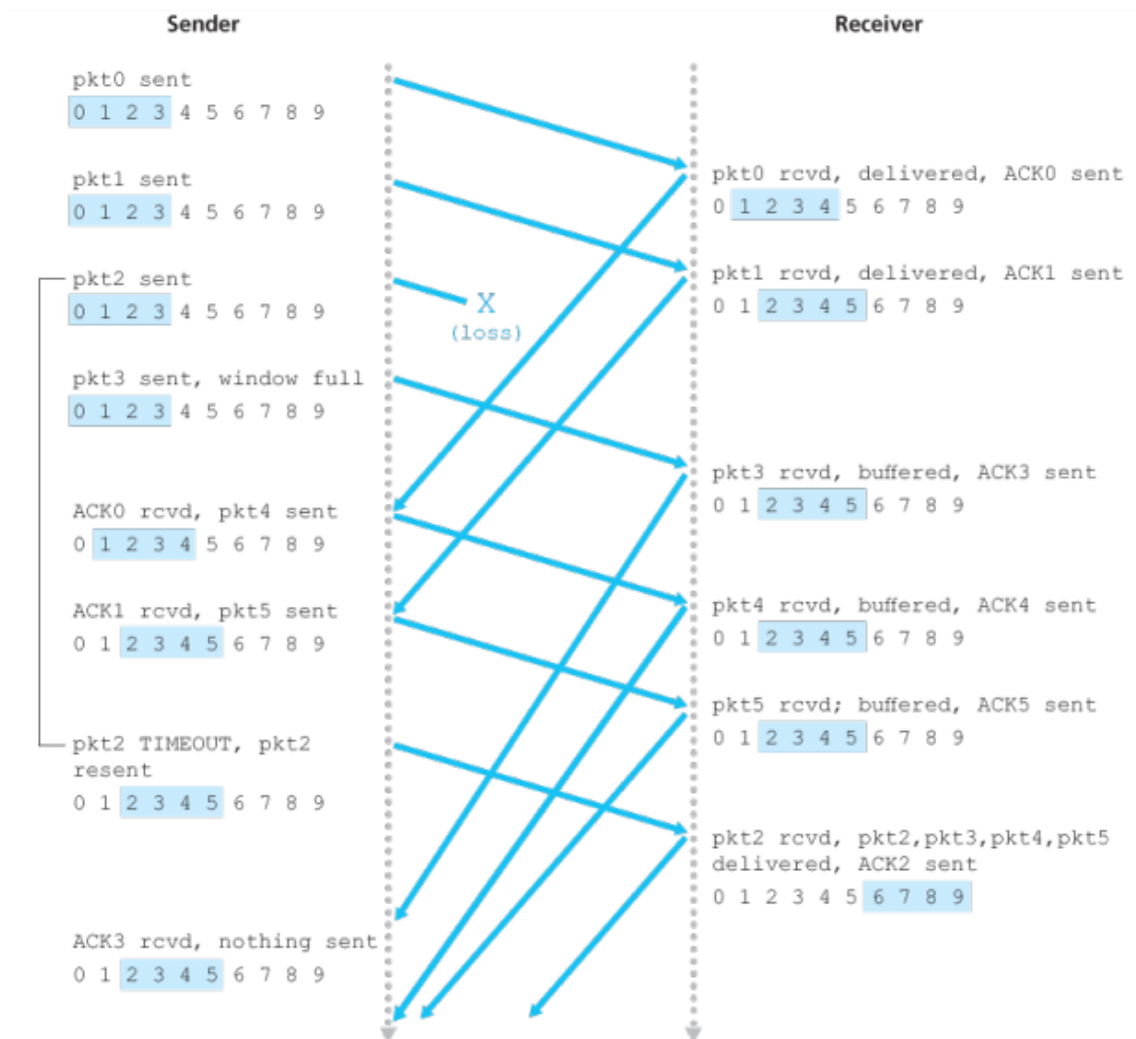


Figure 3.26 SR operation

Para que funcione tiene que cumplirse que el tamaño de la ventana sea menor o igual a la mitad del espacio de números de secuencia.

Se dice que el protocolo TCP es *connection-oriented* porque antes de que un proceso pueda comenzar a enviar datos a otro, debe establecerse la comunicación entre ambos (handshaking). Esta primera comunicación sirve para inicializar variables de estado asociadas a la conexión.

Una conexión TCP provee un servicio *full-duplex*: si hay una conexión TCP entre un proceso A en un host y un proceso B en otro host, entonces los datos de la capa de aplicación pueden viajar de A a B al mismo tiempo que datos viajan de B a A.

Una conexión TCP es siempre *point-to-point*: entre un único *sender* y un único *receiver*.

La conexión se establece primero con un *three-way handshake* (esto es el cliente envía un segmento, el servidor responde con otro segmento y el cliente vuelve a enviar otro segmento que puede o no contener un payload). Establecida la conexión, el cliente envía información a través del socket. Pasado el socket, los datos quedan en manos de TCP del lado del cliente. TCP dirige los datos al buffer de envío. Cada tanto, TCP va a tomar chunks de datos del buffer y se los va a pasar a la capa de red. La cantidad máxima de datos que puede tomar y enviar está limitada por el tamaño máximo del segmento (MSS-maximum segment size). TCP empareja cada chunk de data con un header, formando **segmentos**. En resumen, una conexión TCP consiste de buffers, variables y sockets en ambos hosts.

Estructura de un segmento:

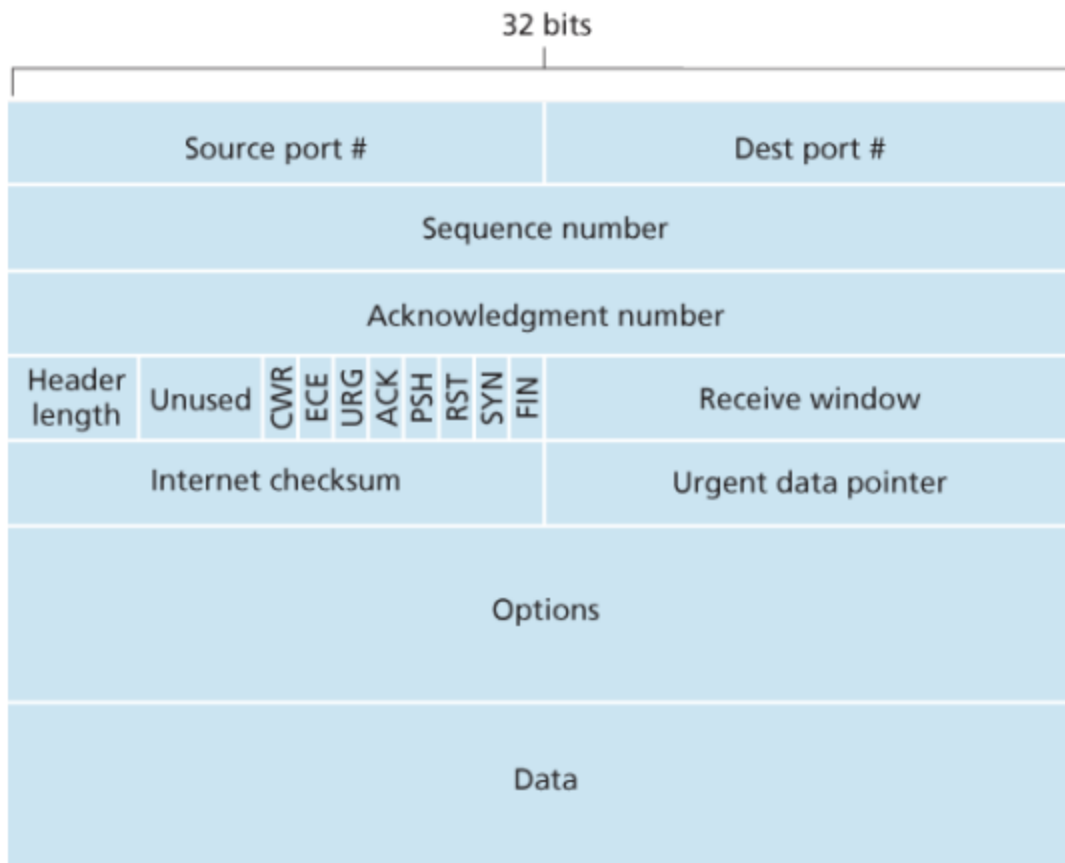


Figure 3.29 TCP segment structure

El **número de secuencia** y el **número de acknowledgment** lo usan el sender y receiver para garantizar la transferencia confiable de datos. *The acknowledgment number that Host A puts in its segment is the sequence number of the next byte Host A is expecting from Host B.*

Suppose that Host A has received all bytes numbered 0 through 535 from B and suppose that it is about to send a segment to Host B. Host A is waiting for byte 536 and all the subsequent bytes in Host B's data stream. So Host A puts 536 in the acknowledgment number field of the segment it sends to B.

Suppose that Host A has received one segment from Host B containing bytes 0 through 535 and another segment containing bytes 900 through 1,000. For some reason Host A has not yet received bytes 536 through 899. In this example, Host A is still waiting for byte 536 (and beyond) in order to re-create B's data stream. Thus, A's next segment to B will contain 536 in the acknowledgment number field. [en este caso, donde llegan segmentos desordenados hay dos opciones: 1) descartar el que viene desordenado, 2) guardarlo y esperar que llegue el faltante para llenar el hueco. La 2 es más eficiente.]

Se dice que TCP provee *cumulative acknowledgments* porque solo reconoce bytes desde el primer byte faltante en el stream.

El campo **receive windows** se usa para controlar el flujo (indica la cantidad de bytes que el receiver está dispuesto a aceptar).

El campo **header length** especifica la longitud del header, la longitud es variable porque depende del campo options (que típicamente viene vacío).

El campo options se usa cuando el sender y receiver negocian el tamaño máximo del segmento.

El bit ACK indica que el valor cargado en el campo de acknowledgment es válido.

Los bits RST, SYN y FIN son para el setup y teardown de la conexión.

Los bits CWR y ECE se usan para notificar congestión.

El bit PSH indica al receiver que debe pasar los datos inmediatamente a la capa superior.

El bit URG indica que hay datos marcados como urgentes por la capa superior del sender.

TCP usa el mecanismo de timeout/retransmisión para los segmentos perdidos.

Recordar que el servicio IP no es confiable: no puede garantizar que los datos lleguen, ni que lleguen en orden, ni que no se corrompan en el camino. Pero TCP sí debe garantizar eso, para ello tiene que construir un servicio de transferencia confiable por sobre el servicio IP.

Una recomendación es que se use un solo timer de retransmisión (porque tener uno por cada segmento no reconocido, implica mucho overhead)

```

/* Assume sender is not constrained by TCP flow or congestion control, that data from above is less
than MSS in size, and that data transfer is in one direction only. */

NextSeqNum=InitialSeqNumber
SendBase=InitialSeqNumber

loop (forever) {
    switch(event)

        event: data received from application above
            create TCP segment with sequence number NextSeqNum
            if (timer currently not running)
                start timer
            pass segment to IP
            NextSeqNum=NextSeqNum+length(data)
            break;

        event: timer timeout
            retransmit not-yet-acknowledged segment with
                smallest sequence number
            start timer
            break;

        event: ACK received, with ACK field value of y
            if (y > SendBase) {
                SendBase=y
                if (there are currently any not-yet-acknowledged segments)
                    start timer
            }
            break;

    } /* end of loop forever */

```

Figure 3.33 Simplified TCP sender

(SendBase es el numero de secuencia del ultimo segmento reconocido)

Cada vez que TCP retransmite un segmento, duplica el intervalo de timeout. Esto provee una especie de control de congestión limitado.

Uno de los problemas de las retransmisiones triggereadas por un timeout, es que el periodo de timeout puede ser relativamente largo. Existe una manera que tiene TCP para saber que un paquete fue perdido antes de que termine el timeout → ACK's duplicados. Un ACK duplicado es un ACK de un paquete del cual el sender ya recibió reconocimiento. Cuando el receiver recibe un segmento con un número de secuencia mayor al esperado, detecta que falta un segmento (por perdida o por desorden); ante esto, reenvía una señal ACK por el último byte de datos recibido correctamente. Si el sender recibe 3 ACKs del mismo segmento, interpreta que el segmento siguiente al que fue ACKd tres veces, se perdió y hace un *fast retransmit*, o sea retransmite antes de que termine el timeout.

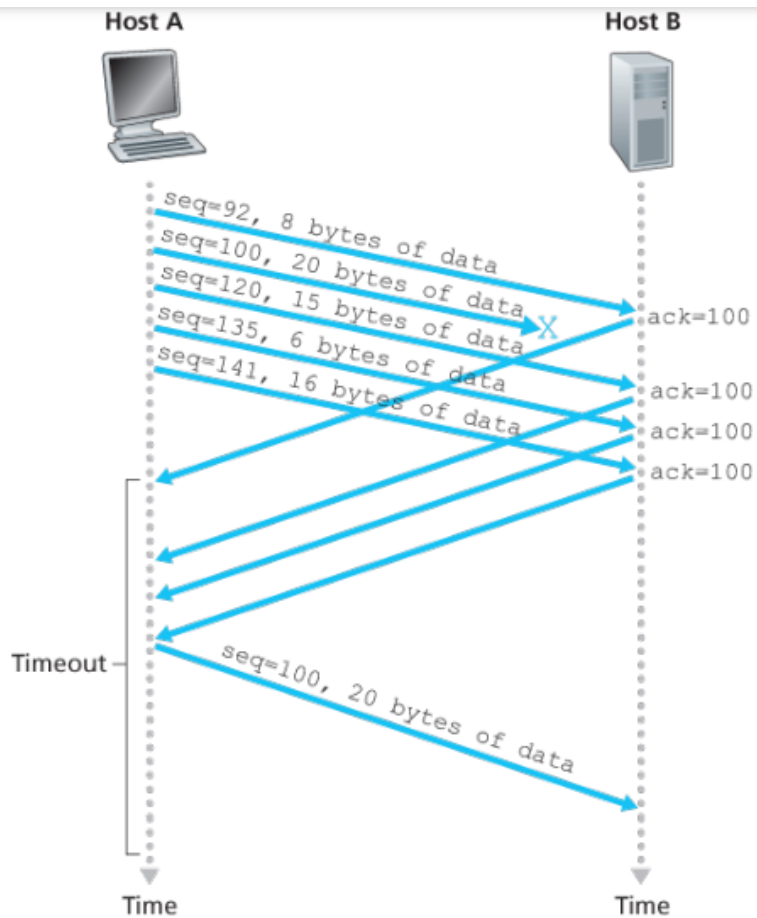


Figure 3.37 Fast retransmit: retransmitting the missing segment before the segment's timer expires

```

event: ACK received, with ACK field value of y
    if (y > SendBase) {
        SendBase=y
        if (there are currently any not yet
            acknowledged segments)
            start timer
    }

```



```

        else { /* a duplicate ACK for already ACKed
                segment */
            increment number of duplicate ACKs
                received for y
            if (number of duplicate ACKS received
                for y==3)
                /* TCP fast retransmit */
                resend segment with sequence number y
            }
        break;

```

TCP se asemeja más al tipo *Go-Back-N*, aunque conserva ciertas diferencias.

Consider also what happens when the sender sends a sequence of segments 1, 2, . . . , N, and all of the segments arrive in order without error at the receiver. Further suppose that the acknowledgment for packet gets lost, but the remaining acknowledgments arrive at the sender before their respective timeouts. In this example, GBN would retransmit not only packet n, but also all of the subsequent packets TCP, on the other hand, would retransmit at most one segment, namely, segment n. Moreover, TCP would not even retransmit segment n if the acknowledgment for segment arrived before the timeout for segment n.

Existe también una versión modificada de TCP donde provee *selective acknowledgment*. Esto significa que el receiver puede reconocer segmentos no-ordenados de manera selectiva. Combinado con *selective retransmission*, hace que TCP se asemeje más al tipo *Selective-Repeat*.

Se dice que el mecanismo de errores de TCP es un híbrido de GBN y SR.

TCP provee un servicio de control de flujo para que el sender no colapse el buffer del receiver. Esto se logra haciendo que el sender mantenga una variable llamada **receive window**. Esta ventana le da al sender una idea de cuánto espacio disponible hay en el

buffer del receiver. Para que el sender pueda saber esa información, existe el campo *receive window* en los headers del segmento.

Supongamos que el host A le envía algo al host B. El host B va a tener un receive buffer de tamaño `RcvBuffer`, tal que $\text{LastByteRecv} - \text{LastByteRead} \leq \text{RcvBuffer}$. Receive window (`rwnd`) se setea con la cantidad de espacio libre, $\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRecv} - \text{LastByteRead})$. En cada segmento que envía B a A, le indica cuanto espacio tiene disponible. El host A "vigila" dos variables: `LastByteSent` y `LastByteAced`. Manteniendo que $\text{LastByteSent} - \text{LastByteAced} \leq \text{rwnd}$, se asegura que no haya overflow en el buffer de B.

Podría suceder que se llene el buffer de B, se avise al host A, pero después no se tenga nada más que enviar desde B hasta A como para avisarle si se libera espacio. O sea, A quedaría bloqueado porque nunca se enteraría cuando hay espacio de nuevo. Para solucionar eso, se hace que A le siga mandando segmentos de un byte a B, así B los reconoce y va informando si se libera o no espacio.

Procedimiento para establecer una conexión TCP:

1. El cliente envía un segmento con el bit SYN en 1 y un número de secuencia inicial (random).
2. El servidor envía un segmento con el bit SYN en 1, setea su número de secuencia inicial y setea el acknowledgment number como `client_sqn+1`
3. El cliente envía otro segmento donde confirma la recepción del segmento del servidor, setea el bit SYN en 0. Si tiene que enviar payload, lo envía.

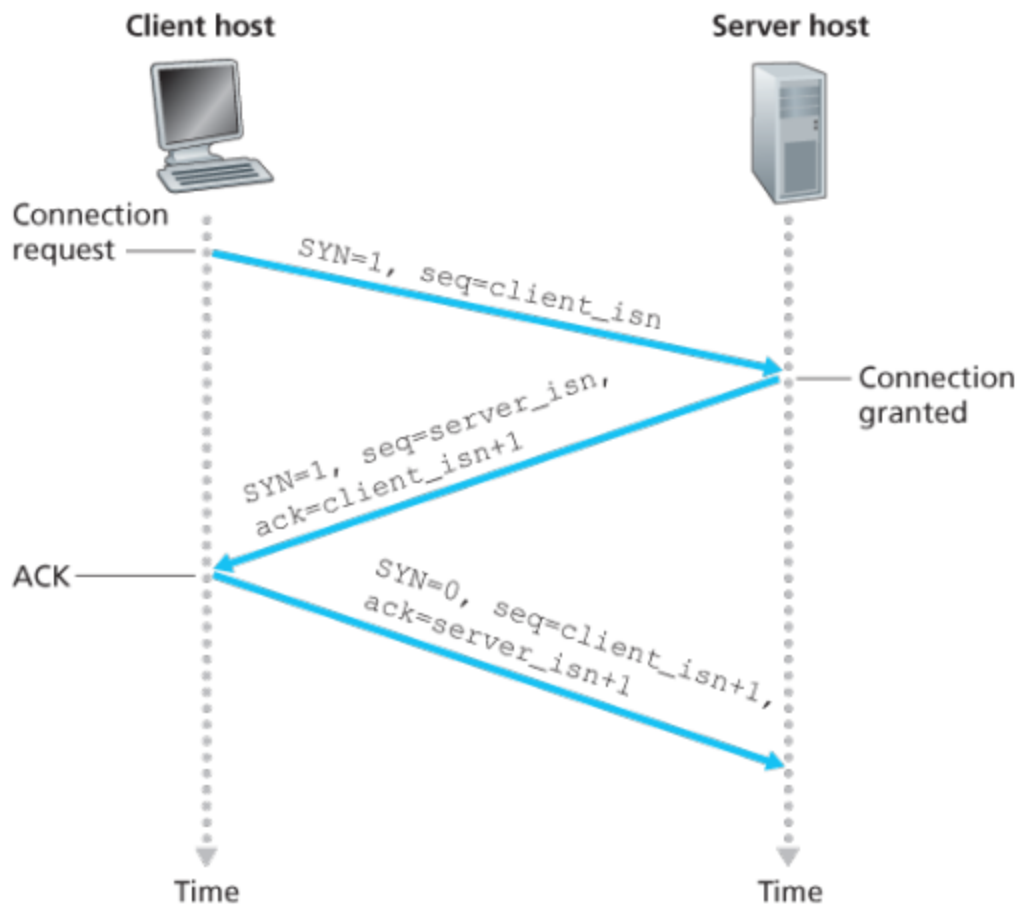
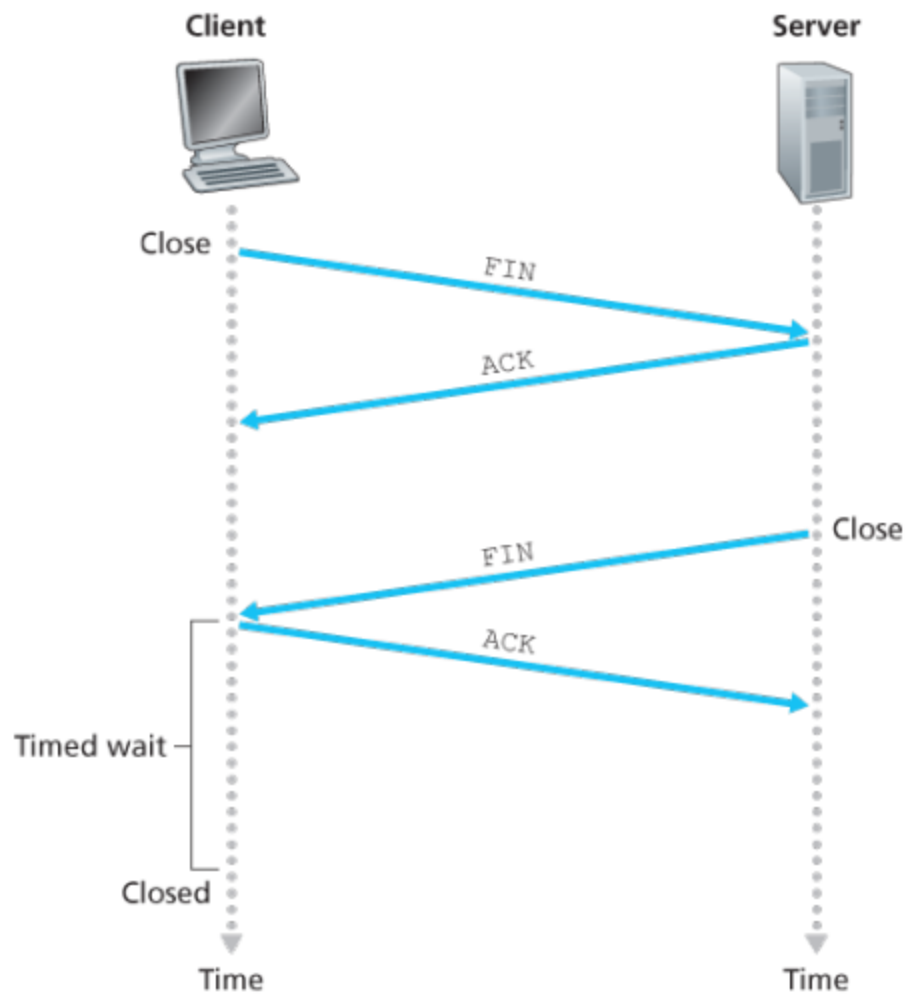


Figure 3.39 TCP three-way handshake: segment exchange

Para cerrar una conexión TCP:

1. La aplicación cliente ejecuta un comando `close`. El cliente TCP envía un segmento con el bit FIN en 1.
2. El servidor envía un segmento donde lo reconoce y setea el bit FIN en 1.
3. Finalmente, el cliente reconoce el segmento de shutdown del servidor. Se desalocan los buffers y variables.



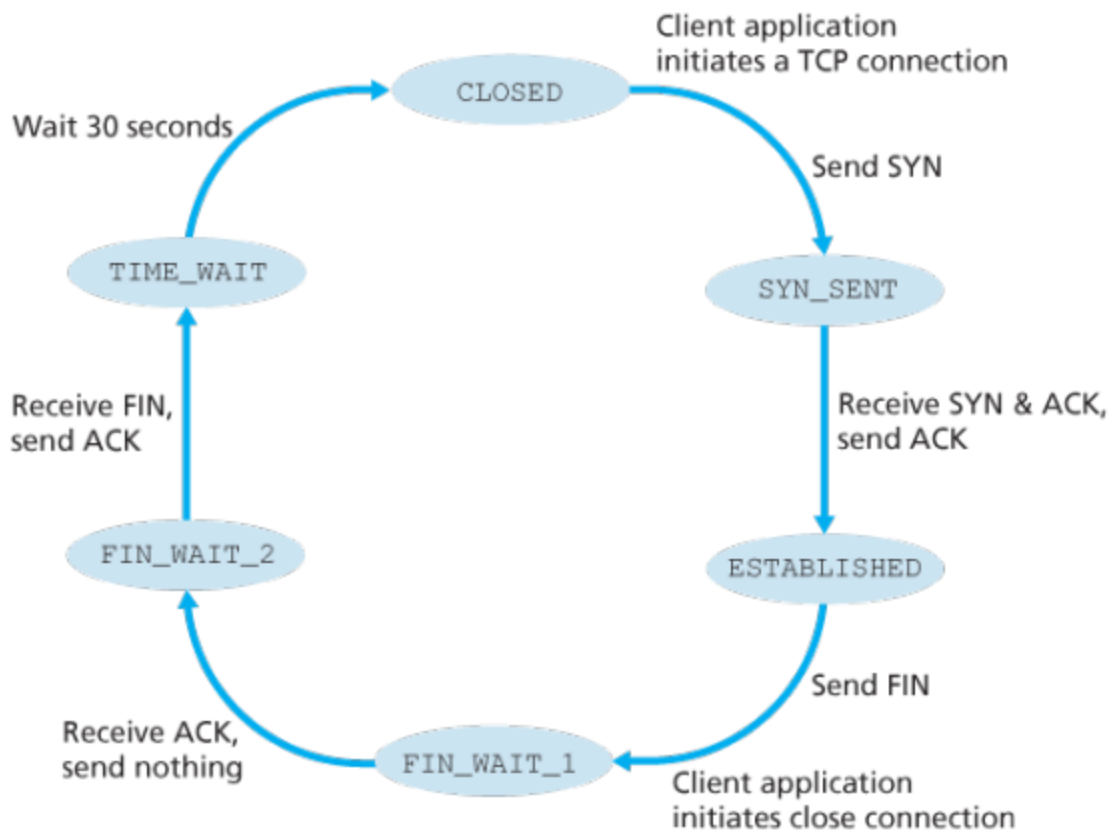


Figure 3.41 A typical sequence of TCP states visited by a client TCP

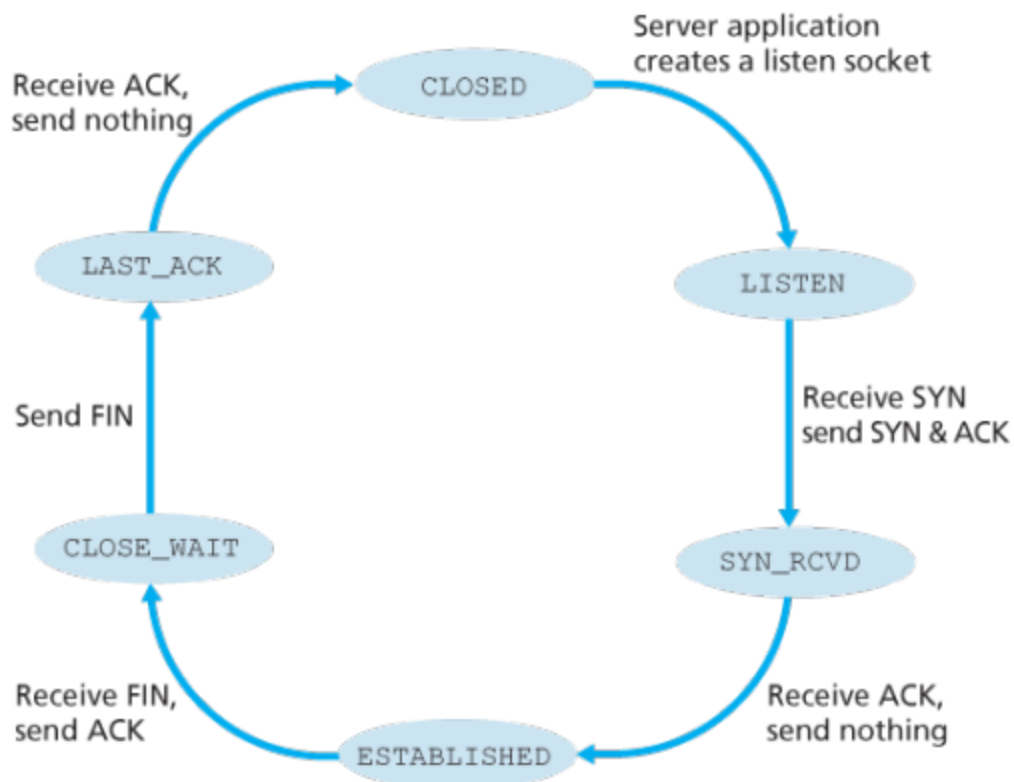


Figure 3.42 A typical sequence of TCP states visited by a server-side TCP

Cuando un host envía un segmento con el bit RST en 1, le está diciendo que no tiene un socket para ese segmento y que no lo reenvíe.

Algunos escenarios en los que ocurre congestión:

1. Dos hosts que comparten un router con buffer infinito

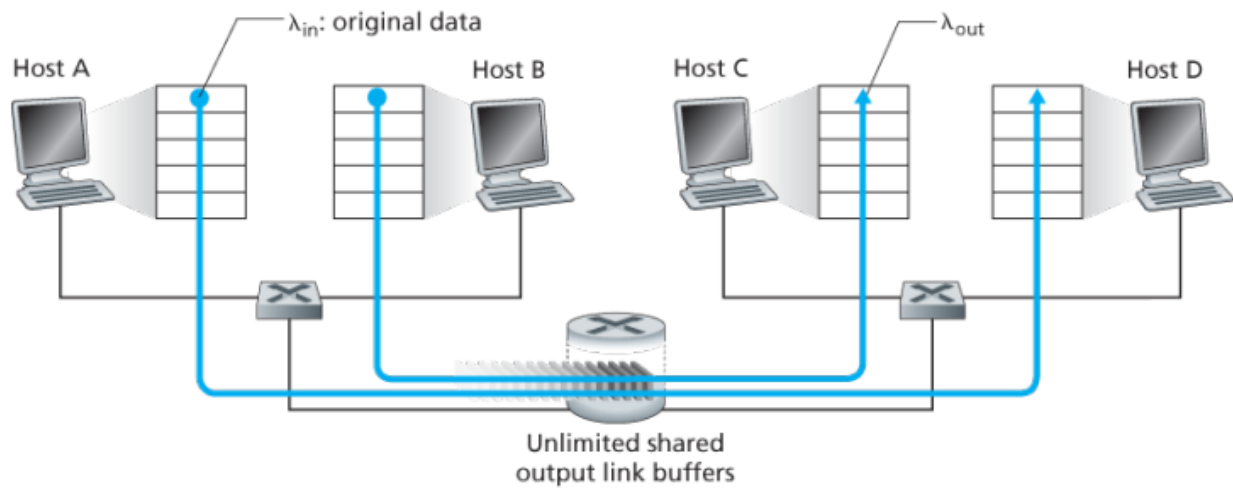
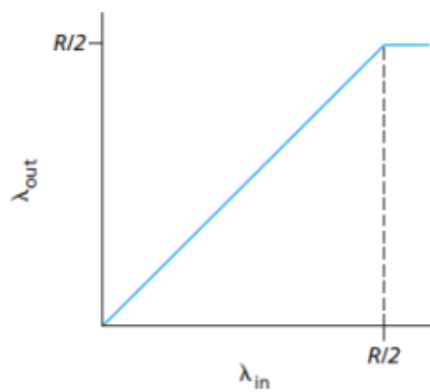
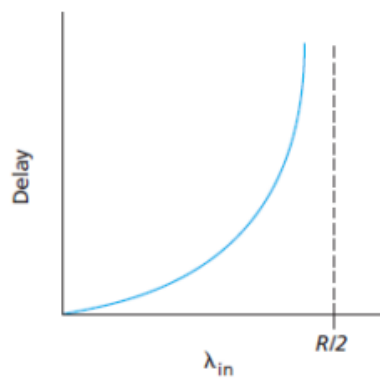


Figure 3.43 Congestion scenario 1: Two connections sharing a single hop with infinite buffers



a.

Figure 3.44 Congestion scenario 1: Throughput and delay as a function of host sending rate



b.

El primer gráfico es el throughput por conexión (cantidad de bytes por segundo en el receiver) en función del rate de envío del sender. Con un sending rate entre 0 y $R/2$ (R es la capacidad del link), el throughput va a ser igual al rate de envío. Pero cuando el sending rate supera $R/2$, el throughput queda en $R/2$ porque está limitado por la capacidad del linker que está siendo utilizado por otra conexión además de esta.

En el segundo gráfico se ve que a medida que el rate de envío se acerca al valor de $R/2$, aumenta el delay porque aumenta la cantidad de paquetes encolados (este es un costo de la congestión).

2. Dos hosts y un router con buffer finito

Que el buffer sea finito significa que cuando se llene, los paquetes nuevos se van a perder. Pero también implica una conexión confiable, o sea que si pierde un paquete, se va a retransmitir.

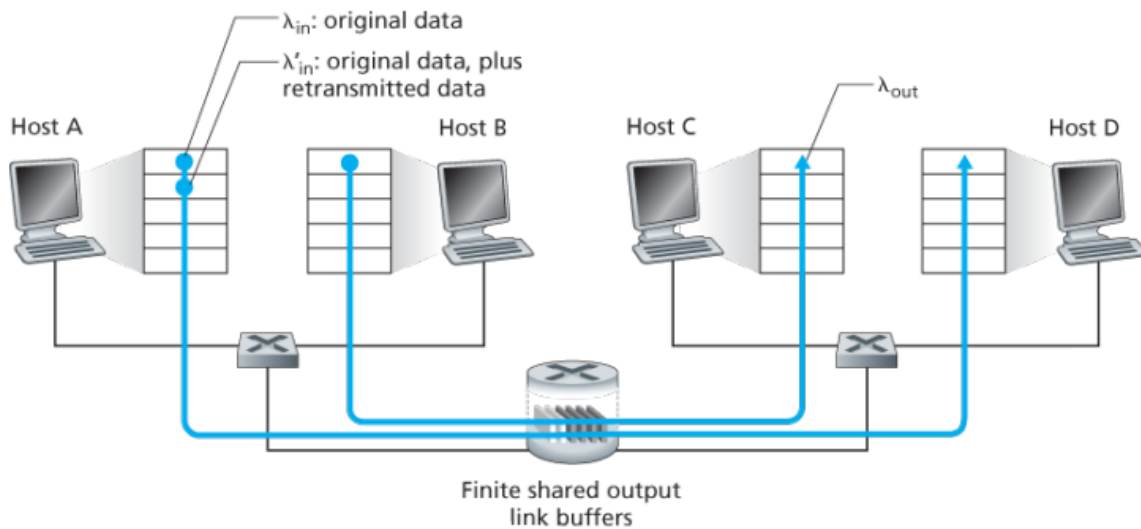
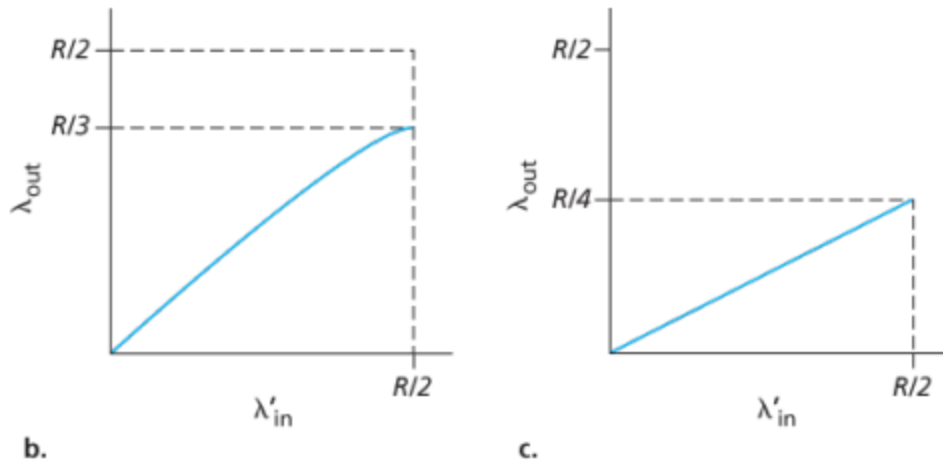


Figure 3.45 Scenario 2: Two hosts (with retransmissions) and a router with finite buffers



En el gráfico de la izquierda se asume que el sender retransmite paquetes solo cuando sabe que fueron perdidos. Un costo de una red congestionada es que el sender debe realizar retransmisiones para compensar por los paquetes perdidos por buffer overflow.

En el gráfico de la derecha se asume que el sender puede cortar el timeout prematuramente y retransmitir paquetes que están demorados pero no perdidos. Ese es otro costo de la red congestionada, retransmisiones innecesarias que usan el bandwidth del link para reenviar copias innecesarias de paquetes.

3. 4 hosts, routers con buffers finitos y multihop paths

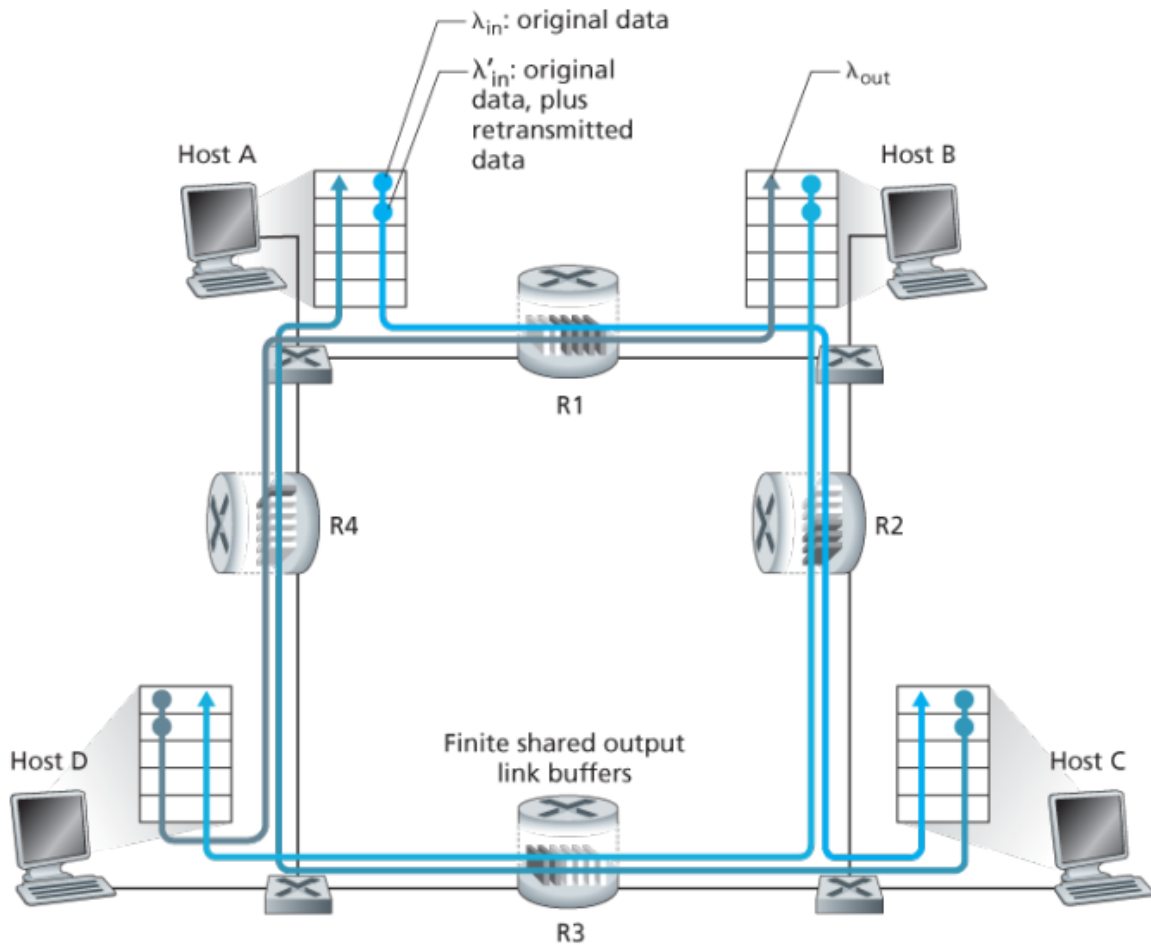


Figure 3.47 Four senders, routers with finite buffers, and multihop paths

Acá se ve otro costo de soltar un paquete debido a la congestión: cuando un paquete se suelta en medio de un path, la capacidad de transmisión que fue usada en todos los links para llevar ese paquete a donde ahora fue arrojado, fue desperdiciada.

El approach que toma TCP para controlar la congestión es básicamente reducir o aumentar el rate de envío según qué tan congestionada esté la conexión.

TCP lleva registro de una variable que denominamos *congestion window* (cwnd). Esta ventana es la que limita el rate al que se puede enviar información desde el sender. Debe suceder que

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\}.$$

The constraint above limits the amount of unacknowledged data at the sender and therefore indirectly limits the sender's send rate. To see this, consider a connection for which loss and packet transmission delays are negligible. Then, roughly, at the beginning of every RTT, the constraint permits the sender to send $cwnd$ bytes of data into the connection; at the end of the RTT the sender receives acknowledgments for the data. Thus the sender's send rate is roughly $cwnd/RTT$ bytes/sec. By adjusting the value of $cwnd$, the sender can therefore adjust the rate at which it sends data into its connection.

Cómo percibe TCP que la red está congestionada? Lo que pasa cuando se congestiona es que se pierden paquetes en el camino, entonces ante un timeout o tres arribos de ACK por un mismo segmento, TCP va a percibir que hay congestión. A su vez, si no hay congestión lo va a percibir al recibir todos los ACK, y ante esto puede incrementar el congestion window. Como TCP utiliza los ACK para triggerear el aumento o decremento de la ventana, se dice que es **self-clocking**.

Algunos guidelines:

- A lost segment implies congestion, and hence, the TCP sender's rate should be decreased when a segment is lost.
- An acknowledged segment indicates that the network is delivering the sender's segments to the receiver, and hence, the sender's rate can be increased when an ACK arrives for a previously unacknowledged segment.
- Bandwidth probing. The TCP sender's behavior is perhaps analogous to the child who requests (and gets) more and more goodies until finally he/she is finally told "No!", backs off a bit, but then begins making requests again shortly afterwards.

Capítulo 4: The Network Layer - Data Plane

El principal rol de la capa de red es mover paquetes de un host enviador a un host receptor. Se identifican dos funciones importantes:

- **Forwarding:** cuando un paquete llega al enlace de entrada de un router, el router debe mover el paquete al enlace de salida apropiado. Toma muy poco tiempo, típicamente en la medida de nanosegundos. Se suele implementar en hardware.
- **Routing:** se debe determinar la ruta o el camino tomados por los paquetes a medida que viajan del enviador al receptor. Toma un poco más de tiempo, en la medida de los segundos. Se suele implementar en software.

Routing es planear todo el camino desde el sender al receiver. Forwarding es dentro de un router determinar cuál va a ser la salida.

Un elemento clave es la **tabla de forwarding**, en el header de cada paquete se evalúan algunos campos con los que se indexa esta tabla. El valor almacenado en la tabla para dicho índice, indica cuál es la salida a tomar.

El algoritmo de routing es el que determina el contenido de las tablas de forwarding.

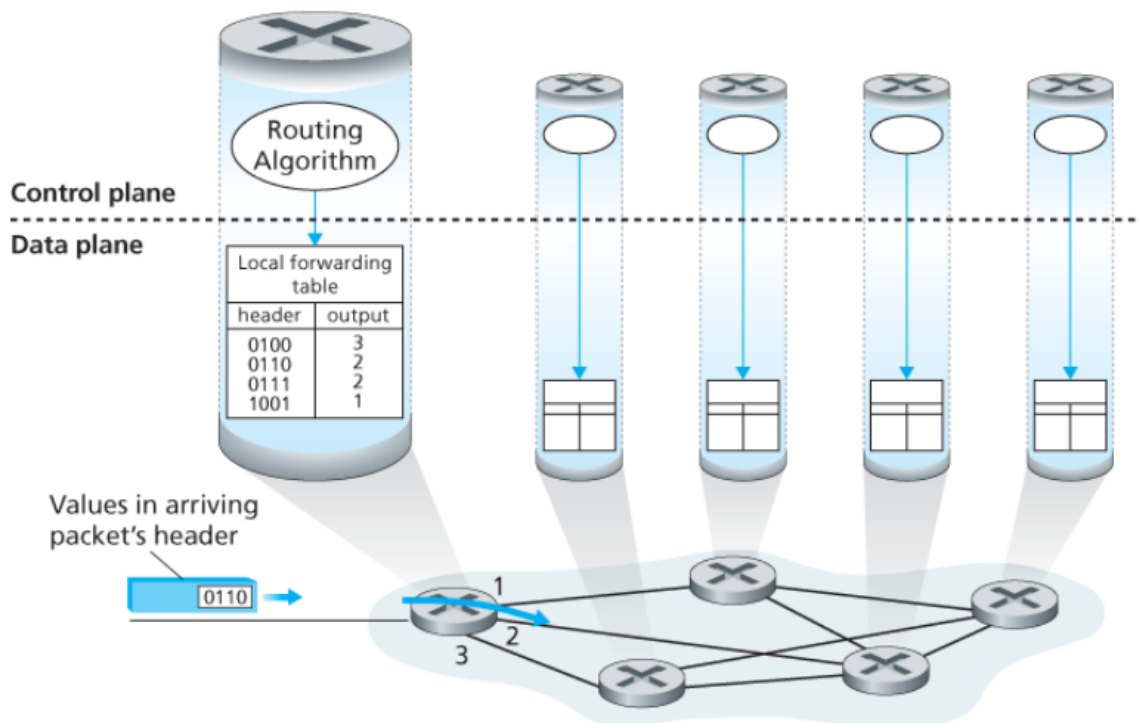


Figure 4.2 Routing algorithms determine values in forward tables

En la siguiente figura, se muestra otro approach donde un controlador remoto distribuye las tablas de forwarding para cada router. → **SDN** (software-defined networking), se llama así porque el controlador que interactúa con los routers está implementado en software en data centers remotos.

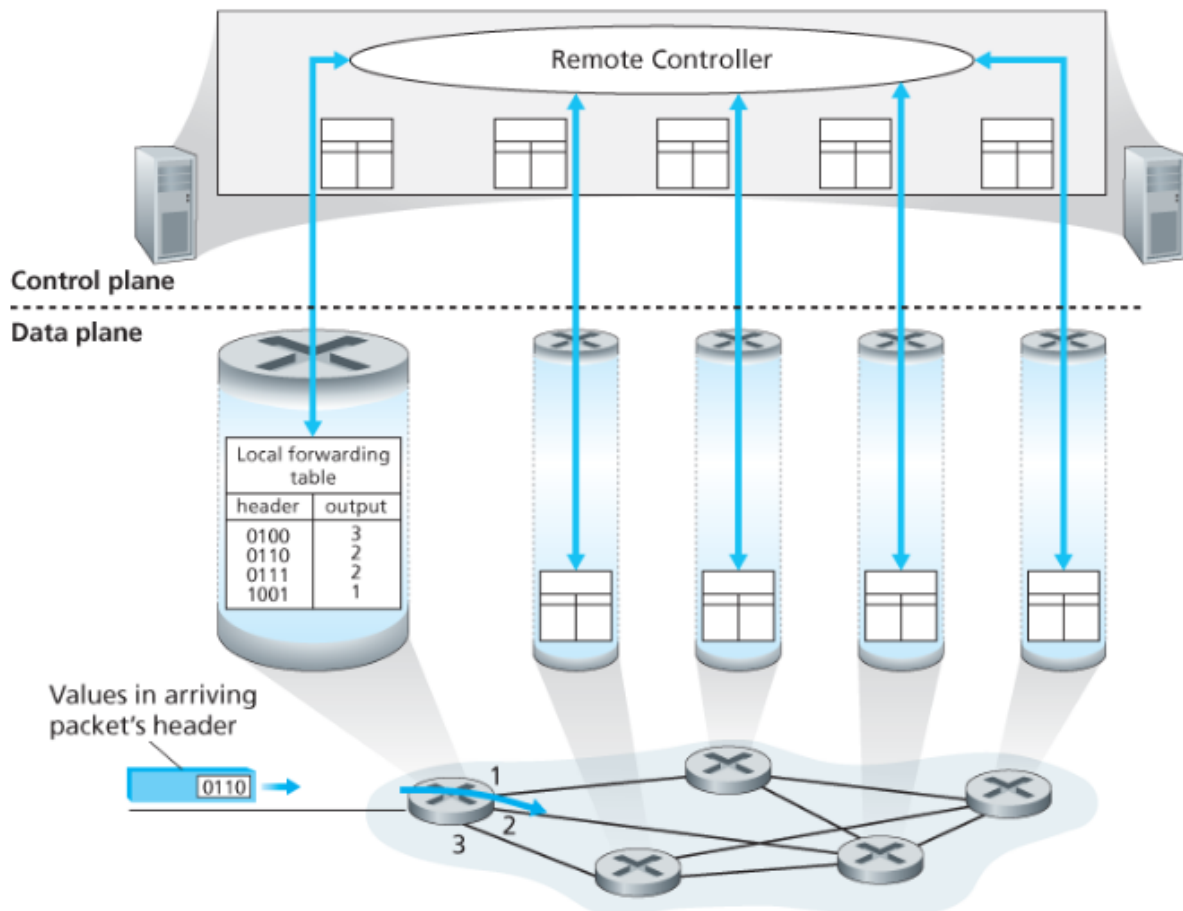


Figure 4.3 A remote controller determines and distributes values in forwarding tables

El modelo de servicio de red (network service model) define las características de la entrega de paquetes entre el host envíador y el host receptor. Algunos de los servicios que pueden incluir:

- **Entrega garantizada.** Un paquete enviado eventualmente va a llegar al receptor.
- **Entrega garantizada con retraso limitado.** Además de garantizar la entrega, especifica el tiempo de demora.
- **Entrega ordenada.** Los paquetes llegan en el orden en que fueron enviados.

- **Minimo bandwidth garantizado.** Este servicio emula el comportamiento de un enlace de transmisión de un determinado bit rate. Siempre que el host transmita bits por debajo del rate especificado, los paquetes van a llegar a destino.
- **Seguridad.** Se podrían encriptar los datagramas en la fuente y desencriptar al llegar a destino para preservar la confidencialidad de los datos.

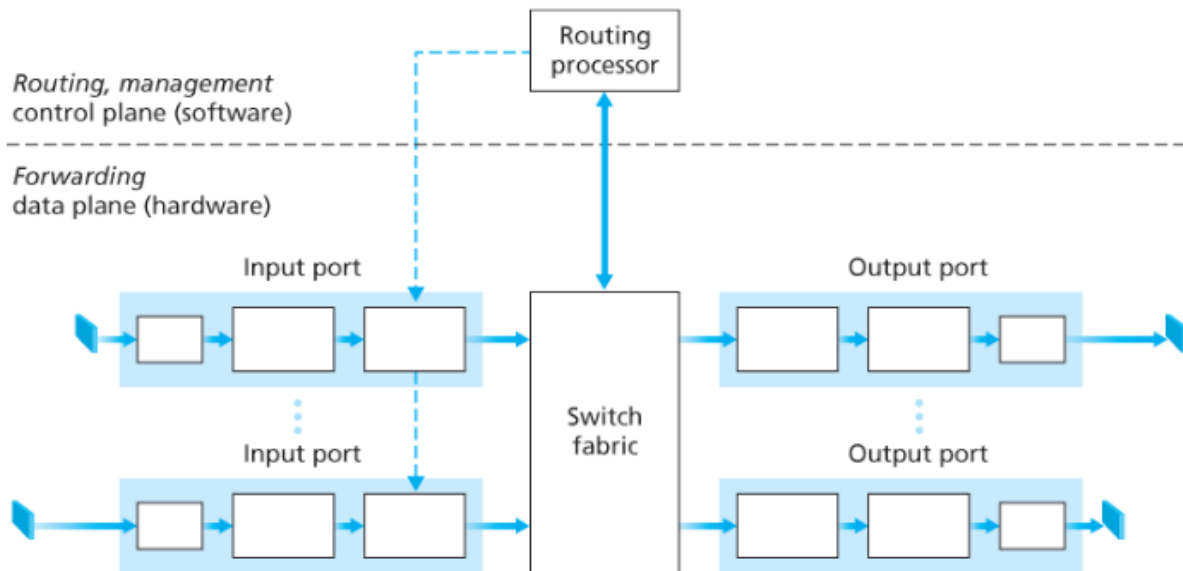


Figure 4.4 Router architecture

Dentro de un router se identifican cuatro componentes:

- **Puertos de entrada** (input ports): Cumple la función física de "terminar" el enlace físico entrante en el router. También cumple funciones de la capa de enlace, funciones de lookup. Acá es donde se consulta a la tabla de forwarding.
- **Switching fabric:** Conecta los inputs con los outputs.
- **Puertos de salida** (output ports): Almacena los paquetes que recibe del switch fabric y los transmite al enlace de salida implementando funciones de la capa de enlace y de la capa física.
- **Routing processor:** Ejecuta las funciones del plano de control. En routers tradicionales, ejecuta protocolos de routing, mantiene tablas de routing e información de estado de los enlaces. En routers SDN, es responsable de la comunicación con el controlador remoto.

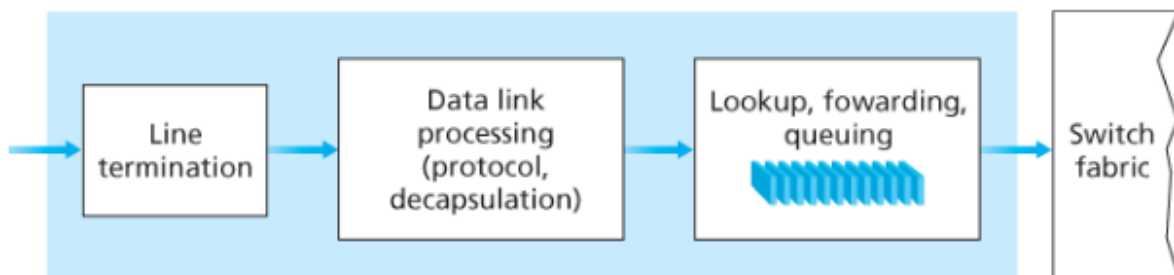


Figure 4.5 Input port processing

No escalaría que en la tabla de forwarding haya una fila por cada dirección, ya que las direcciones son de 8 bits.. lo que se puede hacer entonces, es agrupar las direcciones por prefijo, por ejemplo:

Prefix	Link Interface
<i>11001000 00010111 00010</i>	<i>0</i>
<i>11001000 00010111 00011000</i>	<i>1</i>
<i>11001000 00010111 00011</i>	<i>2</i>
Otherwise	3

Si una dirección matchea con más de un prefijo, el router usa la regla del prefijo más largo (longest prefix matching rule), o sea busca el prefijo más largo que matchee con la dirección.

En algunos diseños, a un paquete se le puede bloquear temporalmente el pase al fabric switch si paquetes de otros inputs lo están usando. En tal caso, el paquete queda encolado y programado para pasar al switch eventualmente.

Además del lookup, en el router se ejecutan otras acciones importantes: procesos de la capa física y de enlace, chequeo de los campos de número de versión, checksum y time-to-live (los últimos dos se reescriben en esta etapa) y la actualización de los contadores relativos al manejo de la red.

Nota: estos pasos que hace el router, donde primero "matchea" algo y después toma una acción, es un caso de la abstracción **match plus action** implementada en varios dispositivos de redes.

El **switching** se puede hacer de distintas maneras:

- **Por memoria** (switching via memory): los primeros routers eran computadoras tradicionales, con el switch bajo control directo de la CPU. Los puertos de input y output funcionaban como dispositivos de entrada y salida en un sistema operativo tradicional. El puerto de entrada, avisaba mediante una señal de interrupción la llegada de un paquete, luego el paquete se copiaba a la memoria del procesador. El procesador, entonces, extraía de los headers la dirección destino y a partir de la tabla de forwarding, copiaba el paquete al puerto de salida. En este escenario, si la bandwidth de la memoria soporta operaciones con un máximo de B paquetes por segundo, entonces el rate total al que los paquetes pueden ser transferidos de la entrada a la salida es $B/2$. Además, los paquetes no pueden ser transferidos al mismo tiempo, ya que la memoria solo puede hacer una operación de lectura/escritura por vez.

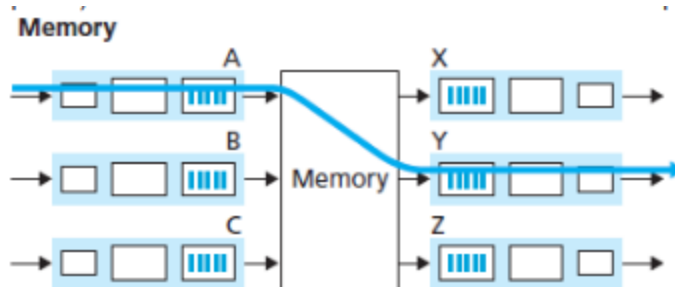
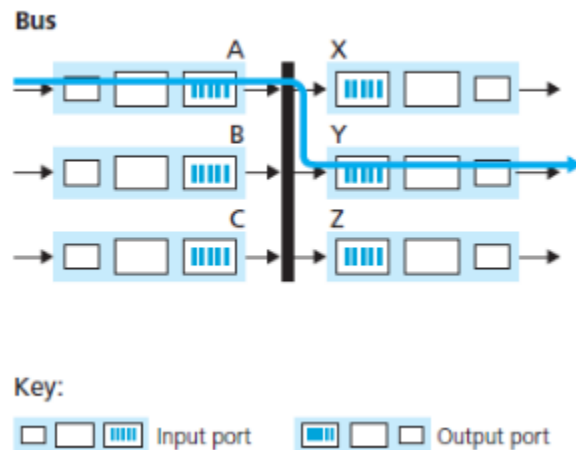
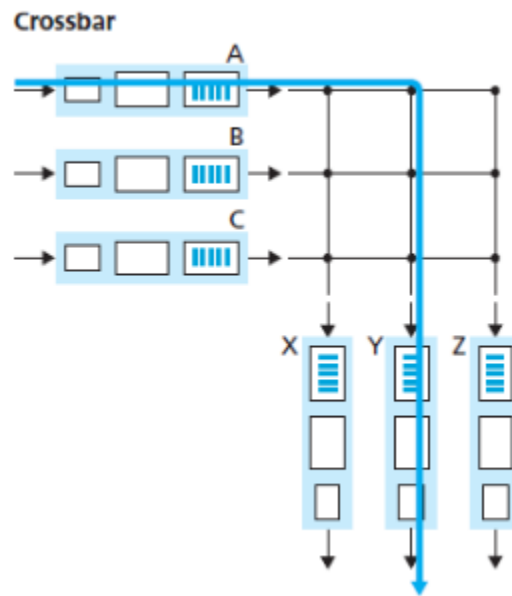


Figure 4.6 Three switching techniques

- **Via bus**: el puerto de entrada transfiere el paquete directamente al puerto de salida mediante un bus compartido, sin intervención del procesador de routing. Esto típicamente se hace pre-apendeando un label (o header) al paquete indicando el puerto de salida y transmitiendo el paquete al bus. Todos los puertos de salida reciben el paquete, pero solo se lo queda aquel que matchee con el label. Por el bus puede cruzar un paquete por vez, por ello la velocidad del router está limitada a la velocidad del bus.



- **Via red interconectada** (interconnection network): una solución para la limitación del bandwidth que ofrece utilizar un único bus compartido, es un switch de tipo **crossbar**. Consiste en $2N$ buses que conectan N puertos de entrada con N puertos de salida. Cada bus vertical intersecta cada bus horizontal en un crosspoint que puede ser abierto o cerrado en cualquier momento por el controlador del switch fabric. Cuando llega un paquete al puerto A que tiene que ser transmitido al puerto Y, el controlador cierra la intersección A-Y y envía el paquete por ese bus. Notar que aca, el puerto A podría mandar al mismo tiempo un paquete al puerto X porque son distintos buses. O sea, que **se pueden enviar paquetes en paralelo**. Además, es non-blocking, a menos que dos paquetes de distinta entrada quieran ir al mismo puerto de salida.



Se pueden formar colas de paquetes tanto en los puertos de entrada como en los de salida. La longitud de las colas dependerá de la velocidad del switch, del tráfico y de la velocidad de la línea. El problema es que a medida que se encolen más paquetes, mayor la chance de que se pierdan paquetes si se llena la memoria del router.

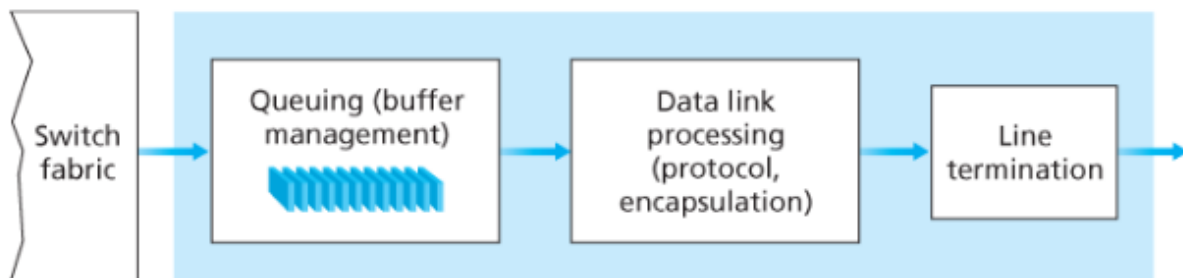


Figure 4.7 Output port processing

Suppose that the input and output line speeds (transmission rates) all have an identical transmission rate of R_{line} packets per second, and that there are N input ports and N output ports. To further simplify the discussion, let's assume that all packets have the same fixed length, and that packets arrive to input ports in a synchronous

manner. That is, the time to send a packet on any link is equal to the time to receive a packet on any link, and during such an interval of time, either zero or one packets can arrive on an input link. Define the switching fabric transfer rate R_{switch} as the rate at which packets can be moved from input port to output port. If R_{switch} is N times faster than R_{line} , then only negligible queuing will occur at the input ports. This is because even in the worst case, where all N input lines are receiving packets, and all packets are to be forwarded to the same output port, each batch of N packets (one packet per input port) can be cleared through the switch fabric before the next batch arrives.

Qué pasa si el switch no es tan rápido como para transferir todos los paquetes sin delay? En ese caso, se pueden formar colas en los puertos de entrada. Una consecuencia de esto es lo que se conoce como **head-of-the-line (HOL) blocking**, un paquete encolado en un puerto entrada tiene que esperar porque está bloqueado por otro paquete que está antes que él.

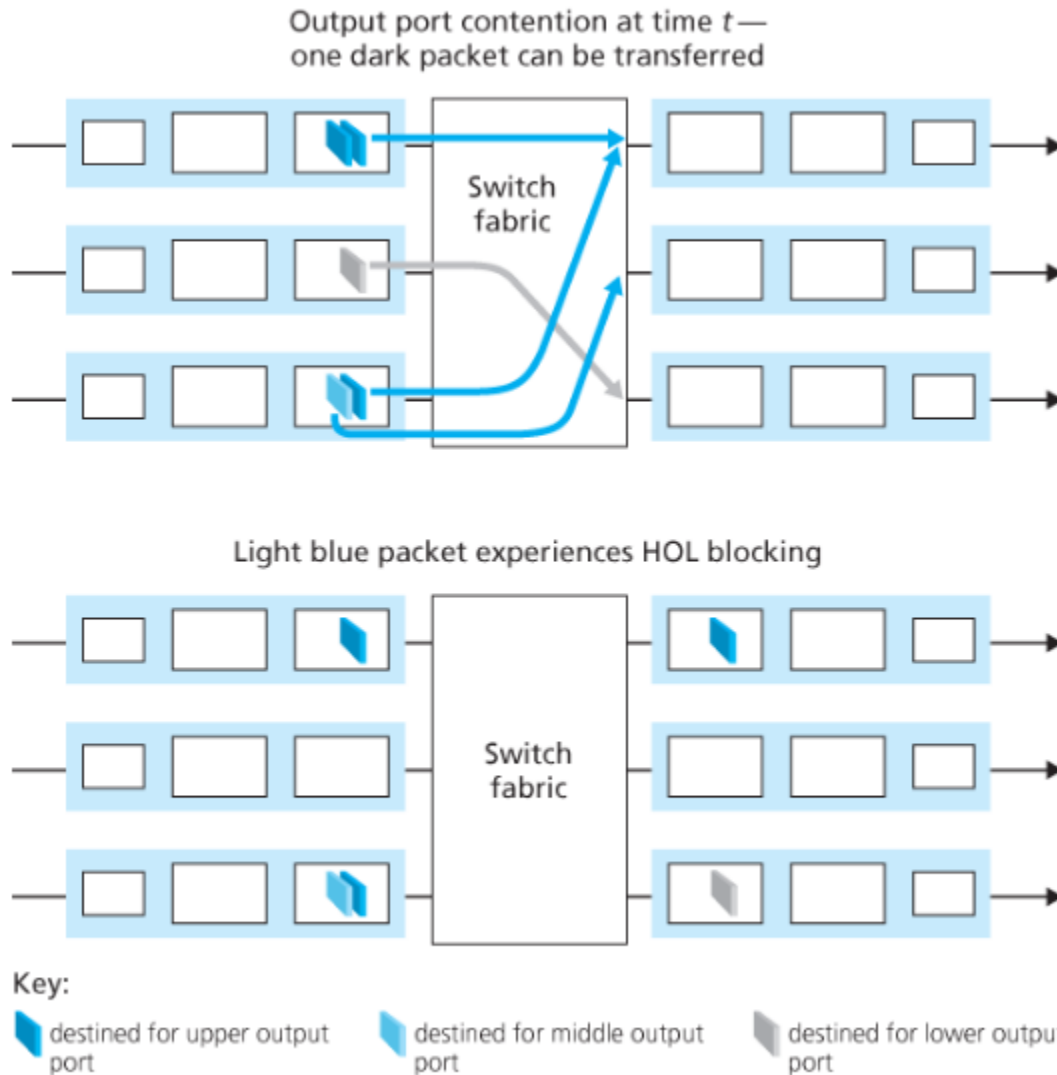


Figure 4.8 HOL blocking at and input-queued switch

Suponiendo que R_{switch} es N veces más rápido que R_{line} y que todos los paquetes van al mismo puerto de salida. En este caso, en el tiempo que lleva mover un paquete al enlace de salida, N paquetes nuevos van a llegar al puerto de salida. Como el puerto de salida solo puede transmitir de a un paquete, los N paquetes nuevos van a tener que hacer cola. Una consecuencia de esto, es que el puerto de salida va a tener que elegir qué paquete transmitir primero al enlace de salida (packet scheduler).

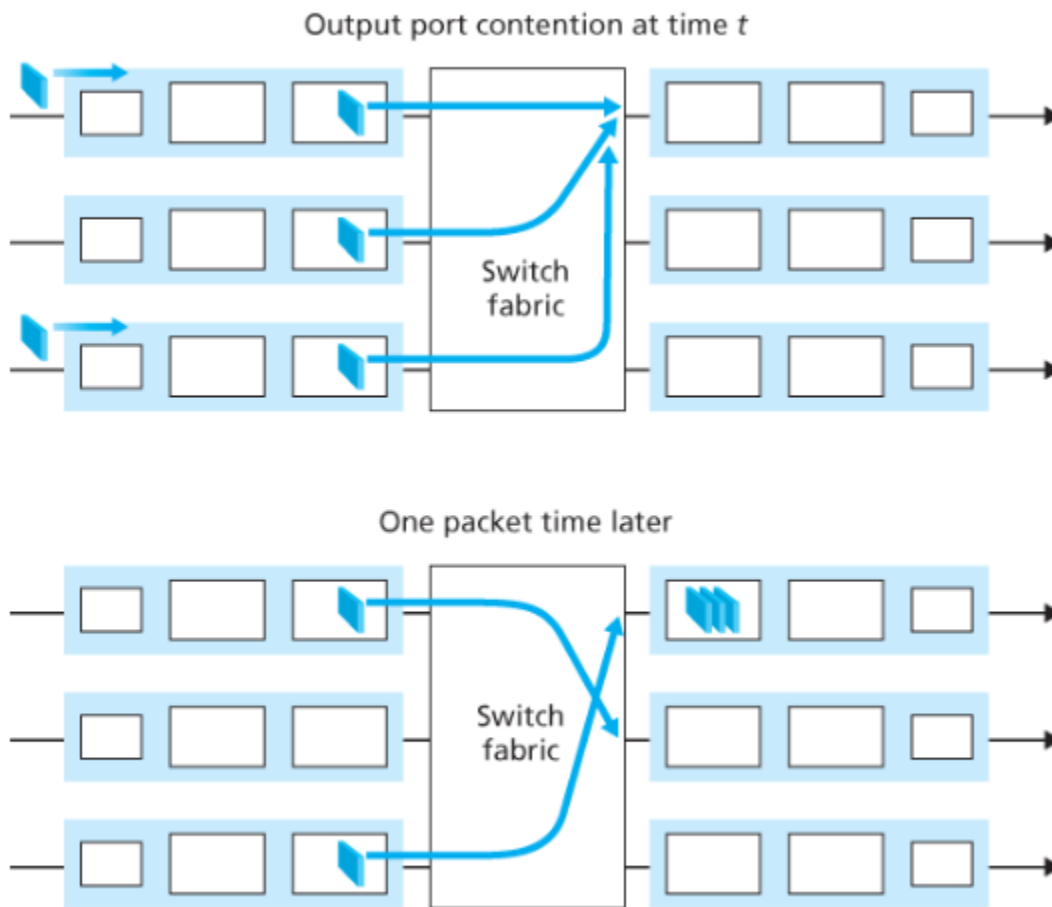


Figure 4.9 Output port queuing

Cuando no hay más memoria para paquetes, se debe tomar la decisión de droppear el paquete nuevo o eliminar uno o más paquetes encolados.

First-In-First-Out (FIFO): los paquetes que llegan a la cola del puerto de salida, esperan para ser transmitidos si el enlace está ocupado. Si el buffer no tiene más espacio para paquetes entrantes, la política de "descarte" de la cola va a ser la encargada de decidir si debe descartarse el paquete nuevo u otro de los que ya estaban en la cola. El modelo FIFO va transmitiendo los paquetes en el orden en que llegan.

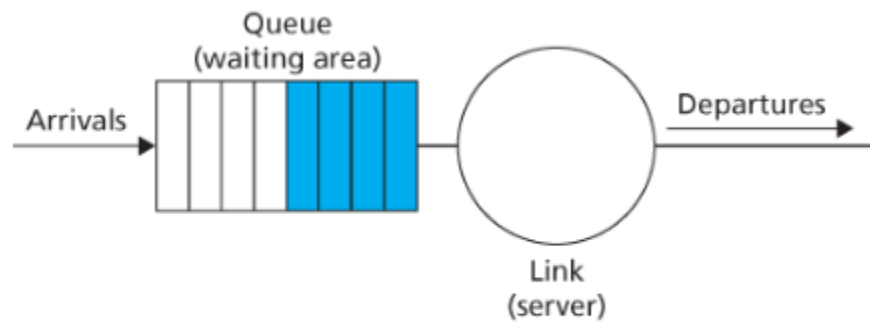


Figure 4.10 FIFO queueing abstraction

Priority Queuing: los paquetes entrantes son clasificados en clases de prioridad a medida que llegan a la cola. Un operador de red podría configurar una cola para que los paquetes que llevan información del manejo de red tengan mayor prioridad, así como paquetes con mensajes de tipo real-time por sobre mensajes tipo email. Tipicamente, cada clase de prioridad tiene su propia cola. Cuando se elige el paquete a transmitir, se transmite el de la cola con mayor prioridad. Dentro de una misma prioridad, se usa FIFO.

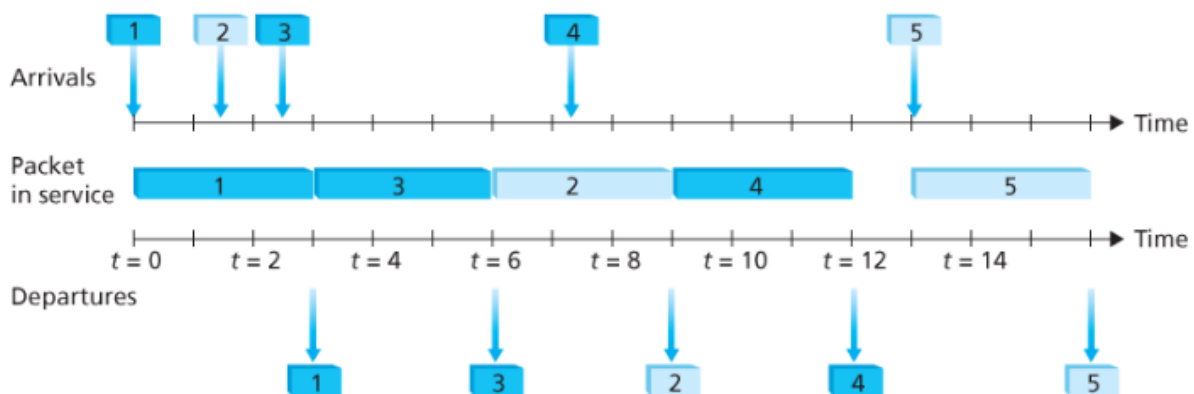


Figure 4.13 The priority queue in operation

Round Robin and Weighted Fair Queuing (WFQ): los paquetes se dividen en colas de prioridad, pero se va alternando el servicio entre las colas. Así ningún paquete queda demasiado tiempo esperando en una cola. WFQ se diferencia de Round Robin en que cada clase puede recibir una cantidad de servicio distinta en algún intervalo de tiempo (o sea que tienen distinto peso).

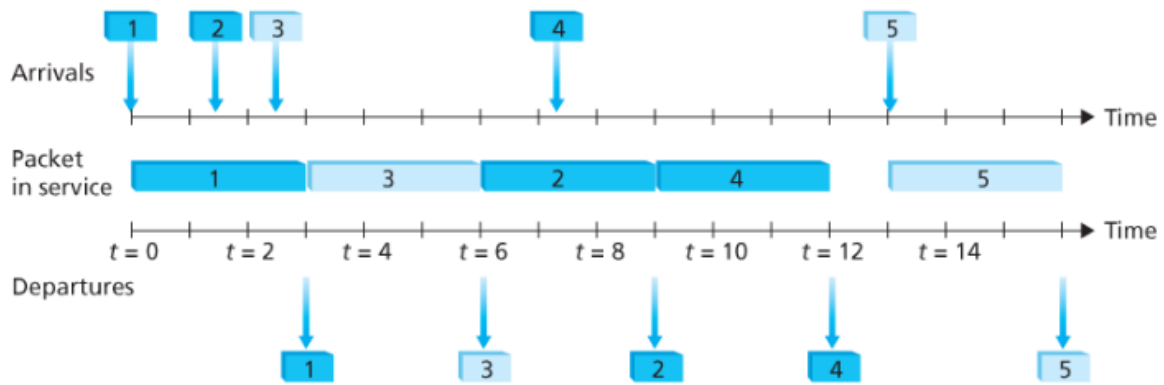


Figure 4.14 The two-class robin queue in operation

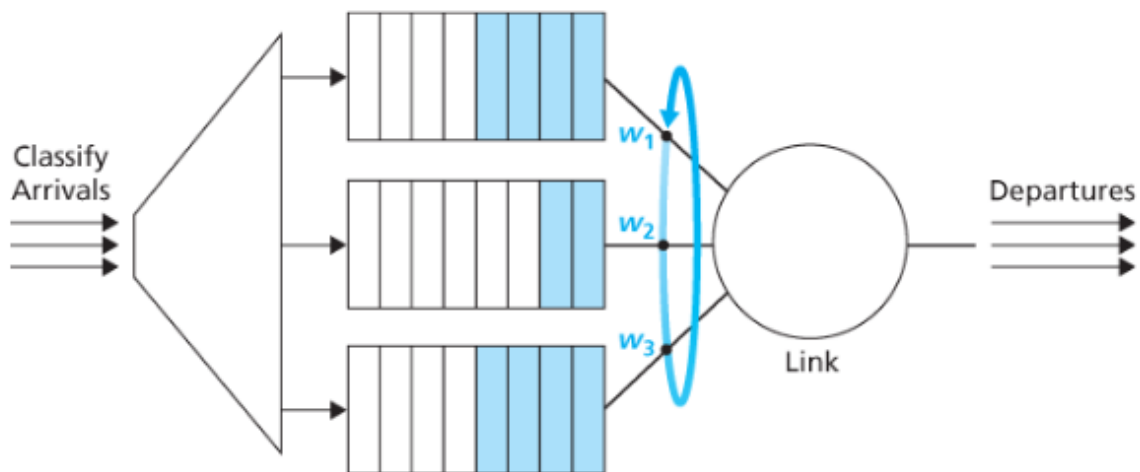


Figure 4.15 Weighted fair queueing

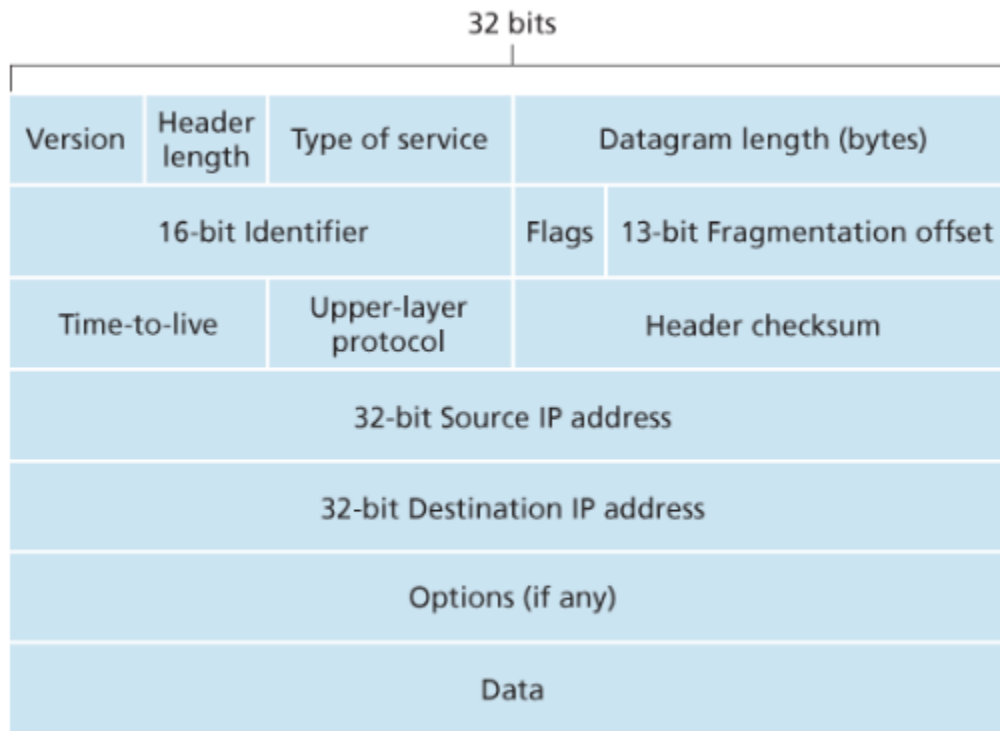


Figure 4.16 IPv4 datagram format

En la capa de red los paquetes son **datagramas**.

- **Version number:** a partir de este número, el router puede determinar cómo debe interpretar los siguientes datos del datagrama.
- **Header length:** se usa para poder identificar dónde comienza el payload.
- **Type of service:** se usa para poder distinguir entre distintos tipos de datagramas, por ej. uno real-time (tipo telefono) de uno no real-time (como FTP).
- **Datagram length:** es el largo total en bytes del header+data.
- **Identifier, flags, fragmentation offset:** tienen que ver con la fragmentación IP (tema para futuro).
- **Time-to-live:** para asegurar que los datagramas no queden circulando para siempre en la red. Si el TTL llega a 0, el router debe droppear el datagrama.
- **Protocol:** en general se usa cuando el datagrama llega al destino final. Indica el protocolo de capa de transporte al que se debe pasar el datagrama. Por ejemplo, un valor 6 indica TCP, un valor 17 indica UDP. Este número es análogo al número

de puerto de la capa de transporte. El número de protocolo es el *glue* entre la capa de red y de transporte, mientras que el número de puerto es el de la capa de transporte y la de aplicación.

- **Header checksum:** para detectar errores, igual que en la capa de transporte.
- **Source and destination IP addresses**
- **Options:** permiten extender el header. Se usan raramente.
- **Data (payload)**

La máxima cantidad de data que puede llevar un enlace se denomina MTU (maximum transmission unit). Cada datagrama IP se encapsula dentro de frame de capa de enlace, para ser transportado de un router a otro; entonces el MTU limita mucho el tamaño del datagrama IP, el problema es que cada enlace puede usar un protocolo de enlace distinto y estos protocolos pueden tener distintos MTU's. Para ejemplificar: si yo soy el router y me llega un datagrama de un tamaño, pero el enlace al que debe ir soporta un tamaño más chico, qué hago? lo voy a tener que dividir en partes más chicas, estas porciones de datagramas se llaman **fragmentos**. Los fragmentos tienen que ser reensamblados antes de llegar a la capa de transporte destino. Para mantener el core de la red simple, se decidió que este reensamble se lleve a cabo en los hosts.

Para que el host destino pueda identificar los fragmentos entre los datagramas y reconstruirlos, los diseñadores de IP incluyeron los headers identification, flag, and fragmentation offset. Cuando se crea el datagrama, el host que envía le pone un número de identificación además de las direcciones fuente y destino. Cuando el router necesita fragmentar un datagrama, le pone a cada fragmento el número de identificación y las direcciones fuente y destino. Así, cuando al host destino le llegan los datagramas, puede identificar si hay fragmentos por el número de identificación. Para que pueda saber que recibió el último fragmento de un datagrama, se le pone el flag en 0 al último. Para saber si falta algún fragmento, se usa el offset field para especificar dónde va el fragmento dentro del datagrama original.

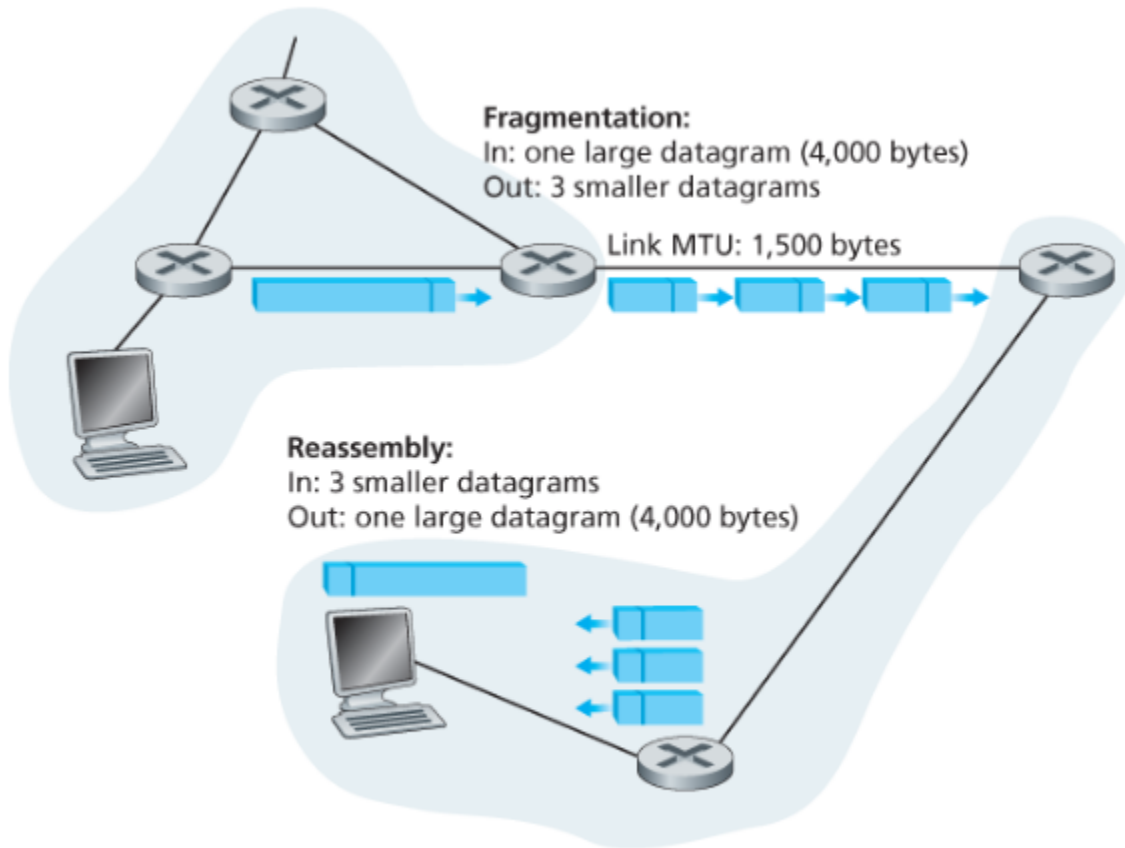


Figure 4.17 IP fragmentation and reassembly

IPv4

Tipicamente, un host tiene un solo enlace con la red. El borde entre el host y el enlace físico se llama **interfaz**. El borde entre un router y cualquiera de sus enlaces, también se llama interfaz (o sea que un router tiene muchas interfaces). IP requiere que cada interfaz de host y router tenga su interfaz. O sea que la dirección IP está asociada no con el host o el router, sino con una interfaz.

Cada dirección IP tiene 32 bits (4 bytes), o sea que hay una posibilidad de 2^{32} direcciones IP posibles. Tipicamente se escriben en *dotted-decimal notation*, cada byte se escribe en decimal separado de otros bytes por un punto. Por ejemplo:

193.32.216.9 = 11000001 00100000 11011000 00001001

Cada interfaz debe tener una IP única en el mundo. Esta IP no se asigna arbitrariamente, de hecho hay una porción de la IP que se determina por la subred a la que está conectada.

Una red que interconecta distintas interfaces de host y una interfaz de router forma una subred.

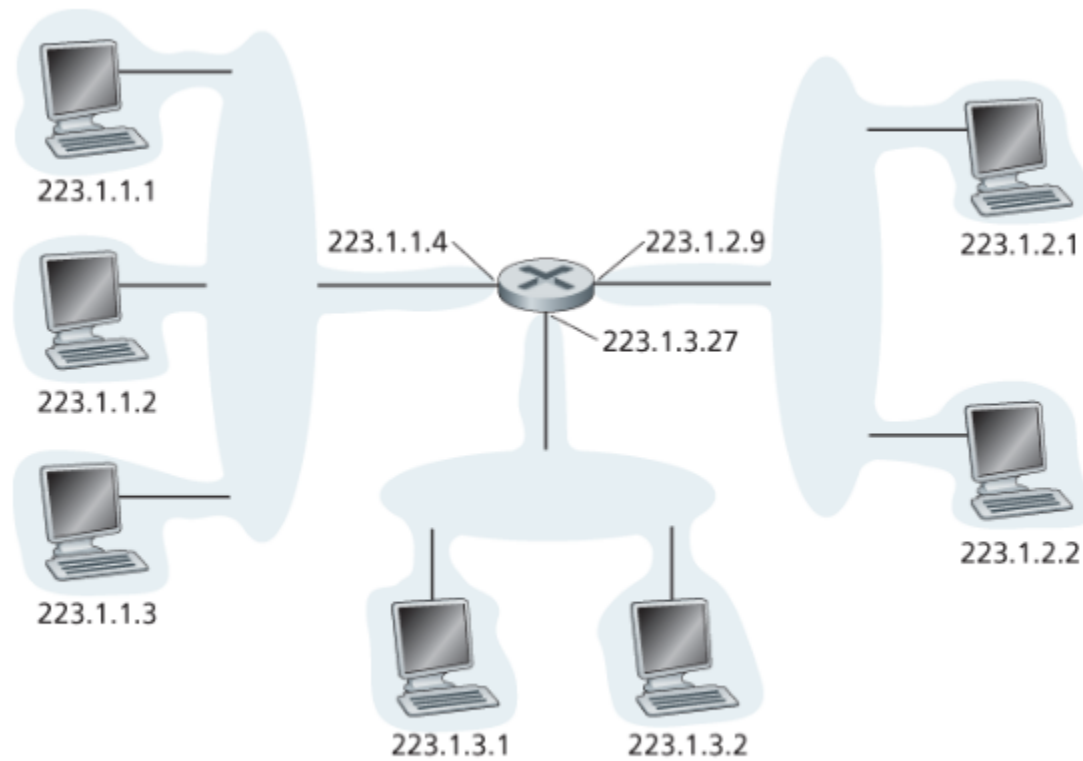


Figure 4.18 Interface addresses and subnets

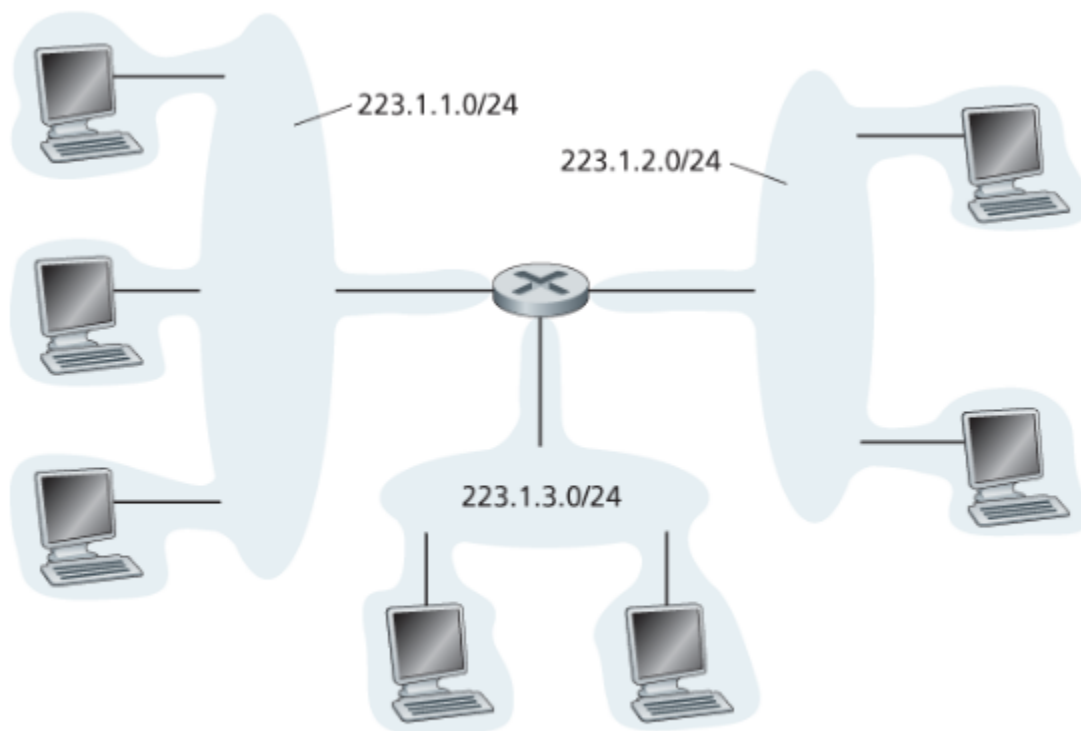


Figure 4.19 Subnet addresses

La notación '/24' se conoce como máscara de subred, indica que los 24 bits más a la izquierda definen la dirección de la subred.

To determine the subnets, detach each interface from its host or router, creating islands of isolated networks, with interfaces terminating the end points of the isolated networks. Each of these isolated networks is called a subnet.

La estrategia de asignación de direcciones de Internet se conoce como **Classless Interdomain Routing (CIDR)**.

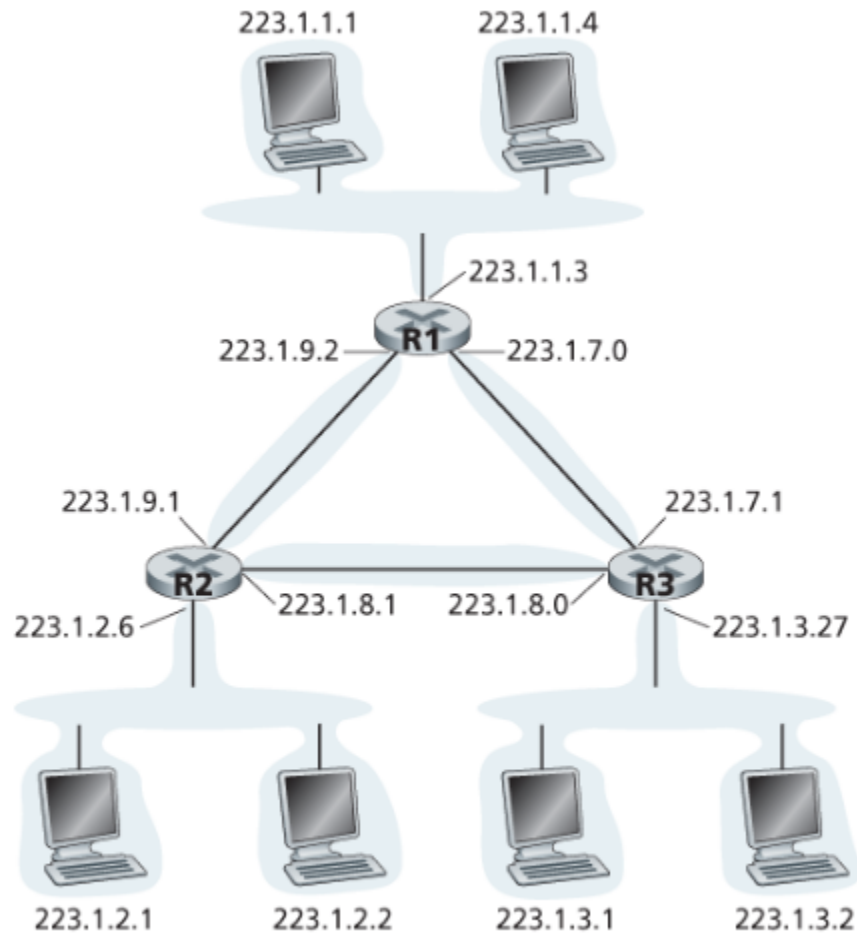


Figure 4.20 Three routers interconnecting six subnets

Cuando un router ajeno a la organización transmite un datagrama cuya dirección pertenece a la organización, solo necesita tener en cuenta los primeros x bits, así se reduce el tamaño de las tablas de forwarding. Los últimos bits solo importan dentro de la organización.

Cuando un host envía un datagrama a la dirección 255.255.255.255, el mensaje va a ser entregado a todos los hosts de la misma subred (broadcast address).

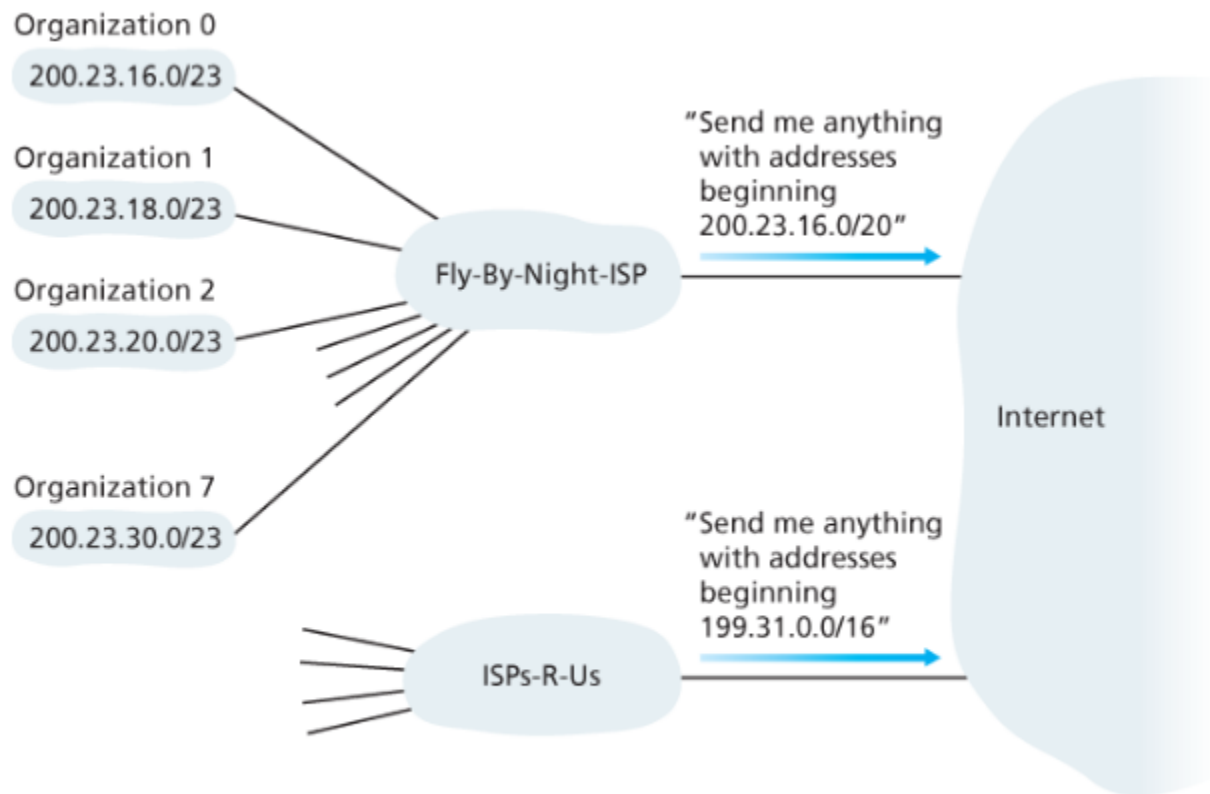


Figure 4.21 Hierarchical addressing and route aggregation

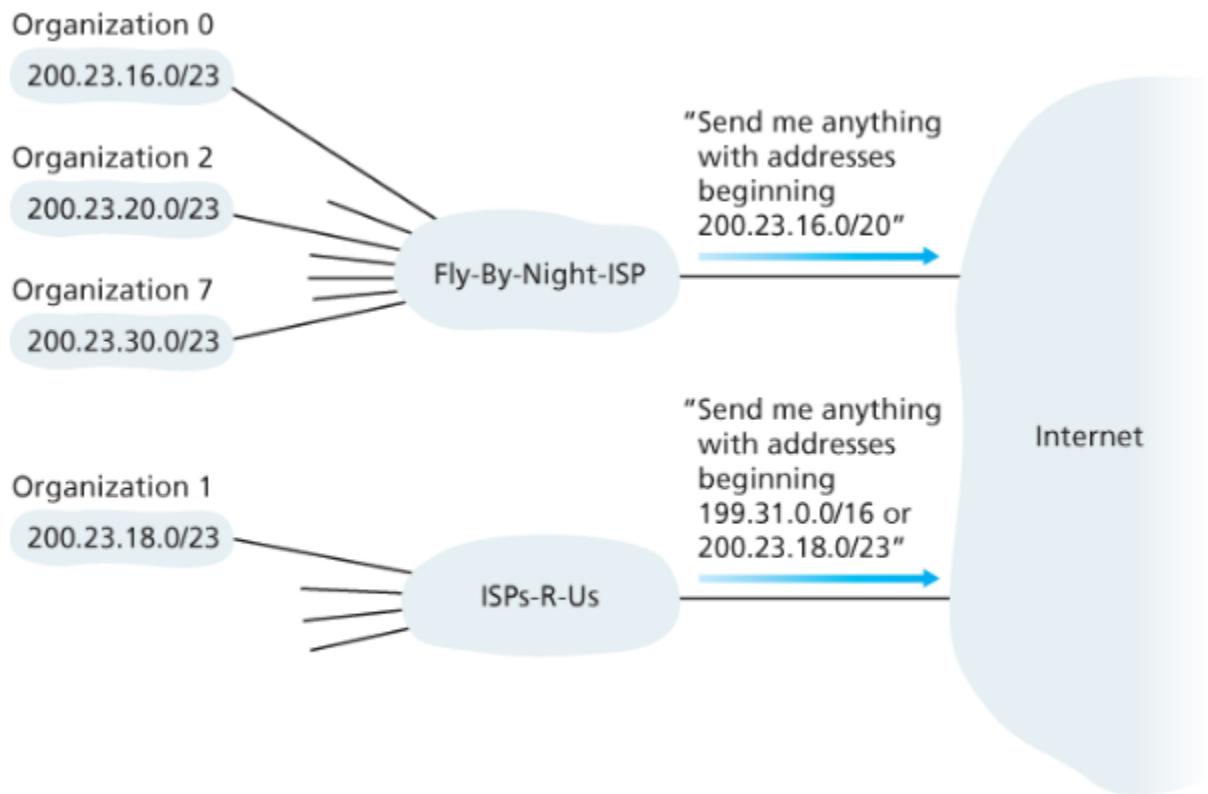


Figure 4.22 ISPs-R-Us has a more specific route to Organization 1

Para poder obtener un bloque de direcciones IP para utilizar dentro de una subred, el administrador de red debe comunicarse con su ISP. El ISP ya tiene bloques de direcciones IP reservadas, entonces debería dividir estos bloques en 8 partes iguales y contiguas para poder darle a hasta 8 organizaciones distintas.

ISP's block: 200.23.16.0/20 11001000 00010111 00010000 00000000

Organization 0 200.23.16.0/23 11001000 00010111 00010000 00000000

Organization 1 200.23.18.0/23 11001000 00010111 00010010 00000000

Organization 2 200.23.20.0/23 11001000 00010111 00010100 00000000

... ..

...

Organization 7 200.23.30.0/23 11001000 00010111 00011110 00000000

Las direcciones IP se manejan bajo la autorización de **Internet Corporation for Assigned Names and Numbers (ICANN)**.

Una vez que el host tiene su bloque de direcciones, puede asignar IP individuales al router y host. Típicamente un administrador de sistema configura manual o remotamente la IP en el router. En el host se puede configurar manual o más común es mediante el **Dynamic Host Configuration Protocol (DHCP)**. DHCP permite que un host obtenga una dirección IP automáticamente. Además, permite que el host sepa su servidor DNS local, su máscara de subred, la dirección del primer router.

DHCP es un protocolo de tipo cliente-servidor.

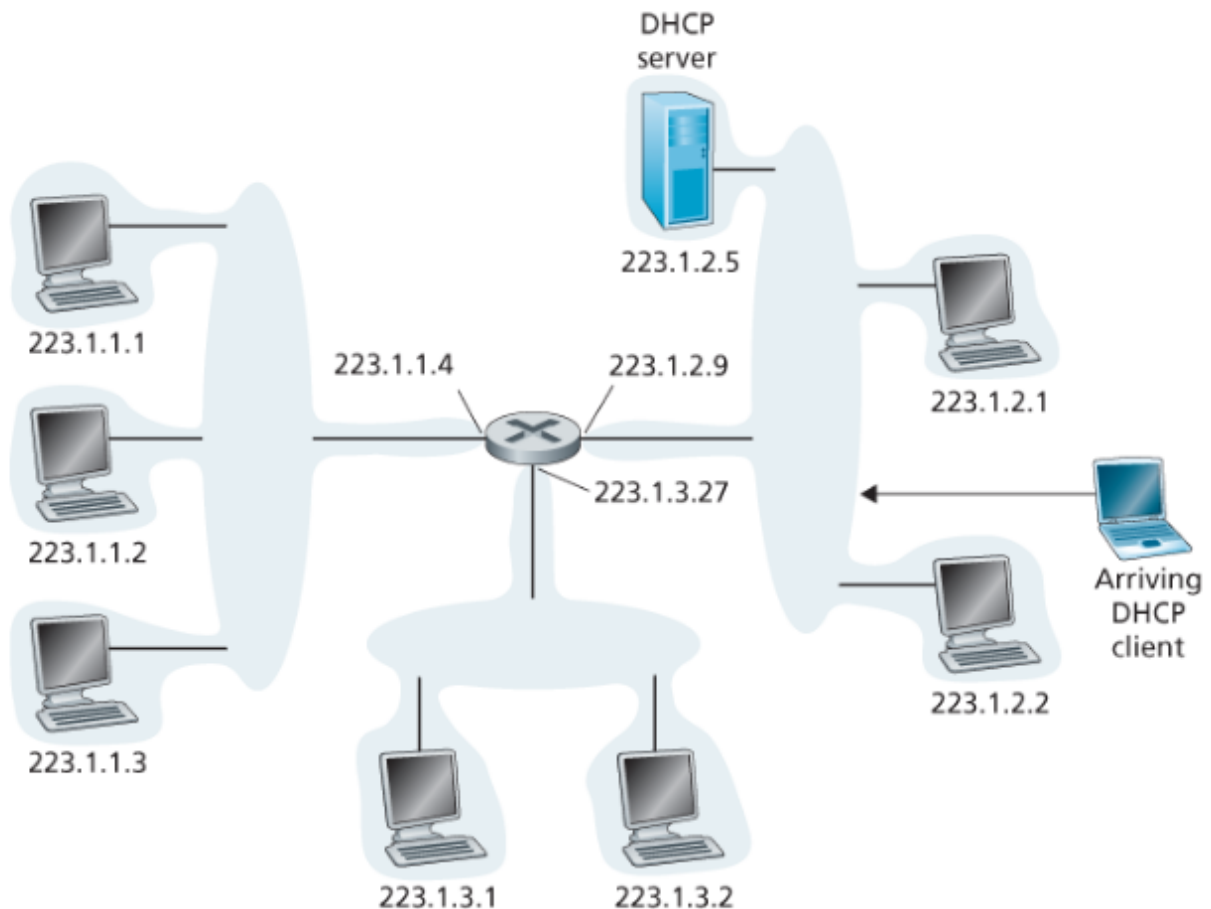


Figure 4.23 DHCP client and server

- **DHCP server discovery:** cuando se conecta un nuevo host, envía un DHCP discovery message dentro de un paquete UDP al puerto 67 a la dirección de broadcast.
- **DHCP server offers:** un servidor DHCP que recibe el mensaje de discovery, responde al cliente con un DHCP offer message que es broadcasteado a toda la subred. Como puede haber varios servidores, el cliente va a tener que elegir. Cada offer contiene: id transaccional del discover message recibido, la IP propuesta para el cliente, la máscara de red y el tiempo por el que la IP va a ser válida.
- **DHCP request:** el cliente va a elegir el servidor enviando un DHCP message request repitiendo los parámetros de configuración.
- **DHCP ACK:** el servidor responde con un DHCP ACK message confirmando los parámetros.

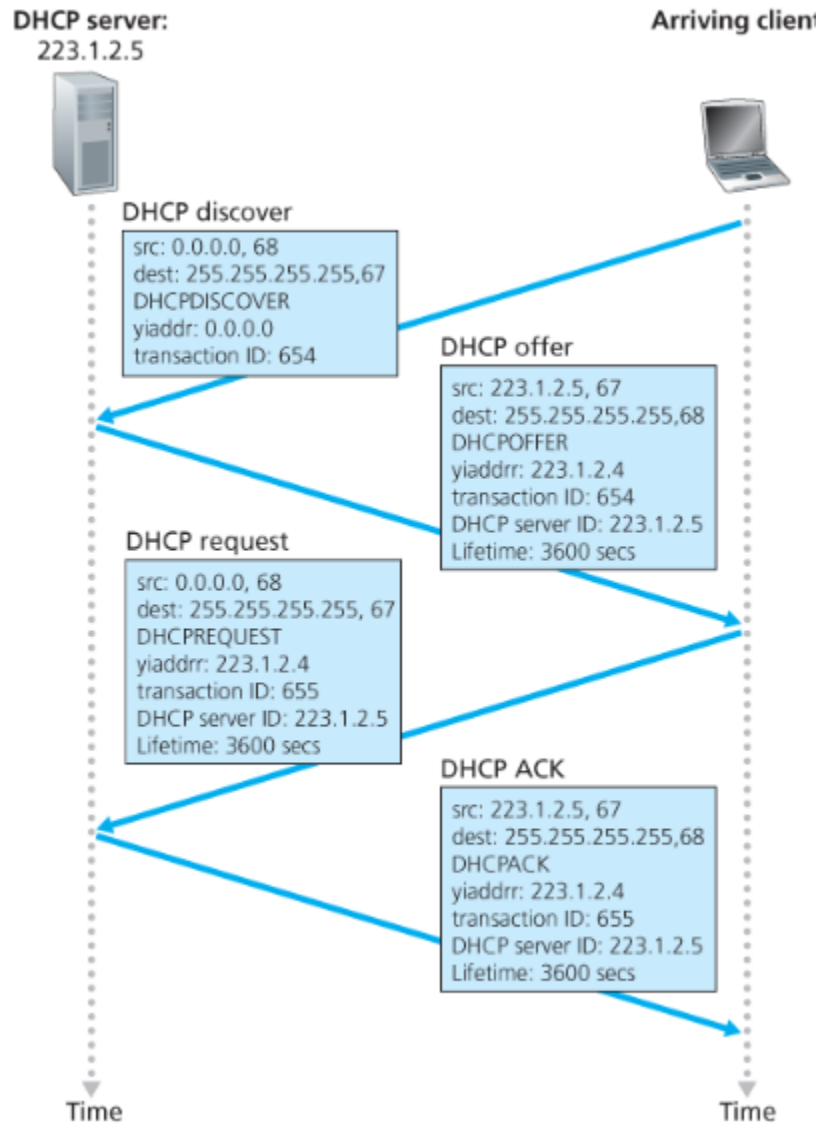


Figure 4.24 DHCP client-server interaction

Un approach para la asignación de direcciones IP es el **network address translation (NAT)**.

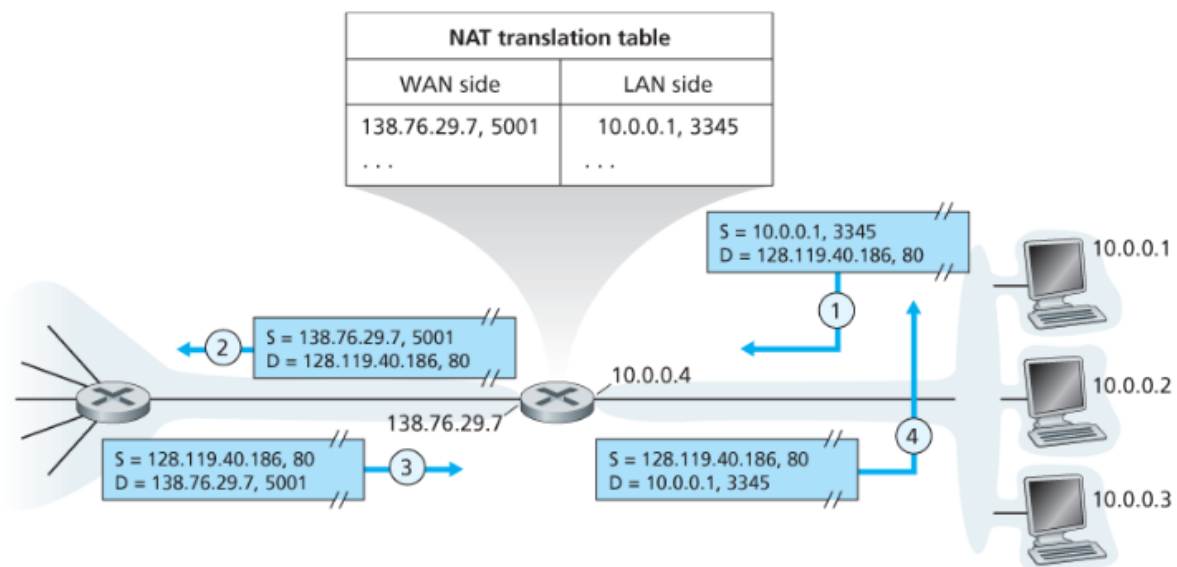


Figure 4.25 Network address translation

El router NAT visto desde fuera se comporta como un único dispositivo con una única IP. Siguiendo el ejemplo de la imagen, todo el tráfico que sale del router local lleva la dirección fuente 138.76.29.7 y todo el tráfico que ingresa, ingresa a la dirección destino 138.76.29.7. Para saber a qué dispositivo tiene que mandar los datagramas, existe la NAT translation table.

Como la cantidad de direcciones de IPv4 estaba quedando corta, se creo la IPv6.

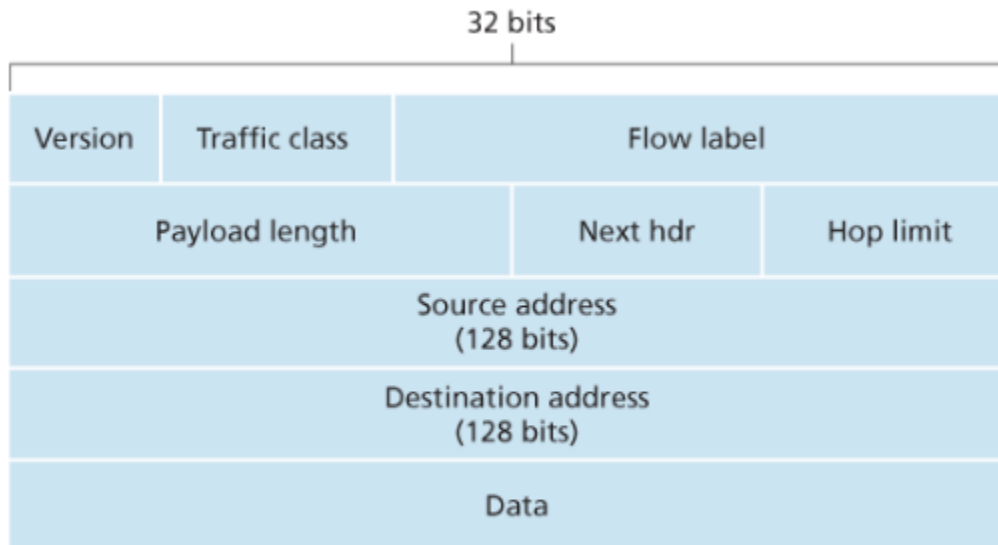


Figure 4.26 IPv6 datagram format

Cambios respecto a la versión anterior:

- **Más direcciones disponibles:** se incrementa el tamaño de las direcciones de 32 a 128 bits, asegurando que el mundo (lit) no se quede sin direcciones IP. Se introduce un nuevo tipo de dirección: **anycast**.
- **Header fijo de 40 bytes:** permite que se procesen más rápido los datagramas en el router.
- **Flow labeling:** se etiquetan los paquetes pertenecientes a determinados flujos, por ejemplo en transmisión de audio o video.

Descripción de campos de IPv6:

- **Version**
- **Traffic class:** como el campo TOS de IPv4, se puede usar para dar prioridad a ciertos datagramas dentro de un flujo.
- **Flow label:** se usa para identificar flujos de datagramas.
- **Payload length**
- **Next header:** identifica el protocolo al que van a ser entregados los datos del datagrama.

- **Hop limit:** se va decrementando en uno cada vez que un router lo forwardea. Cuando llega a 0, se descarta el datagrama.
- **Source and destination addresses**
- **Data:** el payload. Cuando el datagrama llega a destino, se elimina el payload del datagrama y se pasa al protocolo especificado en el campo 'Next header'.

Algunos campos que estaban en IPv4 pero **no** en IPv6:

- **Fragmentations/reassembly:** no se permite la fragmentación y ensamblado en los routers, solo puede hacerse en la fuente o destino. Si un datagrama es muy grande, se droppea y manda un mensaje de error "Packet Too Big" de vuelta al sender. Se hace esto para ahorrar tiempos de procesamiento.
- **Header checksum:** se sentía redundante considerando que en la capa de transporte y enlace ya existe el chequeo del checksum.
- **Options:** es uno de los posibles 'next header's.

Cómo se lleva a cabo la transición de IPv4 a IPv6? via **tunneling**. La idea básicamente es encapsular la información del datagrama IPv6 en el payload de un datagrama IPv4, así dentro del router pueden viajar como IPv4, pero en los extremos se dismantela como IPv6 (se identifica que debe hacerse esto a partir del protocol number field).

Logical view



Physical view

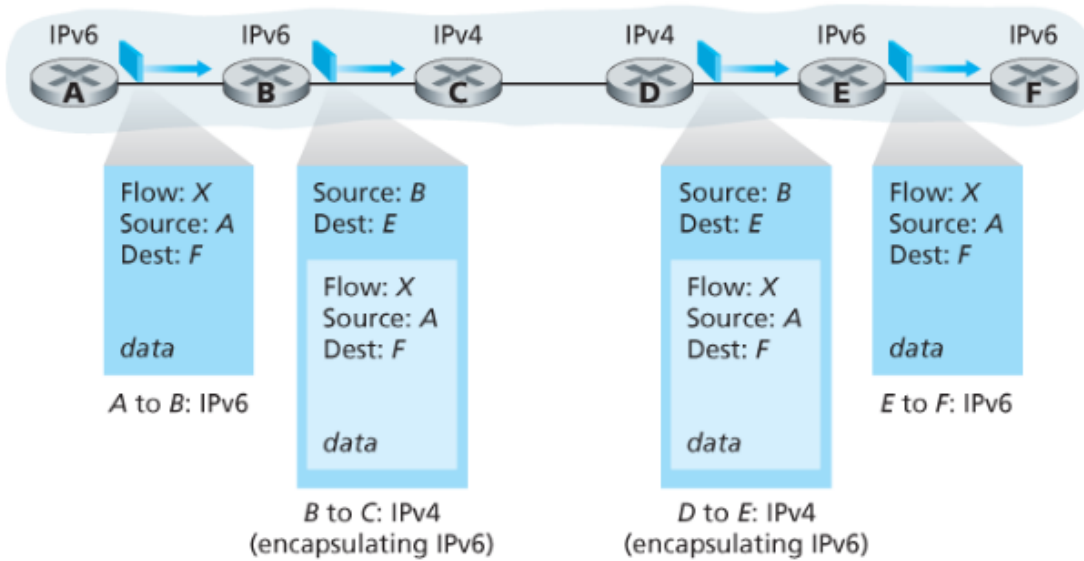


Figure 4.27 Tunneling

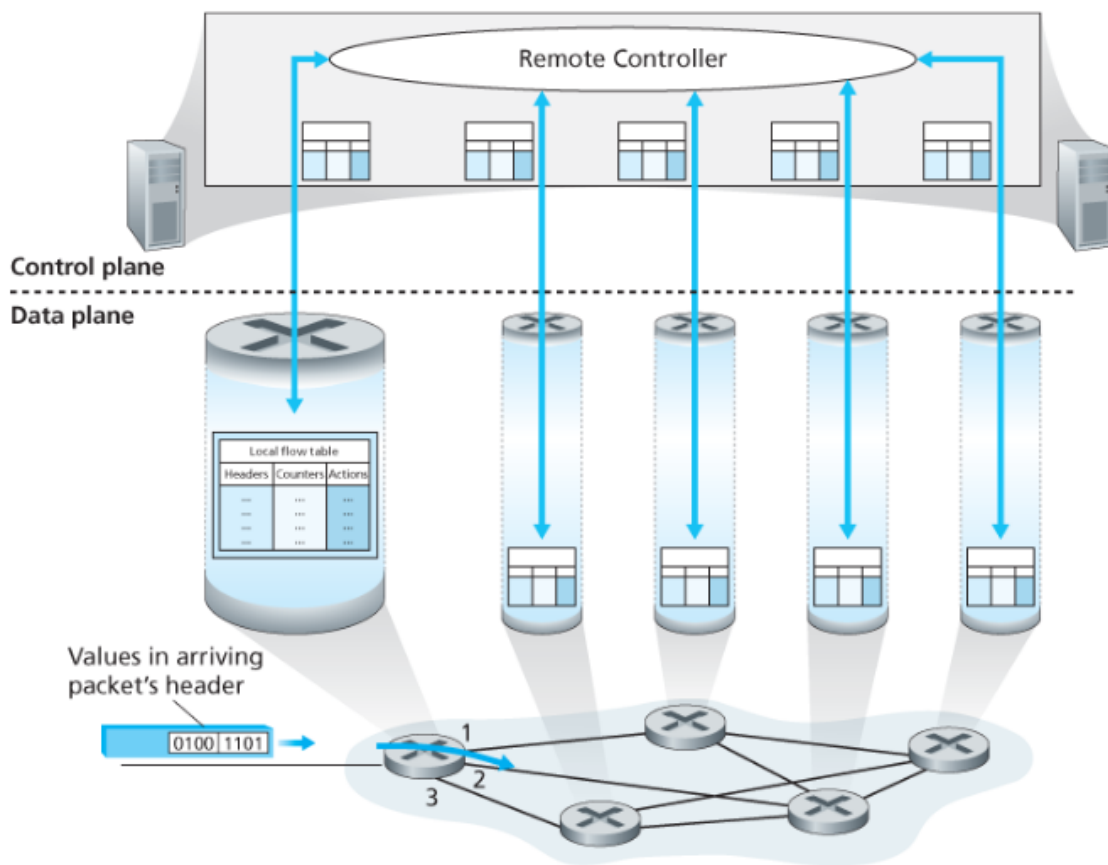


Figure 4.28 Generalized forwarding: Each packet switch contains a match-plus-action table that is computed and distributed by a remote controller

Cada entrada de la tabla de forwarding, conocida como **flow table**, incluye:

- Un set de valores para el campo headers, con los que van a matchear los paquetes entrantes.
- Un set de contadores que se van actualizando a medida que matchean los paquetes con las entradas de la tabla.
- Un set de acciones a tomar cuando un paquete matchee.

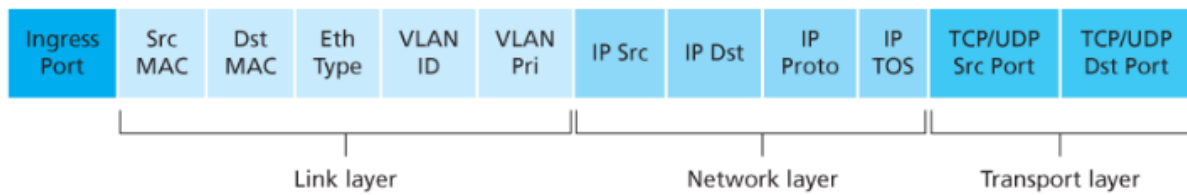


Figure 4.29 Packet matching fields, OpenFlow 1.0 flow table

Capítulo 5: The network layer - Control plane

Hay dos approaches para la instalación y mantenimiento de las tablas de forwarding y de flujo.

- **Per-router control:** un algoritmo se ejecuta en cada router. Cada router tiene un componente que se comunica con componentes de otros routers para computar los valores de la tabla de forwarding.

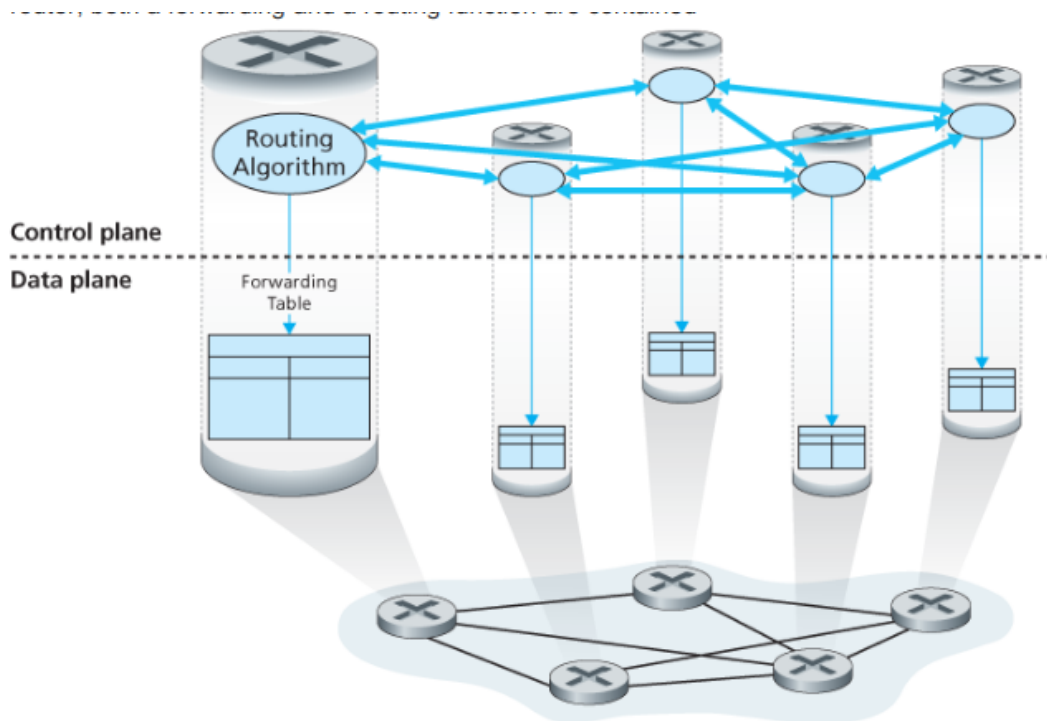


Figure 5.1 Per-router control: Individual routing algorithm components interact in the control plane

- **Logically centralized control:** un controlador centralizado computa y distribuye las tablas de forwarding que va a usar cada router.

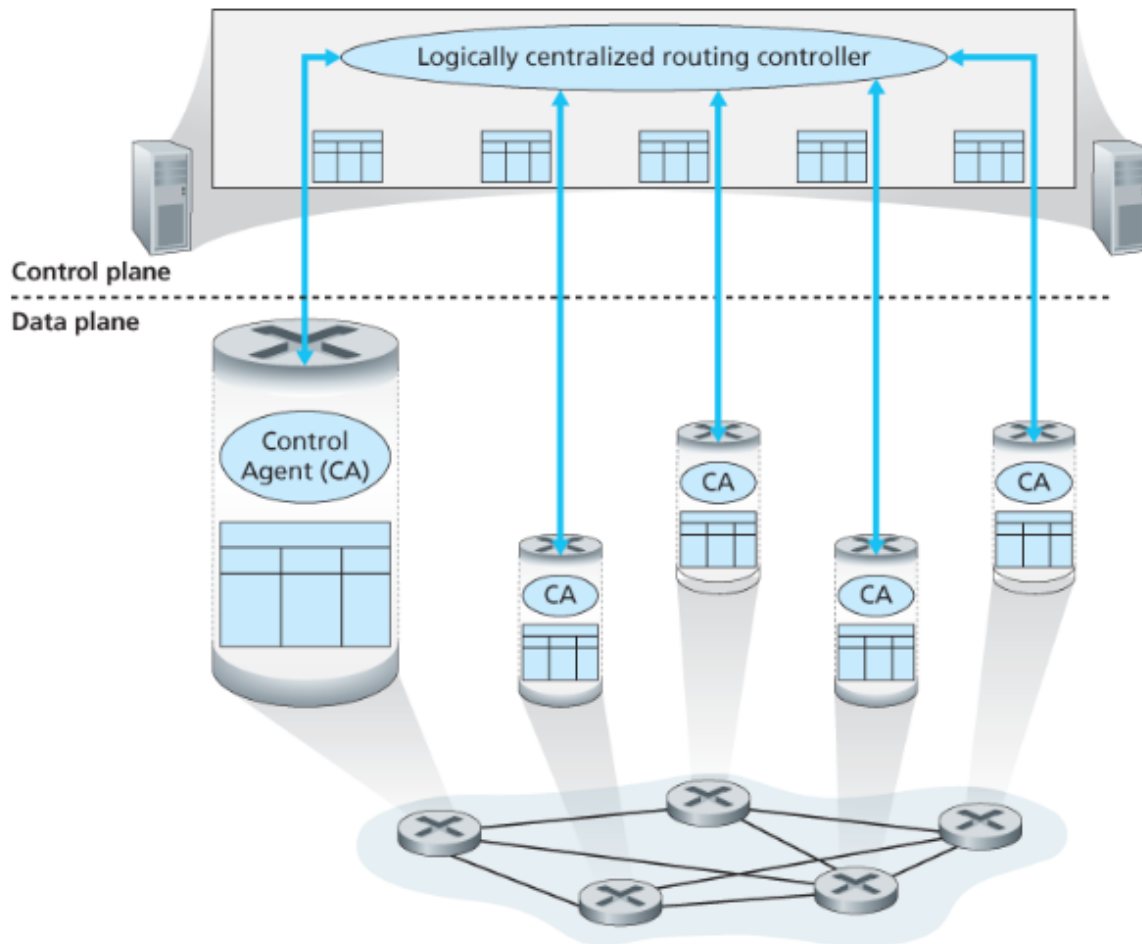


Figure 5.2 Logically centralized control: A distinct, typically remote, controller interacts with local control agents (CAs)

Para plantear los problemas de routing, se puede pensar en grafos donde cada nodo es un router y cada arista es un enlace físico. Además cada arista tiene un peso que representa el costo. Típicamente, el costo refleja la distancia física, la velocidad o el costo monetario del enlace.

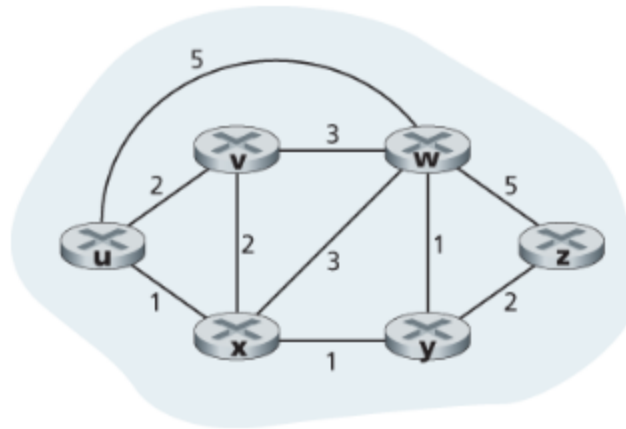


Figure 5.3 Abstract graph model of a computer network

El objetivo natural para un algoritmo de routing es el de encontrar el camino menos costoso.

Una manera de clasificar los algoritmos de routing es definiendo si son centralizados o no.

- **Algoritmo de routing centralizado:** busca el camino menos costoso conociendo la red completa. Es decir, el algoritmo toma como input la conectividad entre todos los nodos y aristas. Algoritmos con estados globales se conocen como **link-state algorithms**.
- **Algoritmo de routing descentralizado:** el cálculo del camino menos costoso lo hace cada router de manera iterativa. Un nodo conoce sobre sus aristas, después va intercambiando información con los nodos vecinos y así iterativamente hasta saber lo suficiente para calcular el camino.

Una segunda clasificación de los algoritmos es si son **estáticos** o **dinámicos**. En los estáticos, las rutas cambian muy lentamente, en general por intervención humana. En los dinámicos, las rutas cambian a medida que aumenta el tráfico en la red o por cambio de topología(?).

Una tercera clasificación es por si son **load-sensitive** o **load-insensitive**. En un algoritmo load-sensitive, el costo del enlace puede variar dependiendo de la congestión de la red.

Algoritmo Link-State (LS)

En este tipo de algoritmo, la topología de red y todos los costos de enlace son conocidos. Esto se logra haciendo que todos los nodos broadcasteen paquetes link-state a todos los demás nodos de la red. Los algoritmos que se usan son el de **Dijkstra** o el de Prim. El algoritmo de Dijkstra es iterativo, tiene la particularidad que después de la k-ésima iteración, los caminos menos costosos van a ser conocidos por k nodos destino, y entre esos caminos menos costosos esos k caminos van a tener los k costos más pequeños.

Notación:

$D(v)$: cost of the least-cost path from the source node to destination v as of this iteration of the algorithm.

$p(v)$: previous node (neighbor of v) along the current least-cost path from the source to v .

N' : subset of nodes; v is in N' if the least-cost path from the source to v is definitively known.

El algoritmo consiste en una inicialización seguida por un loop que se ejecuta N veces, si N es la cantidad de nodos en la red.

```
Initialization:
   $N' = \{u\}$ 
  for all nodes  $v$ 
    if  $v$  is a neighbor of  $u$ 
      then  $D(v) = c(u, v)$ 
    else  $D(v) = \infty$ 

Loop:
  find  $w$  not in  $N'$  such that  $D(w)$  is a minimum
  add  $w$  to  $N'$ 
  update  $D(v)$  for each neighbor  $v$  of  $w$  and not in  $N'$ :
     $D(v) = \min(D(v), D(w) + c(w, v))$ 

  /* new cost to  $v$  is either old cost to  $v$  or known
    least path cost to  $w$  plus cost from  $w$  to  $v$  */
  until  $N' = N$ 
```

Ejemplo:

Table 5.1 Running the link-state algorithm on the network in **Figure 5.3**

step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

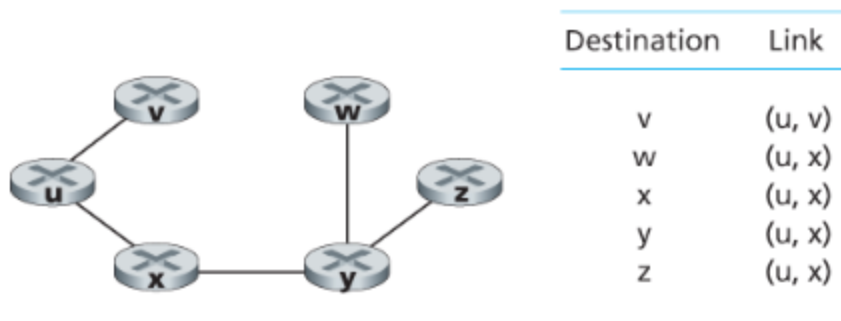


Figure 5.4 Least cost path and forwarding table for node u

La complejidad del algoritmo es $O(n^2)$

Algoritmo Distance-Vector (DV)

Es iterativo, asincrónico y distribuido. Es distribuido en el sentido que cada nodo recibe información sobre sus vecinos directos, hace cálculos y distribuye los resultados devuelta a los vecinos. Es iterativo en el sentido de que el algoritmo continúa hasta que no hay más intercambio de información entre vecinos. Es asincrónico porque no necesita que los nodos operen lockeándose entre ellos.

Ecuación de Bellman-Ford:

$$dx(y) = \min_v \{c(x, v) + dv(y)\}$$

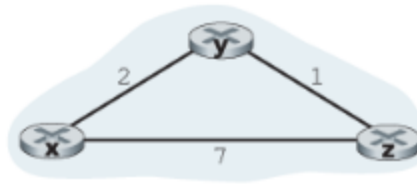
```

Initialization:
  for all destinations y in N:
    D (y)= c(x, y)/* if y is not a neighbor then c(x, y)=  $\infty$  */
  for each neighbor w
    D (y) = ? for all destinations y in N
  for each neighbor w
    send distance vector D = [D (y): y in N] to w

loop
  wait (until I see a link cost change to some neighbor w or
        until I receive a distance vector from some neighbor w)
  for each y in N:
    D (y) = min {c(x, v) + D (y)}
  if Dx(y) changed for any destination y
    send distance vector D = [D (y): y in N] to all neighbors

forever

```



Node x table

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

Node y table

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

		cost to		
		x	y	z
from	x	0	2	7
	y	2	0	1
	z	7	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

Node z table

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

		cost to		
		x	y	z
from	x	0	2	7
	y	2	0	1
	z	3	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

.....> Time

Falta de la pág. 437 a 441

Intra-AS Routing

Hasta ahora se vio la red de routers como algo “homogeneo” donde todos los routers tienen que saber la información de todos los demás routers. Esto tiene dos problemas clarísimos:

- Escalabilidad. Se requeriria muchisimo overhead para tener la info de todo internet!

- Autonomía. Internet es una red de ISPs, donde cada ISP tiene su propia red de routers. Cada ISP va a querer operar cómo le plazca.

Por estos motivos, se organizan los routers en lo que se denomina sistemas autónomos (ASs - autonomous systems). Cada AS consiste de un grupo de routers que están bajo el mismo control administrativo. Los routers que están dentro de un mismo AS, corren el mismo algoritmo de routing y tienen información sobre los demás routers. El algoritmo de routing que se ejecuta dentro de un sistema autónomo se llama **intra-autonomous system routing protocol**.

Open Shortest Path First (OSPF)

Es un protocolo de link-state que utiliza “flooding of link-state information” y el algoritmo de Dijkstra para el camino menos costoso. Cada router construye un grafo del AS entero, luego ejecuta el algoritmo de Dijkstra para determinar el camino más corto a **todas** las subredes. Los routers broadcastean la información a todos los demás routers, no solo a sus vecinos. Algunas de las ventajas/avances que presenta OSPF:

- Seguridad: Los intercambios dentro de OSPF pueden ser autenticados para prevenir intrusos maliciosos.
- Múltiples caminos con el mismo costo: Cuando hay muchos caminos a un mismo destino con el mismo costo, OSPF permite que se puedan usar múltiples caminos.
- Soporte integrado para ruteo unicast y multicast
- Permite jerarquía dentro de un AS

Routing Among the ISPs: BGP

Cuando se routea un paquete dentro de un AS, la ruta la define el protocolo de intra-AS routing. Pero, un paquete pasa por varios ASs, para eso se necesita un **protocolo de inter-AS routing**. En internet, se utiliza el protocolo **BGP** (Border Gateway Protocol). Es el protocolo que hace de pegamento entre las miles de ISPs que tiene internet.

BGP provee a cada router:

- Obtener un prefijo de accesibilidad para la información de ASs vecinos. Una subred “avisa” de su existencia y BGP se asegura de que todos los demás ASs sepan sobre esta subred.
- Determinar la mejor ruta a los prefijos.

Para cada AS, cada router puede ser un **gateway router** o un **internal router**. Un gateway router es el que está en el “borde” de un AS y se comunica con uno o más routers de otras ASs. Un internal router se conecta únicamente con routers y hosts del mismo AS.

En BGP, pares de routers intercambian información sobre conexiones TCP semipermanentes (usando el puerto 179). Cada conexión TCP se conoce como BGP connection. Una conexión BGP que comunica dos ASs se llama external BGP (eBGP), mientras que una conexión BGP entre routers del mismo AS se conoce como internal BGP (iBGP).

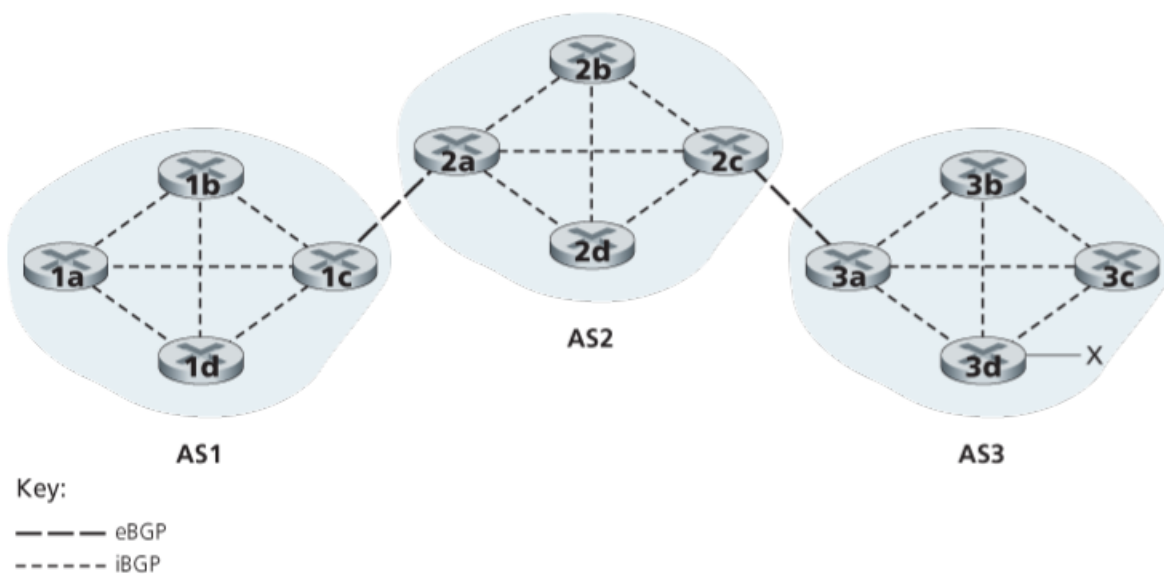


Figure 5.9 eBGP and iBGP connections

In order to propagate the reachability information, both iBGP and eBGP sessions are used. Consider again advertising the reachability information for prefix x to all routers in AS1 and AS2. In this process, gateway router 3a first sends an eBGP message “AS3 x” to gateway router 2c. Gateway router 2c then sends the iBGP message “AS3 x” to all of the other routers in AS2, including to gateway router 2a. Gateway router 2a then sends the eBGP message “AS2 AS3 x” to gateway router 1c. Finally, gateway router

1c uses iBGP to send the message “AS2 AS3 x” to all the routers in AS1. After this process is complete, each router in AS1 and AS2 is aware of the existence of x and is also aware of an AS path that leads to x.

Cuando un router avisa sobre un prefijo a través de la conexión BGP, incluye varios atributos BGP; en la terminología BGP a un prefijo acompañado de sus atributos se lo llama **route**. Los dos atributos más importantes son AS-PATH y NEXT-HOP. AS-PATH contiene la lista de ASs por los que pasó el “aviso”, se usa para detectar y prevenir loops. El NEXT-HOP es la dirección IP de la interfaz del router que comienza el AS-PATH, es básicamente la dirección IP del router más cercano del AS adyacente.

El algoritmo más simple de ruteo es el **hot potato routing**. En este algoritmo, el camino elegido es aquel con el menor costo al router NEXT-HOP.

Example: router 1b will consult its intra-AS routing information to find the least-cost intra-AS path to NEXT-HOP router 2a and the least-cost intra-AS path to NEXT-HOP router 3d, and then select the route with the smallest of these least-cost paths. For example, suppose that cost is defined as the number of links traversed. Then the least cost from router 1b to router 2a is 2, the least cost from router 1b to router 2d is 3, and router 2a would therefore be selected. Router 1b would then consult its forwarding table (configured by its intra-AS algorithm) and find the interface I that is on the least-cost path to router 2a. It then adds (x, I) to its forwarding table

La idea del hot potato es, visto desde cada AS, quitarse de encima lo más rápido posible a los paquetes sin tener que preocuparse por el costo de los paquetes que ya estan fuera del AS. Se dice que es un algoritmo egoísta porque trata de reducir el costo del propio AS sin preocuparse por los costos externos.

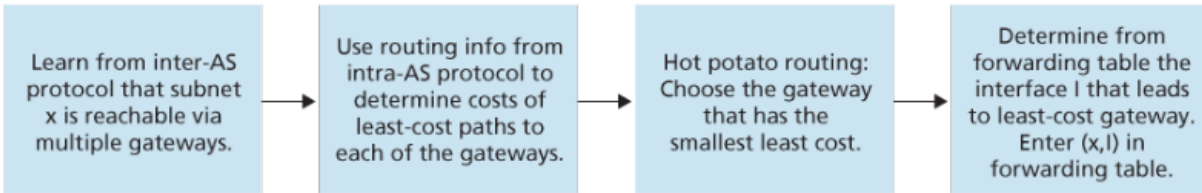


Figure 5.11 Steps in adding outside-AS destination in a router's forwarding table

En la práctica, el algoritmo que se usa es el de **route-selection**. El input de este algoritmo es un set con todas los caminos hacia el prefijo, que ya fueron aprendidos y aceptados por el router. Si hay más de un camino al mismo prefijo, se aplican reglas de eliminación:

- Un camino tiene asignado un atributo llamado 'local preference' tal que se eligen los caminos con el mayor valor.
- De los restantes, se seleccionan los caminos con el AS-PATH más corto.
- Luego se utiliza el algoritmo de hot potato.
- Si siguen quedando más de un camino, se utilizan identificados BGP.

Dato de color: además de el protocolo de ruteo inter-AS, BGP se suele usar para implementar el servicio IP-anycast (comúnmente usado en DNS). Cuando un host envía un datagrama a una dirección anycast, la infraestructura de red buscará el camino más corto hasta uno, y preferiblemente solo uno, de los equipos que aceptan datagramas dirigidos a la dirección anycast utilizada. El algoritmo de BGP provee una manera fácil y natural de hacerlo.

during the IP-anycast configuration stage, the CDN company assigns the same IP address to each of its servers, and uses standard BGP to advertise this IP address from each of the servers. When a BGP router receives multiple route advertisements for this IP address, it treats these advertisements as providing different paths to the same physical location (when, in fact, the advertisements are for different paths to different physical locations). When configuring its routing table, each router will locally

use the BGP route-selection algorithm to pick the “best” (for example, closest, as determined by AS-hop counts) route to that IP address.

Consider what happens when Alice, who knows the IP address of your Web server, sends an IP datagram (e.g., a TCP SYN segment) to that IP address. This datagram will be routed through the Internet, visiting a series of routers in many different ASs, and eventually reach your Web server. When any one of the routers receives the datagram, it is going to look for an entry in its forwarding table to determine on which outgoing port it should forward the datagram. Therefore, each of the routers needs to know about the existence of your company’s /24 prefix (or some aggregate entry). How does a router become aware of your company’s prefix? As we have just seen, it becomes aware of it from BGP! Specifically, when your company contracts with a local ISP and gets assigned a prefix (i.e., an address range), your local ISP will use BGP to advertise your prefix to the ISPs to which it connects. Those ISPs will then, in turn, use BGP to propagate the advertisement. Eventually, all Internet routers will know about your prefix (or about some aggregate that includes your prefix) and thus be able to appropriately forward datagrams destined to your Web and mail servers.

The SDN Control Plane

La arquitectura SDN tiene cuatro características clave:

- **Flow-based forwarding:** El forwarding de paquetes por switches controlados por SDN puede basarse en cualquier número de header field values en la capa de

transporte, de red o de enlace. Es el trabajo de la SDN computar, administrar e instalar las entradas de la tabla de flujo en todos los switches de la red.

- **Separación del plano de datos del plano de control:** El plano de datos consiste en los switches de la red. El plano de control consiste en servidores y software que determinan las tablas de flujo para los switches.
- **Funciones de control de red: externas al plano de datos de los switches.** El SDN se implementa en software, este software se ejecuta en servidores remotos a la red de switches.
- **Red programable:** La red es programable a través de las aplicaciones de control de red que corren en el control-plane. Son el 'cerebro' de la SDN.

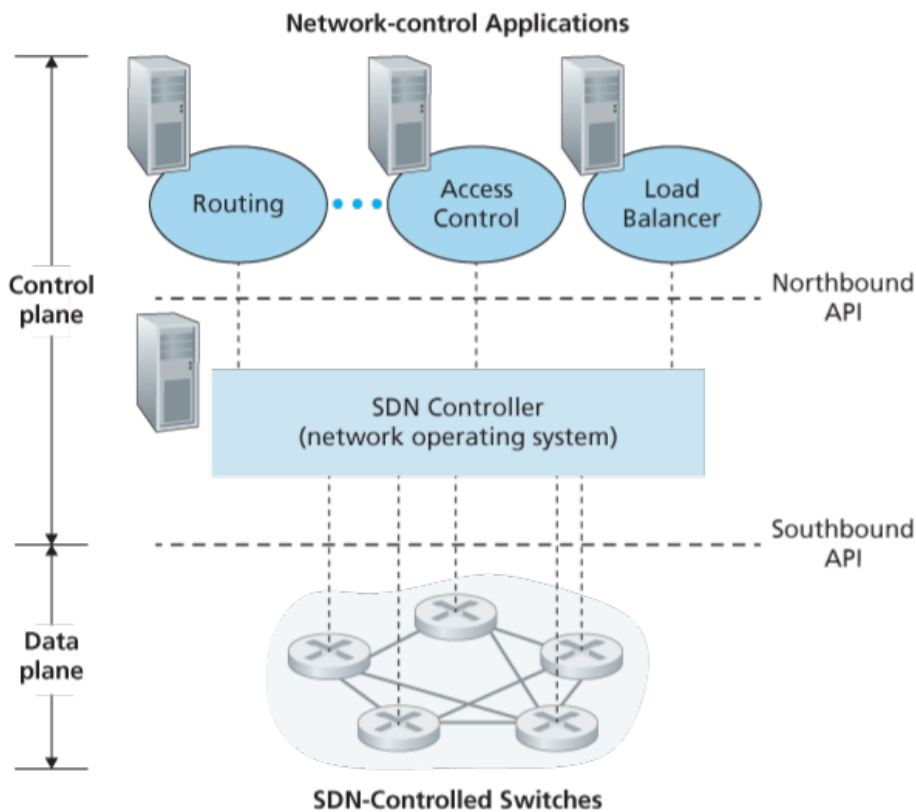


Figure 5.14 Components of the SDN architecture: SDN-controlled switches, the SDN controller, network-control applications

La SDN se divide en dos componentes: el controlador SDN y las aplicaciones de control de red SDN.

La funcionalidad del controlador se puede pensar en tres capas:

- **Capa de comunicación:** Comunicación entre el controlador y los dispositivos de red controlados. Se necesita un protocolo para transferir información del controlador a un dispositivo. Además, un dispositivo debe poder comunicarle eventos observados localmente al controlador. Esta comunicación se la conoce como la interfaz 'southbound'.
- **Capa de gestión de estado en toda la red:** Todas las decisiones que toma la SDN requieren que el controlador tenga toda la información sobre el estado de la red actualizada (información sobre hosts, enlaces, switches y sobre otros dispositivos controlados). Además, debe tener una copia de las tablas de flujo de los switches.
- **Interfaz para la capa de aplicación de control de red:** El controlador interactúa con las aplicaciones de control de red a través de la interfaz 'northbound'. Esta API permite a las aplicaciones de control de red leer/escribir el estado de la red y las tablas de flujo dentro de la capa de gestión de estado.

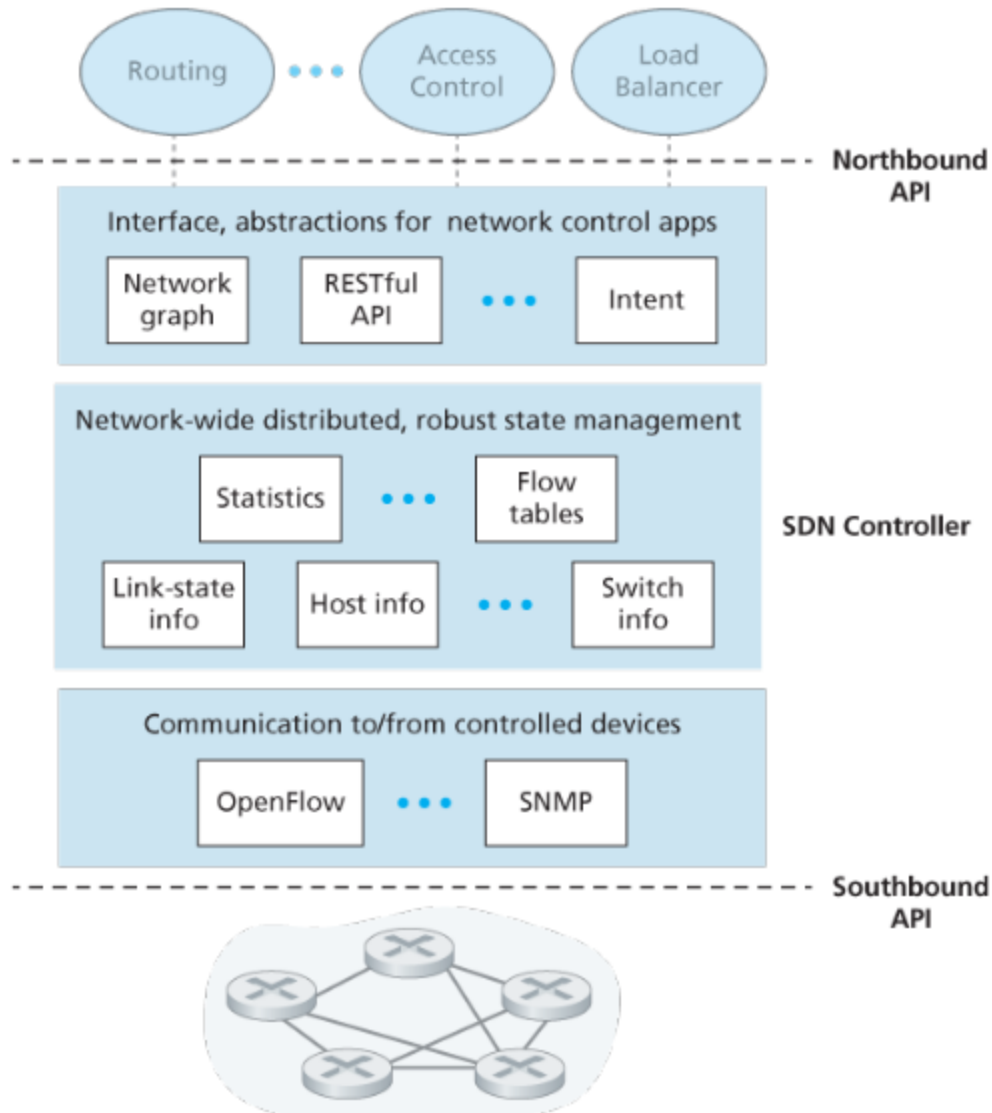


Figure 5.15 Components of an SDN controller

OpenFlow Protocol

Es un protocolo que opera entre un controlador SDN y un switch controlado por SDN.

Entre los mensajes que intercambia estan los siguientes:

- Configuración. Consultar y setear parametros de configuracion de un switch.
- Modify-State. Agregar o eliminar entradas de la tabla de flujo o setear propiedades a los puertos.
- Read-State. Estadísticas y contadores de la tabla de flujo y puertos.

- Send-Packet. Envía un paquete específico a un puerto específico.
- Flow-Removed. Avisa que una entrada de la tabla fue removida.
- Port-Status. Avisa cambios de estado en los puertos.
- Packet-in. Para mandar paquetes al controlador.

ICMP: The Internet Control Message Protocol

El protocolo ICMP lo usan los hosts y routers para comunicarse entre sí información de la capa de red. El caso de uso más típico es el de informe de errores. Los mensajes ICMP viajan dentro de los datagramas IP, como payload de IP, tienen un campo type y otro code y contienen el header y los primeros 8 bytes del datagrama IP que causó que se genere el mensaje. Traceroute está implementado con mensajes ICMP.

Network Management and SNMP

Network management includes the deployment, integration, and coordination of the hardware, software, and human elements to monitor, test, poll, configure, analyze, evaluate, and control the network and element resources to meet the real-time, operational performance, and Quality of Service requirements at a reasonable cost.

Componentes clave:

- El managing server es la aplicación que controla la colección, procesamiento, análisis y/o display de la información.
- El managed device una pieza del equipamiento de red que reside en la managed network. Puede ser un host, router, middlebox, modem, etc.
- Toda la información recolectada se almacena en Management Information Base (MIB). Un objeto MIB puede ser un contador o el número de segmentos UDP recibidos en el host, por ej.
- Un network management agent es un proceso corriendo en el managed device que se comunica con el managing server.

- Un network management protocol corre entre el managing server y los managed devices.

Un ejemplo es SNMP (Simple Network Management Protocol)

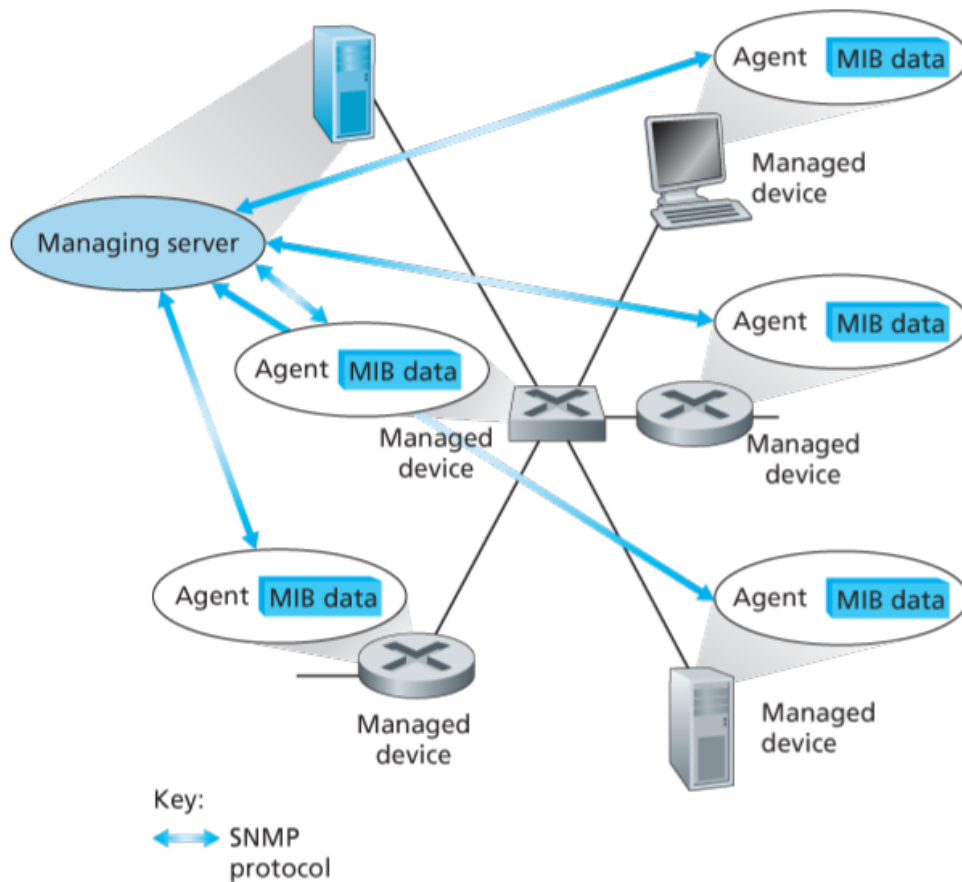


Figure 5.20 Elements of network management: Managing server, managed devices, MIB data, remote agents, SNMP

Capítulo 6: The link layer and LANs

Nos vamos a referir a los dispositivos de la capa de enlace como **nodos**. Los nodos incluyen hosts, routers, switches y puntos de acceso Wi-Fi. A los canales de comunicación que conectan nodos adyacentes los llamamos **links**.

Consider a travel agent who is planning a trip for a tourist traveling from Princeton, New Jersey, to Lausanne, Switzerland. The travel agent decides that it is most convenient for the tourist to take a limousine from Princeton to JFK airport, then a plane from JFK airport to Geneva's airport, and finally a train from Geneva's airport to Lausanne's train station. Once the travel agent makes the three reservations, it is the responsibility of the Princeton limousine company to get the tourist from Princeton to JFK; it is the responsibility of the airline company to get the tourist from JFK to Geneva; and it is the responsibility of the Swiss train service to get the tourist from Geneva to Lausanne. Each of the three segments of the trip is "direct" between two "adjacent" locations. Note that the three transportation segments are managed by different companies and use entirely different transportation modes (limousine, plane, and train). Although the transportation modes are different, they each provide the basic service of moving passengers from one location to an adjacent location. **In this transportation analogy, the tourist is a datagram, each transportation segment is a link, the transportation mode is a link-layer protocol, and the travel agent is a routing protocol.**

Los servicios que puede ofrecer un protocolo de la capa de enlace son:

- **Framing.** Un **frame** consiste de un campo de data donde es insertado el datagrama y de una serie de headers. La estructura del frame la especifica el protocolo.
- **Link access.** El protocolo **medium access control (MAC)** especifica las reglas con las que se transmite un frame a un link. Sirve principalmente cuando muchos nodos comparten un mismo broadcast link.

- **Reliable delivery.** Garantiza que cada datagrama se mueva a través del link sin errores (similar a la idea de TCP). Esto puede lograrse con ACKs y retransmisiones. En general, se usa en enlaces inalámbricos que son los más propensos a errores, no tanto en enlaces cableados.
- **Error detection and correction.** A veces aparecen errores en los bits transmitidos debido a atenuaciones de señales o ruido electromagnético, por ello existen mecanismos para detectar este tipo de errores. En el nodo que transmite se incluyen bits de detección de errores en el frame, luego el nodo receptor tiene que hacer el chequeo de errores (análogo al checksum de la capa de transporte).

Mayormente, la capa de enlace se implementa en un **adaptador de red**, a veces conocido como network interface card (NIC). En el “corazón” del adaptador de red se encuentra el controlador de capa de enlace; generalmente es un único chip que implementa todos los servicios.

Visto del lado de quien envía, el controlador toma un datagrama, lo encapsula en un frame y luego lo transmite al link, siguiendo el protocolo de acceso del link. Visto del lado de quien recibe, el controlador recibe el frame y extrae de allí el datagrama. Si la capa de enlace realiza una detección de errores, entonces es el sending controller el que setea los bits de detección en el header del frame y es el receiver controller el que realiza la detección de error.

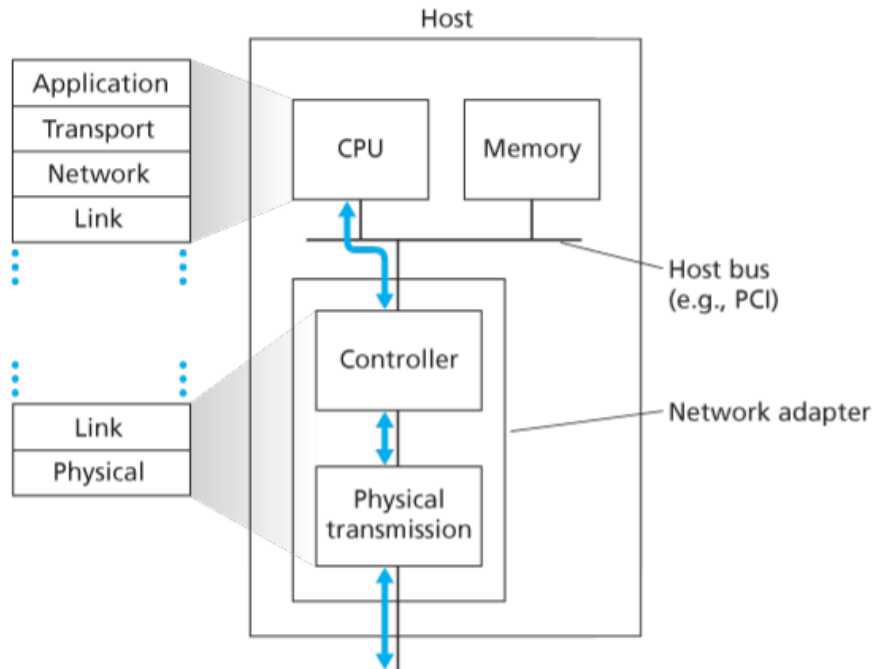


Figure 6.2 **Network adapter:** Its relationship to other host components and to protocol stack functionality

Error-Detection and -Correction Techniques

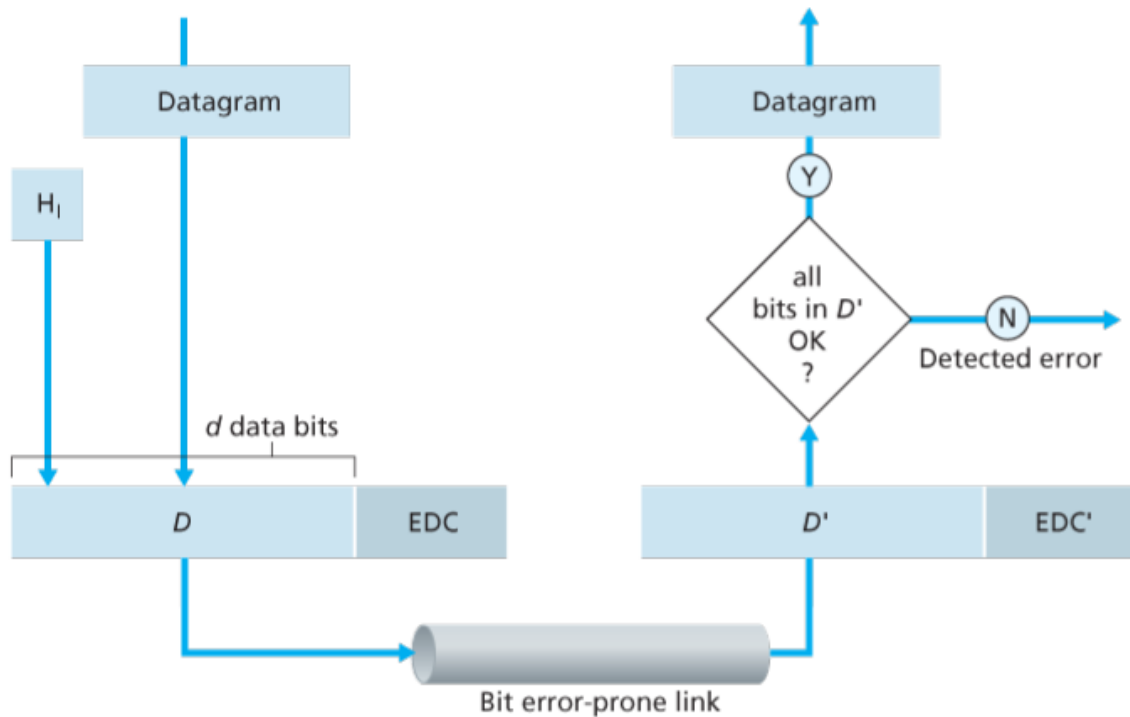


Figure 6.3 Error-detection and -correction scenario

El desafío del receptor es determinar si D es igual a D' , dado que recibió D' y EDC' . Aunque se utilicen técnicas de detección de errores puede que queden errores sin detectar, sucede que en general las técnicas más sofisticadas introducen mucho overhead.

Tres técnicas para detectar errores:

1. Parity checks

Es la forma más simple. Suponiendo que la data que se va a enviar, D , tiene d bits. En un esquema par, el enviador incluye un bit adicional donde elige el valor tal que el número total de 1s en los $d+1$ bits sea par. En un esquema impar, se elegirá tal que $d+1$ sea impar.

El receptor tiene que contar la cantidad de 1s recibidos en los $d+1$ bits, si encuentra un número impar (estando en el esquema par), entonces va a saber que hay un número impar de bits erróneos. Qué pasa si encuentra un número par? no es garantía de nada, podría pasar que sea un caso donde haya errores no detectados y en esta técnica la probabilidad de ocurran errores es muy alta, así que no es la mejor.

Suponiendo que hay un error en los d bits originales, siguiendo esta técnica, el receptor puede identificar cuál es el bit que está mal y así corregirlo. Esta habilidad del receptor de poder detectar y corregir un error se llama **forward error correction** (FEC). Esta técnica se utiliza mucho en dispositivos de audio.

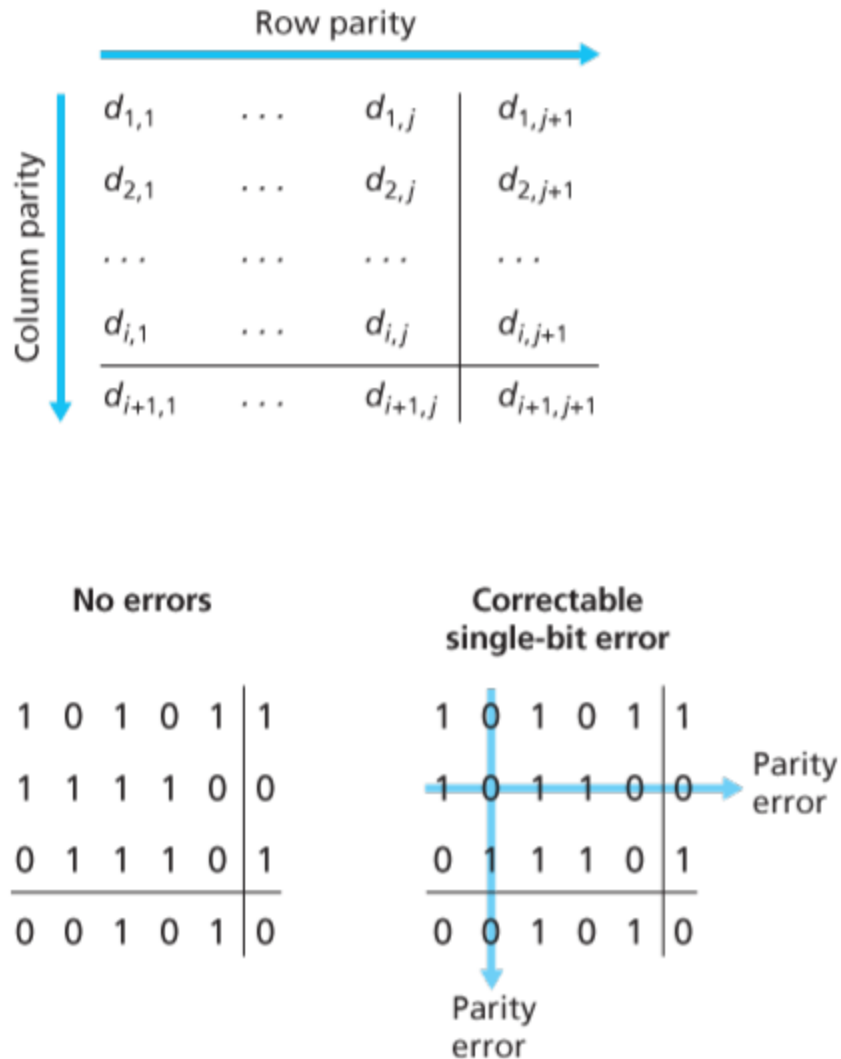


Figure 6.5 Two-dimensional even parity

2. Checksumming methods

Los d bits de datos, se toman como una secuencia de k -bits enteros. Un método simple de checksumming es el de sumar esos k -bits y usar el resultado como el bit de detección de error. El receptor verifica ese checksum calculando el

complemento a 1 de la suma de lo que recibió (incluyendo el checksum) y viendo si el resultado son todos bits de 1. Si hay algún 0, es porque hay un error. Por qué se utilizan checksums tanto en la capa de transporte como en la de enlace? porque los checksums de la capa de transporte están implementados en software, en cambio en la capa de enlace se implementan en hardware que rápidamente pueden resolver operaciones más complejas como CRC (cyclic redundancy checks).

3. Cyclic redundancy checks (CRC)

También conocidos como **polynomial codes** porque se puede pensar en el string de bits como un polinomio con coeficientes 0 y 1. La idea es la siguiente: el emisor y el receptor tienen que ponerse de acuerdo en un patrón de $r+1$ bits, G , llamado **generator**. El bit más significativo de G tiene que ser 1. Para una data D , el emisor va a elegir r bits adicionales que va a concatenar a D tal que el patrón resultante $d+r$ sea exactamente divisible por G usando aritmética de módulo 2.

El receptor va a dividir los $d+r$ bits recibidos por G . Si el resto no es cero, entonces el receptor va a saber que hay un error, sino considerará que la información es correcta.

(info sobre el cálculo que se hace: pág. 505) Multiple Access Links and Protocols



Figure 6.7 A sample CRC calculation

Multiple Access Links and Protocols

Existen dos tipos de enlaces de red: point-to-point y broadcast.

Un enlace **point-to-point** consiste en un único sender en un extremo del enlace y un único receiver en el otro extremo del enlace.

Un enlace **broadcast** puede tener multiples senders y receivers conectados al mismo canal compartido. La idea de broadcast es que cuando un nodo transmite un frame, el canal difunde el frame y cada uno de los otros nodos recibe una copia. Ejemplos son Ethernet y LANs inalámbricas.

Un tema importante es cómo coordinar el acceso de esos multiples nodos al canal compartido, eso se conoce como el **mutiple access problem**. El problema es el de determinar quién “habla” y cuándo (análogo a una clase). Para regular las transmisiones existen protocolos.

Como todos los nodos son capaces de transmitir frames, dos o más nodos pueden transmitir frames al mismo tiempo. Cuando pasa eso, todos los nodos reciben multiples frames al mismo tiempo, o sea, los frames transmitidos *colisionan* en todos los

receivers → se hace un lío, todos los frames implicados en la colisión se pierden → el canal broadcast es desperdiciado. Por eso es necesario coordinar las transmisiones de los nodos activos.

Idealmente un protocolo de multiples accesos de R bits por segundo tiene que tener las siguientes características:

- Cuando un solo nodo tiene data para enviar, ese nodo tiene un throughput de R bps.
- Cuando M nodos tienen data para enviar, cada uno va a tener un throughput de R/M bps.
- El protocolo es descentralizado (no hay un nodo que represente un único punto de falla).
- El protocolo es simple (fácil de implementar).

Los protocolos de multiples accesos se pueden clasificar en tres categorías:

1. **Channel partitioning protocols**

Time-division multiplexing (TDM) y frequency-division multiplexing (FDM) son dos técnicas que pueden usarse para particionar el bandwidth de un canal broadcast entre todos los nodos.

TDM divide el tiempo en time frames y despues divide cada frame en N time slots. Cada time slot es asignado a uno de los N nodos, cuando un nodo tiene un frame para enviar (no confundir este frame con el del time frame), transmite los bits durante su time slot asignado.

Con esta técnica, cada nodo obtiene una transmisión exclusiva de R/N bps en cada frame time. Pero tiene dos desventajas: un nodo es limitado a un rate promedio de R/N aunque sea el único nodo con data y un nodo siempre tiene que esperar por su turno en la secuencia aunque sea el único nodo con data.

FDM es parecido. Divide los R bps en diferentes frecuencias (cada una con un bandwidth de R/N) y asigna cada frecuencia a uno de los nodos. Después crea N canales más chicos de R/N bps.

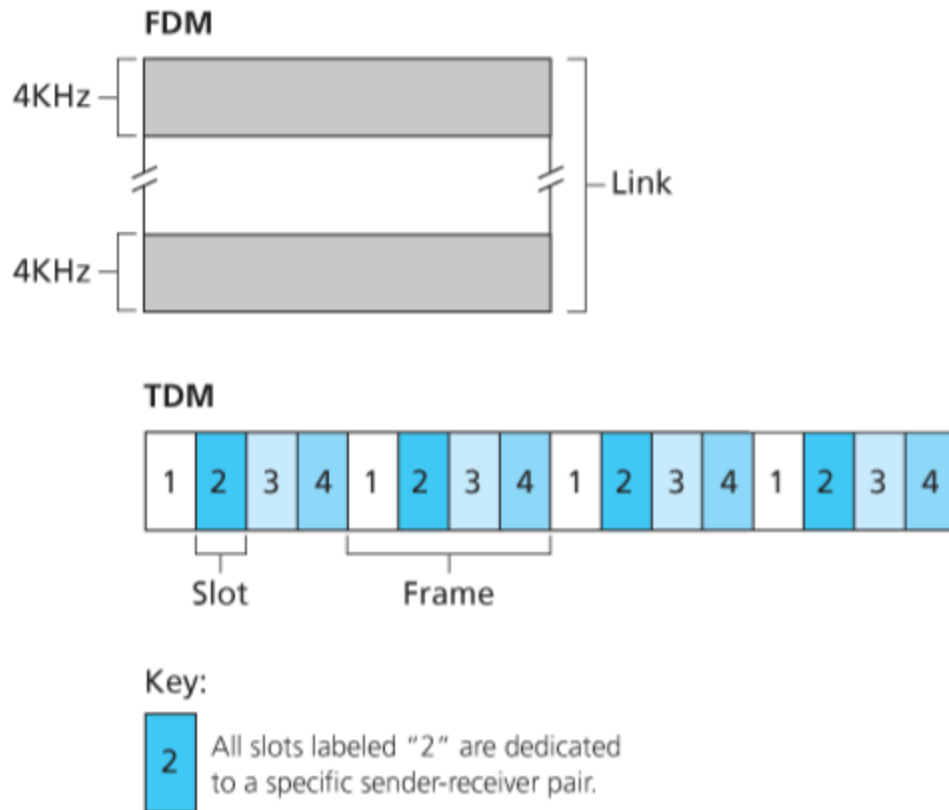


Figure 6.9 A four-node TDM and FDM example

2. Random access protocols

En estos protocolos, el nodo transmite al rate máximo de R bps. Cuando hay una colisión, cada nodo implicado en esa colisión retransmite su frame hasta que ese frame pase sin colisionar. Pero no es que retransmite inmediatamente después de la colisión, sino que espera un tiempo random antes de hacerlo.

Slotted aloha protocol

Se divide el tiempo en slots de L/R segundos, donde L es la cantidad de bits por frame (todos los frames tienen la misma cantidad de bits). Cuando un nodo tiene un frame nuevo para mandar, espera el comienzo del siguiente slot y ahí lo envía. Si hay una colisión, se detecta antes de que termine el slot. El nodo retransmite el frame en cada slot subsecuente con probabilidad p de que se transmita sin colisión; con probabilidad p quiere decir que decide con cierta probabilidad si retransmitir o esperar al slot siguiente. Es un protocolo descentralizado, simple y que permite que

un nodo transmita al máximo rate. El problema que tiene es que no es muy eficiente cuando hay muchos nodos con muchos frames por transmitir.

Aloha

En este protocolo se requiere que todos los nodos transmitan al comienzo de un slot. Apenas llega un frame, se transmite al canal de broadcast. Si hay colisión, se va a retransmitir inmediatamente con probabilidad p .

Carrier sense multiple access (CSMA)

En los anteriores, la decisión de transmitir o no era independiente de los demás nodos. Aquí aparecen dos reglas:

- Listen before speaking: carrier sensing. Un nodo escucha el canal antes de transmitir. Si tiene que transmitir algo, espera hasta detectar que no haya otras transmisiones.
- If someone else begins talking at the same time, stop talking: collision detection. Un nodo que está transmitiendo, escucha el canal mientras transmite.

La performance queda atada al delay de propagación del canal.

3. Taking-turns protocols

Lo que le falta a los protocolos de ALOHA y CSMA es la propiedad de que cuando hay M nodos activos, todos tengan un throughput de R/M bps.

Polling protocol

Requiere que uno de los nodos sea designado como el nodo “maestro”. El nodo maestro va eligiendo cada uno de los nodos al estilo round-robin. Tiene dos desventajas: el delay que introduce esto de estar eligiendo nodos y que si el nodo maestro falla, todo el canal se vuelve inoperativo.

Token-passing protocol

El token es un pequeño frame que se intercambia entre los nodos en un determinado orden. Cuando un nodo recibe el token, se lo queda sólo si tiene datos para transmitir, sino se lo pasa al siguiente nodo. Si el nodo sí tiene datos para transmitir, envía hasta un número máximo de frames y después le pasa el token al nodo siguiente. Esta técnica es descentralizada y eficiente, pero tiene un problema:

la falla de un nodo puede afectar a todo el canal, o si algun nodo se apodera del token, tiene que existir algun mecanismo de recuperacion.

Switched Local Area Networks

MAC Addresses

Además de la/s dirección/es de red, los hosts y routers (mejor dicho, sus adaptadores) tienen multiples direcciones de enlace asociadas. Vale aclarar que los switches no tienen dirección de enlace asociada, solo los hosts y routers. Una dirección de enlace puede llamarse dirección LAN, dirección física o dirección MAC; de ahora en más nos vamos a referir a ella como MAC address.

Una propiedad interesante es que no puede haber dos adaptadores con la misma dirección. La IEEE gestiona el espacio de direcciones MAC. A diferencia de la dirección IP, la dirección MAC no cambia, sin importar el lugar físico en el que se encuentre el adaptador.

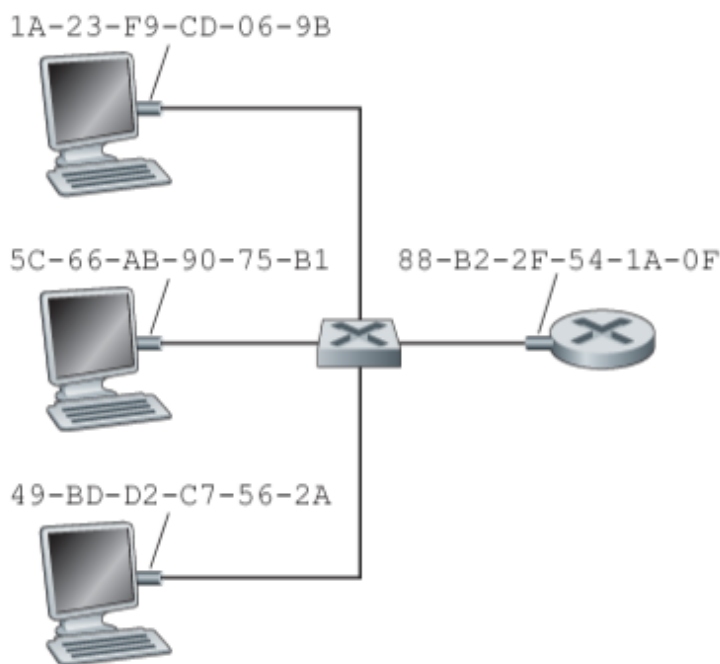


Figure 6.16 Each interface connected to a LAN has a unique MAC address

Cuando un adaptador quiere enviar un frame, inserta la dirección MAC del adaptador destino en el frame y lo manda a la LAN. Entonces cuando un adaptador recibe un

frame, verifica que la dirección destino que tiene coincida con la propia. Si no coincide, lo descarta.

Existe una dirección MAC especial de broadcast para cuando se quiere que el frame lo reciban todos los adaptadores. En LANs con direcciones de 6 bytes es FF-FF-FF-FF-FF-FF.

Address Resolution Protocol (ARP)

Se encarga de la traducción entre direcciones IP y direcciones MAC. ARP recibe como input una dirección IP **que esté en la misma LAN** y devuelve la correspondiente dirección MAC. Esto se necesita para enviar datagramas dentro de una misma subnet porque quien envía solo tiene la dirección IP destino.

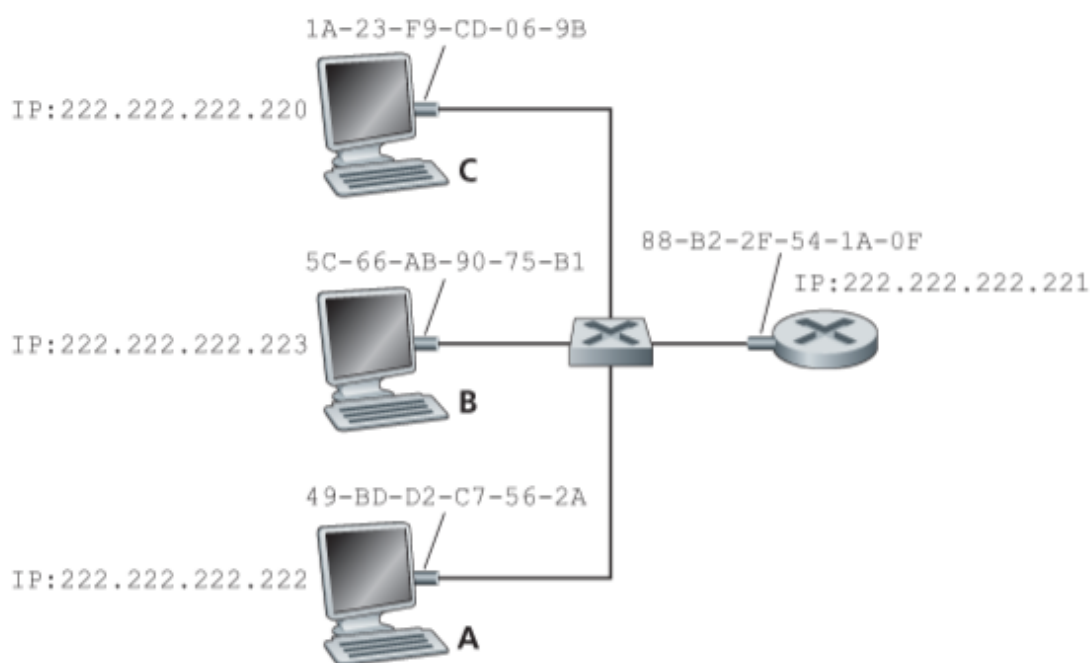


Figure 6.17 Each interface on a LAN has an IP address and a MAC address

IP Address	MAC Address	TTL
222.222.222.221	88-B2-2F-54-1A-0F	13:45:00
222.222.222.223	5C-66-AB-90-75-B1	13:52:00

Figure 6.18 A possible ARP table in 222.222.222.220

Cada host y cada router tienen una tabla ARP en su memoria, conteniendo los mapeos de IP ↔ MAC y un TTL que determina después de cuanto tiempo se tiene que borrar el mapeo de la tabla. Si un host quisiera enviar un datagrama a otro teniendo la IP destino, necesita la dirección MAC entonces primero se fija en la tabla ARP si tiene ese dato, si no lo tiene va a usar el protocolo ARP para calcularlo. Esto último consiste en que quien envía tiene que construir un paquete ARP, un paquete ARP tiene varios campos, entre ellos las direcciones IP y MAC destino y origen. La idea es que pedirle a todos los demás hosts y routers de la subred que chequeen si su dirección IP coincide con la dirección IP destino del paquete ARP, la que coincida va a enviar un paquete ARP de respuesta indicándole el mapeo.

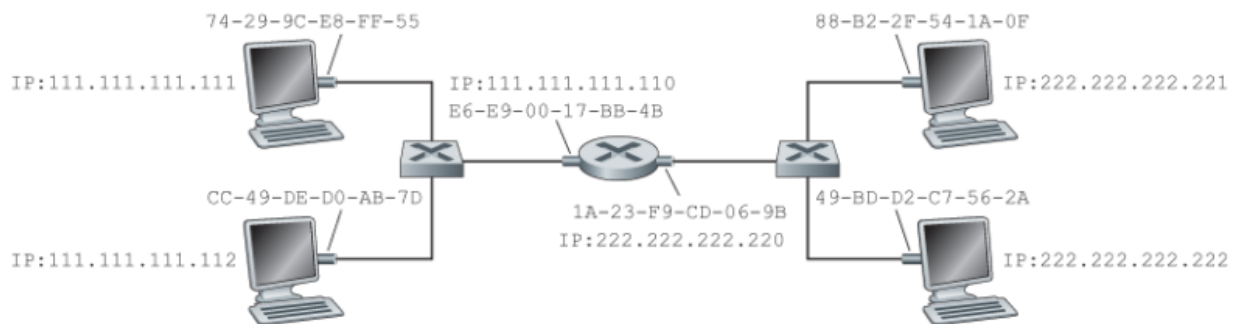


Figure 6.19 Two subnets interconnected by a router

Notar que un router tiene dos adaptadores, dos direcciones IP, dos módulos ARP y cada adaptador tiene su dirección MAC.

Si se quisiera enviar un paquete desde la subnet de la izquierda a la de la derecha, primero se tendría que enviar el datagrama a la interfaz del router 111.111.111.110, entonces al frame se le indicaría la dirección MAC E6-E9-00-17-BB-4B. Para mover el datagrama del router a la interfaz que corresponde, el router chequea en su tabla de forwarding: 222.222.222.220. Esta interfaz le pasa el datagrama a su adaptador que lo va a encapsular en un frame para enviarlo a la subnet2. La dirección MAC destino la obtiene el router de su tabla ARP.

Ethernet

Es la tecnología de LAN cableado más prevalente hoy en día.

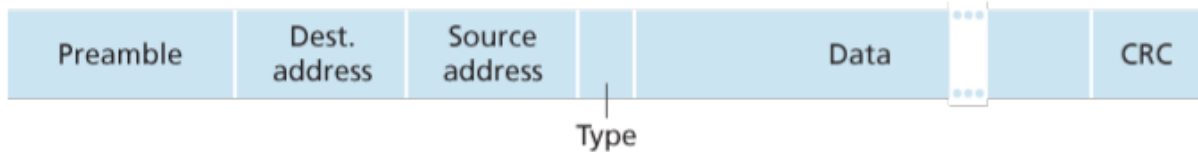


Figure 6.20 Ethernet frame structure

Campos de un frame Ethernet:

- **Data field** (46 a 1500 bytes): contiene el datagrama IP. Si el datagrama excede los 1500 bytes, se va a tener que fragmentar. Si el datagrama contiene menos de 46 bytes, se tiene que rellenar.
- **Destination address** (6 bytes): Contiene la dirección MAC del adaptador destino. Cuando el adaptador destino recibe el frame, le pasa los contenidos del campo de data a la capa de red.
- **Source address** (6 bytes): Contiene la dirección MAC del adaptador que transmite el frame a la LAN.
- **Type field** (2 bytes): Permite multiplexar protocolos de la capa de red (tener en cuenta que se podría usar un protocolo que no sea IP)
- **Cyclic redundancy check (CRC)** (4 bytes): Para detectar errores de bits en el frame.
- **Preamble** (8 bytes): Los primeros 7 bytes sirven para “despertar” a los adaptadores que reciben y sincronizar sus relojes con el de quien envía.

Todas las tecnologías Ethernet proveen un servicio connectionless a la capa de red, o sea, no se necesita de un handshaking para enviar algo de un adaptador a otro. Además, el servicio es unreliable, si un frame falla el CRC el adaptador directamente lo descarta, sin avisar si lo recibió bien o no.

Link-layer switches

El switch es transparente a los hosts y routers de una subnet, es decir que un host/router envía un frame sin saber que en el medio va a pasar por un switch.

Los switches tienen una función de **filtrado** donde se determina si un frame debe ser reenviado a alguna interfaz o descartado. También tienen una función de **reenvío** donde se determinan las interfaces a las que será enviado un frame.

Estas funciones se hacen en la **switch table**, esta tabla contiene entradas para algunos de los hosts y routers de la LAN. Estas entradas son: una dirección MAC, la interfaz de switch que lleva a la dirección MAC y el tiempo en que se creó la entrada en la tabla.

Address	Interface	Time
62-FE-F7-11-89-A3	1	9:32
7C-BA-B2-B4-91-10	3	9:36
****	****	****

Figure 6.22 Portion of a switch table for the uppermost switch in [Figure 6.15](#)

Cada vez que llega un frame al switch hay tres posibilidades:

1. No existe la entrada para su dirección MAC. En tal caso el switch hace broadcaste del frame.
2. Existe la entrada, pero tiene asociada la misma interfaz en la que está, entonces como no hay que reenviarlo a otra interfaz, se descarta (filtrado).
3. Existe la entrada y tiene asociada una interfaz distinta a la que ya está, el switch hace el reenvío mandando el frame por el buffer de salida que precede a la interfaz destino.

Una propiedad del switch es que se construye automática y dinámicamente, y de forma autónoma → **self-learning**. Inicialmente la tabla está vacía, a medida que se reciben frames, se carga en la tabla la información correspondiente. El switch elimina de la tabla direcciones de las que no recibió ningún frame luego de un x período de tiempo.

Ventajas de los switches:

- **Elimination of collisions.** Los switches tienen un buffer donde van guardando los frames, nunca transmiten más de un frame por segmento.
- **Heterogeneous links.** Como el switch aísla a los enlaces entre sí, diferentes enlaces pueden operar a distintas velocidades y sobre distintos medios.
- **Gestión.** Si detecta que un adaptador está funcionando mal, puede desconectarlo automáticamente. También recolectan estadísticas del uso de bandwidth,

colisiones, tráficos, etc.

Switches vs routers

Aunque son fundamentalmente diferentes, los administradores de red suelen tener que decidir entre uno o el otro.

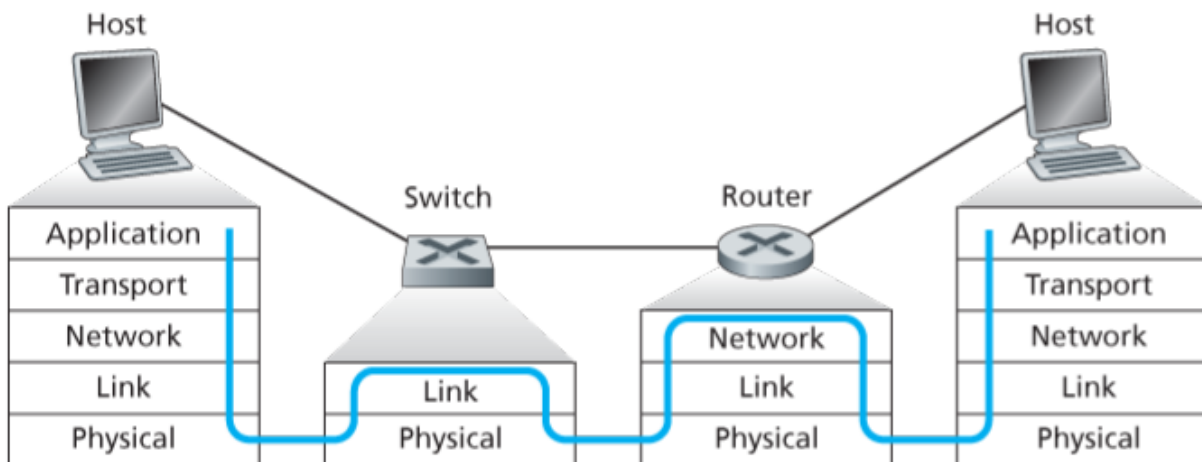


Figure 6.24 Packet processing in switches, routers, and hosts

Table 6.1 Comparison of the typical features of popular interconnection devices

	Hubs	Routers	Switches
Traffic isolation	No	Yes	Yes
Plug and play	Yes	No	Yes
Optimal routing	No	Yes	No

En general, a redes chicas les alcanza con usar switches para localizar el trafico y ahorrarse la configuración de IPs, pero redes grandes con miles de hosts tienen también routers. Los routers proveen un aislamiento más robusto del tráfico, control de “broadcast storms” y un uso más inteligente de las rutas.

Virtual Local Area Networks (VLANs)

Generalmente las LANs institucionales están configuradas jerárquicamente:

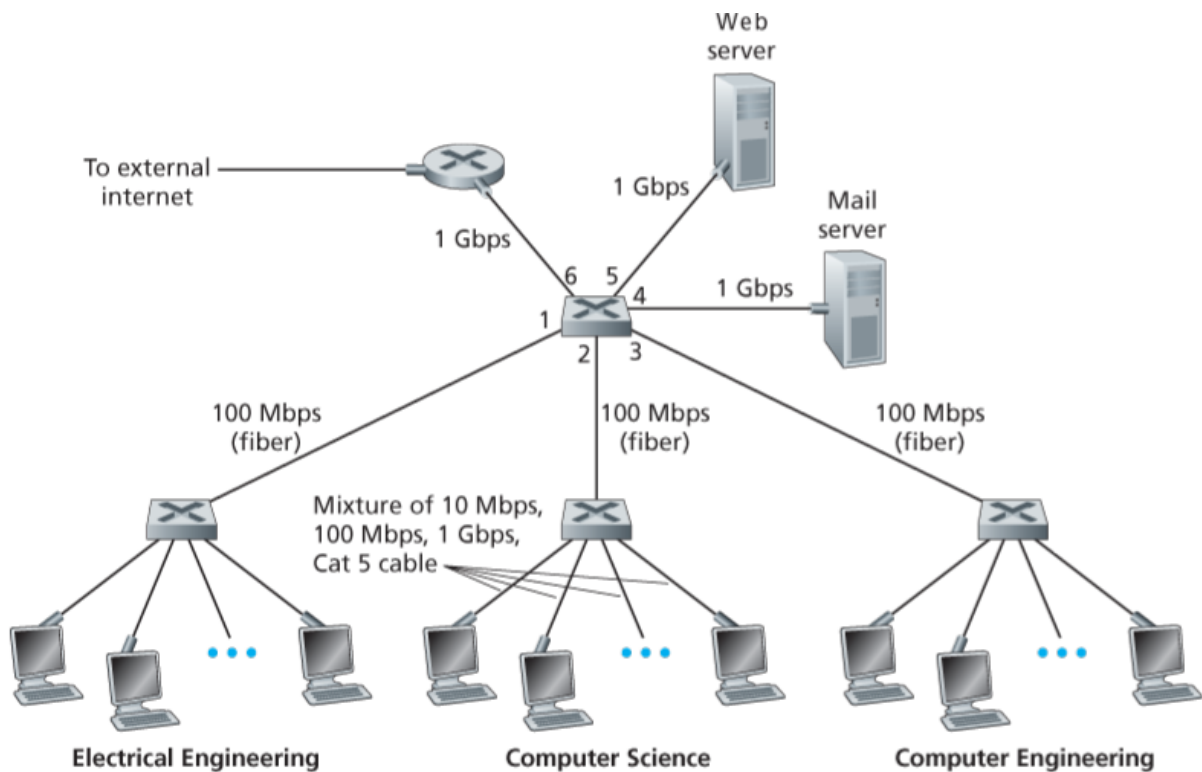


Figure 6.15 An institutional network connected together by four switches

Esto presenta tres desventajas:

- Lack of traffic isolation. El tráfico de broadcast tiene que atravesar toda la red. Esto se podría solucionar reemplazando el switch del medio por un router.
- Inefficient use of switches
- Managing users. Si un empleado se mueve entre distintos grupos, el cableado físico tiene que cambiarse para conectar al empleado a un switch diferente.

Estas dificultades se pueden resolver utilizando un switch que soporte VLANs. Esto es un switch que permite multiples redes locales virtuales definidas en un unica infraestructura de red local física.

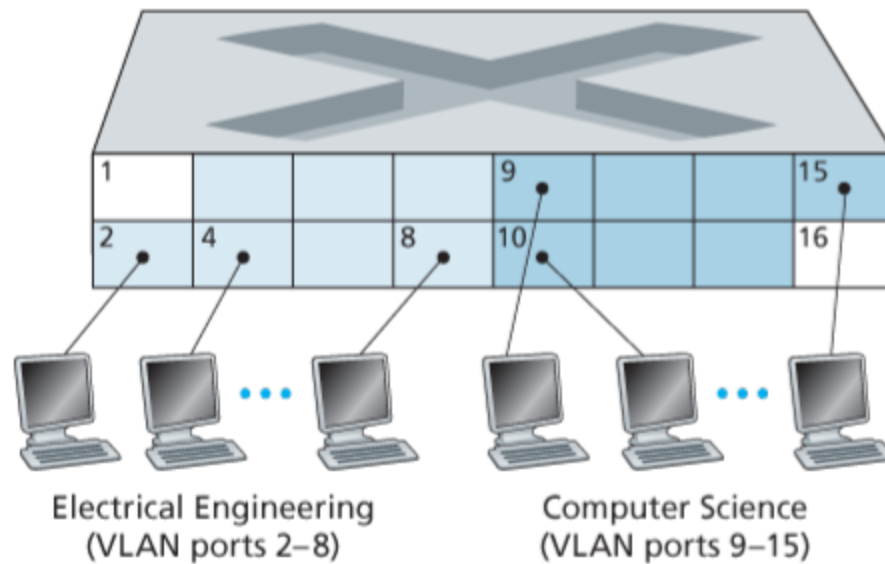


Figure 6.25 A single switch with two configured VLANs

Link Virtualization: A Network as a Link Layer

Multiprotocol Label Switching (MPLS)

La idea de MPLS es etiquetar datagramas permitiendo a los routers reenviar datagramas en base a etiquetas de un largo fijo en vez de direcciones IP.

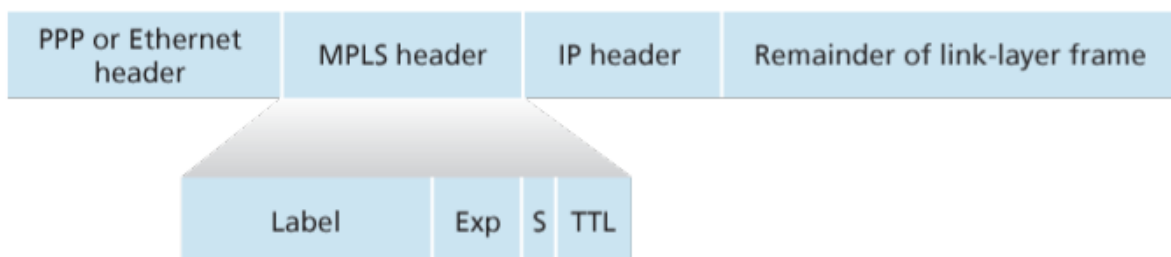


Figure 6.28 MPLS header: Located between link- and network-layer headers

Un frame MPLS solo puede ser transmitido entre routers que lo soporten.

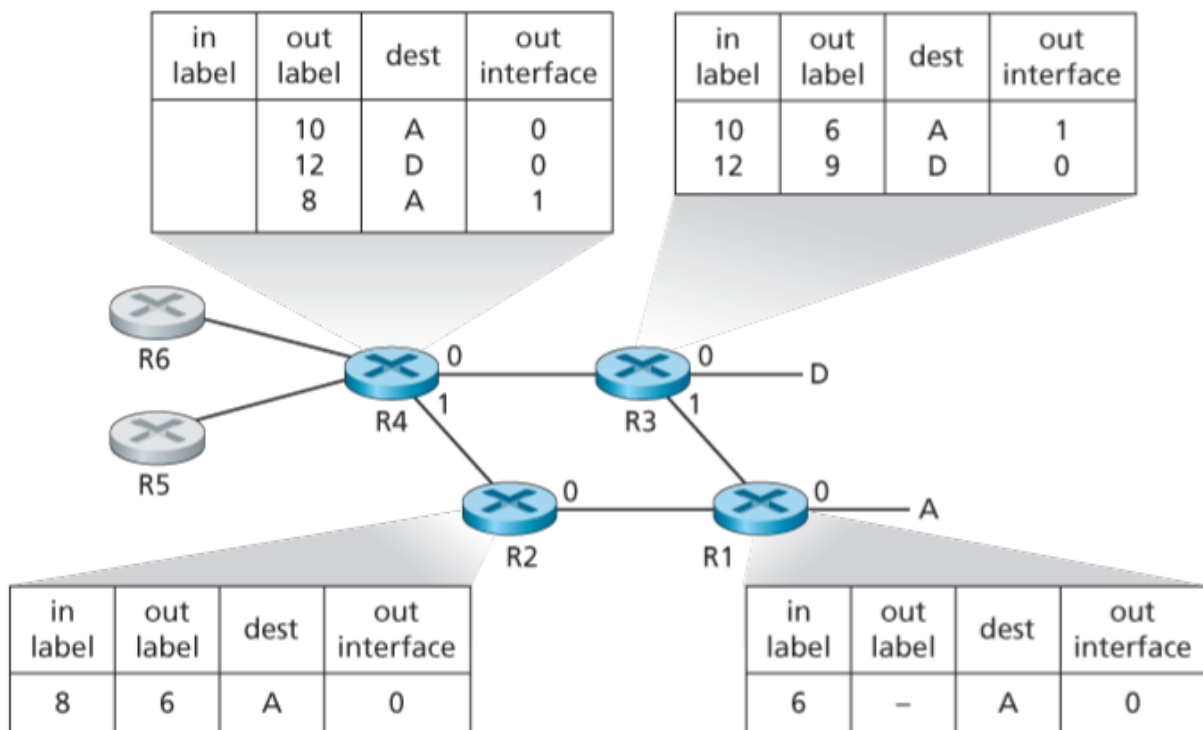


Figure 6.29 MPLS-enhanced forwarding

La ventaja que trae es que provee la habilidad de reenviar paquetes por rutas que no serían posibles siguiendo un protocolo IP estandar, esto es una forma de “ingeniería de trafico” donde se puede sobrescribir el ruteo IP y forzar el tráfico por otros caminos. También se usa MPLS para implementar VPNs.

Data Center Networking

Cada data center tiene su propia red que interconecta los hosts entre sí e interconecta al data center con Internet. Esta red soporta dos tipos de tráfico: entre clientes externos y hosts internos, y entre hosts internos. Para manejar los flujos con clientes externos existen los border routers.

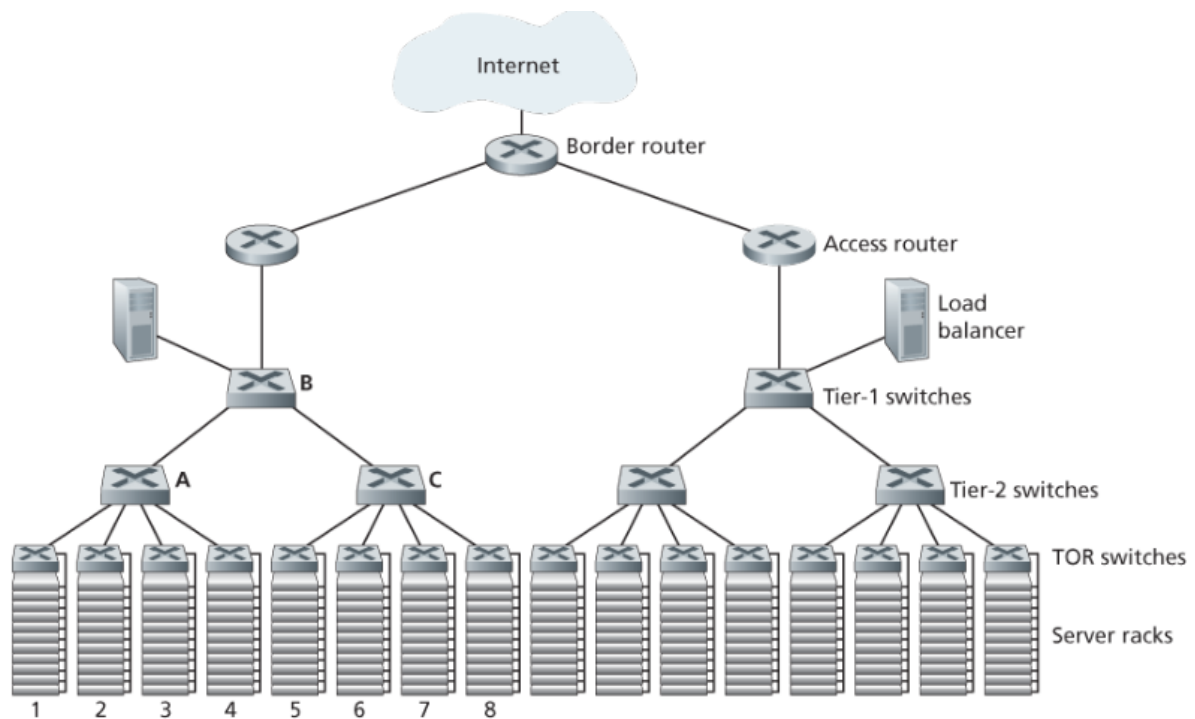


Figure 6.30 A data center network with a hierarchical topology

Tres propiedades:

- **Load balancing.** Todas las solicitudes externas pasan primero por un balanceador de carga cuyo trabajo es distribuir las solicitudes de los hosts, balanceando la carga entre los hosts en función de su carga actual. Además, provee una función tipo NAT traduciendo la IP pública a la IP interna, así se previene a los clientes de contactar directamente al host.
- **Hierarchical architecture.** En general tienen una jerarquía de routers y switches como se ve en la figura 6.30.

Retrospective: A Day in the Life of a Web Page Request

Case: a student, Bob, connects a laptop to his school's Ethernet switch and downloads a Web page.

Let's suppose that Bob boots up his laptop and then connects it to an Ethernet cable connected to the school's Ethernet switch, which in turn is connected to the school's router, as shown in Figure 6.32.

The school's router is connected to an ISP, in this example, comcast.net. In this example, comcast.net is providing the DNS service for the school; thus, the DNS server resides in the Comcast network rather than the school network. We'll assume that the DHCP server is running within the router, as is often the case.

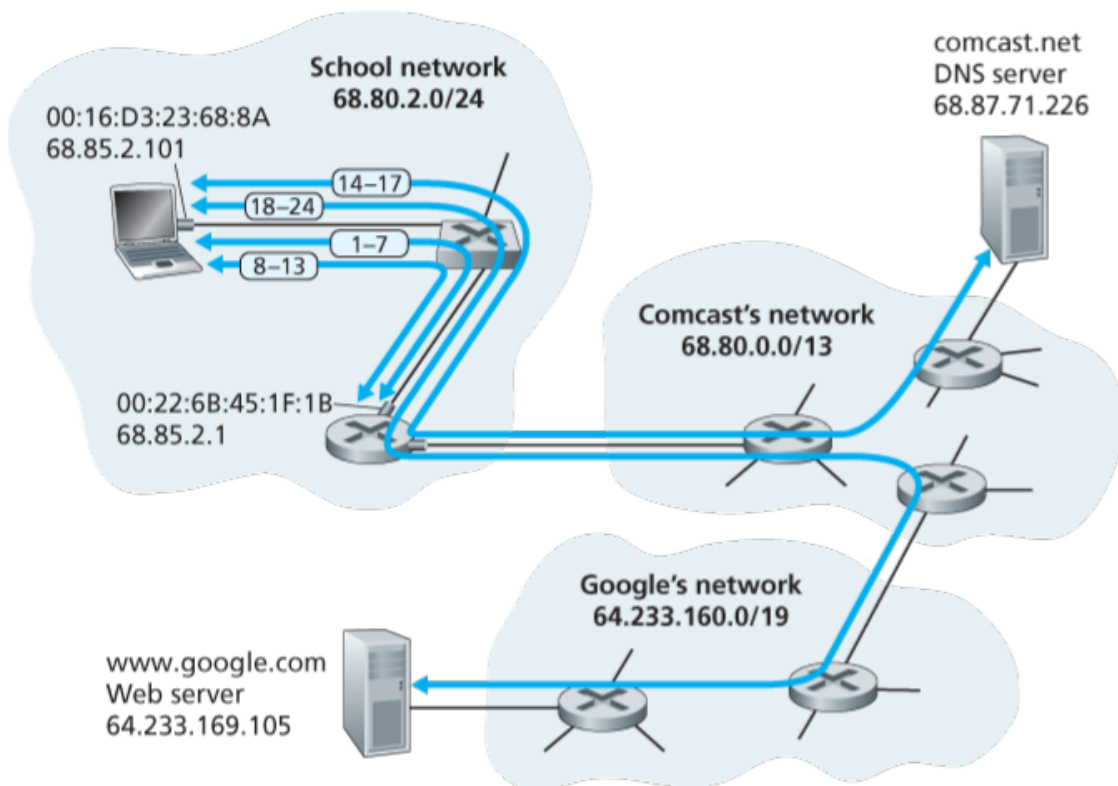


Figure 6.32 A day in the life of a Web page request: Network setting and actions

When Bob first connects his laptop to the network, he can't do anything (e.g., download a Web page) without an IP address. Thus, the first network-related action taken by Bob's laptop is to run the DHCP protocol to obtain an IP address, as well as other information, from the local DHCP server:

1. The operating system on Bob's laptop creates a DHCP request

message and puts this message within a UDP segment with destination port 67 (DHCP server) and source port 68 (DHCP client). The UDP segment is then placed within an IP datagram with a broadcast IP destination address (255.255.255.255) and a source IP address of 0.0.0.0, since Bob's laptop doesn't yet have an IP address.

2. The IP datagram containing the DHCP request message is then placed within an Ethernet frame (Section 6.4.2). The Ethernet frame has a destination MAC addresses of FF:FF:FF:FF:FF:FF so that the frame will be broadcast to all devices connected to the switch (hopefully including a DHCP server); the frame's source MAC address is that of Bob's laptop, 00:16:D3:23:68:8A.

3. The broadcast Ethernet frame containing the DHCP request is the first frame sent by Bob's laptop to the Ethernet switch. The switch broadcasts the incoming frame on all outgoing ports, including the port connected to the router.

4. The router receives the broadcast Ethernet frame containing the DHCP request on its interface with MAC address 00:22:6B:45:1F:1B and the IP datagram is extracted from the Ethernet frame. The datagram's broadcast IP destination address indicates that this IP datagram should be processed by upper layer

protocols at this node, so the datagram's payload (a UDP segment) is thus demultiplexed up to UDP, and the DHCP request message is extracted from the UDP segment. The DHCP server now has the DHCP request message.

5. Let's suppose that the DHCP server running within the router can allocate IP addresses in the CIDR block 68.85.2.0/24. In this example, all IP addresses used within the school are thus within Comcast's address block. Let's suppose the DHCP server allocates address 68.85.2.101 to Bob's laptop. The DHCP server creates a DHCP ACK message containing this IP address, as well as the IP address of the DNS server (68.87.71.226), the IP address for the default gateway router (68.85.2.1), and the subnet block (68.85.2.0/24) (equivalently, the "network mask"). The DHCP message is put inside a UDP segment, which is put inside an IP datagram, which is put inside an Ethernet frame. The Ethernet frame has a source MAC address of the router's interface to the home network (00:22:6B:45:1F:1B) and a destination MAC address of Bob's laptop (00:16:D3:23:68:8A).

6. The Ethernet frame containing the DHCP ACK is sent (unicast) by the router to the switch.

Because the switch is self-learning and previously received an

Ethernet frame

(containing the DHCP request) from Bob's laptop, the switch knows to forward a frame

addressed to 00:16:D3:23:68:8A only to the output port leading to Bob's laptop.

7. Bob's laptop receives the Ethernet frame containing the DHCP ACK, extracts the IP datagram from the Ethernet frame, extracts the UDP segment from the IP datagram, and extracts the DHCP ACK message from the UDP segment. Bob's DHCP client then records its IP address and the IP address of its DNS server. It also installs the address of the default gateway into its IP forwarding table. Bob's laptop will send all datagrams with destination address outside of its subnet 68.85.2.0/24 to the default gateway. At this point, Bob's laptop has initialized its networking components and is ready to begin processing the Web page fetch.

When Bob types the URL for www.google.com into his Web browser, he begins the long chain of events that will eventually result in Google's home page being displayed by his Web browser. Bob's Web browser begins the process by creating a TCP socket that will be used to send the HTTP request to www.google.com. In order to create the socket, Bob's laptop will need to know the IP address of www.google.com.

8. The operating system on Bob's laptop thus creates a DNS query message, putting the string "www.google.com" in the question section of the DNS message. This DNS

message is then placed within a UDP segment with a destination port of 53 (DNS server). The UDP segment is then placed within an IP datagram with an IP destination address of 68.87.71.226 (the address of the DNS server returned in the DHCP ACK in step 5) and a source IP address of 68.85.2.101.

9. Bob's laptop then places the datagram containing the DNS query message in an Ethernet frame. This frame will be sent (addressed, at the link layer) to the gateway router in Bob's school's network. However, even though Bob's laptop knows the IP address of the school's gateway router (68.85.2.1) via the DHCP ACK message in step 5 above, it doesn't know the gateway router's MAC address. In order to obtain the MAC address of the gateway router, Bob's laptop will need to use the ARP protocol.

10. Bob's laptop creates an ARP query message with a target IP address of 68.85.2.1 (the default gateway), places the ARP message within an Ethernet frame with a broadcast destination address (FF:FF:FF:FF:FF:FF) and sends the Ethernet frame to the switch, which delivers the frame to all connected devices, including the gateway router.

11. The gateway router receives the frame containing the ARP query message on the interface to the school network, and finds that the target IP address of 68.85.2.1 in the ARP message matches the IP address of its interface. The gateway router thus prepares an ARP reply,

indicating that its MAC address of 00:22:6B:45:1F:1B corresponds to IP address 68.85.2.1. It

places the ARP reply message in an Ethernet frame, with a destination address of 00:16:D3:23:68:8A (Bob's laptop) and sends the frame to the switch, which delivers the frame to Bob's laptop.

12. Bob's laptop receives the frame containing the ARP reply message and extracts the MAC address of the gateway router (00:22:6B:45:1F:1B) from the ARP reply message.

13. Bob's laptop can now (finally!) address the Ethernet frame containing the DNS query to the gateway router's MAC address. Note that the IP datagram in this frame has an IP destination address of 68.87.71.226 (the DNS server), while the frame has a destination address of 00:22:6B:45:1F:1B (the gateway router). Bob's laptop sends this frame to the switch, which delivers the frame to the gateway router.

14. The gateway router receives the frame and extracts the IP datagram containing the DNS query. The router looks up the destination address of this datagram (68.87.71.226) and determines from its forwarding table that the datagram should be sent to the leftmost router in the Comcast network in Figure 6.32. The IP datagram is placed inside a link-layer frame appropriate for

the link connecting the school's router to the leftmost Comcast router and the frame is sent over this link.

15. The leftmost router in the Comcast network receives the frame, extracts the IP datagram, examines the datagram's destination address (68.87.71.226) and determines the outgoing interface on which to forward the datagram toward the DNS server from its forwarding table, which has been filled in by Comcast's intra-domain protocol (such as RIP, OSPF or IS-IS) as well as the Internet's inter-domain protocol, BGP.

16. Eventually the IP datagram containing the DNS query arrives at the DNS server. The DNS server extracts the DNS query message, looks up the name www.google.com in its DNS database, and finds the DNS resource record that contains the IP address (64.233.169.105) for www.google.com. (assuming that it is currently cached in the DNS server). Recall that this cached data originated in the authoritative DNS server for googlecom. The DNS server forms a DNS reply message containing this hostname-to-IP-address mapping, and places the DNS reply message in a UDP segment, and the segment within an IP datagram addressed to Bob's laptop (68.85.2.101).

This datagram will be forwarded back through the Comcast network to the school's router and from there, via the Ethernet switch to Bob's laptop.

17. Bob's laptop extracts the IP address of the server www.google.com from the DNS message. Finally, after a lot of work, Bob's laptop is now ready to contact the www.google.com server!

18. Now that Bob's laptop has the IP address of www.google.com, it can create the TCP socket that will be used to send the HTTP GET message to www.google.com. When Bob creates the TCP socket, the TCP in Bob's laptop must first perform a three-way handshake with the TCP in www.google.com. Bob's laptop thus first creates a TCP SYN segment with destination port 80 (for HTTP), places the TCP segment inside an IP datagram with a destination IP address of 64.233.169.105 (www.google.com), places the datagram inside a frame with a destination MAC address of 00:22:6B:45:1F:1B (the gateway router) and sends the frame to the switch.

19. The routers in the school network, Comcast's network, and Google's network forward the datagram containing the TCP SYN toward www.google.com, using the forwarding table in each router, as in steps 14–16 above. Recall that the router forwarding table entries governing forwarding of packets over the inter-domain link between the Comcast and Google networks are determined by the BGP

protocol.

20. Eventually, the datagram containing the TCP SYN arrives at www.google.com. The TCP SYN message is extracted from the datagram and demultiplexed to the welcome socket associated with port 80. A connection socket is created for the TCP connection between the Google HTTP server and Bob's laptop. A TCP SYNACK segment is generated, placed inside a datagram addressed to Bob's laptop, and finally placed inside a link-layer frame appropriate for the link connecting www.google.com to its first-hop router.

21. The datagram containing the TCP SYNACK segment is forwarded through the Google, Comcast, and school networks, eventually arriving at the Ethernet card in Bob's laptop. The datagram is demultiplexed within the operating system to the TCP socket created in step 18, which enters the connected state.

22. With the socket on Bob's laptop now (finally!) ready to send bytes to www.google.com, Bob's browser creates the HTTP GET message containing the URL to be fetched. The HTTP GET message is then written into the socket, with the GET message becoming the payload of a TCP segment. The TCP segment is placed in a datagram and sent and delivered to www.google.com as in steps

18–20 above.

23. The HTTP server at www.google.com reads the HTTP GET message from the TCP socket, creates an HTTP response message, places the requested Web page content in the body of the HTTP response message, and sends the message into the TCP socket.

24. The datagram containing the HTTP reply message is forwarded through the Google, Comcast, and school networks, and arrives at Bob's laptop. Bob's Web browser program reads the HTTP response from the socket, extracts the html for the Web page from the body of the HTTP response, and finally (finally!) displays the Web page!

Capítulo 7: Wireless and Mobile networks

En una red inalámbrica podemos identificar los siguientes elementos:

- **Hosts inalámbricos:** son los que corren las aplicaciones. Puede ser una laptop, pc de escritorio, smartphone, etc. Pueden o no ser móviles.
- **Enlaces inalámbricos:** Cada host se conecta a una *base station* o a otro host a través de un enlace de comunicación inalámbrico.
- **Estación base:** Es una parte clave. Es responsable de enviar y recibir datos desde y hacia un host inalámbrico asociado a la estación. Muchas veces es responsable de coordinar la transmisión de múltiples hosts. Que un host esté asociado a una estación base significa que 1) el host está dentro de la distancia de comunicación inalámbrica de la base y 2) el host usa esa base para transmitir datos entre él y la red más grande. A los hosts asociados a una base se los refiere como que están operando en **modo infraestructura**. Cuando un host móvil se mueve fuera del

range de su estación, va a cambiar la base a la que está asociado, este proceso se llama **handoff**.

- **Infraestructura de red:** Es la red más grande con la que un host inalámbrico va a querer comunicarse.

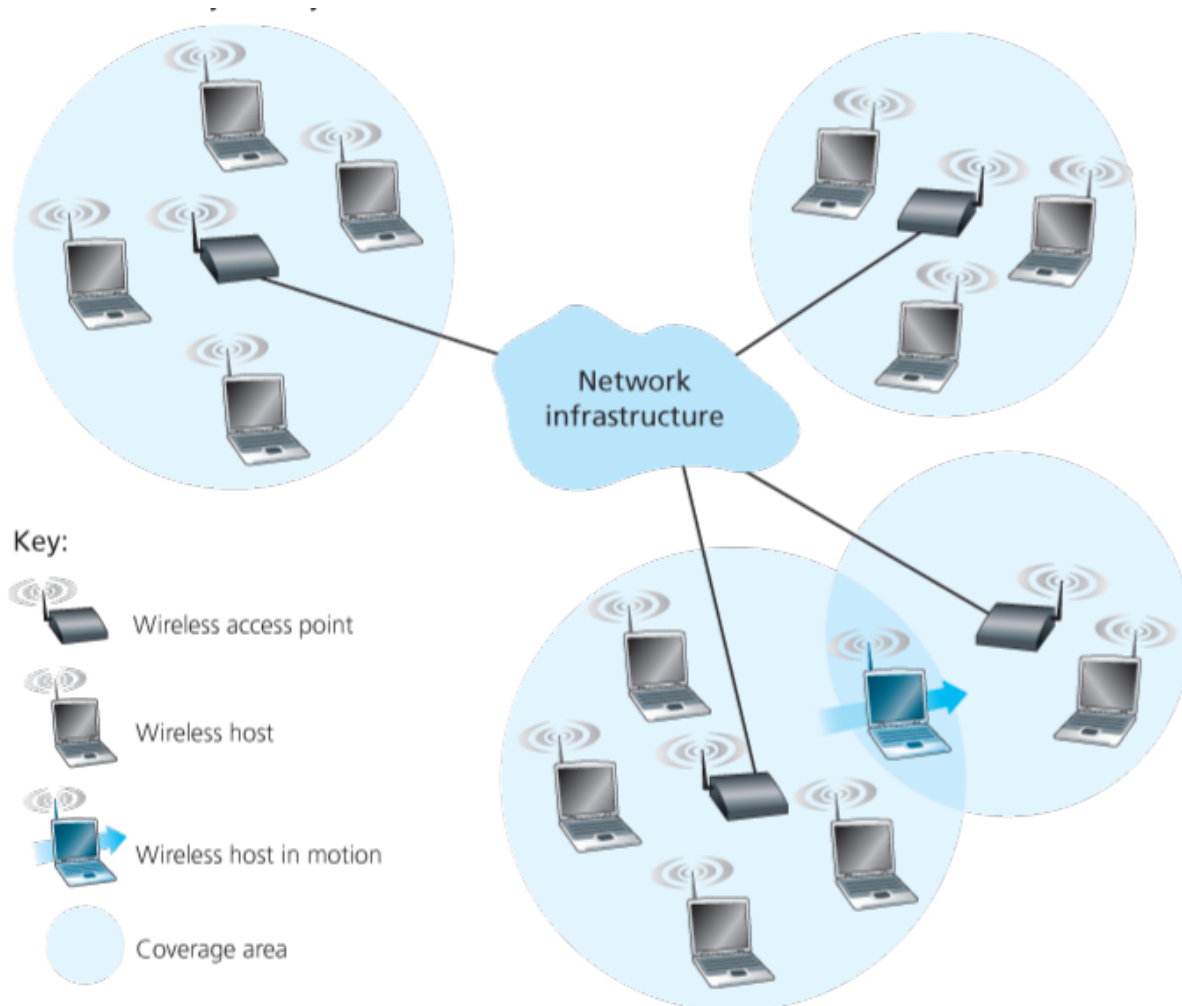


Figure 7.1 Elements of a wireless network

En el nivel más alto, se puede clasificar a las redes inalámbricas en base a dos criterios: 1) si un paquete cruza exactamente un hop inalámbrico o varios, y 2) si hay una infraestructura tal como una estación base en la red.

Single-hop, infrastructure-based: Estas redes tienen una base conectada a una red más grande cableada. Todas las comunicaciones son entre la base y un host inalámbrico sobre un único hop inalámbrico. Ej: 4G LTE.

Single-hop, infrastructure-less: Estas redes no tienen una base conectada a una red inalámbrica. Sin embargo, uno de sus nodos puede coordinar transmisiones de los otros nodos. Ej: bluetooth que conecta dispositivos.

Multi-hop, infrastructure-based: Tienen una red base conectada (wired) a una red más grande. Ej: sensores inalámbricos.

Multi-hop, infrastructure-less: No tienen estación base, los nodos pueden tener que retransmitir sus mensajes entre varios nodos para que lleguen a destino. Los nodos pueden ser móviles.

Wireless Links and Network Characteristics

Al momento de comparar las redes cableadas con las inalámbricas, hay que poner el foco en la capa de enlace. Algunas diferencias entre ambas redes son:

- **Disminución de la intensidad de la señal.** La radiación electromagnética se atenúa a medida que viaja por la materia; a medida que aumenta la distancia entre sender y receiver, la señal se va dispersando.
- **Interferencia de otras fuentes.** Las fuentes de radio que transmiten en la misma banda de frecuencia van a interferir entre sí, además del ruido electromagnético.
- **Propagación multipath.** Ocurre cuando porciones de la onda electromagnética se reflejan sobre objetos y sobre el suelo tomando caminos de distintas longitudes entre el sender y el receiver; esto hace que se “difumine” (blurring) la señal recibida en el receiver.

El host que recibe la señal inalámbrica, recibe una señal electromagnética que es una combinación de la señal original degradada y ruido que se introdujo del ambiente. La **SNR** (signal-to-noise ratio) es una medida relativa de la intensidad de la señal recibida, en general se mide en dB. A mayor SNR, más fácil será quitarle el ruido a la señal transmitida.

Algunas características importantes de la capa física:

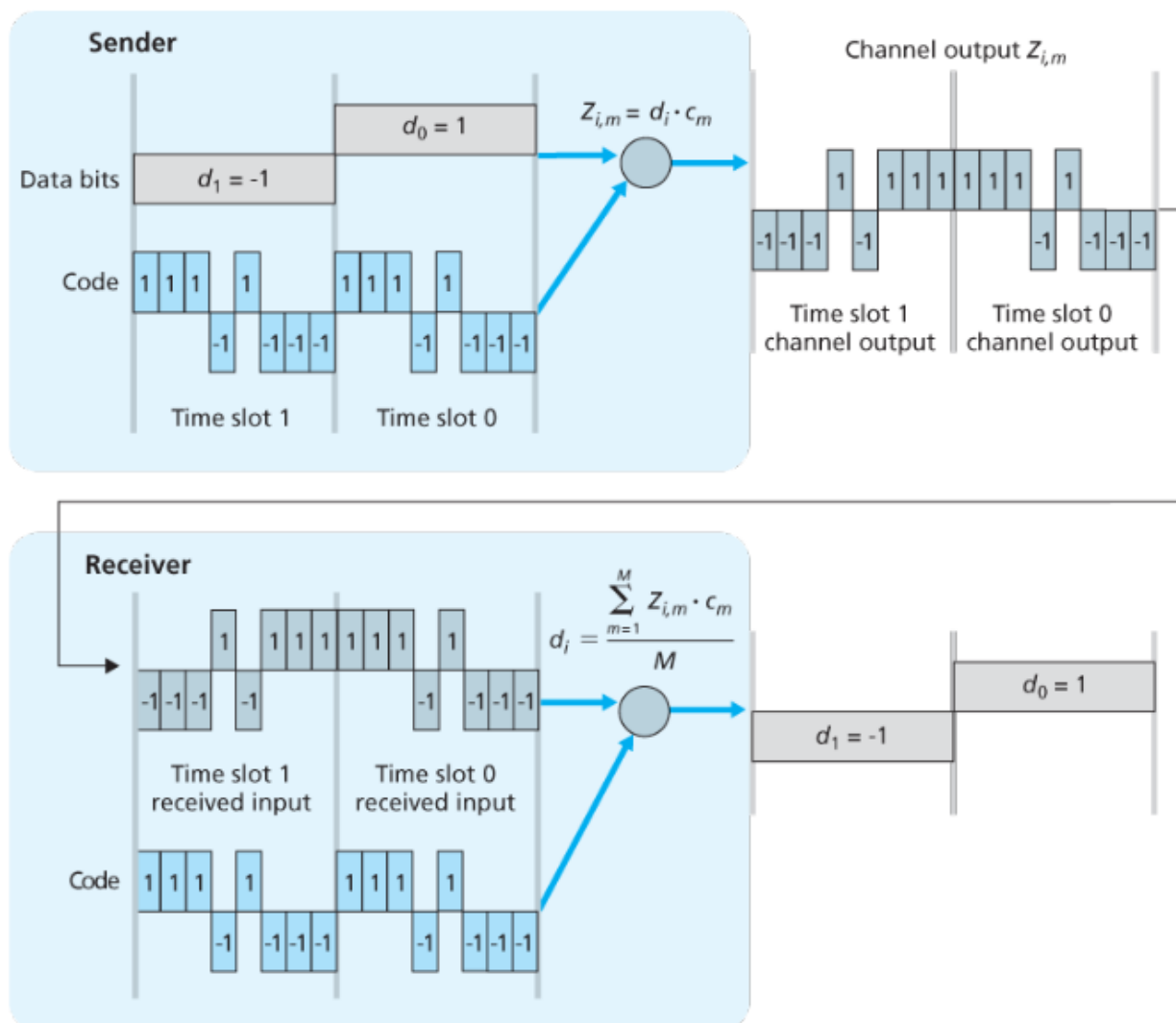
- Para un esquema de modulación dado, a mayor SNR, menor BER (bit error rate). Esto es incrementando la potencia de transmisión.
- Para un SNR dado, una técnica de modulación con un rate de transmisión de bits mayor va a tener mayor BER.

- Se puede usar una selección dinámica de la técnica de modulación para adaptarla a distintas condiciones. El SNR puede cambiar, producto de la movilidad o de cambios en el ambiente.

Protocolo CDMA

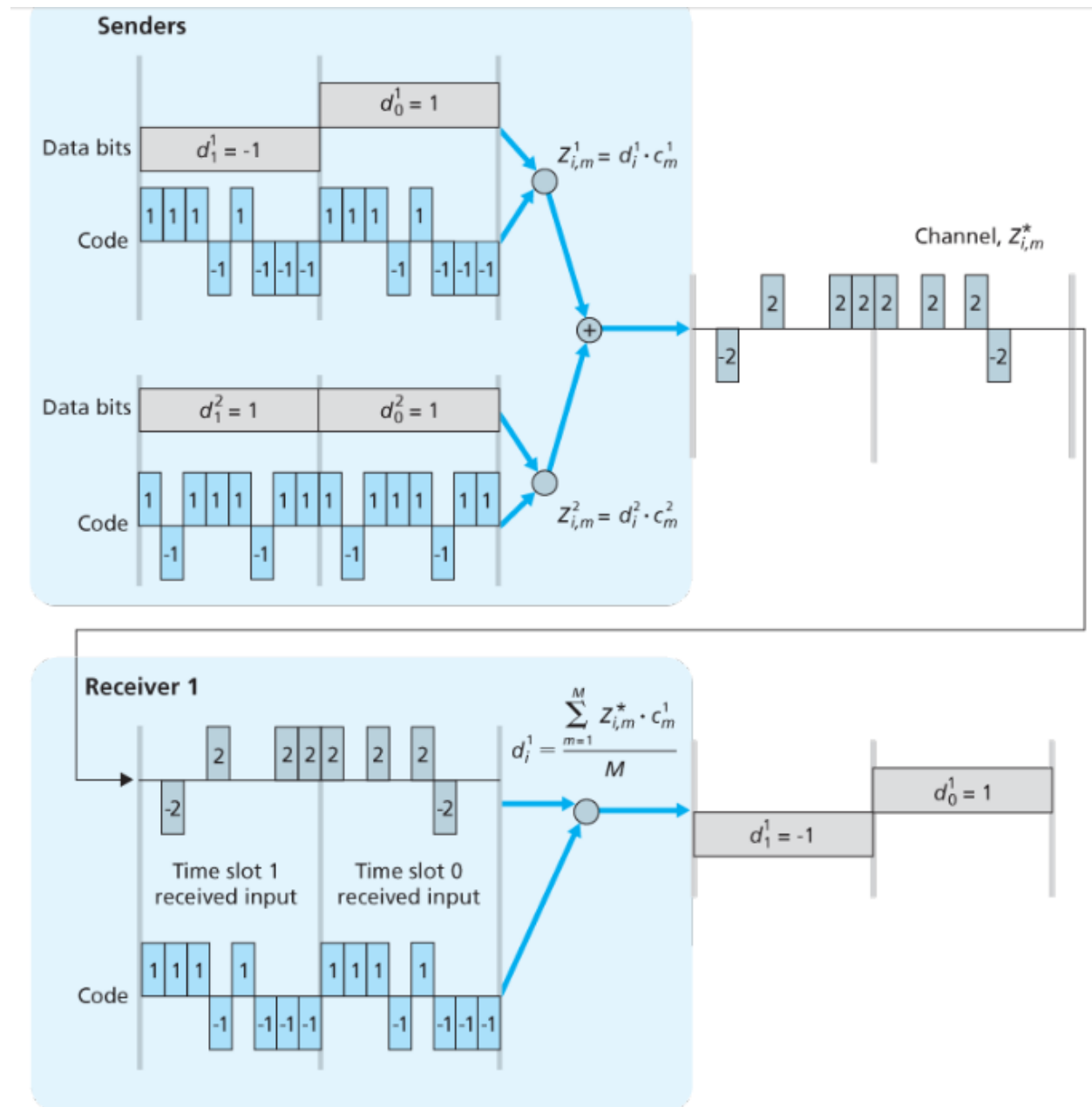
Cada bit enviado se codifica mutiplicandolo por una señal que cambia a un rate mucho más rápido que la secuencia original de bits.

Suponiendo que cada bit requiere un slot de tiempo, d_i es el valor del bit en el slot i ésimo, y se representa el bit con valor 0 con un -1. Cada slot se divide en M (en este caso 8) mini-slots. El código CDMA consiste en una secuencia de M valores que pueden ser 1 o -1.



Para resolver el caso en el que interfieren señales de varios senders, CDMA trabaja bajo la suposición de que los bits transmitidos en interferencia son aditivos. Entonces si se reciben tres señales, se va a tomar la suma de sus valores.

CDMA es un protocolo de particionamiento, particiona el espacio de código y le asigna a cada nodo una parte.



WiFi: 802.11 Wireless LANs

Existen varios standards 802.11 de LANs inalámbricas, pero todas comparten algunas características: usan el mismo protocolo de acceso al medio (CSMA/CA), usan la misma estructura de frames para la capa de enlace, tienen la habilidad de reducir el rate de transmisión para alcanzar mayores distancias y tienen compatibilidad hacia atrás. Las diferencias importantes las tienen en la capa física, difieren en el rango de frecuencia y el rate de datos.

Table 7.1 Summary of IEEE 802.11 standards

Standard	Frequency Range	Data Rate
802.11b	2.4 GHz	up to 11 Mbps
802.11a	5 GHz	up to 54 Mbps
802.11g	2.4 GHz	up to 54 Mbps

802.11n	2.5 GHz and 5 GHz	up to 450 Mbps
802.11ac	5 GHz	up to 1300 Mbps

The 802.11 Architecture

El bloque fundamental es el **basic service set (BSS)**. Un BSS contiene una o más estaciones inalámbricas y una estación base central que se conoce como **access point (AP)**. En una red doméstica típica, hay un AP y un router que conecta el BSS a Internet. En dispositivos Ethernet, cada estación inalámbrica 802.11 tiene una dirección MAC de 6 bytes guardada en el firmware del adaptador de la estación. Además, cada AP tiene una dirección MAC para su interfaz inalámbrica. Estas direcciones MAC las administra la IEEE y son (teóricamente) únicas en el mundo.

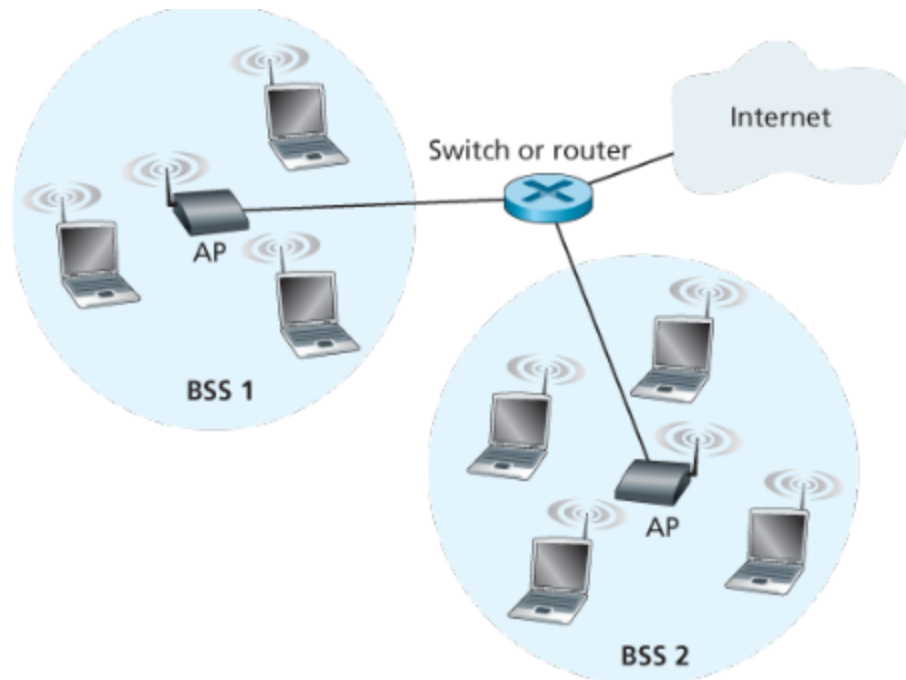


Figure 7.7 IEEE 802.11 LAN architecture

Las LANs inalámbricas que despliegan APs se les llama *infraestructura de LANs inalámbricas*, donde la infraestructura serían las APs junto a la infraestructura de Ethernet cableada que interconecta las APs y el router.

Existen también las **redes ad hoc**, estas son redes sin control central y sin conexiones con el “mundo exterior” (por ej. entre laptops de gente aproximada).

En 802.11, cada estación inalámbrica necesita asociarse a un AP antes de empezar a recibir o enviar datos. Cuando un administrador de red instala una AP, le asigna una o dos palabras **Service Set Identifier (SSID)** al AP; además asigna un número de canal al AP. En 802.11 el rango de frecuencia es 2.4-2.4835 GHz, dentro de esos 85 MHz se definen 11 canales parcialmente solapados.

Un **WiFi jungle** es una ubicación física donde una estación inalámbrica recibe una señal lo suficientemente potente de dos o más APs. Cada AP estará ubicada en una subred IP distinta y tendrá asignada su canal.

El estandar de 802.11 requiere que la AP envíe periódicamente **beacon frames**, cada uno de estos frames incluye el SSID de la AP y la MAC address. Entonces, un dispositivo inalámbrico que entra en una zona de WiFi jungle va a escanear los 11 canales buscando beacon frames de las APs que estén por ahí y va a seleccionar una

para asociarse (asociar=crear un cable virtual entre la AP y el dispositivo). El algoritmo con el cual se selecciona una AP es elección de los diseñadores del firmware y software del dispositivo.

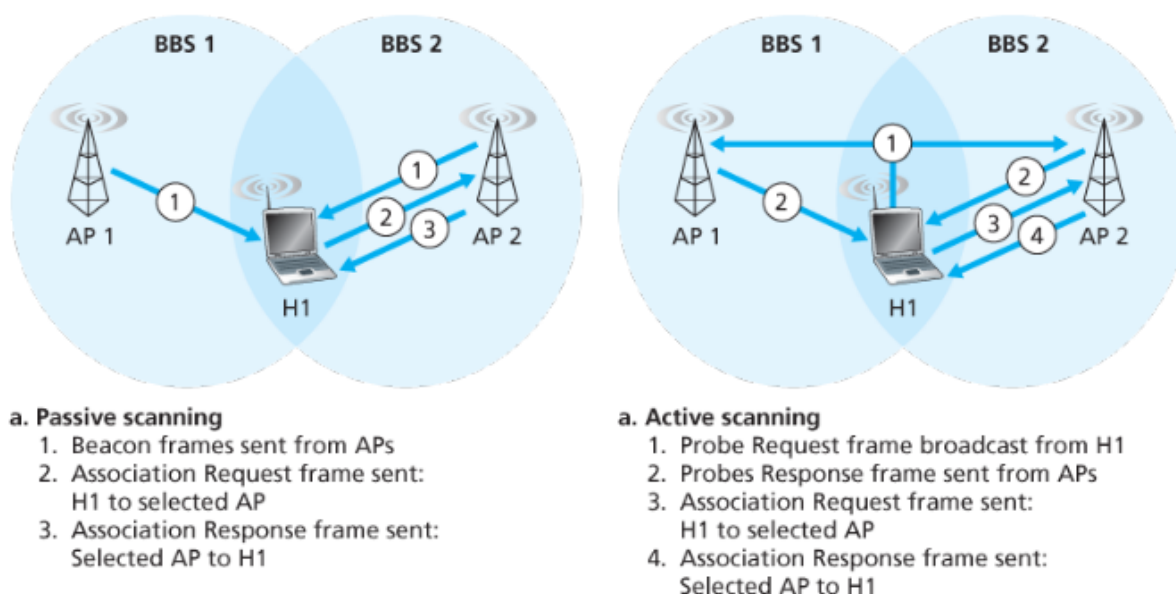


Figure 7.9 Active and passive scanning for access points

El proceso de escanear canales y esperar beacon frames se conoce como **passive scanning**.

Active scanning es cuando el dispositivo envía por broadcast un frame a todos los APs que están dentro del rango, los APs responden y ahí el dispositivo elige entre los APs que respondieron. Notar que en el scanning activo es necesario el segundo handshake porque los APs que respondieron al dispositivo, no saben cuál es el AP que el dispositivo va a elegir.

Una vez que el dispositivo se asocia a un AP, va a querer unirse a la subred a la que pertenece el AP. En general, envía un mensaje DHCP a la subred a través de la AP para obtener la IP, y una vez que obtiene esa IP el resto del mundo va a ver al dispositivo simplemente como un host más con una IP de esa subred.

A veces se requiere una identificación del dispositivo para poder asociarse a una AP. Un approach es a partir de la MAC address, otro es mediante un usuario y contraseña; en ambos casos la AP utiliza un server de autenticación.

The 802.11 MAC Protocol

Se necesita un protocolo de acceso multiple para coordinar las transmisiones ya que seguramente haya muchos dispositivos queriendo enviar datos al mismo tiempo por el mismo canal. Como se vio en el Cap. 6 hay tres tipos de protocolos: canales particionados, random access y por turnos. En 802.11 se utiliza random access, el protocolo se conoce como **CSMA with collision avoidance (CSMA/CA)**. Cada estación (dispositivo o AP) chequea el canal antes de transmitir, y si detecta que está ocupado se abstiene de transmitir.

A diferencia de Ethernet, 802.11 utiliza técnicas de collision-avoidance (en vez de collision detection), además usa un esquema de acknowledgment/retransmisión de capa de enlace.

Lo del collision-avoidance es por dos motivos: primero que es costoso construir hardware que detecte colisiones ya que la señal recibida suele ser pequeña en comparación a la potencia de transmisión (recordar que requiere enviar y recibir una señal al mismo tiempo); segundo porque el adaptador no puede detectar todas las colisiones por el problema de la hidden terminal y fading.

Una vez que una estación comienza a enviar un frame, lo envía entero, no hay vuelta atrás.

Respecto a los ACKs de la capa de enlace → cuando una estación recibe un frame con CRC, espera un período de tiempo conocido como Short Inter-frame Spacing (SIFS) y luego envía un frame ACK. Si la estación que está transmitiendo no recibe nada dentro de un determinado tiempo, asume que hubo un error y retransmite el frame usando el protocolo CSMA/CA para acceder al canal. Si el ACK no se recibe después de X número de retransmisiones, se descarta el frame.

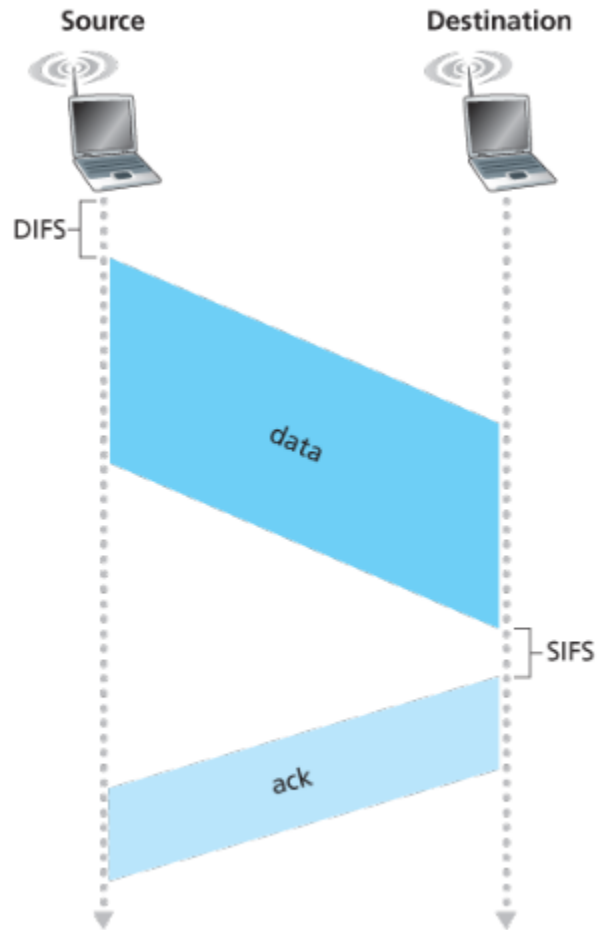


Figure 7.10 802.11 uses link-layer acknowledgments

Ahora si, el protocolo CSMA/CA, suponiendo que una estación tiene un frame para transmitir:

1. Si inicialmente la estación detecta que el canal está inactivo, transmite el frame después de un período de tiempo llamado Distributed Inter-frame Space (DIFS).
2. Sino, elige un valor random utilizando "retroceso exponencial binario" y va contando hacia atrás en tanto perciba al canal inactivo, si lo percibe ocupado el contador queda frizado.
3. Cuando el contador llega a 0, la estación transmite todo el frame y espera el ACK.
4. Si recibe el ACK, interpreta que el frame fue recibido correctamente. Si tiene otro frame para enviar, repite el protocolo desde el paso 2. Si no recibe el ACK, vuelve a

la fase de retroceso del punto 2 eligiendo el valor random de un intervalo más grande.

El hecho de tener un contador aún cuando se detecta que el canal está inactivo es para evitar colisiones, sino podría pasar que dos dispositivos detecten lo mismo al mismo tiempo y habría colisión → no se quiere esto porque se va a transmitir el frame entero.

El protocolo MAC 802.11 incluye un esquema para evitar colisiones en la presencia de terminales ocultas. Las terminales ocultas son un problema porque llevan a colisiones, si por ej. la terminal H1 está transmitiendo a AP y la terminal H2 también quiere transmitir algo, no va a detectar la transmisión de H1 y va a terminar en una colisión.

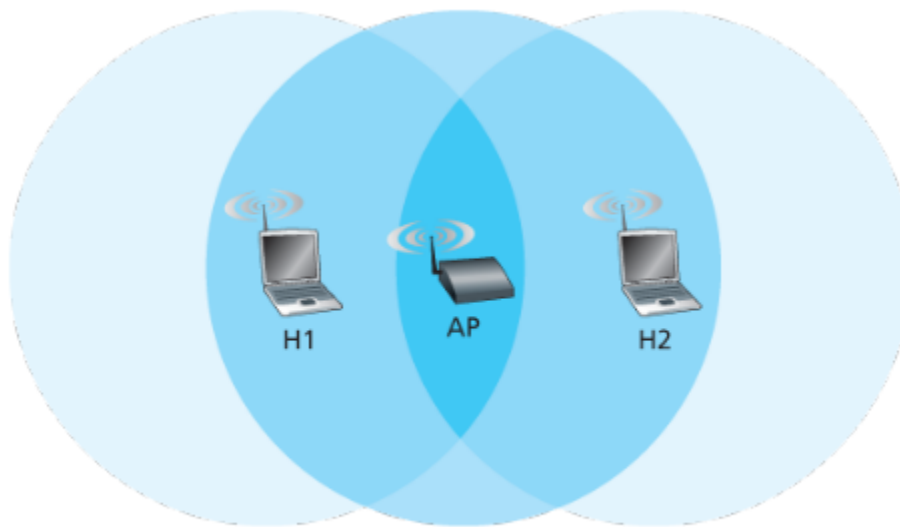


Figure 7.11 Hidden terminal example: H1 is hidden from H2, and vice versa

La solución que propone 802.11 es utilizar un Request to Send (RTS) y Clear to Send (CTS) para reservar el acceso al canal. Quien vaya a hacer un envío debe primero enviar un RTS al AP indicando el tiempo que va a requerir la transmisión tanto del frame como del ACK, el AP luego va a enviar por broadcast un CTS. Este CTS le da el permiso para transmitir a quien lo solicito y le avisa a los demás que no deben enviar nada durante ese período de tiempo.

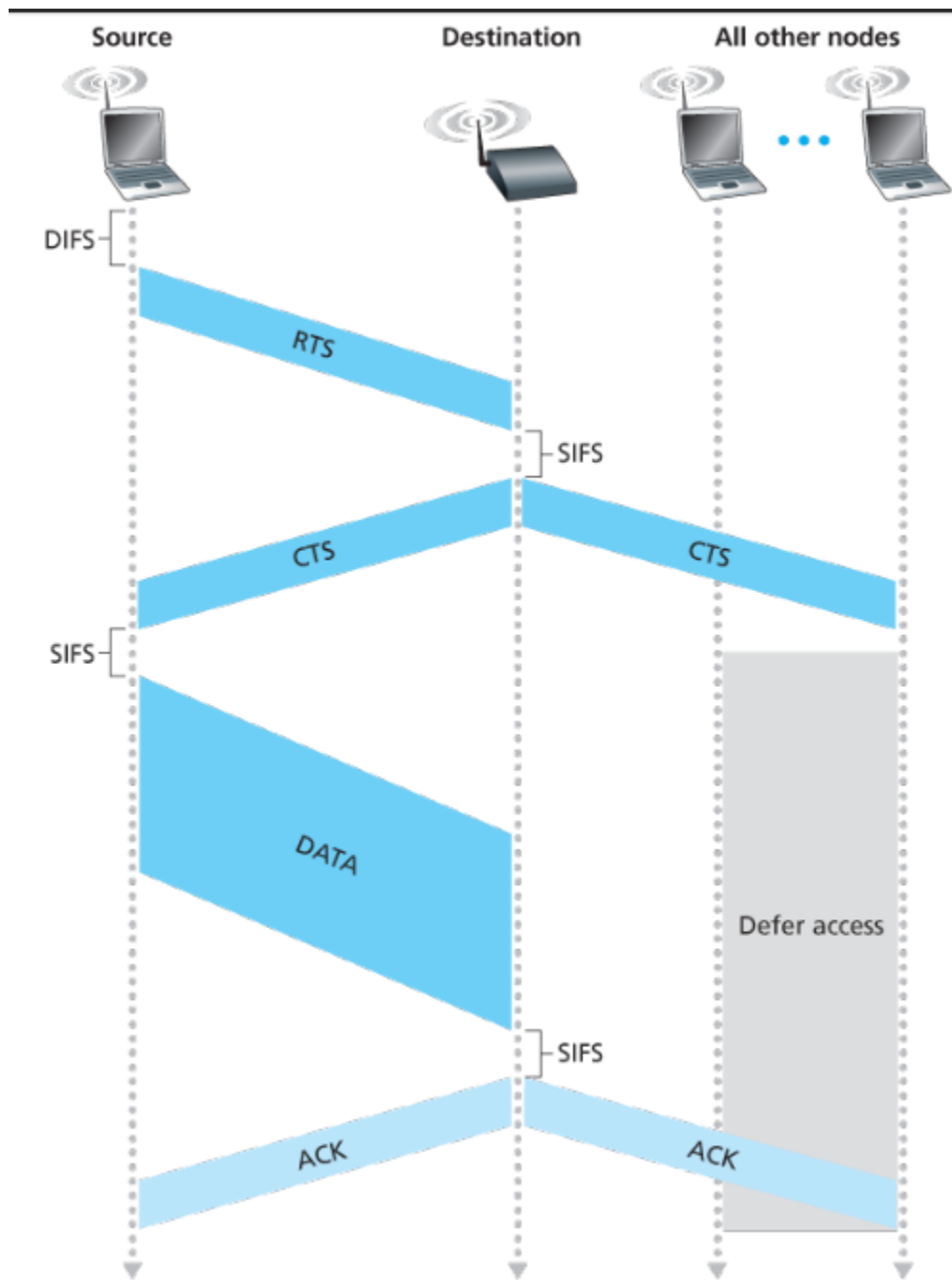


Figure 7.12 Collision avoidance using the RTS and CTS frames

La desventaja de este protocolo es que suma delay y consume recursos del canal. Por este motivo, solo se usa para enviar un frame de datos grande.

The IEEE 802.11 Frame

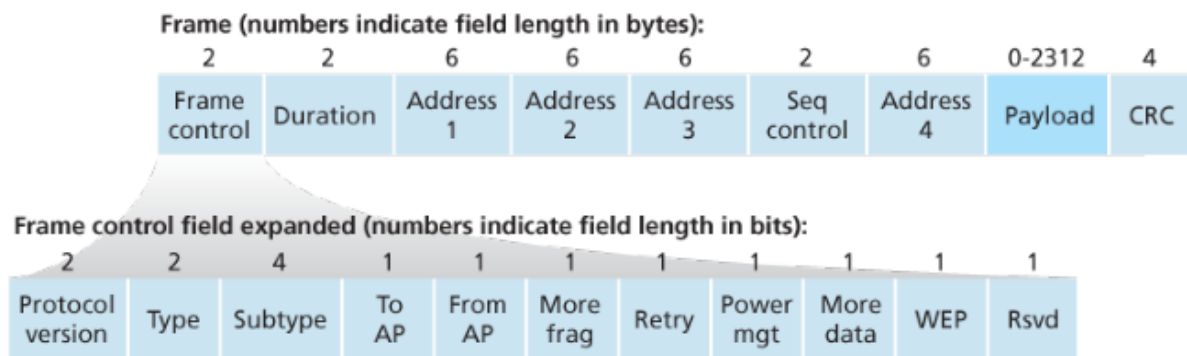


Figure 7.13 The 802.11 frame

El payload en general consiste del datagrama IP o un paquete ARP.

Es más común encontrar errores de bits en redes inalámbricas que en las cableadas, así que el CRC toma importancia acá.

Hay cuatro campos para la dirección. Tres de ellos se usan para internetworking y la cuarta es para mover frames en el modo ad hoc.

Address 2 es la MAC address de la estación que transmite el frame.

Address 1 es la MAC address de la estación inalámbrica que recibe el frame.

Address 3 contiene la MAC address de la interfaz del router (recordar BSS es parte de la subred que se conecta a otras subredes por la interfaz del router). Esta dirección es importante → más detalle en pág. 604

El campo duration es el tiempo que necesita para transmitir el frame y el ACK.

El Frame control involucra varias cosas, las más importantes son: type y subtype para distinguir la asociación (CTS, RTS, ACK y data frames); to y from para definir el significado de los diferentes campos de address y WEP define si se está utilizando encriptación o no.

Mobility in the Same IP Subnet

Con el fin de aumentar el rango físico de una LAN, algunas compañías despliegan varios BSSs dentro de la misma subred IP. Esto aumenta la movilidad entre los BSSs, si por ejemplo H1 quiere pasar de BSS1 a BSS2 lo que va a pasar es que H1 detecta una señal débil de AP1 y empieza a buscar una señal más potente, recibe los beacon frames de AP2 y entonces se desasocia de AP1 y se asocia con AP2 manteniendo su IP y sus conexiones TCP. Los switches se enteran del cambio porque recordan que son

self-learning y van actualizando la tabla de forwarding automáticamente. Un “truco” es que AP2 mandé un frame Ethernet por broadcast con la dirección origen de H1 al switch justo después de asociarse y ahí el switch actualiza su tabla.

Cellular Internet Access

Cellular Network Architecture, 2G: Voice Connections to the Telephone Network

El término “celular” viene por el hecho de que a una región cubierta por una red celular se la divide en áreas llamadas “celdas”.

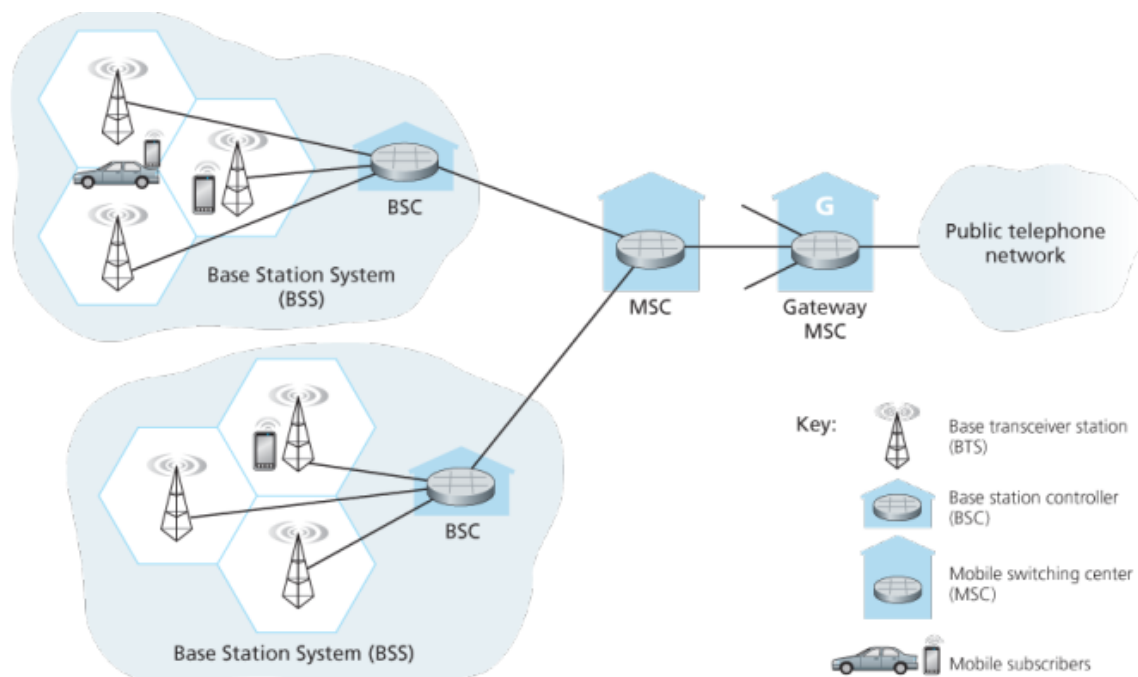


Figure 7.18 Components of the GSM 2G cellular network architecture

Cada celda contiene un **base transceiver station (BTS)** que transmite y recibe señales de las estaciones móviles de la celda. Algunos sistemas ponen un solo BTS en una esquina intersección de tres celdas con antenas que le den servicio a las tres celdas.

El estandar GSM para celulares 2G usa una combinación FDM/TDM. Esto es que el canal se particiona en un número de sub-bandas de frecuencia. Dentro de cada sub-banda el tiempo se divide en frames y slots, o sea que si un canal se divide en F sub-bandas y el tiempo en T slots, el canal va a soportar F.T llamados simultáneos.

El BSC (base station controller) ofrece decenas de BTSs. Su rol es el de alocar canales de radio BTS para los suscriptores móviles. El BSC y sus BTSs conforman un BSS (base station system).

3G Cellular Data Networks: Extending the Internet to Cellular Subscribers

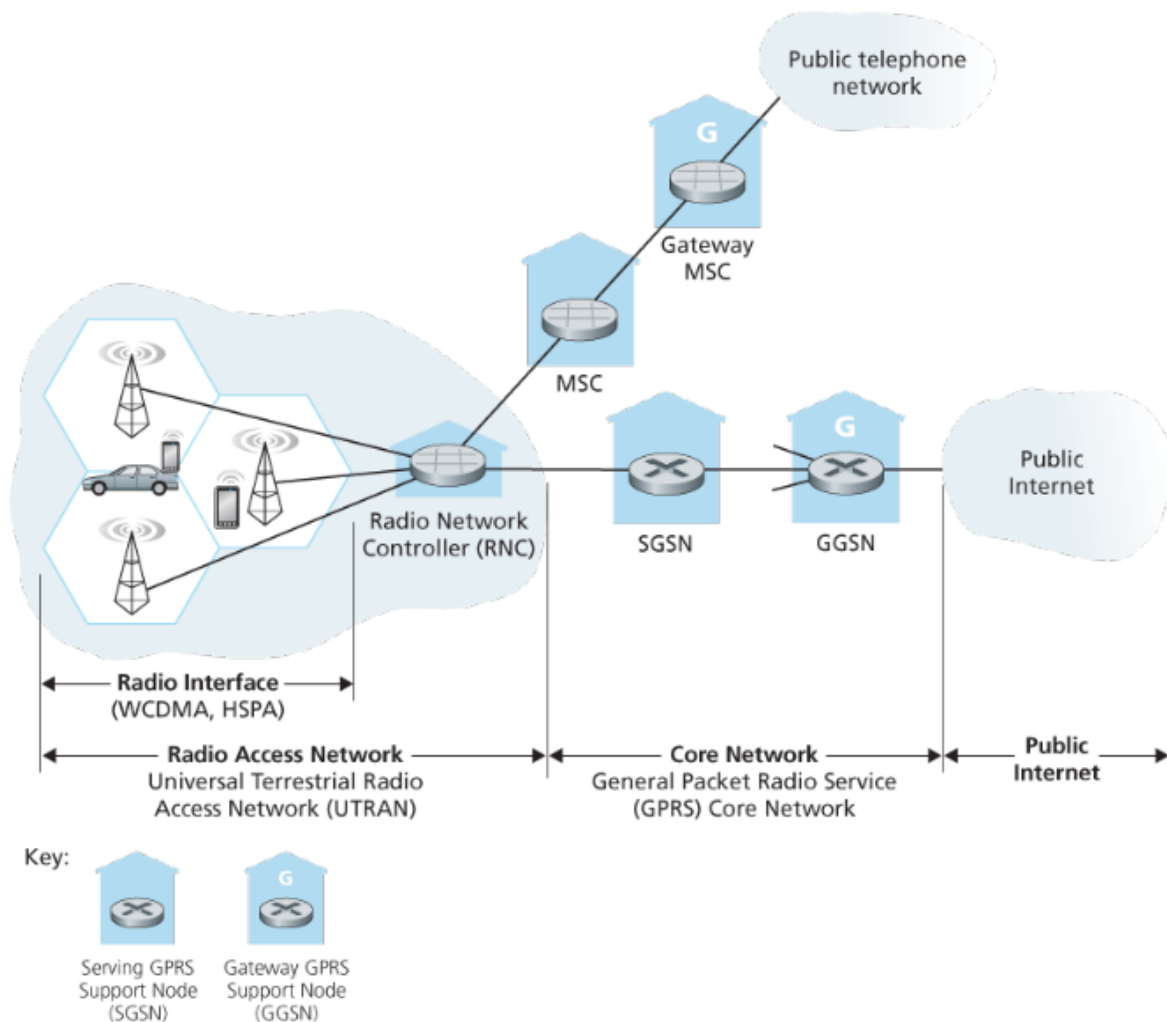


Figure 7.19 3G system architecture

El núcleo de una red de datos celular conecta las redes de acceso por radio a Internet. El approach de quienes diseñaron 3G fue: dejar el core GSM intacto y agregar funcionalidad en paralelo a la red de voz celular existente. En una red 3G existen dos tipos de nodos: **Serving GPRS Support Nodes (SGSNs)** y **Gateway GPRS Support Nodes (GGSNs)**. SGSN es responsable de entregar datagramas a/desde nodos

móviles en la red de acceso por radio a la que está adjunta la SGSN. GGSN hace de gateway, conectando muchas SGSNs a Internet.

La red 3G va a ser el primer hop inalámbrica que veamos como usuarios 3G. El **Radio Network Controller (RNC)** controla varias estaciones transceptoras similares a las de 2G. RNC se conecta con la red de voz celular con conmutación de circuitos (ke) via MSC y con Internet con conmutación de paquetes via SGSN. Entonces, mientras que el servicio de voz y el de datos usan distintas redes core, comparten el primer y último hop de la radio access network.

On to 4G: LTE

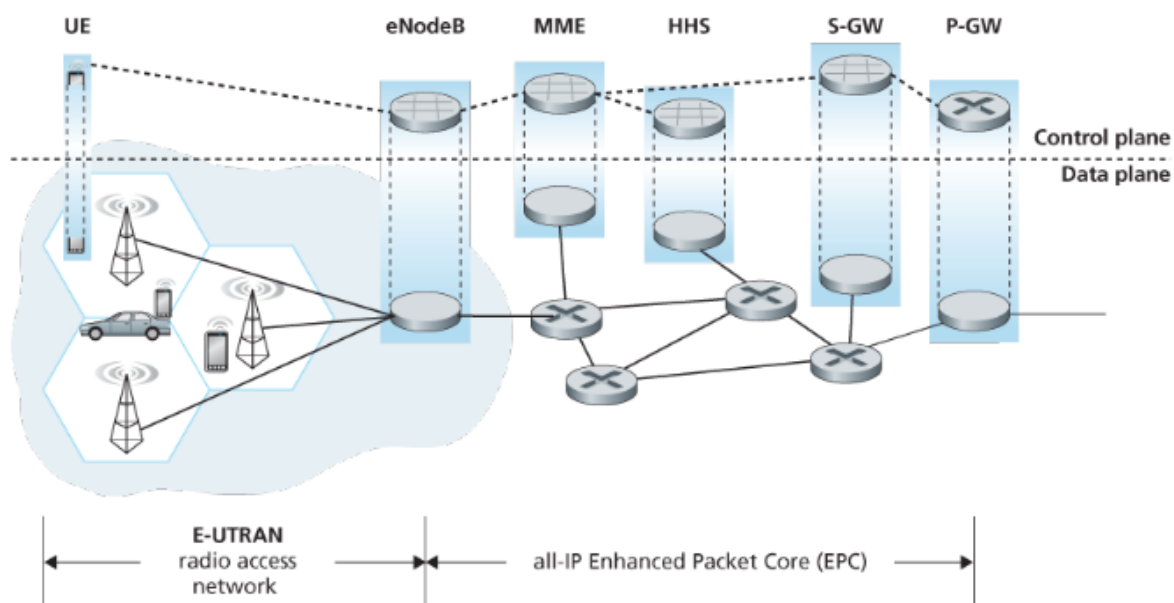


Figure 7.20 4G network architecture

Algunas características:

- **A unified, all-IP network.** Tanto la voz como los datos viajan en datagramas IP hacia/desde el dispositivo inalámbrico al gateway y luego al gateway del paquete que conecta el borde 4G con el resto de la red.
- **A clear separation of the 4G data plane and 4G control plane.**
- **A clear separation between the radio access network, and the all-IP-core network.** Los datagramas que llevan la data de usuario se reenvían entre el usuario (UE, user equipment) y el gateway (P-GW) por una red interna IP 4G al Internet exterior.

Los componentes principales son:

- **eNodeB**: El rol de su data plane es el de reenviar datagramas entre UE y P-GW. Los datagramas UE se encapsulan en eNodeB y van tipo túnel al P-GW a través del **EPC (Enhances Packet Core)** de la red 4G. En el plano de control, el eNodeB maneja el registro y el tráfico de señalización de movilidad en nombre del UE.
- **Packet Data Network Gateway (P-GW)**: asigna direcciones IP a los UEs y hace los QoS (quality of services). Encapsula/desencapsula datagramas (cuando hay reenvios con UE)
- **Serving Gateway (S-GW)**: todo el tráfico de UE pasa por acá, es el “punto de anclaje” de la movilidad del plano de datos. Acá se aplican las funciones de cobro/facturación e interceptación de tráfico legal.
- **Mobility Management Entity (MME)**: realiza la gestión de la conexión y la movilidad en nombre de los UE residentes en la celda que controla.
- **Home Subscriber Server (HSS)**: contiene información del UE incluida la información de autenticación, perfiles de QoS y capacidades de roaming.

Mobility Management: Principles

Un nodo móvil es aquel que cambia su punto de conexión a la red con el tiempo.

- **Desde el punto de vista de la capa de red, qué tan móvil es un usuario?**: un usuario que carga con una laptop que tiene una tarjeta de interfaz de red inalámbrica, no se ve como un usuario móvil desde la red. Es más, si el usuario se asocia desde el mismo punto de acceso sin importar el lugar, no va a ser un usuario móvil ni para la capa de enlace. Ahora, un usuario conectado desde una autopista a 100km/h queriendo mantener una conexión TCP sí es móvil.

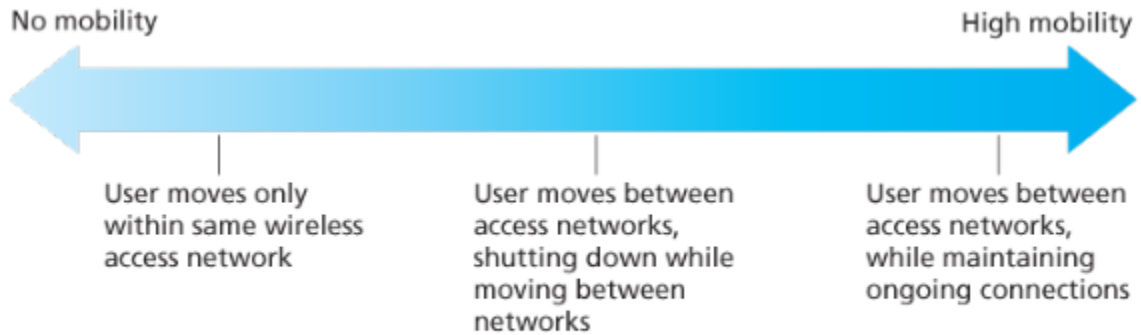


Figure 7.22 Various degrees of mobility, from the network layer's point of view

- **Qué tan importante es que la dirección de un nodo móvil sea fija?:** Recordar del Cap. 3 que una aplicación de internet necesita saber la dirección IP y el puerto de la entidad remota con la que se está comunicando, entonces si una entidad móvil es capaz de conserva su IP mientras se mueve, es como que la movilidad se hace invisible desde el punto de vista de la aplicación. En el caso de un usuario que vuelve del trabajo y se conecta desde su casa para funciones principalmente del tipo cliente-servidor, no tendría sentido que mantenga la IP.
- **Qué infraestructura cableada de soporte está disponible?:** Las ad hoc

A twenty-something adult moving out of the family home becomes mobile, living in a series of dormitories and/or apartments, and often changing addresses. If an old friend wants to get in touch, how can that friend find the address of her mobile friend? One common way is to contact the family, since a mobile adult will often register his or her current address with the family. The family home, with its permanent address, becomes that one place that others can go as a first step in communicating with the mobile adult. Later communication from the friend may be either indirect (for example, with mail being sent first to the parents' home and then forwarded to the mobile adult) or direct (for example, with the friend using the address obtained from the parents to send mail directly to her mobile friend).

En redes, el hogar permanente de un nodo móvil es la **home network**, y el **home agent** es la entidad dentro de la home network que opera las funciones de 'gestión' de movilidad. La red en la que el nodo está residiendo es el **foreign (or visited) network**, y dentro está el **foreign agent**. Un **correspondent** es una entidad que quiere comunicarse con el nodo móvil.

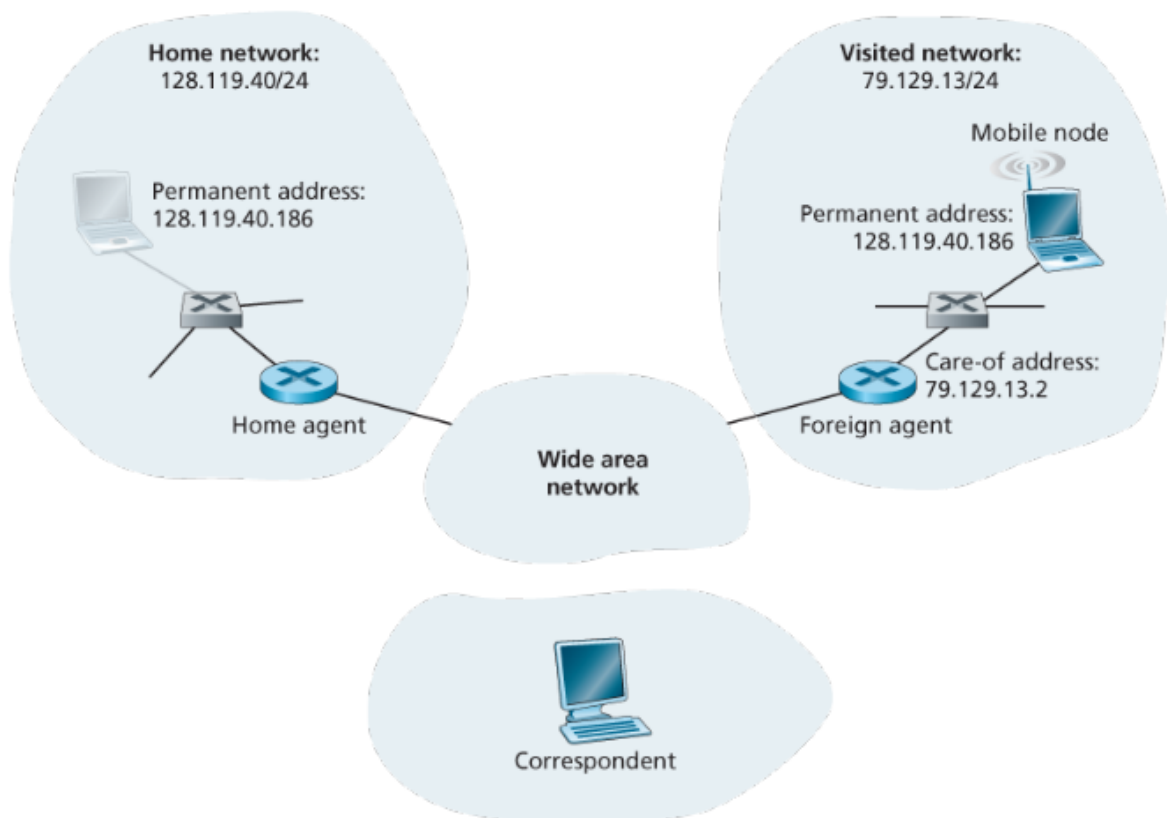


Figure 7.23 Initial elements of a mobile network architecture

Cuando un nodo está residiendo en una foreign network, todo el tráfico que iba dirigido a la dirección permanente del nodo necesita ser redirigida al foreign. Esto se puede lograr de varias maneras. La primera es que el foreign le avise a las demás redes que el nodo móvil está residiendo ahí, esto requiere pocos cambios al sistema de routing existente, se puede hacer por el intercambio de intra e interdomain. El foreign le avisa a sus vecinos y sus vecinos propagan la información. Una desventaja es que no escala, si la gestión de la movilidad fuera responsabilidad del router, tendría que guardar millones de nodos en las tablas.

La otra opción es mover la funcionalidad de movilidad desde el core de la red hasta el borde. Para esto se usaría el home agent, este tendría que trackear el nodo donde

reside el nodo móvil. Para eso se necesita un protocolo entre ellos tal que se garantice la información actualizada del nodo móvil.

Una tarea del foreign agent es la de crear una care-of-address (COA) o foreign address para el nodo móvil, entonces el nodo va a tener dos direcciones: la permanente y la COA. Otra tarea es la de avisarle al home agent que el nodo móvil está en su red y que le asignó un COA.

Routing indirecto

El correspondent envía un datagrama a la home network, ignora si el nodo está ahí o no. Los home agents tienen que estar atentos si reciben datagramas para nodos que están en foreign, así los intercepta y los reenvía al nodo móvil en un proceso de dos pasos: 1. el datagrama se envía al foreign agent a partir del COA y 2. es reenviado del foreign agent al nodo.

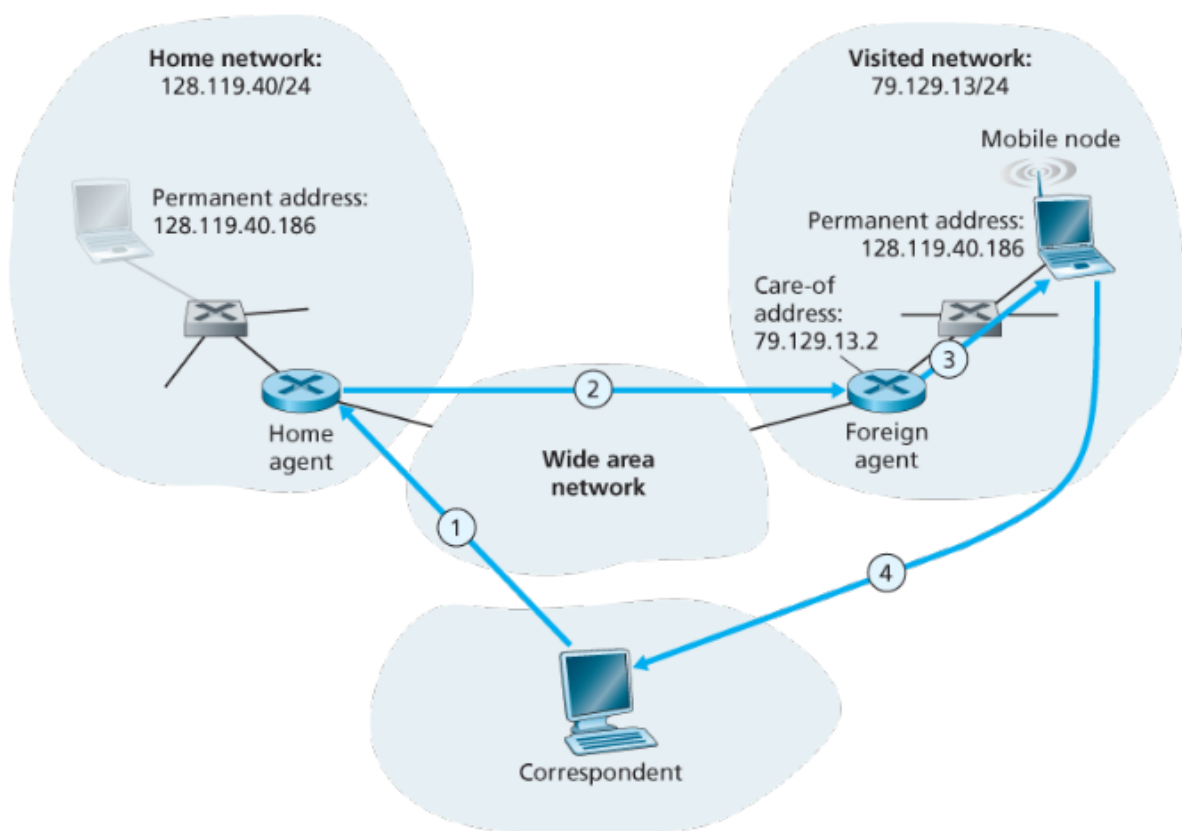


Figure 7.24 Indirect routing to a mobile node

El home agent debe encapsular el datagrama original.

El nodo móvil puede mandarle datagramas directamente al correspondent usando su dirección permanente como origen.

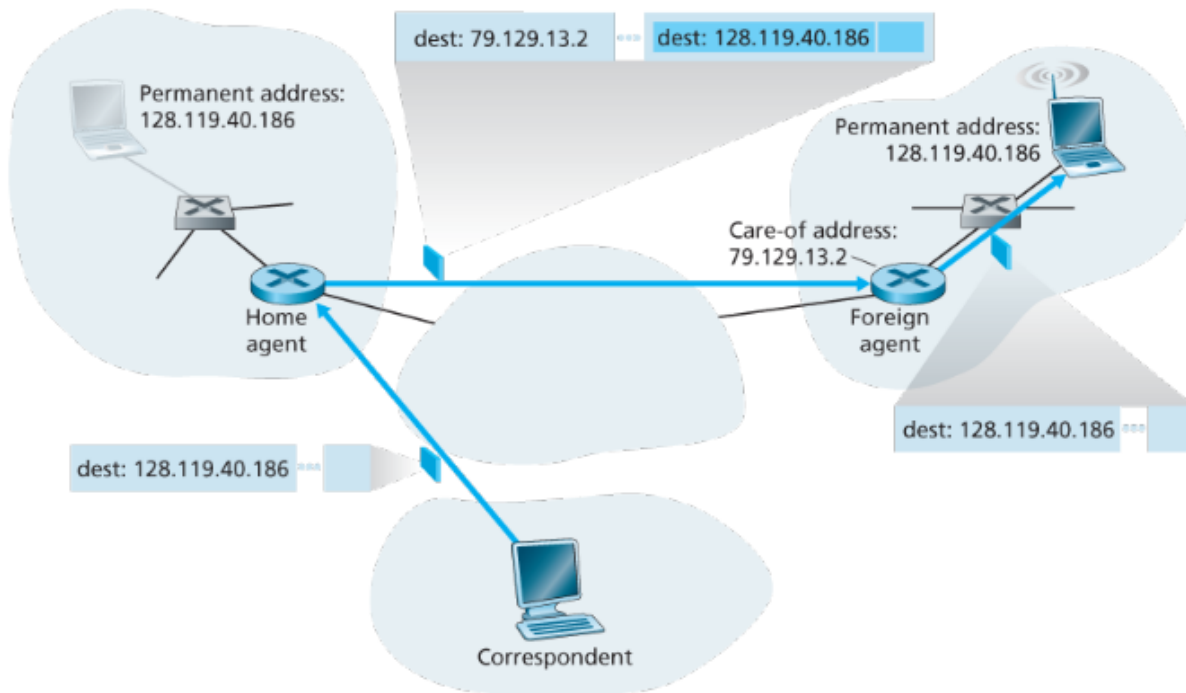


Figure 7.25 Encapsulation and decapsulation

Routing directo

El routing indirecto es ineficiente por el problema de enrutamiento triangular (*imagine a mobile user who is visiting the foreign network of a colleague. The two are sitting side by side and exchanging data over the network. Datagrams from the correspondent (in this case the colleague of the visitor) are routed to the mobile user's home agent and then back again to the foreign network!*).

En el routing directo, existe un correspondent agent que aprende el COA del nodo móvil; el agent le manda una query al home agent pidiéndole el COA.

Dos problemas nuevos: se necesita un protocolo mobile-user location para que el correspondent pueda hacer la query; y qué pasa si el nodo se mueve a un segundo foreign? → anchor foreign agent.

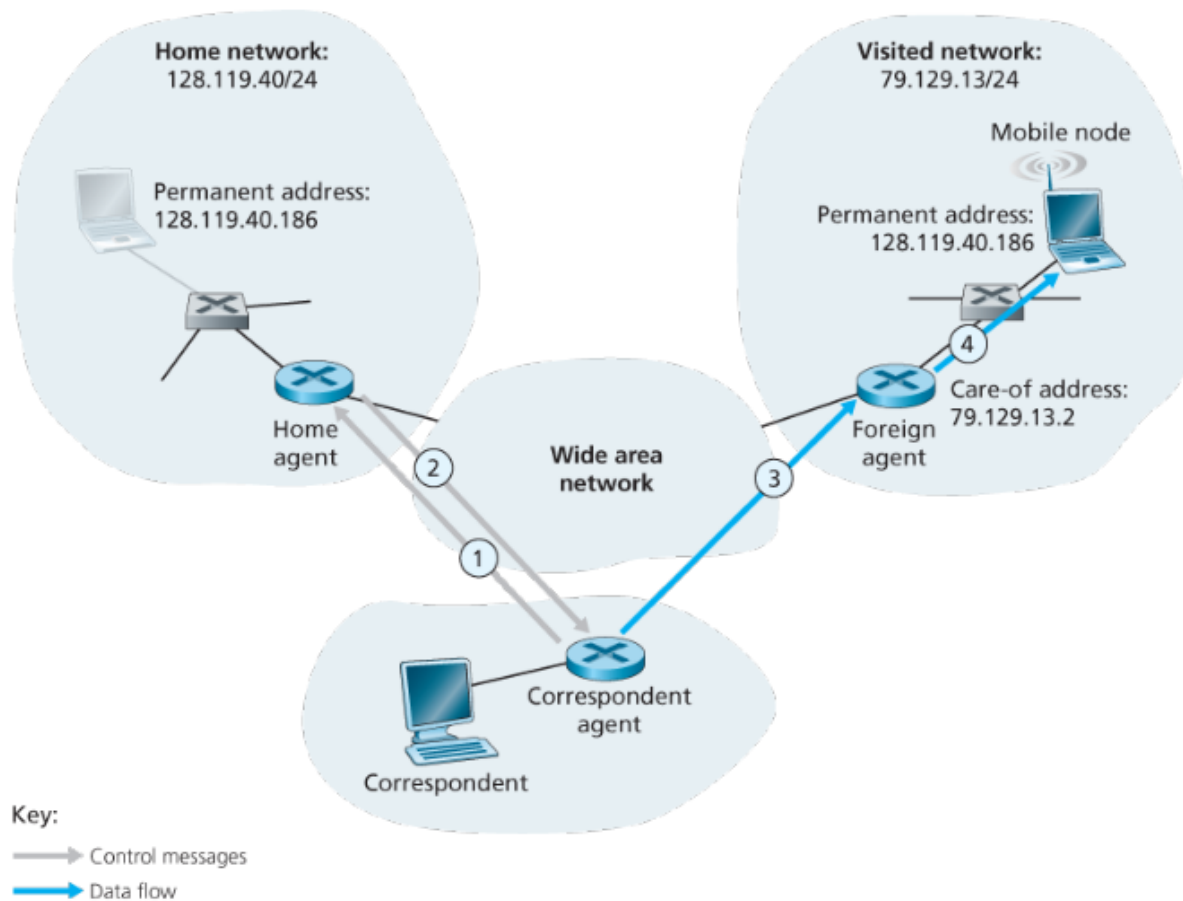


Figure 7.26 Direct routing to a mobile user

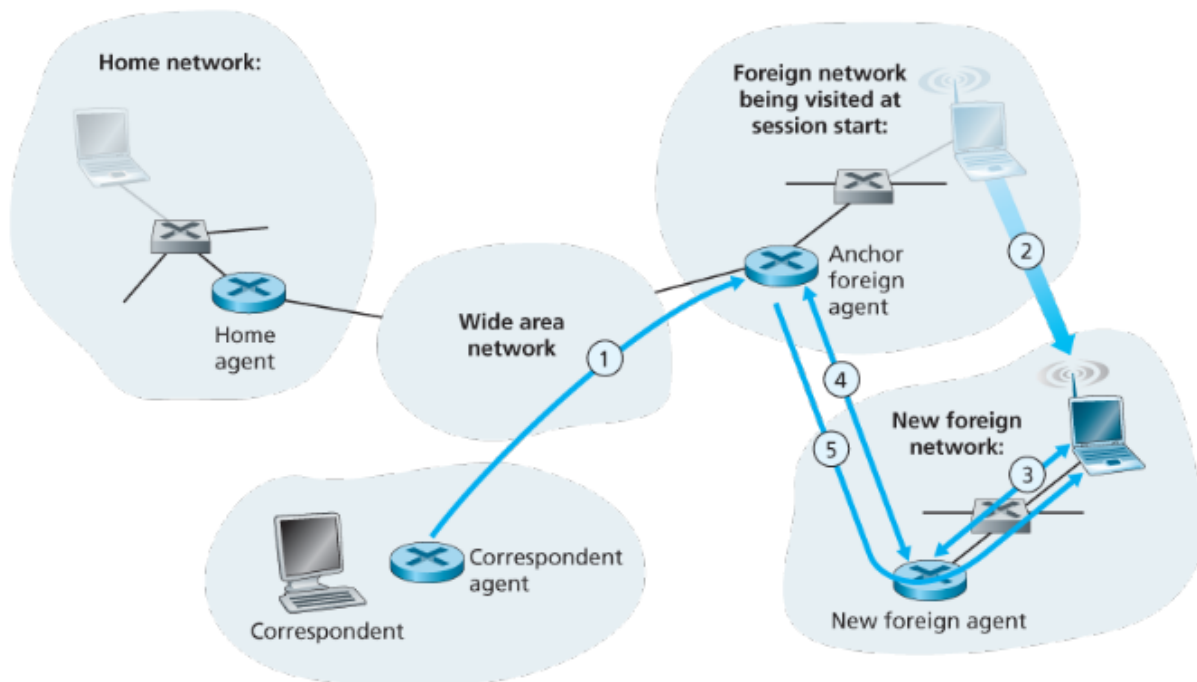


Figure 7.27 Mobile transfer between networks with direct routing

Mobile IP

Consiste principalmente de tres piezas:

- **Agent discovery.** Se definen protocolos que van a usar los home y foreign agents para avisar sobre sus servicios y viceversa, protocolos que usaran los nodos moviles para solicitar servicios.
- **Registration with the home agent.** Protocolos para registrar o quitar COAs
- **Indirect routing of datagrams.** Se define la forma en la que se reenvían los datagramas (encapsulacion, manejo de errores, etc)

Agent discovery

Es el proceso de encontrar un nuevo foreign agent con una nueva dirección de red. Puede ser con **agent advertising** (mensaje broadcast ICMP de tipo 9) o con **agent solicitation** (mensaje broadcast ICMP de tipo 10).

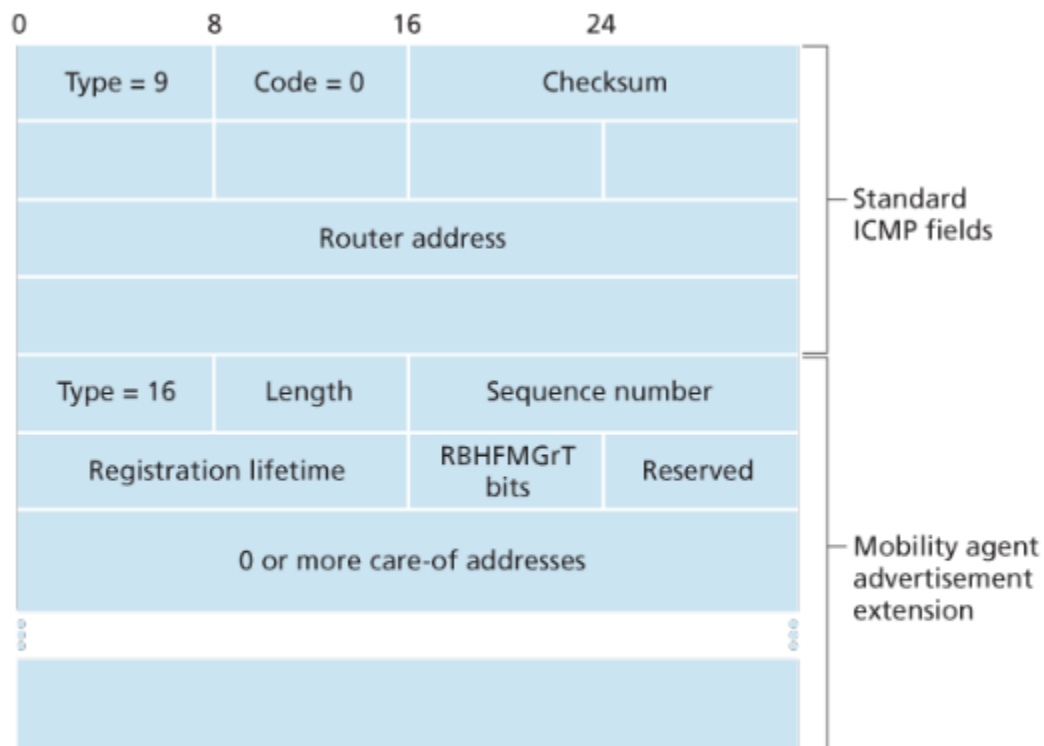


Figure 7.28 ICMP router discovery message with mobility agent advertisement extension

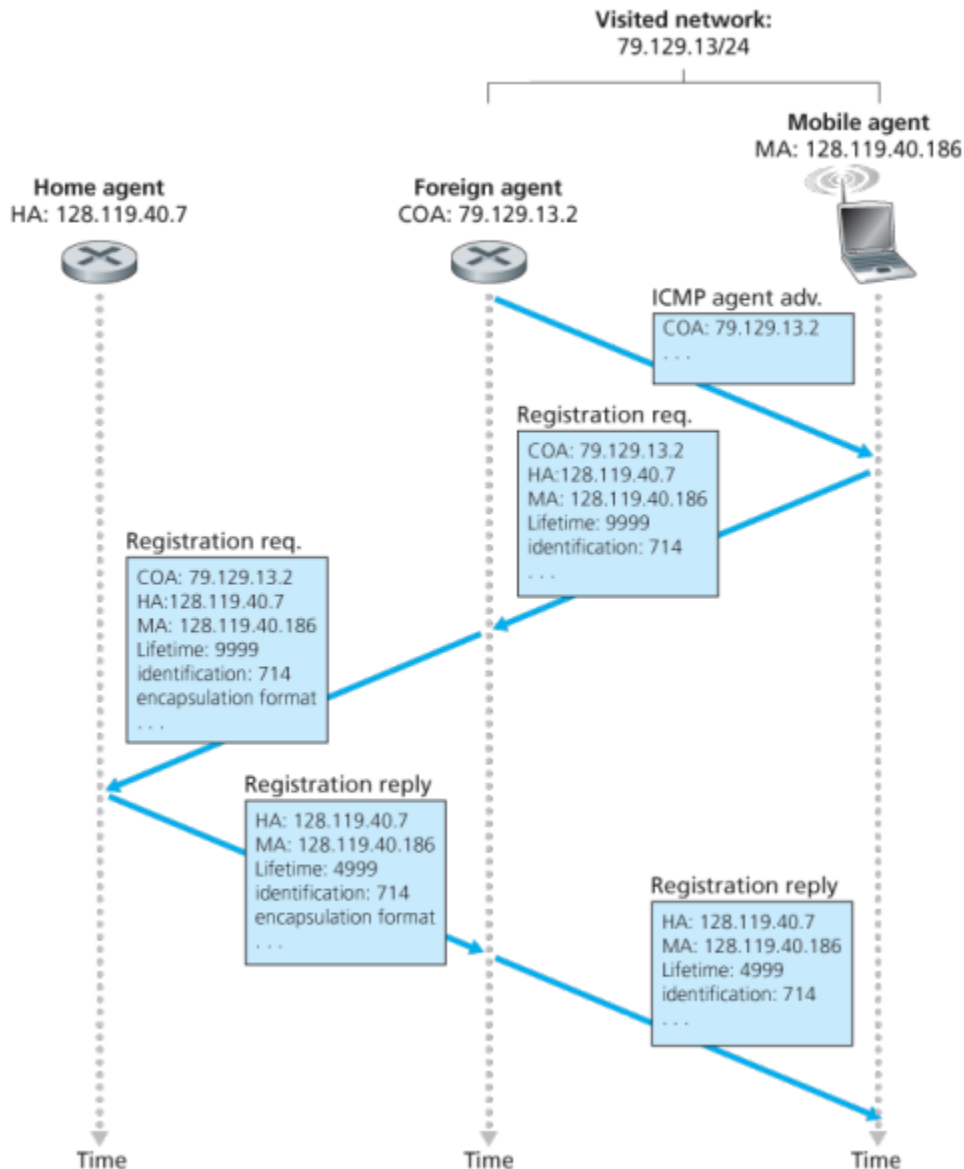
Los campos más importantes de la extensión (en advertising) son: home agent bit (H), foreign agent bit (F), registration required bit (R), M G encapsulation bits, COA fields.

Registration with the Home Agent

- Following the receipt of a foreign agent advertisement, a mobile node sends a mobile IP registration message to the foreign agent. The registration message is carried within a UDP datagram and sent to port 434. The registration message carries a COA advertised by the foreign agent, the address of the home agent (HA), the permanent address of the mobile node (MA), the requested lifetime of the registration, and a 64-bit registration identification. The requested registration lifetime is the number of seconds that the registration is to be valid. If the registration is not renewed at the home agent within the specified lifetime, the

registration will become invalid. The registration identifier acts like a sequence number and serves to match a received registration reply with a registration request, as discussed below

2. The foreign agent receives the registration message and records the mobile node's permanent IP address. The foreign agent now knows that it should be looking for datagrams containing an encapsulated datagram whose destination address matches the permanent address of the mobile node. The foreign agent then sends a mobile IP registration message (again, within a UDP datagram) to port 434 of the home agent. The message contains the COA, HA, MA, encapsulation format requested, requested registration lifetime, and registration identification.
3. The home agent receives the registration request and checks for authenticity and correctness. The home agent binds the mobile node's permanent IP address with the COA; in the future, datagrams arriving at the home agent and addressed to the mobile node will now be encapsulated and tunneled to the COA. The home agent sends a mobile IP registration reply containing the HA, MA, actual registration lifetime, and the registration identification of the request that is being satisfied with this reply.
4. The foreign agent receives the registration reply and then forwards it to the mobile node.



Managing Mobility in Cellular Networks

GSM usa un approach de ruteo indirecto. Aca el home network va a ser el proveedor de telefonía del celular. Las responsabilidades de las redes home y visited son distintas:

- El home tiene una base de datos conocida como **home location register (HLR)** que contiene el número de celular permanente y el perfil de cada uno de sus usuarios. También tiene la información de la ubicación de los usuarios.
- La visited network tiene una base conocida como **visitor location register (VLR)** que tiene una entrada por cada usuario móvil que actualmente está en una porción

de la red servida por el VLR.

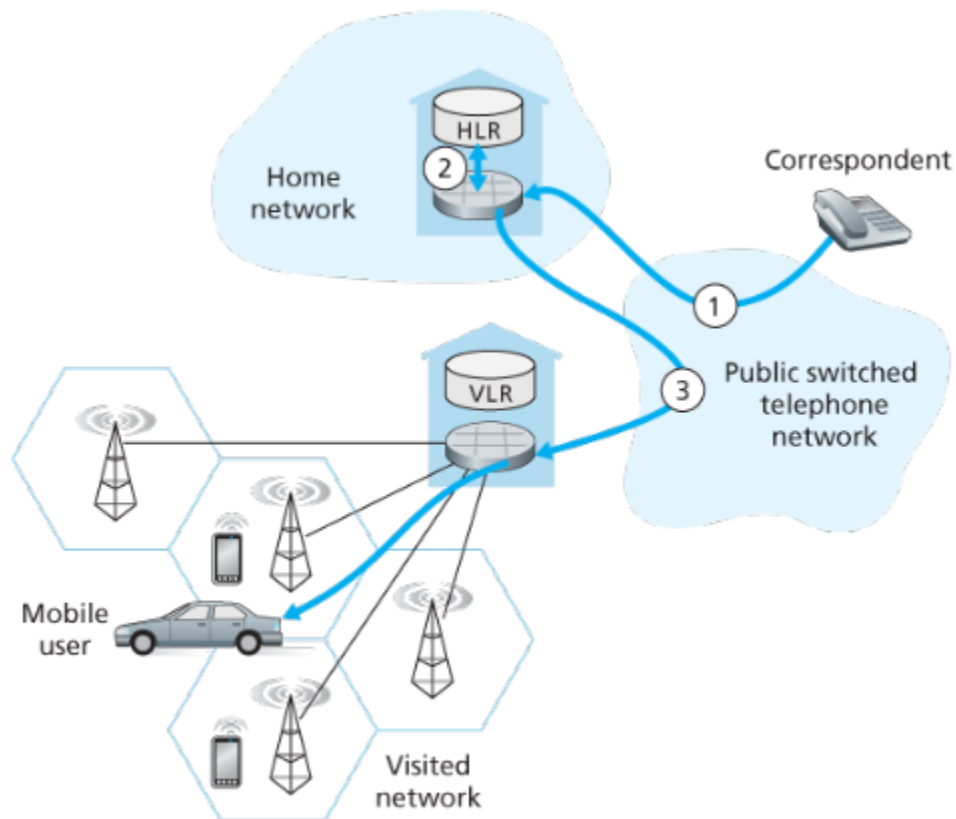


Figure 7.30 Placing a call to a mobile user: Indirect routing

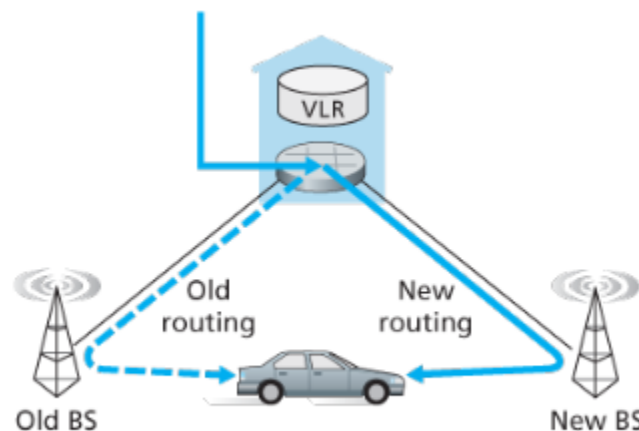
1. The correspondent dials the mobile user's phone number. This number itself does not refer to a particular telephone line or location (after all, the phone number is fixed and the user is mobile!). The leading digits in the number are sufficient to globally identify the mobile's home network. The call is routed from the correspondent through the PSTN to the home MSC in the mobile's home network. This is the first leg of the call.
2. The home MSC receives the call and interrogates the HLR to determine the location of the mobile user. In the simplest case, the HLR returns the mobile station roaming number (MSRN), which we will refer to as the roaming number. Note that this number is different from

the mobile's permanent phone number, which is associated with the mobile's home network. The roaming number is ephemeral: It is temporarily assigned to a mobile when it enters a visited network. The roaming number serves a role similar to that of the care-of address in mobile IP and, like the COA, is invisible to the correspondent and the mobile. If HLR does not have the roaming number, it returns the address of the VLR in the visited network. In this case (not shown in Figure 7.30), the home MSC will need to query the VLR to obtain the roaming number of the mobile node. When a mobile telephone is switched on or enters a part of a visited network that is covered by a new VLR, the mobile must register with the visited network. The visited VLR, in turn, sends a location update request message to the mobile's HLR.

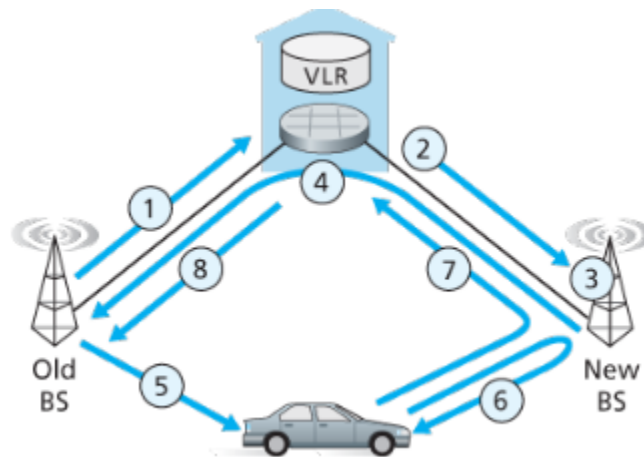
3. Given the roaming number, the home MSC sets up the second leg of the call through the network to the MSC in the visited network. The call is completed, being routed from the correspondent to the home MSC, and from there to the visited MSC, and from there to the base station serving the mobile user.

Handoffs in GSM

Un handoff ocurre cuando una estación móvil cambia su asociación de una estación base a otra durante una llamada.



Los motivos pueden ser varios, por ej. que la señal se haya deteriorado y sea muy débil o que esté saturada la celda.



1. The old base station (BS) informs the visited MSC that a handoff is to be performed and the BS (or possible set of BSs) to which the mobile is to be handed off.
2. The visited MSC initiates path setup to the new BS, allocating the resources needed to carry the rerouted call, and signaling the new BS that a handoff is about to occur.
3. The new BS allocates and activates a radio channel for use by the mobile.
4. The new BS signals back to the visited MSC and the old BS that the visited-MSC-to-new-BS path has been established and that the mobile should be informed of the impending handoff. The new BS provides all of the information that the mobile will need to associate with the new BS.
5. The mobile is informed that it should perform a handoff. Note that up until this point, the mobile has been blissfully unaware that the network has been laying the groundwork (e.g., allocating a channel in the new BS and allocating a path from the visited MSC to the new BS) for a handoff.
6. The mobile and the new BS exchange one or more messages to fully activate the new channel in the new BS.
7. The mobile sends a handoff complete message to the new BS, which is forwarded up to the visited MSC. The visited MSC then reroutes the ongoing call to the mobile via the new BS.

8. The resources allocated along the path to the old BS are then released.

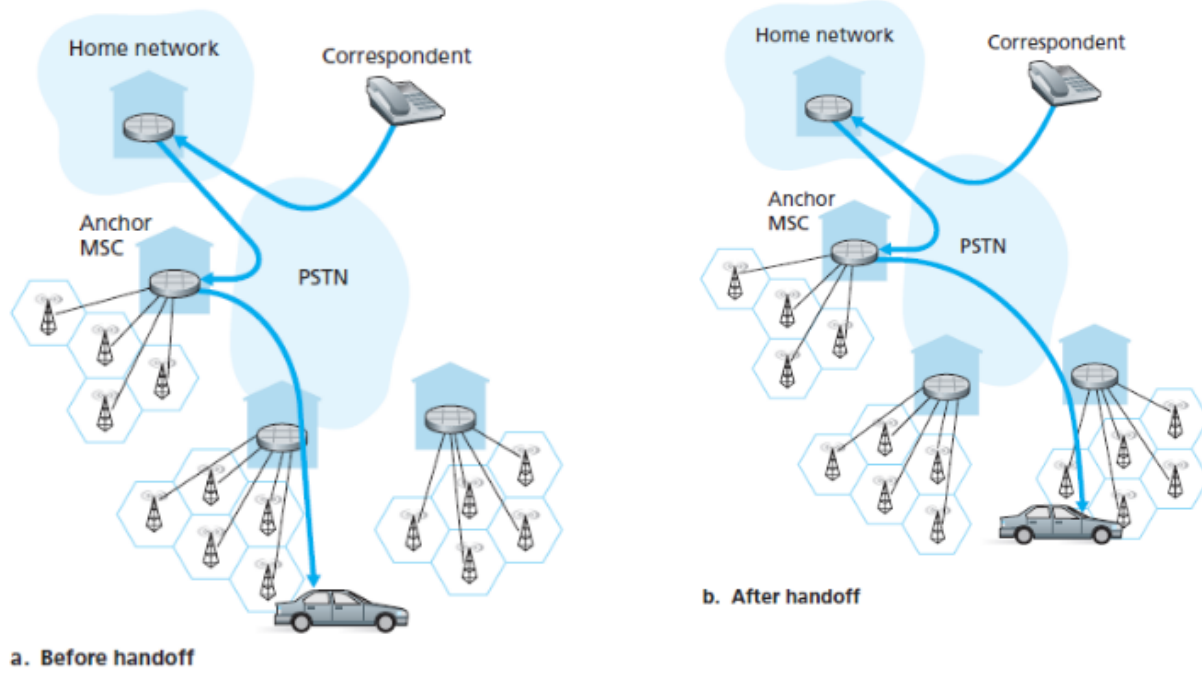


Figure 7.33 Rerouting via the anchor MSC

Table 7.2 Commonalities between mobile IP and GSM mobility

GSM element	Comment on GSM element	Mobile IP element
Home system	Network to which the mobile user's permanent phone number belongs.	Home network
Gateway mobile switching center or simply home MSC, Home location register (HLR)	Home MSC: point of contact to obtain routable address of mobile user. HLR: database in home system containing permanent phone number, profile information, current location of mobile user, subscription information.	Home agent
Visited system	Network other than home system where mobile user is currently residing.	Visited network
Visited mobile services switching center, Visitor location register (VLR)	Visited MSC: responsible for setting up calls to/from mobile nodes in cells associated with MSC. VLR: temporary database entry in visited system, containing subscription information for each visiting mobile user.	Foreign agent
Mobile station roaming number (MSRN) or simply roaming number	Routable address for telephone call segment between home MSC and visited MSC, visible to neither the mobile nor the correspondent.	Care-of address

Capítulo 8: Security in Computer Networks

What Is Network Security?

Se identifican las siguientes propiedades para una comunicación segura:

- **Confidencialidad.** Únicamente el sender y el receiver deberían poder entender el contenido del mensaje transmitido. Esto requiere que el mensaje sea encriptado tal que si alguien interviene, no lo entienda.

- **Integridad del mensaje.** Que el contenido no sea alterado en el camino, ni por accidente ni intencionalmente.
- **Autenticación.** Tanto el sender como el receiver tendrían que poder confirmar la identidad del otro, poder asegurar que la otra entidad es quien dice ser.
- **Seguridad operacional.** Casi todas las organizaciones están ligadas a la internet pública, por lo tanto pueden ser comprometidas. Se necesitaría algún detector de intrusos o firewalls.

Principles of Cryptography

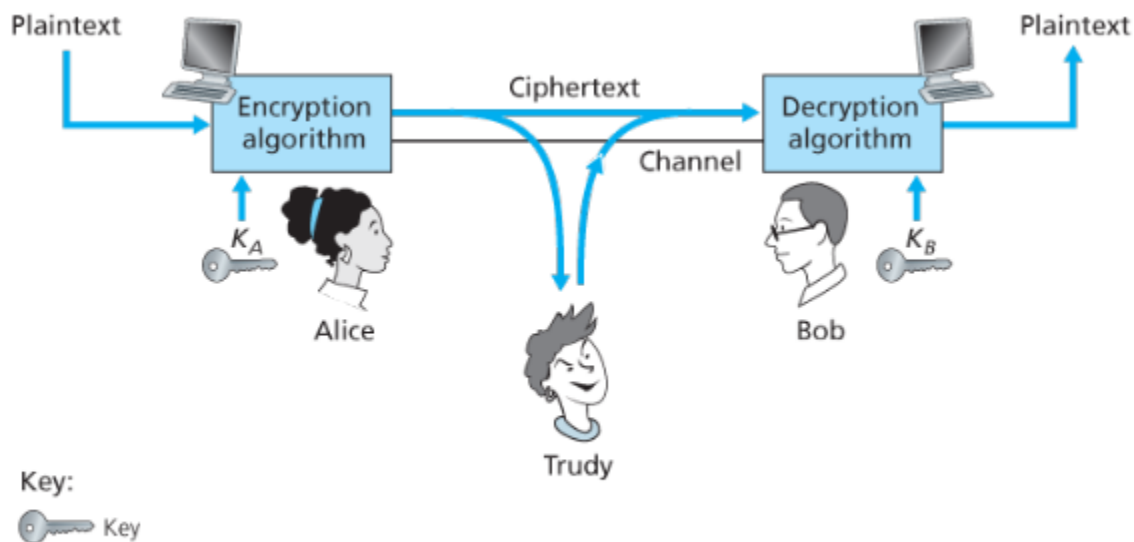


Figure 8.2 Cryptographic components

El mensaje original se conoce como **plaintext**, al encriptarlo con un algoritmo de encriptación pasa a ser un **ciphertext**. En general, las técnicas de encriptación son conocidas, lo que hace que un intruso no pueda decifrar el mensaje aún conociendo el método de encriptación son las **keys**. Hay dos tipos de sistemas: **symmetric key systems** y **public key systems**.

Symmetric key cryptography

Algunos escenarios en los que un intruso podría romper la encriptación:

- Ciphertext-only attack. En algunos casos, el intruso puede tener acceso sólo al ciphertext, sin conocimiento sobre el contenido del mensaje. Un análisis estadístico puede ayudar en este tipo de ataques.

- Known-plaintext attack. Si el intruso conociera algunas palabras que aparecen en el mensaje, podría determinar la encriptación de ciertas letras.
- Chose-plaintext attack. En este ataque, el intruso puede elegir el plaintext y obtener su correspondiente ciphertext.

Plaintext letter:	a b c d e f g h i j k l m n o p q r s t u v w x y z
Ciphertext letter:	m n b v c x z a s d f g h j k l p o i u y t r e w q

Figure 8.3 A monoalphabetic cipher

Plaintext letter:	a b c d e f g h i j k l m n o p q r s t u v w x y z
$C_1(k = 5)$:	f g h i j k l m n o p q r s t u v w x y z a b c d e
$C_2(k = 19)$:	t u v w x y z a b c d e f g h i j k l m n o p q r s

Figure 8.4 A polyalphabetic cipher using two Caesar ciphers

Block Ciphers: se usa en varios protocolos de Internet tipo SSL (for secure TCP), PGP (for secure email) y IPsec. El mensaje a encriptar se procesa en bloques de k bits y cada bloque se encripta independientemente. El número de posibles mapeos es 2^k !

Table 8.1 A specific 3-bit block cipher

input	output	input	output
000	110	100	011
001	111	101	010
010	101	110	000
011	100	111	001

Es difícil de implementar porque para un k medianamente bueno (≥ 64) se necesitarían guardar muchos valores (2^{64} !). Lo que hacen es usar funciones que simulan tablas permutadas aleatoriamente.

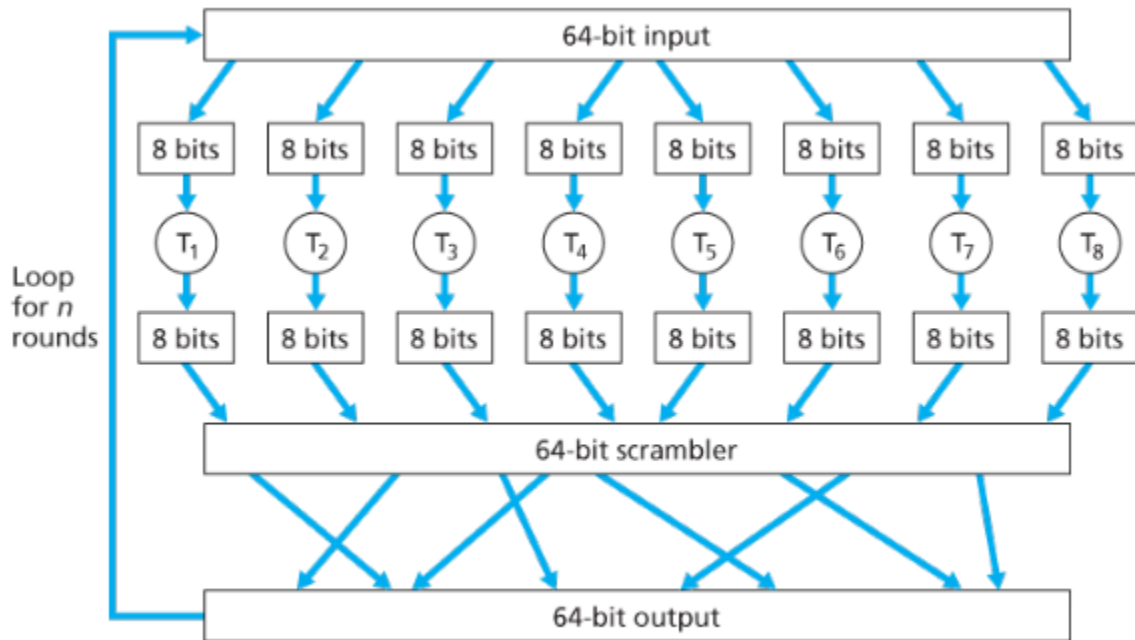


Figure 8.5 An example of a block cipher

Cipher-Block Chaining: Un problema de los block ciphers es que puede que dos o más bloques sean idénticos, o sea que habría idénticos ciphertext y a partir de ahí, el intruso podría adivinar o descifrar el resto. Para solucionarlo lo que se hace es que el sender cree un número de k -bits random $r(i)$ para cada bloque i y calcule $c(i) = KS(m(i) \oplus r(i))$ [$m(i)$ es el bloque del plaintext, $c(i)$ es el bloque del ciphertext], a partir de ahí por más que sea el mismo texto el ciphertext va a ser distinto.

Ahora por cada cipher bit, se tiene que mandar un random bit $\Rightarrow$ se duplica el bandwidth necesario; por lo que usualmente lo que se utiliza es la técnica **Cipher Block Chaining (CBC)** $\rightarrow$ se envía un único número random con el primer mensaje y luego el sender y el receiver arman los bloques con el número subsecuente:

1. Before encrypting the message (or the stream of data), the sender generates a random k -bit string, called the Initialization Vector (IV). Denote this initialization vector by $c(0)$. The sender sends the IV to the receiver in cleartext.
2. For the first block, the sender calculates $m(1) \oplus c(0)$, that is, calculates the exclusive-or of the first block of cleartext with the IV. It then runs the result through

the block-cipher algorithm to get the corresponding ciphertext block; that is, $c(1) = KS(m(1) \oplus c(0))$. The sender sends the encrypted block $c(1)$ to the receiver.

- For the i th block, the sender generates the i th ciphertext block from $c(i) = KS(m(i) \oplus c(i-1))$.

Public Key Encryption

Una dificultad de la encriptación simétrica es que ambos involucrados precisan saber la key, pero eso requiere comunicación!

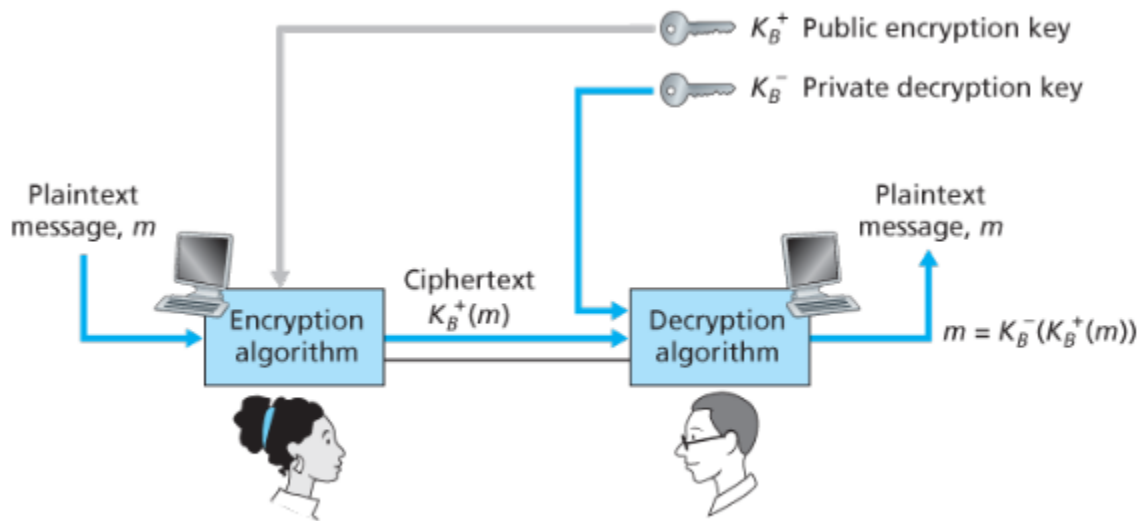


Figure 8.6 Public key cryptography

La idea es que en vez de que ambos compartan una clave privada, el receptor tenga dos claves: una pública y otra privada (sólo la sabe el receptor). El sender primero obtiene la clave pública del receiver y la utiliza para encriptar el mensaje. El receiver lo descripta utilizando la clave privada.

Dos problemas con esto son: 1. chosen-plaintext attack porque se conoce el algoritmo y la clave pública para encriptar cualquier mensaje y 2. cualquiera podría hacerse pasar por el sender y enviar un mensaje.

RSA → tiene dos componentes interrelacionados: 1. la elección de la public key y la private key, 2. el algoritmo de encriptación y descriptación.

To generate the public and private RSA keys, Bob performs the following steps:

1. Choose two large prime numbers, p and q . How large should p and q be? The larger the values, the more difficult it is to break RSA, but the longer it takes to perform the encoding and decoding. RSA Laboratories recommends that the product of p and q be on the order of 1,024 bits.
2. Compute $n=pq$ and $z=(p-1)(q-1)$.
3. Choose a number, e , less than n , that has no common factors (other than 1) with z . (In this case, e and z are said to be relatively prime.) The letter e is used since this value will be used in encryption.
4. Find a number, d , such that $ed-1$ is exactly divisible (that is, with no remainder) by z . The letter d is used because this value will be used in decryption. Put another way, given e , we choose d such that $ed \bmod z = 1$.
5. The public key that Bob makes available to the world, KB^+ , is the pair of numbers (n, e) ; his private key, KB^- , is the pair of numbers (n, d) .

La encriptación del sender se hacer haciendo: $c = m^e \bmod n$

Para descriptarlo, $m = c^d \bmod n$

Message Integrity and Digital Signatures

La necesidad de la integridad de mensajes es que quien reciba el mensaje pueda verificar que el sender fue quien realmente lo creó y que nadie haya alterado el contenido del mensaje en el camino.

Cryptographic Hash Functions

Una función hash toma un input m y le aplica un hash $H(m)$. Para que sea criptográfico tiene que cumplirse que sea inviable computacionalmente encontrar dos mensajes distintos x e y tal que $H(x)=H(y)$. En criollo, que sea inviable para un intruso reemplazar el mensaje por otro que tenga aplicado el mismo hash.

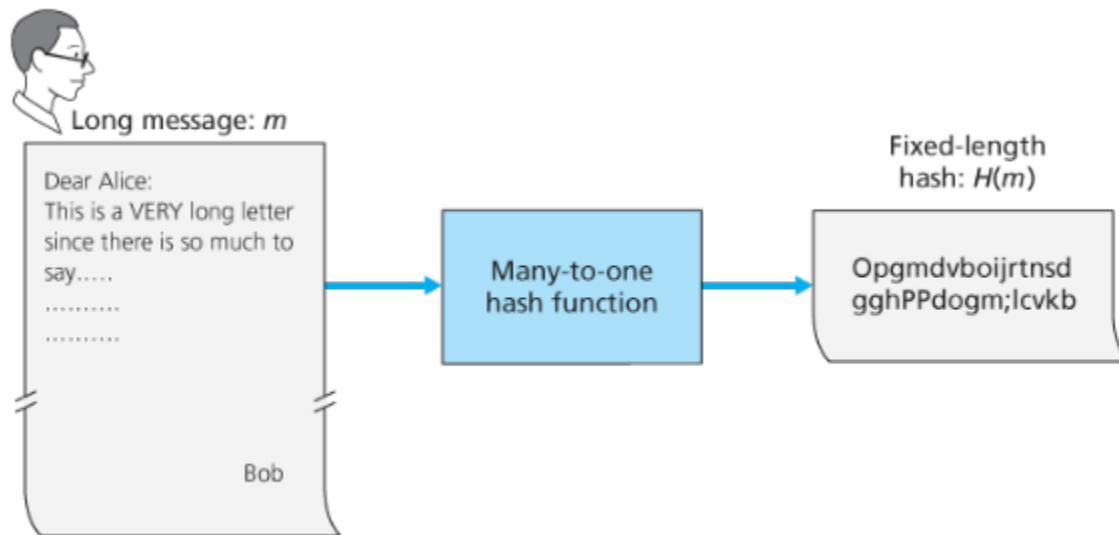


Figure 8.7 Hash functions

El checksum, por ejemplo, es malísimo porque es fácil encontrar otro mensaje que de igual checksum. Algunos de los algoritmos más usados actualmente son MD5 y SHA-1.

Message Authentication Code

Para obtener integridad del mensaje, ambas partes van a tener que compartir una clave s , esta clave es la **authentication key**.

1. El sender crea el mensaje m , le concatena s tal que crea $m+s$ y calcula el hash $H(m+s)$, este último se llama message authentication code (MAC).
2. El sender concatena el MAC al mensaje original, creando un mensaje extendido $(m, H(m+s))$ y se lo envía al receiver.
3. El receiver recibe (m, h) y conociendo s , calcula el MAC $H(m+s)$. Si $H(m+s)=h \Rightarrow$ se concluye que todo está ok.

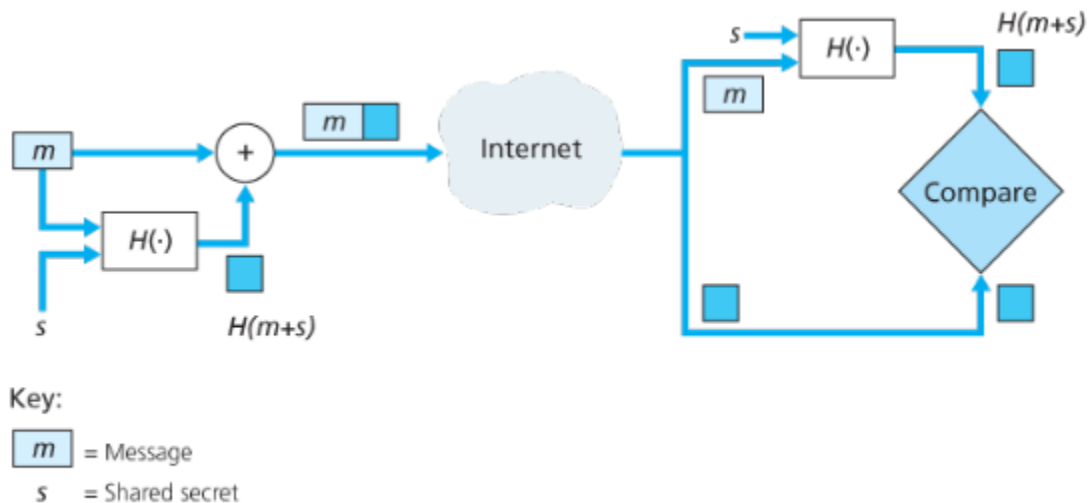


Figure 8.9 Message authentication code (MAC)

Cómo se distribuye esta clave compartida? Físicamente por el administrados de red o encriptandola.

Digital Signatures

La idea es que se pueda firmar/autenticar algo de manera verificable e infalsificable.

Lo que se puede hacer es que quien firma lo haga con su clave privada, esta clave se encripta. Quien lo recibe debe desencriptarlo usando la clave pública y tiene que pasar que $KB + (KB - (m)) = m$. Si eso no se cumple es porque no está todo ok.

Un problema es que es costoso computacionalmente $\Rightarrow$ aparecen las funciones hash. Entonces, el sender firma el mensaje hasheado en vez del mensaje original, o sea, $KB - (H(m))$; esto es mejor porque en general $H(m)$ es mucho más chico que m .

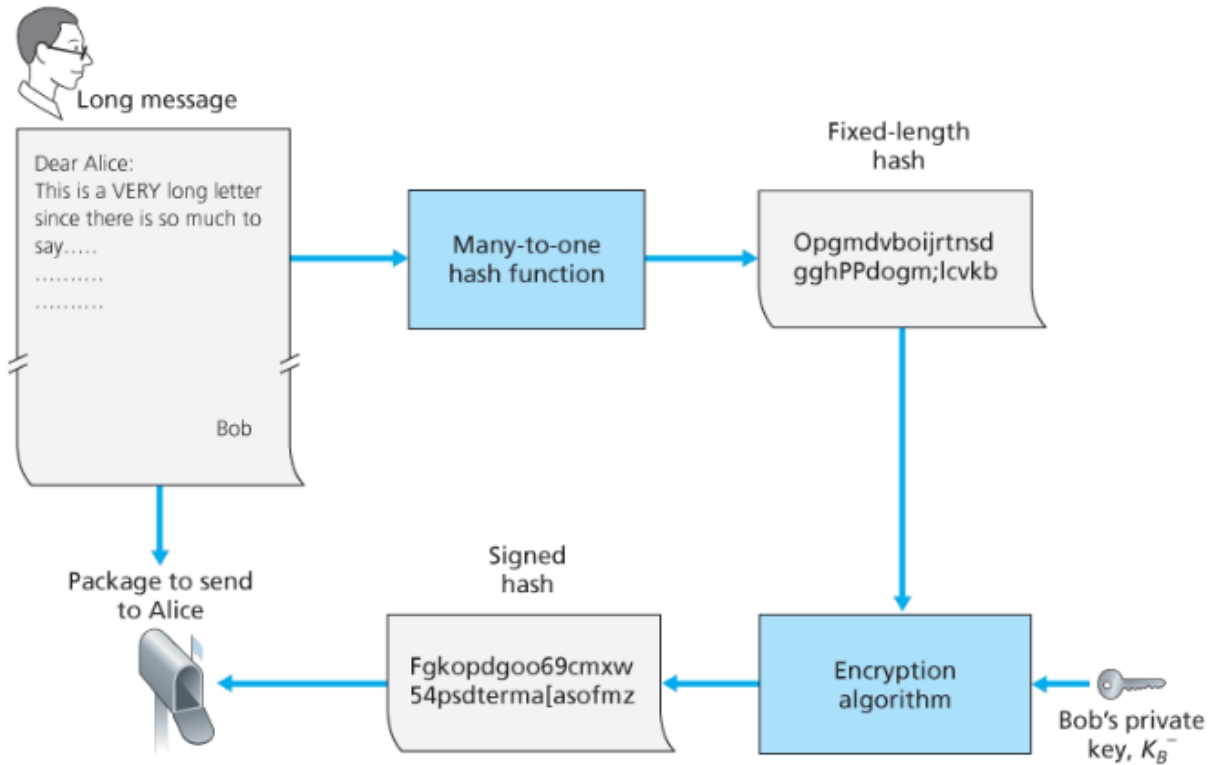


Figure 8.11 Sending a digitally signed message

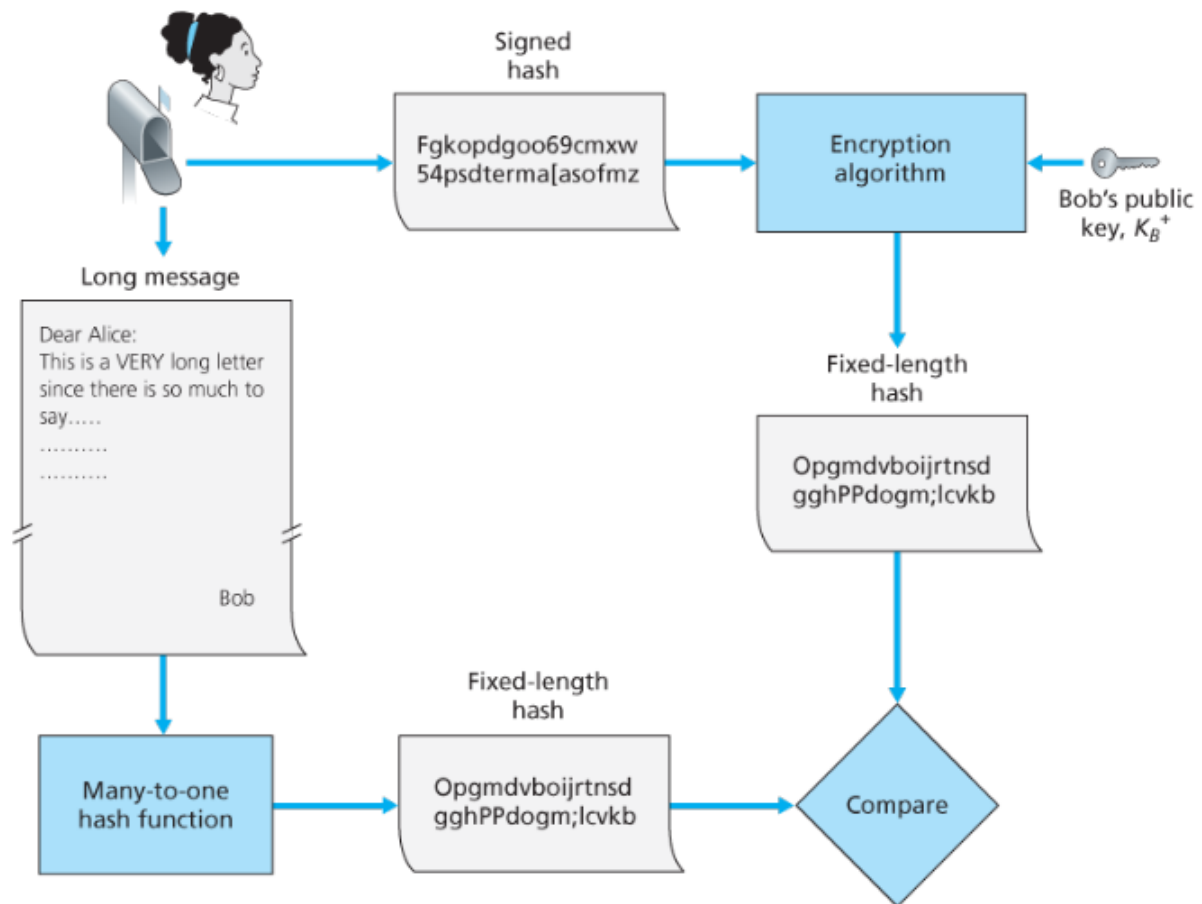


Figure 8.12 Verifying a signed message

Una aplicación importante de las firmas digitales es la de la **certificación de claves públicas (public key certification)**, esto es certificar que una clave pública pertenece a una entidad específica.

Caso:

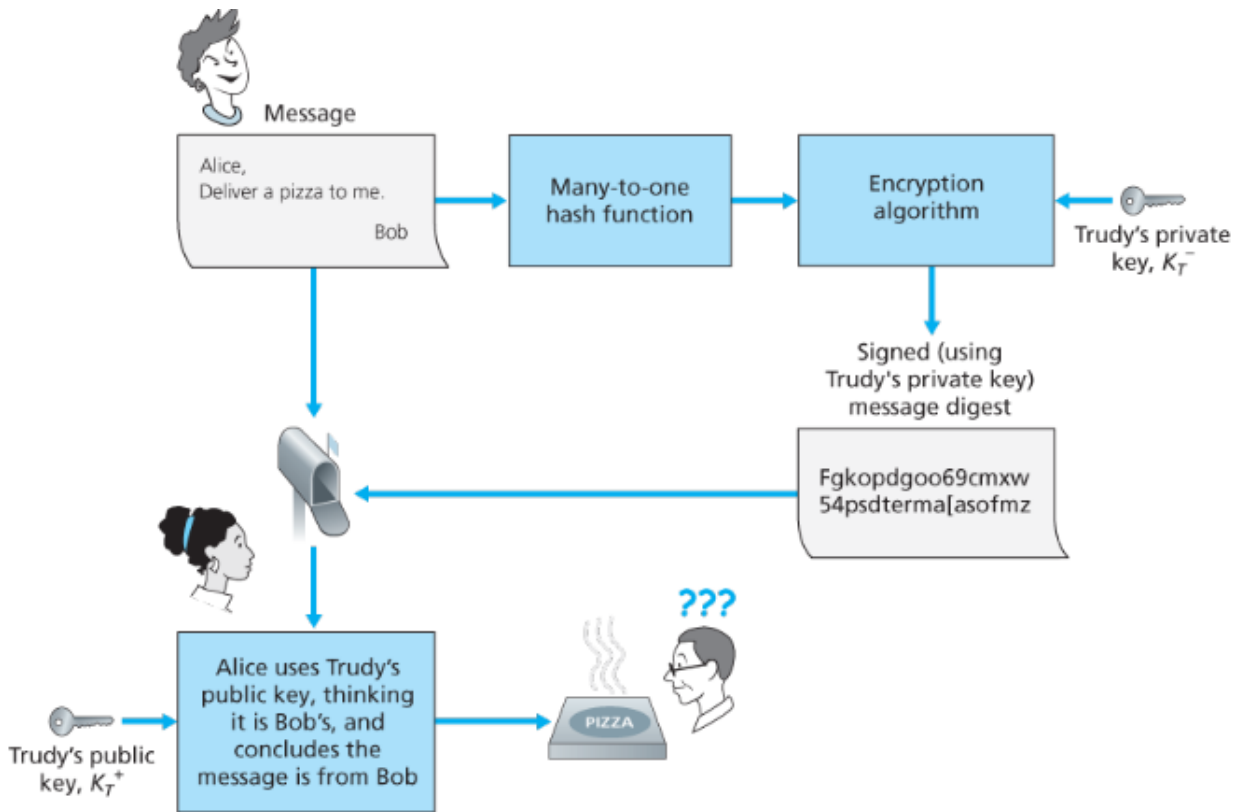


Figure 8.13 Trudy masquerades as Bob using public key cryptography

Para que sea útil usar public key cryptography, tiene que poder verificarse la identidad de quien envía. En general, quien vincula una clave pública con una entidad es el **Certification Authority (CA)**, su trabajo es validar identidades y emitir certificados.

1. Un CA verifica que una persona es quien dice ser. No hay procedimientos estándar para hacerlo, uno confía en el CA. O sea que puedes confiar en la identidad del ente tanto cómo confíes en el CA que lo certifica.
2. El certificado que expide contiene la clave pública y algún identificador único global del dueño de la clave. El certificado está firmado digitalmente por la CA.

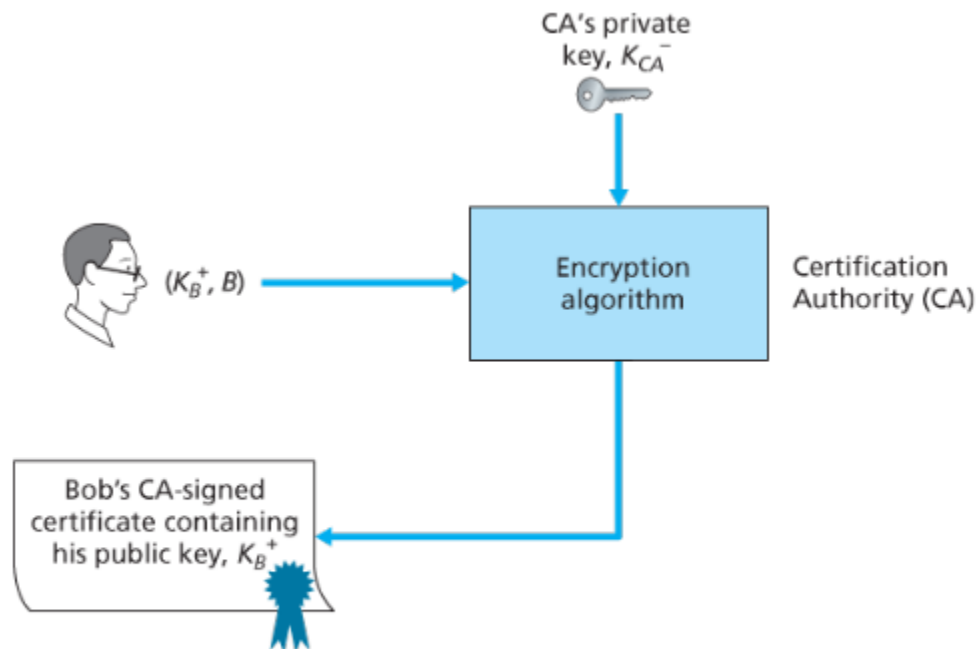


Figure 8.14 Bob has his public key certified by the CA

End-Point Authentication

Es el proceso en el que una entidad le prueba a otra su identidad mediante una computer network, por ejemplo, un usuario probándole su identidad a un servidor de email. Para las autenticaciones dentro de la red se necesitan protocolos que se ejecuten previo a la comunicación entre las partes.

Authentication Protocol ap1.0

La forma más simple es en la que el sender le envía un mensaje diciéndole quién es... pero no hay manera de que el receiver compruebe la identidad.

Authentication Protocol ap2.0

Podría verificarse a partir de la dirección IP, el receiver lo puede chequear en el datagrama. Pero podría caer en IP spoofing → se crea un datagrama con cualquier dirección IP que se quiera y ahí te haces pasar por otra entidad.

Authentication Protocol ap3.0

Utilizando una password secreta. El riesgo está si el intruso espía las comunicaciones del sender y logra averiguar su contraseña. Se podría encriptar la contraseña (ap3.1)

pero igualmente el intruso podría obtener la contraseña encriptada y a partir de allí hacerse pasar por otra identidad.

Authentication Protocol ap4.0

El problema del anterior termina siendo que el receiver no puede saber que el sender está en “tiempo real” del otro lado de la comunicación o si el mensaje que está recibiendo es un playback de la autenticación previa. Algo similar pasaba con TCP (un servidor TCP no quiere aceptar una conexión si el SYN es una copia de un SYN más viejo), se resolvía enviando un sequence number nuevo que no haya sido usado en un tiempo, el servidor respondía con un ACK reconociendo el mismo sequence number.

Un **nonce** es un número que el protocolo utiliza una **única** vez.

1. El sender envía un mensaje al receiver
2. El receiver elige un nonce, R y se lo envía al sender
3. El sender encripta el nonce usando una symmetric secret key K_{A-B} y lo envía de vuelta al receiver. Es el hecho de que el sender conoce K_{A-B} y la usa para encriptar un valor, el que hace que el receiver sepa que el sender es quien dice ser. El nonce asegura que está ocurriendo todo en vivo
4. El receiver descrypta el mensaje. Si el nonce descryptado coincide con el que había enviado originalmente, queda probada la autenticidad del sender

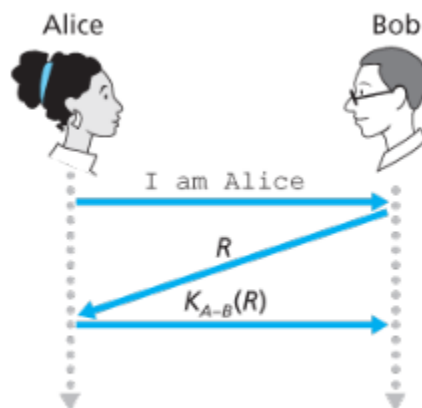


Figure 8.18 Protocol ap4.0 and a failure scenario

Securing E-Mail

Por qué se proveería seguridad en más de una capa? Primero porque que haya seguridad en la capa de red, no quita que se necesite hacer algún tipo de autenticación en la aplicación de un e-commerce, por ejemplo. Segundo porque es más facil agregar seguridad en capas más altas mientras se espera que se despliegue/desarrolle seguridad en las capas más bajas.

En el diseño de un sistema de emails seguro, se desea: confidencialidad, autenticación del sender, integridad del mensaje y autenticación del receiver.

Para la confidencialidad se utiliza public key cryptography con una session key $\Rightarrow$

1. el sender selecciona de manera random una session key simétrica K_S
2. encripta el mensaje m con la clave simétrica
3. encripta la clave simétrica con la clave pública del receiver (K_B^+)
4. arma un paquete concatenando el mensaje encriptado y la clave simétrica encriptada
5. envía el paquete al receiver

El receiver utiliza su clave privada K_B^- para obtener K_S y utiliza K_S para descryptar el mensaje m .

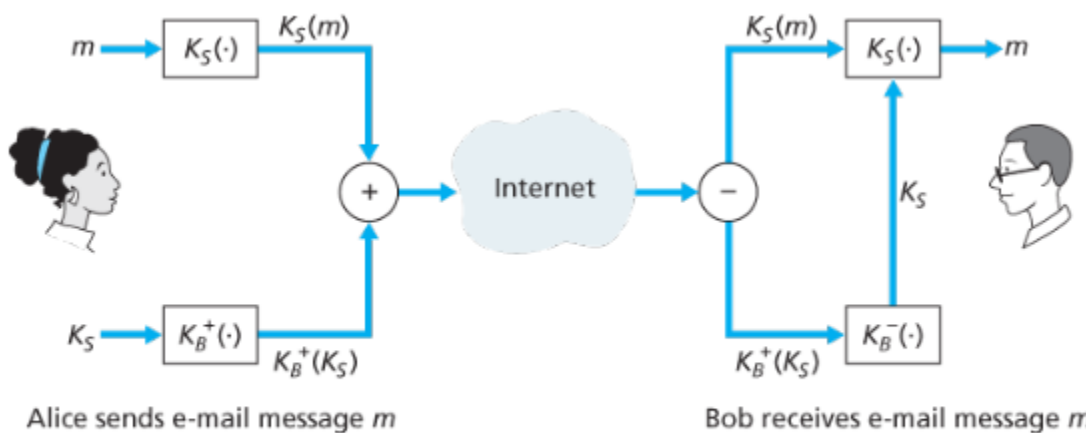


Figure 8.19 Alice used a symmetric session key, K_S , to send a secret e-mail to Bob

Para la autenticación e integridad se usan firmas digitales y mensajes “resumidos”.

1. el sender le aplica una función hash al mensaje para obtener el mensaje resumido
2. firma el mensaje hasheado con su clave privada

3. concatena al mensaje original la firma
4. envía el paquete al receiver

El receiver aplica la clave pública al paquete que recibió y compara el resultado con su mensaje hashado.

Combinando ambos métodos se obtiene un mecanismo que provee confidencialidad, autenticación e integridad →

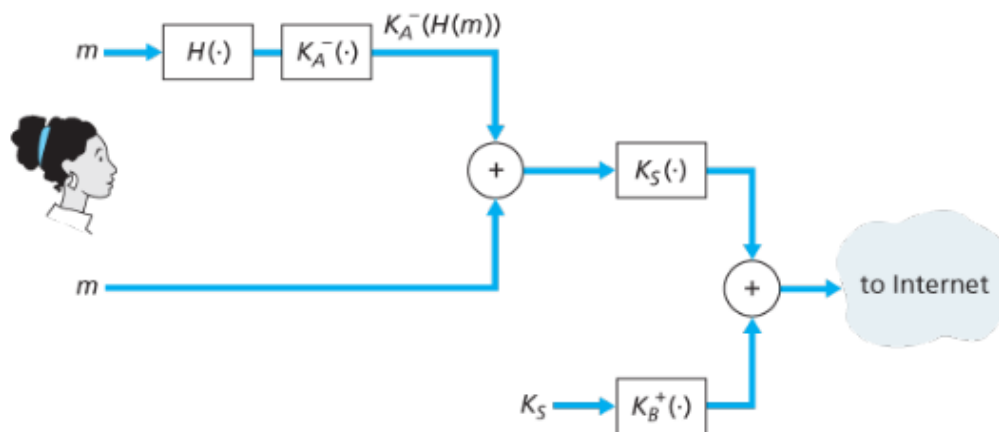
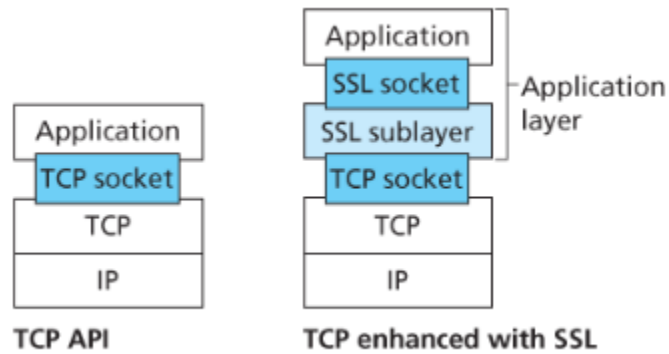


Figure 8.21 Alice uses symmetric key cyptography, public key cryptography, a hash function, and a digital signature to provide secrecy, sender authentication, and message integrity

Así como está ambas partes necesitan conocer la clave pública de la otra → podría haber un ataque tipo masquerade, que se puede resolver certificando las claves públicas.

Securing TCP Connections: SSL

SSL (Secure Sockets Layer) se utiliza para asegurar conexiones TCP, por ejemplo para pagos online. Provee confidencialidad, integridad de los datos, autenticación de cliente y servidor.



En una versión simplificada de SSL se podrían distinguir tres fases: handshake, key derivation y data transfer.

En el handshake se establece la conexión TCP, se verifica que el servidor sea realmente el servidor y se le envía al servidor una clave secreta que van a usar cliente-servidor para generar las claves simétricas.

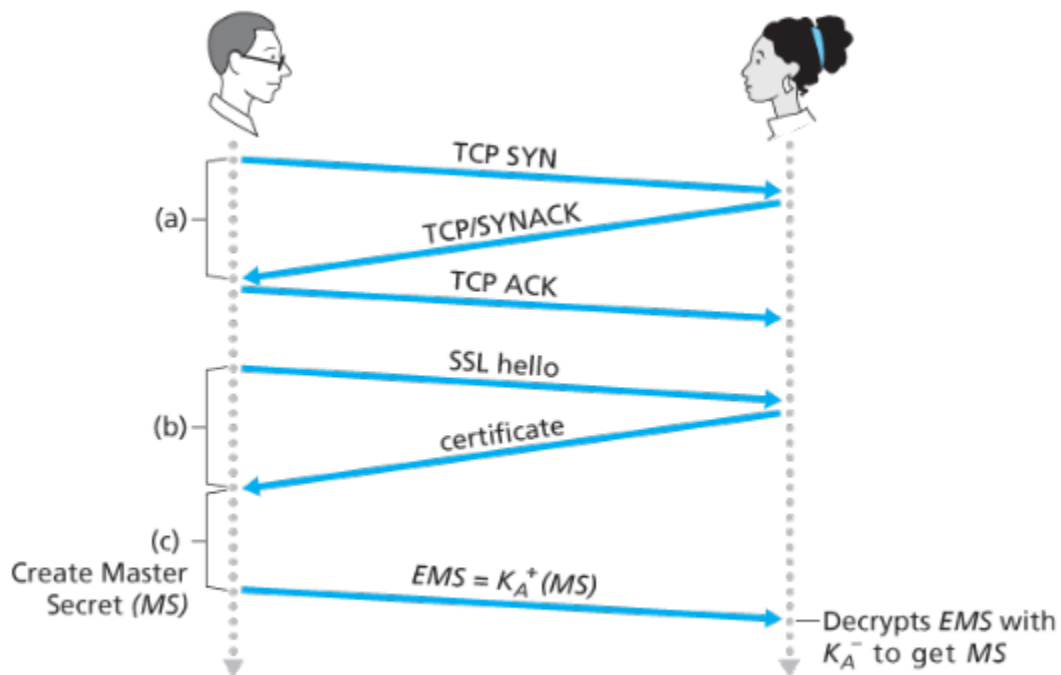


Figure 8.25 The almost-SSL handshake, beginning with a TCP connection

En SSL posta durante el handshake se puede acordar el algoritmo criptográfico. Además, durante el handshake ambas partes se envían nonces que se van a usar para el key derivation.

1. *The client sends a list of cryptographic algorithms it supports, along with a client nonce.*
2. *From the list, the server chooses a symmetric algorithm (for example, AES), a public key algorithm (for example, RSA with a specific key length), and a MAC algorithm. It sends back to the client its choices, as well as a certificate and a server nonce.*
3. *The client verifies the certificate, extracts the server's public key, generates a Pre-Master Secret (PMS), encrypts the PMS with the server's public key, and sends the encrypted PMS to the server.*
4. *Using the same key derivation function (as specified by the SSL standard), the client and server independently compute the Master Secret (MS) from the PMS and nonces. The MS is then sliced up to generate the two encryption and two MAC keys. Furthermore, when the chosen symmetric cipher employs CBC (such as 3DES or AES), then two Initialization Vectors (IVs)—one for each side of the connection—are also obtained from the MS. Henceforth, all messages sent between client and server are encrypted and authenticated (with the MAC).*
5. *The client sends a MAC of all the handshake messages.*
6. *The server sends a MAC of all the handshake messages.*

Los últimos dos pasos son para verificar que no haya habido manipulación durante el handshake.

Los nonces se usan para defenderse del ataque “connection replay” y los sequence number se usan para defenderse de la reproducción de paquetes individuales durante la sesión en curso.

En key derivation tanto cliente como servidor utilizan la clave secreta para generar cuatro claves nuevas:

EB (session encryption key for data sent from client to server),

MB (session MAC key for data sent from client to server),

EA (session encryption key for data sent from server to client),

MA (session MAC key for data sent from server to client)

En data transfer se envían los datos por la conexión TCP ya establecida.



Figure 8.26 Record format for SSL

El campo type indica si el mensaje es un handshake o contiene datos de la aplicación. También se usa para cerrar la conexión.

Network-Layer Security: IPsec and Virtual Private Networks

IPsec asegura datagramas entre dos entidades cualquiera de capa de red, incluyendo hosts y routers. Muchas instituciones lo usan para crear VPNs (virtual private networks).

Confidencialidad en la capa de red implica que los payloads de todos los datagramas sean encriptados, esto puede ser un segmento TCP/UDP, un mensaje ICMP, etc; se dice que es un “blanket coverage”. Además de confidencialidad, IPsec provee mecanismos para la integridad, autenticación y prevención de ataques de tipo replay.

IPsec and Virtual Private Networks (VPNs)

Una institución que abarca varias regiones geográficas, en general va a querer tener su propia red IP tal que sus hosts y servidores puedan enviarse información de manera segura. Para esto, podría desplegar su propia red física → private network. Pero eso puede ser muy costoso porque se necesita costear y mantener toda la infraestructura física, lo que se hace es crear una VPN. La VPN encripta todo el tráfico antes de enviarlo a la internet pública.

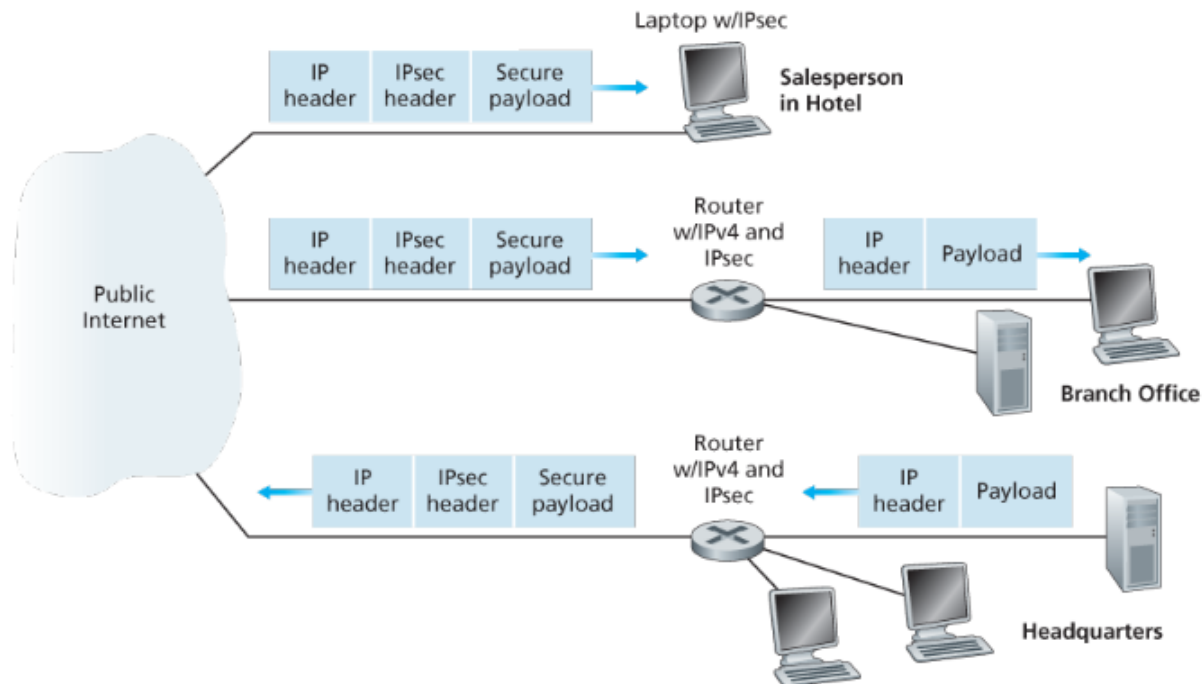


Figure 8.27 Virtual private network (VPN)

Lo que pasa entre el router y el host es que un datagrama común se transforma en un datagrama tipo IPsec (o viceversa según el sentido del mensaje). Este datagrama IPsec tiene los mismos headers que un datagrama normal para que sea tratado como tal, pero el payload guarda los headers del IPsec (todo encriptado) para que después se pueda procesar. Cuando el datagrama llega al host, el SO lo desencripta y lo pasa desencriptado a la capa superior (por ej. a TCP/UDP).

The AH and ESP Protocols

Dentro del protocolo IPsec hay dos protocolos principales: **Authentication Header (AH)** y **Encapsulation Security Payload (ESP)**. El protocolo AH provee autenticación del origen e integridad de datos pero no provee confidencialidad. El protocolo ESP provee autenticación del origen, integridad de datos y confidencialidad. En general es este último protocolo el que se utiliza.

Security Associations

Antes de enviar datagramas IPsec, las entidades fuente y destino deben crear una conexión lógica en la capa de red. Esta conexión es una **security association (SA)**. Es una conexión unilateral desde la fuente al destino.

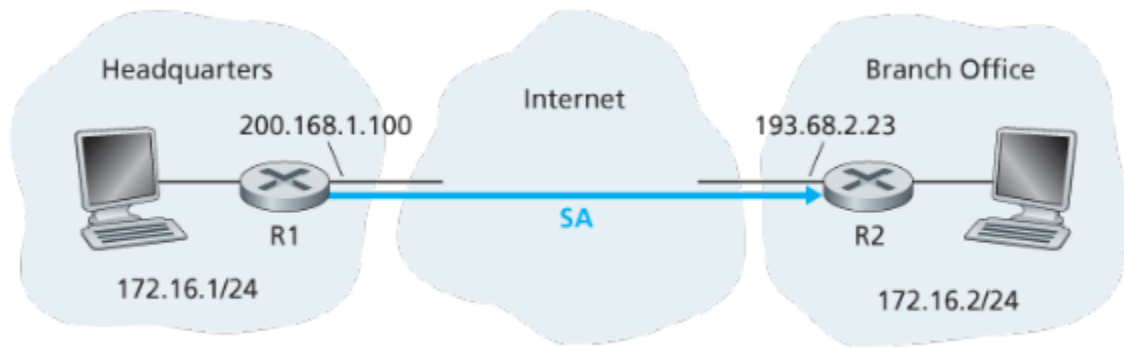


Figure 8.28 Security association (SA) from R1 to R2

R1 y R2 van a mantener cierta información sobre el SA, a saber:

- Un identificador de 32 bits conocido como **Security Parameter Index (SPI)**
- La interfaz del SA origen (200.168.1.100) y la del destino (193.68.2.23)
- El tipo de encriptación
- La clave de encriptación
- El tipo de chequeo de integridad
- La clave de autenticación

Con esta información es que se van a encriptar/desencriptar los datagramas.

The IPsec Datagram

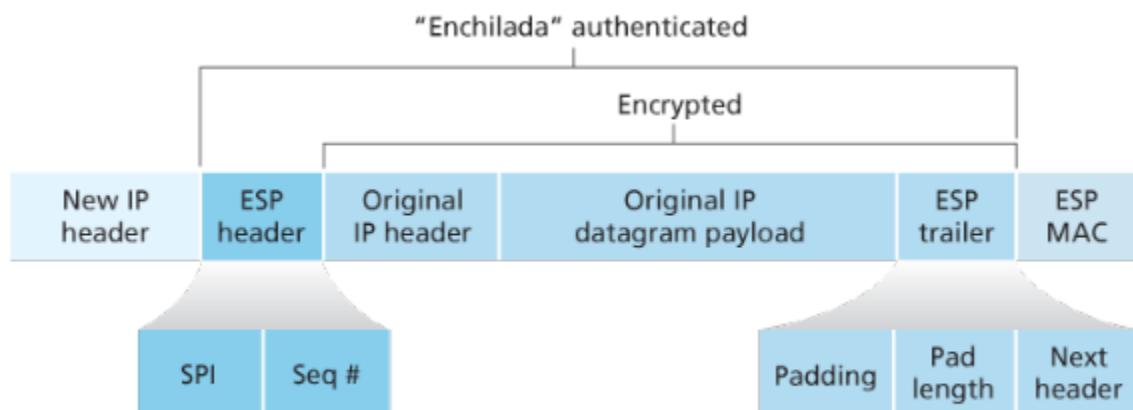


Figure 8.29 IPsec datagram format

Receta para convertir el datagrama original en uno IPsec:

- Appends to the back of the original IPv4 datagram (which includes the original header fields!) an “ESP trailer” field
- Encrypts the result using the algorithm and key specified by the SA
- Appends to the front of this encrypted quantity a field called “ESP header”; the resulting package is called the “enchilada”
- Creates an authentication MAC over the whole enchilada using the algorithm and key specified in the SA,
- Appends the MAC to the back of the enchilada forming the payload
- Finally, creates a brand new IP header with all the classic IPv4 header fields (together normally 20 bytes long), which it appends before the payload.

El protocol number se setea en “50” para indicar que es de tipo IPsec ESP.

Securing Wireless LANs

Wired Equivalent Privacy (WEP)

El protocolo WEP se diseñó para proveer autenticación y encriptación de datos entre un host y un punto de acceso inalámbrico utilizando el approach de shared symmetric key. WEP no especifica un algoritmo para el manejo de la key, se asume que ya fue acordado entre el host y punto de acceso por otra vía. La autenticación se lleva a cabo de la siguiente manera:

1. Un host inalámbrico solicita autenticación por un punto de acceso
2. El punto de acceso responde con un nonce de 128 bytes
3. El host encripta el nonce utilizando la clave compartida
4. El punto de acceso desencripta el nonce encriptado por el host

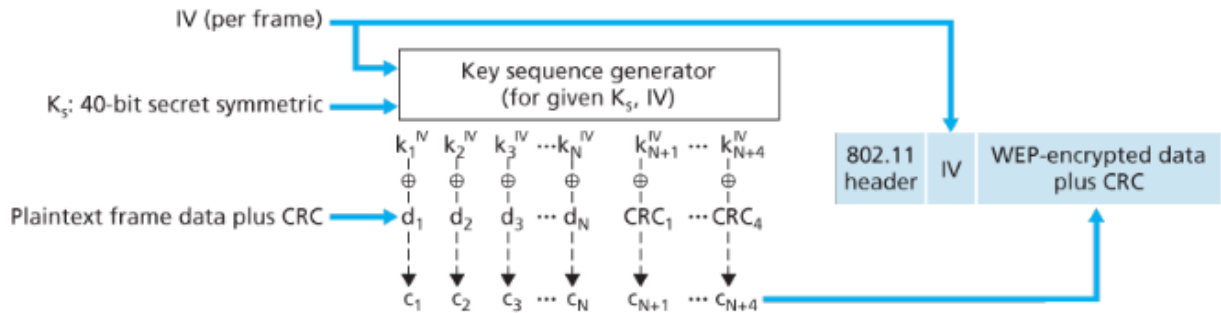


Figure 8.30 802.11 WEP protocol

IEEE 802.11i

Presenta mejoras en los mecanismos de seguridad con respecto a WEP. WEP provee una encriptación relativamente débil, una única manera de autenticación y ningún mecanismo de distribución de claves.

802.11i define un servidor de autenticación con el que se puede comunicar el punto de acceso. Opera en cuatro fases:

1. **Discovery.** El AP anuncia su presencia y las formas de autenticación y encriptación que puede proveerle al nodo cliente. El cliente solicita la manera específica de autenticación y encriptación que desea.
2. **Mutual authentication and Master Key (MK) generation.** En esta fase, el AP hace de "posta" reenviando mensajes entre el cliente y el servidor de autenticación. EAP-TLS utiliza claves públicas para la autenticación de ambas partes (cliente y serv. auth) y para la generación de la MK.

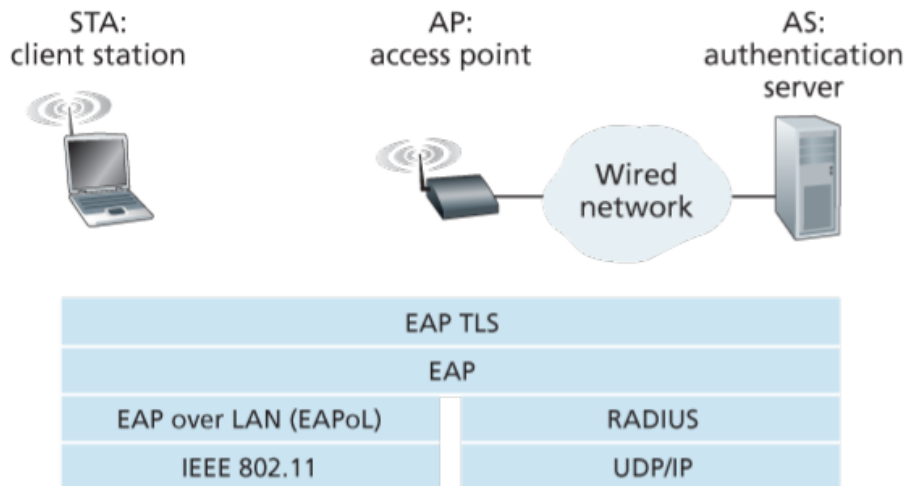


Figure 8.32 EAP is an end-to-end protocol. EAP messages are encapsulated using EAPoL over the wireless link between the client and the access point, and using RADIUS over UDP/IP between the access point and the authentication server

3. **Pairwise Master Key (PMK) generation.** La MK es una clave secreta que solo conocen el cliente y el servidor de autenticación, se utiliza para generar una segunda clave que es la PMK. Entonces, el servidor de auth le envía la PMK al AP y ahí recién el AP y el cliente van a estar compartiendo una clave y ya se dan por autenticados.
4. **Temporal Key (TK) generation.** Con la PMK, el cliente y el AP pueden generar claves temporales que van a utilizar durante la comunicación. La TK se utiliza para realizar el cifrado a nivel de enlace de los datos enviados a través del enlace inalámbrico y a un host remoto arbitrario.

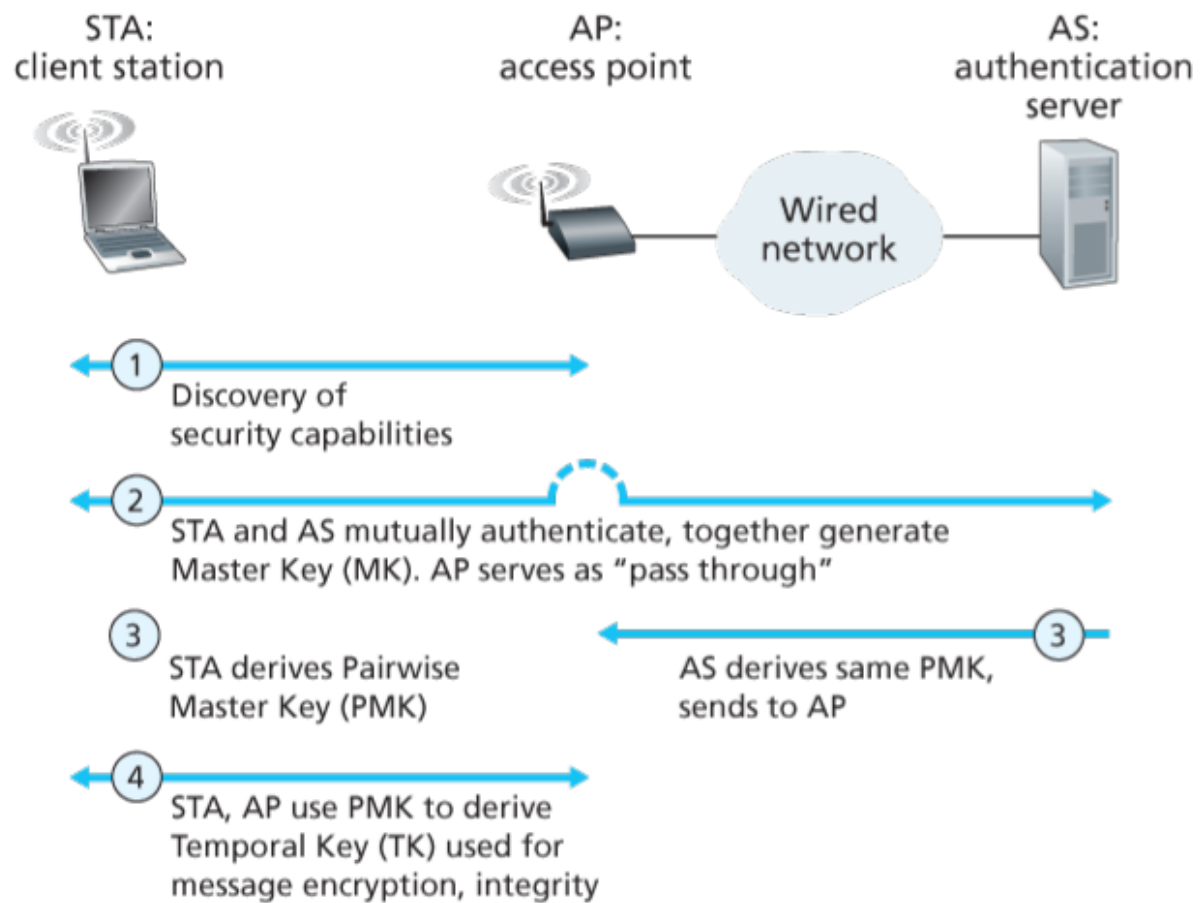


Figure 8.31 802.11i: Four phases of operation

Operational Security: Firewalls and Intrusion Detection Systems

En una red informática, cuando el tráfico que entra o sale de una red se comprueba (security-checked), registra, descarta o reenvía, lo hacen dispositivos operativos conocidos como firewalls, sistemas de detección de intrusos (IDS) y sistemas de prevención de intrusos (IPS).

Firewalls

Es una combinación de hardware y software que aísla la red interna de una organización de Internet, permitiendo pasar algunos paquetes y bloqueando otros. Un firewall permite que un administrador de red controle el acceso. Tiene tres objetivos:

- Todo el tráfico de afuera hacia adentro y viceversa, pasa por el firewall
- Solo el tráfico autorizado tiene permitido el pase

- El firewall no puede ser traspasado

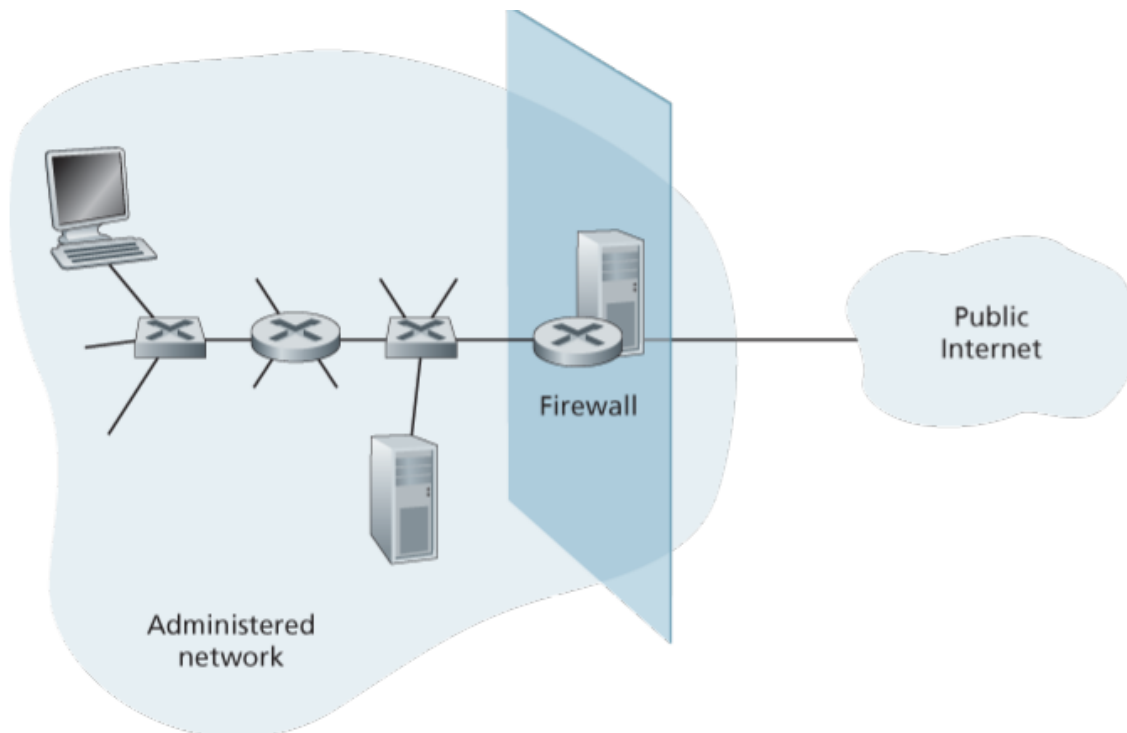


Figure 8.33 Firewall placement between the administered network and the outside world

Se los clasifica en tres categorías:

- Traditional packet filters

Se filtran paquetes en el gateway router. En general el filtro se hace en base a:

- IP origen o dirección destino
- El tipo de protocolo en el datagrama IP: TCP, UDP, OSPF, ICMP...
- Puerto origen o destino TCP o UDP
- Flags TCP: SYN, ACK, etc
- Tipo de mensaje ICMP
- Reglas para datagramas
- Reglas para router interfaces

Table 8.5 Policies and corresponding filtering rules for an organization's network 130.207/16 with Web server at 130.207.244.203

Policy	Firewall Setting
No outside Web access.	Drop all outgoing packets to any IP address, port 80.
No incoming TCP connections, except those for organization's public Web server only.	Drop all incoming TCP SYN packets to any IP except 130.207.244.203, port 80.
Prevent Web-radios from eating up the available bandwidth.	Drop all incoming UDP packets—except DNS packets.
Prevent your network from being used for a smurf DoS attack.	Drop all ICMP ping packets going to a "broadcast" address (eg 130.207.255.255).
Prevent your network from being tracerouted.	Drop all outgoing ICMP TTL expired traffic.

Las reglas de firewall se implementan en routers con listas de acceso de control.

- **Stateful packet filters**

Se trackean las conexiones TCP y con ese conocimiento se aplican filtros. Se chequea contra la tabla si un paquete nuevo es parte de una conexión ya establecida, por ejemplo.

Table 8.7 Connection table for stateful filter

source address	dest address	source port	dest port
222.22.1.7	37.96.87.123	12699	80
222.22.93.2	199.1.205.23	37654	80
222.22.65.143	203.77.240.43	48712	80

Table 8.8 Access control list for stateful filter

action	source address	dest address	protocol	source port	dest port	flag bit	check conxion
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any	
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK	X
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	—	
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	—	X
deny	all	all	all	all	all	all	

- **Application gateway**

Toman decisiones a partir de application data. Es un servidor por el que tiene que pasar toda la data de aplicaciones.

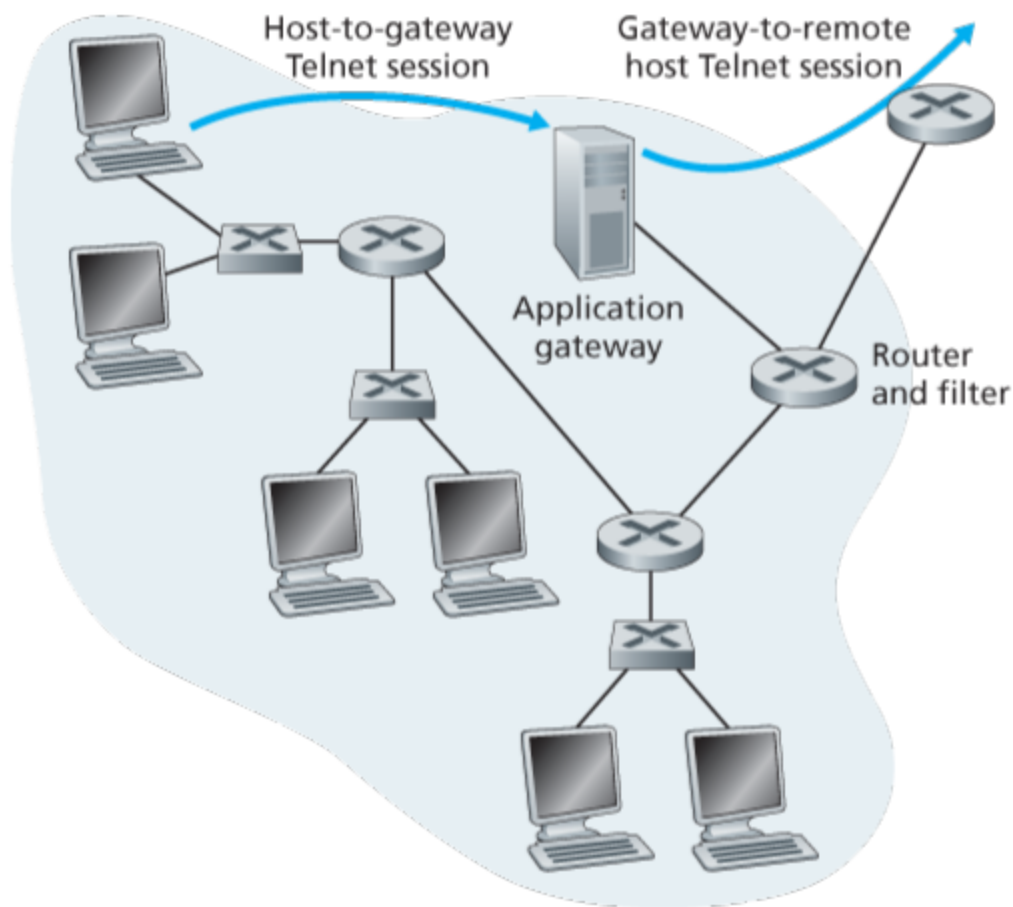
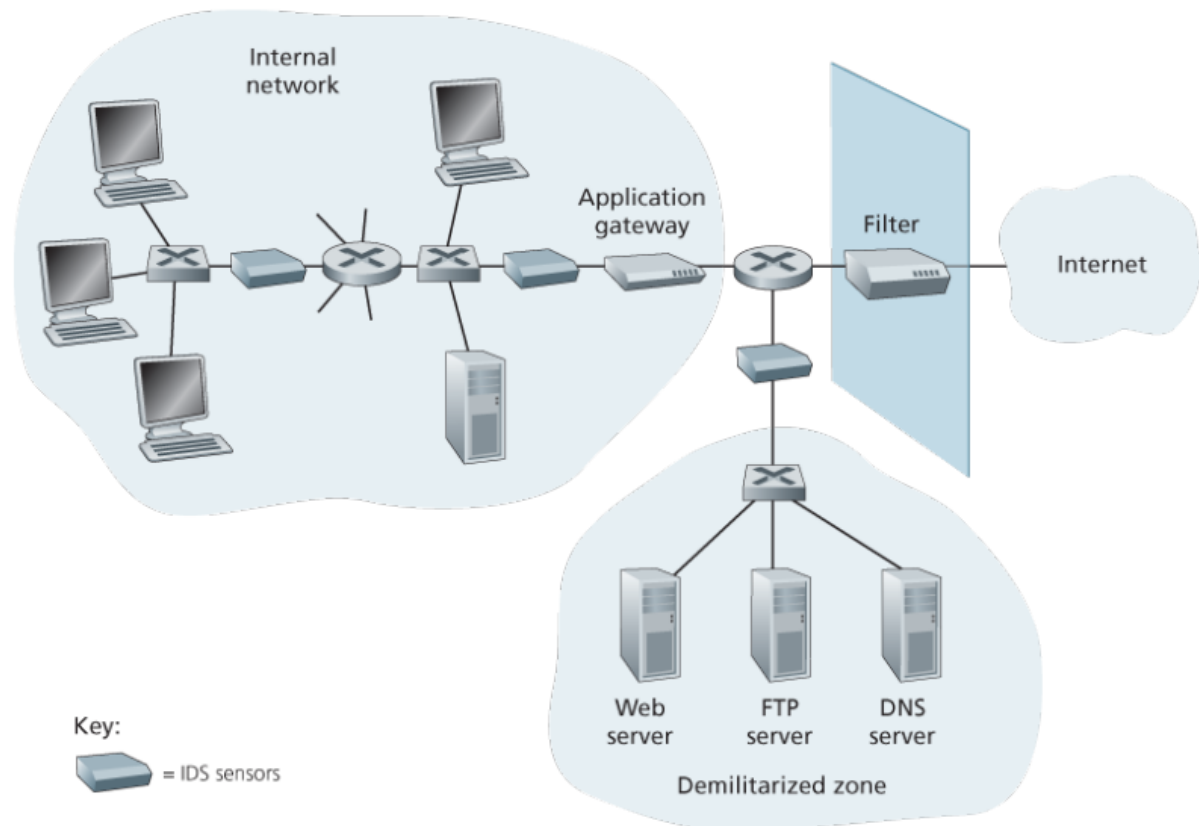


Figure 8.34 Firewall consisting of an application gateway and a filter

Una desventaja es que se necesita un application gateway por cada aplicación. No tiene buena performance si toda la data tiene que retransmitirse al gateway.

Intrusion Detection Systems

Para detectar varios ataques se necesita hacer un **deep packet inspection**, es decir mirar la data que carga el paquete más allá de los headers. Un dispositivo que genera alertas cuando observa tráfico potencialmente malicioso, se llama intrusion detection system (IDS). Un dispositivo que filtra tráfico sospechoso, se llama intrusion prevention system (IPS).



Los IDS se clasifican en: signature-based systems y anomaly-based systems.

La primera mantiene una base de datos de firmas de ataques. Cada firma es un set de reglas pertenecientes a una actividad de intrusión.

La segunda crea un perfil del tráfico a partir de la observación del tráfico en operaciones normales. Busca paquetes estadísticamente inusuales.